

FROM C TO RUST · PRODUCTION-READY REFERENCE

Rust 使用大全

面向 C 语言学习者的 14 讲系统进阶

覆盖 14 大主题:数据类型 → 运算符 → 流程控制 → 函数 → 内存管理 →
面向对象 → 文件I/O → 网络 → 并发异常 → 泛型与元编程 →
反射 → 注解装饰器 → 模块系统 → 类型系统进阶

第1章 数据类型与变量 · 第2章 运算符与表达式 · 第3章 流程控制 · 第4章 函数与方法
第5章 内存管理 · 第6章 面向对象 · 第7章 文件I/O · 第8章 网络通信
第9章 并发与异常 · 第10章 泛型与元编程 · 第11章 反射与动态特性
第12章 注解与装饰器 · 第13章 模块系统 · 第14章 类型系统进阶

知识导图 × C 语言对照 × 例题精讲 × 章末实验

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 建立 Rust 内建类型全图:整数 (i8-i128)、浮点 (f32/f64)、bool、char、元组 ()——并与 C 的 char/int/long/float/double ——对照
- 掌握 **let 绑定**:默认不可变、mut 可变、遮蔽(shadowing),并理解"Rust 没有未初始化变量"这条与 C 截然不同的规定
- 熟练写出各类字面量:进制前缀、下划线分隔符、类型后缀、字节字面量、raw 字符串
- 掌握 **const** 与 **static** 的语义,与 C 的 #define/const 对照
- 理解"Rust 没有隐式类型转换":as 的截断语义与 TryFrom/TryInto 的安全转换
- 分清 String 与 &str 两种字符串,知道"长度"与"字符数"的区别(UTF-8)
- 入门数组、元组、Vec、强类型枚举与 type 别名;埋下伏笔:Rust 的类型系统里带着"所有权"信息 (第5章展开)

💡 从 C 到 Rust:本章主题如何迁移

C 里一个 int 是多少字节?标准只保证"至少 16 位",在不同平台上可能是 16、32 或 64 位;而 Rust 的类型名直接把位宽写在名字里: i32 就一定是 32 位有符号整数。位宽、有无符号、字节序一目了然,这是 Rust 给 C 程序员的第一份礼物——类型自述其大小。

C 的局部变量不初始化就读,是未定义行为(UB),它可能打印出任何垃圾值;Rust 直接**禁止**未初始化变量存在——let x; 之后不使用、或未赋初值就使用,都是编译错误。把 UB 变成编译错误,是贯穿全书的主线:第5章所有权、第9章并发安全,走的都是同一条路。

C 的类型转换是"隐式 + 悄悄截断": int x = 300; char c = x; 得到一个 44,编译器最多给个警告。Rust 的态度是**零隐式转换**:所有转换必须写出来, as 负责"我就是要截断", TryFrom 负责"可能失败请告诉我"。类型转换从此成为显式决策,而不是潜伏的 bug。

Rust 的 String 之于 C 的 char[],相当于"自带长度的动态字符数组":不需要 strlen 数结尾、不需要 strcpy 防溢出、不需要担心 strcat 写穿缓冲区——内存安全是编译器默认行为,不是程序员的责任。

最后请记住本章最关键的一句伏笔: Rust 的每个类型不仅描述"值的形状",还描述"值的所有权归属"。比如 String 拥有自己的数据,而 &str 只是借用别人的数据。第5章将把这个"带所有权的类型系统"彻底讲透——你现在看到的每一条规则,都是在为那一刻铺路。

📦 前置知识自查

- C 的基本类型:char/short/int/long/float/double 及其字节数与范围
- C 的变量声明与初始化、#define 与 const 的差别
- C 的整型/浮点字面量、转义序列(\n \\ \")
- C 的强制转换 (int)x 与隐式转换(char c = 300 会发生什么)
- C 的字符数组 char s[] = "abc"、strlen / strcpy 的用法
- C 的数组、struct、enum、typedef

本章学习路线

1. **第一阶段(约1.5小时,建立直觉)**:通读 §1.1–§1.3,把"类型表"与"let 绑定"对照 C 记住,随手改改示例代码
2. **第二阶段(约1小时,攻转换)**:重点做 §1.4 的转换练习,把 C 的"隐式"习惯彻底改掉
3. **第三阶段(约1小时,练字符串与复合类型)**:§1.5–§1.7,亲手写字符串拼接与 Vec 增删
4. **验收标准**:不看资料,能背出类型表、写出 10 种字面量、解释"为什么没有未初始化变量"

本章知识导图

- §1.1 类型总览:与 C 类型——对照
 - 整数家族 i8–i128 / 浮点 f32/f64 / bool / char / 单元类型 ()
- §1.2 变量绑定:let、不可变默认、mut、遮蔽、无未初始化
- §1.3 字面量与常量:进制、分隔符、后缀、const 与 static
- §1.4 类型转换:零隐式转换、as 截断、TryFrom/TryInto
- §1.5 字符串:&str 与 String、UTF-8 长度 vs 字符数
- §1.6 复合类型入门:数组、元组、Vec、切片
- §1.7 枚举与类型别名:强类型枚举、带值枚举、type
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§1.1 类型总览:与 C 类型——对照

1.1.1 整数家族:类型名自带位宽

C 的 `int` 位宽依赖平台, `long` 在 64 位 Linux 是 8 字节、在 Windows 是 4 字节——跨平台代码里这是反复踩坑的根源。Rust 把"符号 + 位宽"直接写进类型名, **类型与平台无关**:

Rust 类型	C 对应	字节数	取值范围
i8	signed char	1	-128 ~ 127
i16	short	2	-32768 ~ 32767
i32	int	4	±21 亿
i64	long long	8	±9.2×10 ¹⁸
i128	(C 无)	16	超大整数
u8	unsigned char	1	0 ~ 255
u16/u32/u64/u128	unsigned short/int/long long	2/4/8/16	0 ~ 2 ⁿ -1
isize / usize	ptrdiff_t / size_t	8(64 位平台)	与指针同宽
f32	float	4	±3.4×10 ³⁸
f64	double	8	±1.8×10 ³⁰⁸
bool	_Bool / int	1	true / false
char	char(单字节)	4	Unicode 标量值
()	void(占位)	0	只有一个值

对照要点: `isize/usize` 对应 C 的 `size_t`, 用于长度、下标、指针算术结果; `char` 是 4 字节的 Unicode 字符, 与 C 的 1 字节 `char` 完全不同, 字符串用专门的 `String / &str` (§1.5); `bool` 是真正的布尔类型, Rust 没有 C 那种“非零即真”的隐式真值转换。单元类型 `()` 表示“没有任何值”, 相当于 C 里 `void` 返回值的位置。

示例 1-1 观察各类型的大小

```
fn main() {
    // std::mem::size_of 返回类型在内存中占用的字节数
    println!("i8 : {} 字节", std::mem::size_of::<i8>());
    println!("i32 : {} 字节", std::mem::size_of::<i32>());
    println!("u64 : {} 字节", std::mem::size_of::<u64>());
    println!("usize: {} 字节", std::mem::size_of::<usize>());
    println!("f64 : {} 字节", std::mem::size_of::<f64>());
    println!("bool : {} 字节", std::mem::size_of::<bool>());
    println!("char : {} 字节", std::mem::size_of::<char>());
    // 引用类型(指针)在 64 位平台上是 8 字节
    println!("&i32: {} 字节", std::mem::size_of::<&i32>());
    println!("() : {} 字节", std::mem::size_of::<()>());
}
```

1.1.2 布尔类型: C 没有真正的 bool

C 的 `_Bool` (或 `int`) 与整数互通: `if (x)`、`return 0`; 到处是隐式转换。Rust 的 `bool` 只有 `true / false` 两个值, 不能与整数互相转换——`if 1 { }` 是编译错误。好处是: 条件表达式永远只表达“真假”这一件事, 没有 C 里 `if (ptr)`、`if (fd)` 那种依赖约定俗成的写法。

1.1.3 字符类型与编码

C 的 `char` 是 1 字节,存 ASCII 或按地区编码(GBK 等)凑合;Rust 的 `char` 是 4 字节的 Unicode 标量值,'你'、'A'、'😄' 都是合法 `char`。写 Unicode 转义: `'\u{4F60}'` 就是 '你'。字符串默认 UTF-8 编码,每个汉字 3 字节,这与 C 的"一个字符一个字节"的直觉完全不同——§1.5 会专门讲"字节数 ≠ 字符数"。

§1.2 变量绑定:let 与不可变性

1.2.1 声明与初始化:没有"未初始化"状态

C 的 `int x;` 之后直接读 `x`,是 UB:局部变量里是栈上的垃圾值。Rust 的规则是 变量必须在使用前初始化,否则编译不过:

```
fn main() {
    let x = 42;           // 整数默认推导为 i32
    let y: i64 = 42;      // 显式类型注解
    let mut count = 0;    // mut 关键字声明"可变的绑定"
    count += 1;           // 只有 mut 绑定才能被修改

    // let z;             // 编译错误:未初始化就声明
    // println!("{z}");    // 且无法使用未初始化变量
    println!("x = {x}, y = {y}, count = {count}");
}
```

注意 `let` 声明的是**绑定(binding)**,不是 C 意义上的"变量":`let` 把名字绑到一个值上,默认不可变。C 的变量默认可**变**,`const` 需要显式加;Rust 恰恰相反:默认不可变,`mut` 需要显式加。这个"反过来的默认值"是 Rust 最重要的文化之一:编译器默认帮你锁住数据,只有你明确要求才放开。

1.2.2 遮蔽(shadowing):同名新绑定

C 里同名变量在同一个作用域内不能重新声明;Rust 允许 `let` 再次声明同名绑定,新绑定"遮蔽"旧的——旧值不会消失,只是这个名字不再指向它。遮蔽常用于"对同一个值做加工"的链式写法,且**可以在遮蔽时改变类型**:

```
fn main() {
    let name = "zhangsan";    // &str 类型
    let name = name.to_uppercase(); // 遮蔽:现在 name 是 String
    let name = name.len();    // 再遮蔽:现在 name 是 usize
    println!("{name}");       // 9(字节数)

    // 遮蔽与 mut 的区别:遮蔽创建新绑定,旧绑定仍存在
    let x = 5;
    let x = x + 1;            // 新绑定 x = 6
    let x = x * 2;            // 新绑定 x = 12
    println!("{x}");
}
```

⚠ 易错点 1:以为 Rust 也有 C 的"零值默认"或"未初始化垃圾值"

两个都没有。`let x: i32;` 不赋初值就使用,是**编译错误**;全局 `static` 虽然像 C 全局变量一样有零值(0、0.0、false),但普通局部变量绝不自动清零,也绝不允许读未初始化内存。C 的"忘记初始化 → 读到垃圾值"这一类 bug,在 Rust 里被编译器直接消灭。

1.2.3 与 C 的声明对照

C 的写法	Rust 的写法	说明
<code>int x = 5;</code>	<code>let x: i32 = 5;</code> 或 <code>let x = 5;</code>	Rust 可省略类型(推导)
<code>const int N = 10;</code>	<code>const N: i32 = 10;</code>	Rust 的 <code>const</code> 必须注解类型
<code>int *p = NULL;</code>	<code>let p: Option<i32> = None;</code>	Rust 没有 NULL,用 Option(第14章)
<code>char buf[100];</code>	<code>let buf = [0u8; 100];</code>	数组字面量加类型后缀
<code>int x; /* 读它=UB */</code>	<code>let x: i32; /* 编译就拒绝 */</code>	无未初始化状态

§1.3 字面量与常量

1.3.1 整数字面量:进制、分隔符、后缀

Rust 支持 C 的全部进制写法(0x/0o/0b 与十进制),并新增 **下划线数字分隔符** 提升可读性。整数默认 `i32`;需要其他类型时,要么注解,要么写类型后缀:

```
fn main() {
    let a = 255;           // 十进制,默认 i32
    let b = 0xFF;          // 十六进制:255
    let c = 0o377;         // 八进制:255
    let d = 0b1111_1111;   // 二进制:255,下划线只作分隔
    let e = 1_000_000u64;  // 后缀 u64:一百万
    let f = 3.14f32;       // 浮点后缀 f32(默认是 f64)
    let g = 1.2e-3;        // 科学计数法:0.0012
    let h = 0xFFu8;        // 后缀指定无符号小类型
    println!("{a} {b} {c} {d} {e} {f} {g} {h}");
}
```

1.3.2 字符、字节与字符串字面量

```
fn main() {
    let byte = b'A';       // 字节字面量:u8 类型的 65(C 里就是 char)
    let ch = '你';         // char 类型:Unicode 字符
    let esc = '\n';        // 转义序列与 C 相同
    let unicode = '\u{4F60}'; // 按码点写:'你'

    let s = "hello";       // &str:不可变字符串切片
    let raw = r"c:\new\dir"; // raw 字符串:不解析转义,反斜杠原样
    let raw2 = r#"含"引号"的文本"#; // 加 # 包裹,内部可含双引号
    let bytes = b"abc";    // 字节数组:&[u8; 3]
    println!("{byte} {ch} {esc:?} {unicode} {s} {raw} {raw2} {:?}", bytes);
}
```

`r"..."` raw 字符串对 C 程序员是福音:写 Windows 路径、正则表达式时不再需要把每个反斜杠写成 `\\`。需要内部出现双引号时,用 `r#"..."#` (引号数可增减)。

1.3.3 const 与 static:与 C 的对照

C 的 `#define` 是文本替换, `const` 只是"只读变量"; Rust 的 `const` 是真正的编译期常量:值必须在编译期求出来,且每个使用点都会被内联。C 里" `const int` 不能做数组长度"的著名限制,在 Rust 中不存在——`const N: usize = 8;` 直接可用于数组长度。若需要"有固定内存地址的全局变量"(对应 C 的全局变量),用 `static`:

```
// 编译期常量:名字大写、必须注解类型
const MAX_PLAYERS: u32 = 8;
const FONT_SIZE: f64 = 12.0;
const GREETING: &str = "欢迎回来";

// 全局静态变量:有固定地址,生命周期为 'static'
static APP_NAME: &str = "成绩管理系统";

fn main() {
    println!("{APP_NAME} 支持最多 {MAX_PLAYERS} 人,字号 {FONT_SIZE}");
    println!("{GREETING}");
    // 数组长度直接用 const,这在 C 里做不到(除非宏)
    let buffer: [u8; MAX_PLAYERS as usize] = [0; 8];
    println!("buffer 长度:{}, buffer.len()");
}
```

★ 重点:const 与 let 的分工

`const` 用于"编译期就确定的常量"; `let` 用于"运行时才有的变量"——即使 `let x = 42` 也是运行时的绑定。`const` 没有类型推导,必须注解类型;命名约定全大写。C 程序员最容易犯的错:把 `const` 当"只读变量"到处用——Rust 里编译器会告诉你哪些值能进 `const`,进不去的就用 `let`。

§1.4 类型转换

1.4.1 零隐式转换

C 里 `int + double`、`char c = 300`、`if (ptr)` 到处都是隐式转换与截断。Rust 的规则只有一条:不写转换,就没有转换。类型不同的值之间做运算、赋值、比较,一律编译错误——这让"类型错误"在编译期全部暴露,而不是在运行期悄悄变出垃圾值。

```
fn main() {
    let x: i32 = 42;
    // let y: i64 = x;           // 编译错误!i32 不会自动变 i64
    let y: i64 = x as i64;      // as:显式转换,窄→宽,安全
    println!("{y}");

    // 混合运算也必须显式转换
    let a: u32 = 10;
    let b: u64 = 5;
    // let c = a + b;           // 编译错误
    let c = a as u64 + b;       // 先把 a 转成 u64 再相加
    println!("{c}");
}
```

1.4.2 as 的截断语义:与 C 的强制转换一致

`as` 执行的是**底层位级转换**:宽→窄直接截断高位,浮点→整数向零截断。这与 C 的强制转换语义一致——区别在于 Rust 让你把"我知道会截断"写在明面上:

```
fn main() {
    let big: i32 = 300;
    let small: u8 = big as u8;    // 300 % 256 = 44,高位被丢弃
    println!("{}", small);       // 44

    let pi: f64 = 3.99;
    let n: i32 = pi as i32;      // 向零截断:3(不是四舍五入)
    println!("{}", n);

    // 负数转无符号:按位补码语义(与 C 一致)
    let m: i32 = -1;
    let u: u32 = m as u32;       // 4294967295
    println!("{}", u);

    // bool 与整数互转必须显式
    let flag: bool = true;
    let one: u8 = flag as u8;    // 1
    println!("{}", one);
}
```

1.4.3 TryFrom/TryInto:可能失败的转换

`as` 永远成功,但"300 放不进 u8"这种问题它不告诉你。当你想要"成功就取新值、失败就报错"的语义时,用 `TryFrom / TryInto` ——返回 `Result`,第9章会系统讲错误处理,这里先见个面:

```
fn main() {
    let big: i64 = 300;
    match i8::try_from(big) {
        Ok(v) => println!("转换成功:{v}"),
        Err(e) => println!("转换失败:{e}"), // 300 超出 i8 范围
    }

    let x: u32 = 42;
    let x_u8: u8 = x.try_into().unwrap_or(0); // 失败时给默认值 0
    println!("{}", x_u8);
}
```

⚠ 易错点 2:把 C 的"自动转换"习惯带进 Rust

最常见的三类编译错误:① 函数实参类型不匹配——C 里 `int` 传进 `double` 参数自动升格,Rust 必须 `as f64`;② 下标与长度混用——`i32` 做下标,但 `vec.len()` 是 `usize`,得用 `i as usize`;③ 字符与整数混用——`'0'` `as u8` 才等于 48,直接 `'0'` 是 `char` 不是数。见到 "mismatched types" 报错,第一反应就是:补一个显式转换。

§1.5 字符串:&str 与 String

1.5.1 两种字符串:C 的 char[] 一分为二

C 的字符串就是 `char[]` + 结尾 `'\0'`, 没有"字符串"这个类型。Rust 把字符串拆成两个类型:**String**(拥有数据、可增长、可修改)与 **&str**(借用一段文本、不可修改)。字符串字面量 `"abc"` 的类型是 `&str`, 相当于 C 的 `const char*` 字面量——但它自带长度, 不需要 `strlen`:

```
fn main() {
    // &str: 借用了一段文本, 不能修改
    let greeting: &str = "你好, Rust";
    println!("{greeting}");

    // String: 拥有文本的所有权, 可以修改、增长
    let mut name = String::from("张");
    name.push_str("三");           // 追加字符串
    name.push('!');               // 追加单个字符
    println!("{name}");           // 张三!

    // &str 与 String 的转换(这两个转换经常被调用, 免费)
    let s: String = String::from("hello");
    let view: &str = &s;          // String -> &str: 取引用即可
    let owned: String = view.to_string(); // &str -> String: 复制一份
    println!("{view} {owned}");
}
```

1.5.2 长度 vs 字符数: UTF-8 的第一课

`str.len()` 返回**字节数**, `str.chars().count()` 返回**字符数**。因为 UTF-8 里汉字占 3 字节, "你好, Rust" 是 7 个字符、11 个字节。C 程序员习惯"下标访问 = 取字符", Rust 里 `s[1]` 是**编译错误**——必须 `chars().nth(1)`, 而且它是 $O(n)$ 。这是 Rust 对"字符串是字节数组"的诚实: 底层就是字节, 想要字符得明确转换:

```
fn main() {
    let s = "你好, Rust";
    println!("字节数: {}", s.len());           // 11
    println!("字符数: {}", s.chars().count()); // 7

    // 按字节遍历(快, 但看到的是 UTF-8 字节)
    for b in s.bytes().take(3) {
        println!("字节: 0x{:02X}", b);
    }
    // 按字符遍历(正确, 但更慢)
    for ch in s.chars().take(2) {
        println!("字符: {ch}");
    }
    // 注意: 不能像 C 那样 s[0] 取字符——按字节下标会切到半个汉字
}
```

1.5.3 常用 API 速查

```
fn main() {
    // 拼接:format! 最常用,不产生所有权问题
    let a = "hello".to_string();
    let b = "world".to_string();
    let msg = format!("{a}, {b}!");
    println!("{msg}");

    // 查找、截取(按字节范围,务必用字符边界)
    let text = "Rust 语言";
    if let Some(pos) = text.find("语言") {
        println!("找到位置:{pos}");           // 5(字节偏移)
    }
    let sub = &text[..4];                      // 取前 4 字节:"Rust"
    println!("{sub}");

    // 替换、拆分、大小写
    let t2 = text.replace("语言", "编程语言");
    println!("{t2}");
    for part in "a,b,c".split(',') {
        print!("{part} ");
    }
    println();
    println!("{}", "rust".to_uppercase());    // RUST
}
```

⚠ 易错点 3:用 len() 当"字符数",或按字节下标切字符串

"你好".len() 是 6 不是 2。len() 返回字节数;要字符数必须 .chars().count()。更危险的是 &s[0..1] 这种字节切片:如果切在汉字中间(UTF-8 多字节字符的内部),会直接 panic(第9章讲 panic)——C 里你顶多切出一个乱码,至少不崩。这也是为什么 Rust 的字符串下标访问干脆被禁用。

§1.6 复合类型入门:数组、元组、Vec

1.6.1 数组:固定长度,类型 [T; N]

Rust 数组把"长度"写进类型: [i32; 5] 表示 5 个 i32。长度 N 是编译期常量,这与 C 的 int a[5] 类似,但 Rust 数组越界访问是 panic(可预期),不是 C 那种静默内存破坏:

```
fn main() {
    let scores: [i32; 5] = [90, 85, 78, 92, 88];
    let zeros = [0u8; 100];           // 100 个 0(类似 C 的 memset 后)
    println!("第一个:{}", scores[0]);
    println!("长度:{}", scores.len()); // len() 返回 usize
    // println!("{}", scores[10]);     // 越界:运行期 panic,进程退出

    // 数组默认也是"复制"语义——for 遍历逐个拷贝
    for s in scores {
        print!("{}", s);
    }
    println!();
}
```

1.6.2 元组:把不同类型的值打包

元组把多个(可以不同类型的)值合成一个,对应 C 里"为返回多个值而定义的临时 struct",但更轻量。用下标 `t.0` 或解构访问:

```
fn main() {
    // (String, u8, f64):一个学生的三种信息
    let student = (String::from("张三"), 21, 87.5);
    println!("姓名:{}, 年龄:{}, 成绩:{}", student.0, student.1, student.2);

    // 解构:一次拆开
    let (name, age, score) = student;
    println!("{name} 明年 {age} 岁,目标 {score} 分以上");
    // 解构后原元组仍可用(三个字段都是可复制类型时);此处已 move,见第5章
}
```

1.6.3 Vec:动态数组(向量)

`Vec<T>` 是 Rust 的标准动态数组,对应 C 里"自己 malloc 出来的数组 + 手动记录长度"。它能按需增长,离开作用域自动释放——不需要 free,不需要 realloc,不需要防越界:

```
fn main() {
    let mut nums: Vec<i32> = Vec::new();
    nums.push(10);           // 末尾追加
    nums.push(20);
    nums.push(30);
    nums.pop();              // 末尾弹出:30

    let v = vec![1, 2, 3, 4, 5]; // vec! 宏快速构造
    println!("长度:{}, 第3个:{}", v.len(), v[2]);

    // 遍历(第3章细讲 for;此处先感受)
    let mut sum = 0;
    for n in &v {
        sum += n;
    }
    println!("总和:{sum}");
}
```

1.6.4 切片:看数组的一段

`&[T]` 切片是"指向一段连续元素 + 长度"的视图,对应 C 的"指针 + 长度"惯用法,但 长度随视图走,越界由编译器和运行期检查兜底。对 `Vec` 和数组都能取切片:

```
fn main() {
    let v = vec![1, 2, 3, 4, 5, 6];
    let slice: &[i32] = &v[1..4];    // 视图:第2到第4个元素
    println!("切片长度:{}, 首元素:{}, slice.len(), slice[0]");

    // 数组自动支持切片语法
    let arr = [10, 20, 30, 40];
    let head: &[i32] = &arr[..2];    // 省略起点:从头开始
    println!("{head:?}");
}
```

§1.7 枚举与类型别名

1.7.1 强类型枚举:C 的 `enum` 只是整数的"马甲"

C 的 `enum` 本质是 `int`:可以赋任意整数、可以加减、可以和其他枚举互相隐式转换。Rust 的枚举是**独立类型**:不能当整数用、不能与别的枚举混用、也没有 C 那种"默认 0 开始递增"的自动整数值:

```
enum Direction {
    North,
    East,
    South,
    West,
}

fn main() {
    let dir = Direction::North;
    // let n: i32 = dir;        // 编译错误!枚举不能隐式转整数
    // let d = Direction::North + 1; // 编译错误!没有加法

    // 需要"枚举对应的显示文本"时,用 match(第3章)显式映射
    let text = match dir {
        Direction::North => "北",
        Direction::East  => "东",
        Direction::South => "南",
        Direction::West  => "西",
    };
    println!("方向:{text}");
}
```

1.7.2 带值枚举:每个变体可携带数据(第14章的主角)

Rust 枚举的变体可以各自携带数据——这就是"和类型"(sum type)的雏形, `Option` / `Result` 都是它。C 里要模拟这种"要么这个要么那个还带数据",只能靠 `union` + 标志位,既笨拙又不安全:


```
// 消息:要么退出,要么移动(带坐标),要么写文本(带字符串)
enum Message {
    Quit,                                // 不带数据
    Move { x: i32, y: i32 },             // 带两个字段
    Write(String),                       // 带一个 String
}

fn main() {
    let m = Message::Write(String::from("保存成功"));
    match m {
        Message::Quit => println!("退出"),
        Message::Move { x, y } => println!("移动到 ({x}, {y})"),
        Message::Write(text) => println!("写入:{text}"),
    }
}
```

1.7.3 type 别名:给类型起短名字(不产生新类型)

对应 C 的 `typedef`。别名与原名完全等价,编译器不做区分——想要"防混淆的新类型",要用第14章的 `newtype` 模式:

```
type Meters = f64;                      // 别名的用途:缩短长类型名、表达意图
type Callback = fn(i32) -> i32;        // 函数指针类型别名

fn main() {
    let distance: Meters = 100.5;       // 本质还是 f64
    println!("{distance}");
}
```

★ 本章伏笔:类型系统里带着所有权

回顾本章出现的类型: `String` 拥有数据, `&str` 借用数据; `Vec<T>` 拥有堆上的缓冲; `Box<T>` (第5章)拥有堆上的单块数据。也就是说, Rust 的类型天生携带"谁拥有这块内存"的信息——这是 C 的类型系统(指针只是地址,不关心归属)永远表达不了的一维。第5章我们会看到:正是这多出来的一维,让 `use-after-free`、悬垂指针、数据竞争从"运行期事故"变成"编译期错误"。

本章速查表

主题	结论 / 写法
整数类型	i8/i16/i32/i64/i128, u8/u16/u32/u64/u128, isize/usize;默认推导 i32
浮点 / 布尔	f32 / f64(默认);bool 只有 true/false,不与整数互通
字符	char 为 4 字节 Unicode; 'A'、'\u{4F60}';字节字面量 b'A' 是 u8
变量绑定	let x = 5; 不可变; let mut x = 5; 可变; let x: i32 = 5; 注解类型
未初始化	不存在;使用未初始化变量是编译错误(C 是 UB)
遮蔽	let x = x + 1; 创建同名新绑定,可换类型
整数字面量	0x 十六 / 0o 八进 / 0b 二进 / 1_000_000 分隔符 / 42u8 后缀
raw 字符串	r"... " 不解析转义; r#"..."# 内部可含双引号
常量	const N: u32 = 8; 编译期常量,必须注解类型; static 有固定地址
类型转换	无隐式转换; as 截断语义; TryFrom / try_into() 安全转换
字符串	String 可增可改; &str 只读借用; len() 是字节数, chars().count() 是字符数
拼接/格式化	format!("{a}, {b}"), push_str、to_string()
数组 / 元组	[T; N] 定长; (a, b, c) 异类打包,解构取出
Vec	vec![...], push/pop/len/下标;越界 panic
切片	&v[1..4] 视图,自带长度
枚举 / 别名	强类型,不可当整数;变体可带数据; type X = ...

最小必会清单

- 能背出整数/浮点/布尔/字符类型表,并说出与 C 每个类型的对应与差异
- 能解释"为什么 Rust 没有未初始化变量",以及 mut 与遮蔽的区别
- 能写出 6 种以上字面量:十进制、十六进制、八进制、二进制、分隔符、后缀、raw 字符串
- 能说明 const 与 static、与 C 的 #define 的区别
- 能解释"为什么 Rust 禁止隐式类型转换",并正确使用 as 与 try_into()
- 能说出 String 与 &str 的区别,以及"为什么 s[0] 不能取字符"
- 能说出 "你好" 的 len() 与 chars().count() 分别是多少
- 能用数组、元组、Vec 各写一个例子,并说明越界访问的后果
- 能说明 Rust 枚举与 C 枚举的三个本质区别
- 能说出"类型系统里带着所有权"这句话的含义(给一个例子)

自测题

Q 1. C 的 `int` 与 Rust 的 `i32` 有什么本质区别? `long` 与 `i64` 呢?

✅ 参考答案

C 的 `int` 只保证"至少 16 位",实际位宽随平台; `long` 在 64 位 Linux 是 8 字节、Windows 是 4 字节。Rust 的 `i32` 名字即规格:在任何平台都是 32 位有符号。 `long` 对应不到 Rust 的任何一个类型——需要 8 字节有符号就用 `i64`,需要"与指针同宽"就用 `isize`。

Q 2. 下面的 C 代码在 Rust 里怎么写? `int x; printf("%d", x);`

✅ 参考答案

写不出来——这是 UB,编译器拒绝。必须初始化: `let x: i32 = 0; println!("{}", x);`。这正是 Rust 的设计:把 C 里"未初始化 → 垃圾值 → 难以排查"的整类 bug 变成编译错误。

Q 3. `300u16 as u8` 的结果是多少?为什么?C 里 `(char)300` 呢?

✅ 参考答案

44。 `u8` 只能存 0~255, `as` 直接截断高位:300 的二进制去掉高 8 位剩下 44。C 的 `(char)300` 行为与之一致(实现定义,实际相同),但 C 的隐式赋值 `char c = 300;` 也只给警告;Rust 则要求你显式写 `as`,把截断变成可见的决策。

Q 4. "你好, Rust" 的 `len()` 与 `chars().count()` 分别是多少?为什么?

✅ 参考答案

若用全角逗号:3+3+3+4 = 13 字节,7 个字符。每个汉字 3 字节,逗号 3 字节,"Rust" 4 字节;字符数则是 2+1+4 = 7。若用 ASCII 逗号则是 11 字节。要点:`len()` 永远是字节数,与字符数无关。

Q 5. 为什么 Rust 禁止隐式类型转换?这与它的安全目标有什么关系?

✅ 参考答案

隐式转换(尤其截断、有符号↔无符号、浮点精度丢失)是 C 里大量隐蔽 bug 的来源。Rust 要求显式转换,使"类型不匹配"在编译期暴露、让程序员对每次转换负责;与"消除未定义行为"的整体目标一致——能编译错误解决的事,绝不留到运行期。

Q 6. `let x = 5; x = 6;` 能编译吗? `let mut x = 5; let x = 6;` 呢?

✅ 参考答案

第一句编译错误:`x` 绑定默认不可变,不能赋值。第二句可以:先声明可变的 `x=5`,再用 `let` 创建同名新绑定 `x=6`(遮蔽),旧绑定被隐藏。遮蔽不修改原绑定,而是创建新绑定——两者的语义完全不同。

章末实验题:学生信息登记与成绩统计

实验 1:学生信息登记与成绩统计

实验目标:用全章涉及的类型(整数/浮点/布尔/字符/字符串/数组/Vec/元组/枚举/常量/转换/格式化输出)实现一个学生成绩登记统计程序。

实验要求:

- 任务 1:定义年级枚举 Year(覆盖知识点:强类型枚举)与常量(及格线、班级容量)(覆盖:const)
- 任务 2:用 (String, Year, f64, bool) 元组 + Vec 登记不少于 5 名学生(覆盖:元组、Vec、String、浮点、布尔)
- 任务 3:统计总人数、总分、平均分、及格人数与及格率,注意类型转换(覆盖:usize→f64 的 as 转换、算术运算)
- 任务 4:给每名学生打等级 A~F,并标记是否重修,用格式化输出打印成绩单(覆盖:char、if 表达式、format 格式化、遮蔽)
- 任务 5:用字符串拼接生成班级评价语(覆盖:push_str、String)
- 任务 6:运行程序并核对输出与手工计算一致(覆盖:整体验证)

实验环境:装有 Rust 工具链(cargo --version 验证),创建项目 cargo new grade_book,进入目录后 cargo run 运行。

第一步 **一步:** cargo new grade_book 创建项目,把代码写进 src/main.rs

第二步 **二步:**先定义 Year 枚举与两个常量;注意枚举变体命名用驼峰

第三步 **三步:**用 Vec::new() 创建学生列表,push 进 5 名学生;元组字段按 (姓名, 年级, 成绩, 重修) 排列

第四步 **四步:**遍历累加时,先想清楚 sum 的类型;average = sum / total as f64 为什么 total 要转换?

第五步 **五步:**等级判定用 if 表达式链(结果赋给 char 变量),注意每个 if 分支都要返回值

第六步 **六步:**最后核对输出;尝试把及格线改成 80 分,确认输出随之变化

参考答案要点

```
// ===== 实验 1:学生信息登记与成绩统计 =====
// 覆盖知识点:常量(const)、强类型枚举(enum)、元组、Vec、
//          整数/浮点/布尔/字符/字符串、类型转换(as)、格式化输出、遮蔽

const CLASS_SIZE: usize = 5;    // 班级人数(常量)
const PASS_LINE: f64 = 60.0;    // 及格分数线(常量)

/// 学生年级:用强类型枚举代替 C 的 int 魔法数字
enum Year {
    Freshman,    // 大一
    Sophomore,   // 大二
    Junior,       // 大三
    Senior,       // 大四
}

fn main() {
    // 用元组表示一名学生:(姓名, 年级, 成绩, 是否重修)
    let mut students: Vec<(String, Year, f64, bool)> = Vec::new();

    // 登记学生:字符串、枚举、浮点、布尔字面量全覆盖
    students.push((String::from("张三"), Year::Freshman, 88.5, false));
    students.push((String::from("李四"), Year::Sophomore, 59.0, true));
    students.push((String::from("王五"), Year::Junior, 91.0, false));
    students.push((String::from("赵六"), Year::Senior, 45.5, true));
    students.push((String::from("钱七"), Year::Freshman, 73.0, false));

    // ---- 统计:总数、总分、平均分、及格人数 ----
    let total = students.len();           // usize 类型
    let mut sum: f64 = 0.0;
    let mut passed: u32 = 0;              // 无符号整数计数
    for s in &students {
        sum += s.2;                       // 元组下标访问
        if s.2 >= PASS_LINE {
            passed += 1;
        }
    }
    let avg: f64 = sum / total as f64;    // 显式转换:usize -> f64

    // ---- 打印成绩单 ----
    println!("===== 成绩单 =====");
    for s in &students {
        let score = s.2;
        // if 是表达式:每个分支返回一个 char
        let mark: char = if score >= 90.0 { 'A' }
            else if score >= 80.0 { 'B' }
            else if score >= 70.0 { 'C' }
            else if score >= 60.0 { 'D' }
            else { 'F' };
        let result = if s.3 { "(重修)" } else { "" };
        // {:<8} 左对齐占 8 宽;{:5.1} 总宽 5、保留 1 位小数
        println!("{:<8} 成绩 {:5.1} 等级 {} {}", s.0, score, mark, result);
    }
    println!("=====");
    println!("总人数:{total} 平均分:{avg:.1} 及格人数:{passed}");
}
```

```

// ---- 及格率:u32 -> f64 的转换 ----
let pass_rate: f64 = passed as f64 / total as f64 * 100.0;
println!("及格率:{:.1}%", pass_rate);

// ---- 字符串拼接生成班级评价 ----
let mut summary = String::from("班级评价:");
if avg >= 85.0 {
    summary.push_str("优秀");
} else if avg >= 70.0 {
    summary.push_str("良好");
} else {
    summary.push_str("需要努力");
}
println!("{summary}");

// ---- 补充:字符与字节字面量 ----
let first_char: char = '张';
let byte_a: u8 = b'A';
println!("首字符 {first_char},ASCII 码 {byte_a}");
let _ = CLASS_SIZE; // 避免未使用警告(教学示例)
}

```

设计说明:程序刻意把"每个类型"都用一次——常量管配置、枚举管类别、元组管打包、Vec 管动态增长、as 管单位换算、if 表达式管分支取值、format 语法管对齐输出。运行后把及格线改为 80,观察及格人数与评价语的变化,即可验证各任务覆盖的知识点确实在协同工作。

下一章预告:第2章 运算符与表达式——Rust 没有 ++ 与 --,溢出在 debug 模式直接 panic;if/loop/match 都是表达式,还有 C 里没有的 ? 运算符与范围运算符 ..。类型转换的坑刚过,运算的坑马上来。

第2章 运算符与表达式

建议学时:3-4 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 建立 Rust 运算符全景:算术、位、比较、逻辑、赋值、范围——知道哪些与 C 相同、哪些被移除、哪些是新增
- 掌握 **溢出行为的明确规则**: debug 模式 panic, release 模式回绕;以及 wrapping/checked/saturating 三种显式选择
- 理解整数除法与取模的截断语义(与 C 一致,与 Python 不同);负数的 % 到底等于几
- 掌握 **"Rust 没有 ++ 与 --"**,以及自增自减在 C 里造成的两类经典 bug 为何在 Rust 消失
- 掌握位运算全套,并知道 Rust 优先级规则修复了 C 的 `x & 1 == 0` 经典坑
- 掌握 if/loop/match 都是表达式: **C 的三目 ?: 在 Rust 中就是 if 表达式**;赋值是语句不是表达式
- 认识 Rust 特色运算符: ? 错误传播、范围 .. 与 ..=、复合赋值;了解 Rust 规定的求值顺序

💡 从 C 到 Rust:本章主题如何迁移

运算符是 C 程序员最熟悉的领域——但恰恰因为熟悉,C 的坏习惯最容易在这里原样搬进来。Rust 对运算符做的第一件事是**清理 C 的未定义行为**:有符号整数溢出从"UB,一切皆有可能"变成"debug 模式 panic、release 模式回绕"的确定性规则;子表达式的求值顺序从"未指定"变成"明确规定,从左到右"。你在 C 里依赖编译器脾气写的表达式,在 Rust 里要么是编译错误,要么行为确定。

第二件事是**删除和新增**。Rust 没有 ++/--(它们是 C 里 `i = i++ + i++` 这类灾难的温床),没有三目 ?: (if 表达式顶替),但有范围运算符 `1..5`、错误传播运算符 `?`。每个新增运算符都是"消灭一类样板代码"——`?` 消灭的正是 C 里 `if (err) return err;` 的复制粘贴洪流。

第三件事是**表达式化**。C 里 if/while 是语句,只能"做事";Rust 里 if、match、loop、代码块 `{}` 都是表达式,能"算出值"。于是 `int max = a > b ? a : b;` 变成 `let max = if a > b { a } else { b };` ——同样的语义,更少的括号,且**没有赋值语句混进条件里**的可能。

最后提醒一句贯穿全书的风格差异:C 允许 `if (n)`、`if (ptr)` 依赖"非零即真";Rust 的条件必须是 bool, `&& / ||` 永远返回 bool(不像 Python/JS 返回操作数)。刚开始会嫌啰嗦,但一周后你会感谢编译器替你挡住"把指针当布尔用"这类错误。

📦 前置知识自查

- C 的算术/位/逻辑/比较/赋值运算符与优先级表(能默写更好)
- C 的自增自减:前置/后置语义(`i++` 与 `++i`),以及在表达式中混用的危害
- C 有符号溢出是 UB、整数除法向零截断、负数取模的行为
- C 的短路求值 `&& / ||`,以及 `(x & 1) == 0` 为什么要加括号
- C 的三目运算符 `?:` 与逗号运算符

🚀 本章学习路线

1. **第一阶段(约1小时):**读 §2.1 运算符总览与优先级表,对照 C 划出"删除了什么、新增了什么"
2. **第二阶段(约1.5小时):**重点攻 §2.2 溢出与 §2.4 比较逻辑——这两节是 C 程序员最容易踩坑的地方
3. **第三阶段(约1小时):**§2.5–§2.7 表达式化与特色运算符,亲手改写几个 C 表达式
4. **验收标准:**能把 10 个 C 表达式逐一翻译成 Rust,并说出每处翻译的理由

本章知识导图

- §2.1 运算符总览:与 C 的差异清单、完整优先级表
- §2.2 算术运算:溢出规则(debug panic / release 回绕)、除法与取模截断
- §2.3 位运算:& | ^ ! << >>、移位语义、标志位生产场景
- §2.4 比较与逻辑:值比较、浮点比较、短路、无隐式真值
- §2.5 赋值:复合赋值、解构绑定、没有 ++
- §2.6 表达式与语句:if/match/loop 表达式(三目替代)
- §2.7 特色运算符:? 错误传播、范围 ..、求值顺序
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§2.1 运算符总览:与 C 的差异清单

Rust 的运算符绝大部分与 C 同名同义:算术 `+` `-` `*` `/` `%`、位 `&` `|` `^` `!` `<<` `>>`、比较 `==` `!=` `<` `>` `<=` `>=`、逻辑 `&&` `||`、赋值与复合赋值 `=` `+=` `--` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=`。差异集中在下面六点:

C 的特性	Rust 的处理
<code>++</code> / <code>--</code> 自增自减	删除!用 <code>x += 1</code> 替代(§2.5)
三目 <code>a ? b : c</code>	删除!用 if 表达式替代(§2.6)
逗号运算符 <code>a, b</code>	删除!Rust 不需要它
有符号溢出 = UB	确定性:debug panic、release 回绕(§2.2)
求值顺序未指定	明确规定:从左到右(§2.7)
无 <code>?</code> 运算符	新增 <code>?</code> :错误传播(§2.7)
无范围运算符	新增 <code>..</code> 与 <code>..=</code> (§2.7)
无解构赋值	新增 <code>let (a, b) = pair</code> 模式绑定(§2.5)

2.1.1 完整优先级表(从高到低)

优先级	运算符	与 C 的差异
最高	路径 ::、方法调用 .、下标 []、调用 ()	多出 :: 路径运算符
第2	?	Rust 新增(错误传播)
第3	一元:- ! * & &mut	多出借用运算符 &(取引用)
第4	as(类型转换)	C 用强制转换语法
第5	* / %	相同
第6	+ -	相同
第7	< < > >	相同
第8	&	相同
第9	^	相同
第10		相同
第11	== != < > <= >=	相同
第12	&&	相同
第13		相同
第14	=	Rust 新增(范围)
最低	= += -= 等赋值;return break	赋值是语句不是表达式

★ 重点: Rust 的 & 比 == 优先级高, C 则相反

C 的经典坑: `x & 1 == 0` 在 C 中解析为 `x & (1 == 0)`, 恒为 0, 必须写 `(x & 1) == 0`。Rust 的优先级表中 `&` 排在比较运算之上, `x & 1 == 0` 直接按 `(x & 1) == 0` 解析——教科书级的坑被优先级规则修掉了。但请记住一条工程铁律: 位运算与逻辑运算混用时, 永远加括号, 可读性优先于“我知道规则”。

§2.2 算术运算: 溢出与除法取模

2.2.1 溢出: C 的 UB 变成确定性行为

C 的有符号整数溢出是未定义行为: 编译器可能回绕、可能优化掉你的 if、可能产生任何结果。Rust 的规定清晰且可预期: debug 模式(默认 `cargo run`)下溢出直接 panic 并退出; release 模式(`cargo build --release`)下按二进制补码回绕。更关键的是, Rust 提供三种“显式溢出”方法, 把语义写进代码:

```
fn main() {
    // debug 模式:下面这行会 panic(先注释掉)
    // let x: i32 = i32::MAX + 1;

    // 需求是"回绕"(像 C 的 int):用 wrapping
    let a = i32::MAX.wrapping_add(1); // -2147483648
    // 需求是"到边界就停":用 saturating(饱和)
    let b = 255u8.saturating_add(10); // 255,不溢出
    // 需求是"溢出时报告":用 checked(返回 Option)
    let c = 100i32.checked_mul(2); // Some(200)
    let d = 100i32.checked_mul(1_000_000); // None:超过 i32 范围
    println!("{a} {b} {c:?} {d:?}");
}
```

场景	C	Rust
有符号溢出	UB(编译器的自由发挥)	debug panic / release 回绕
无符号回绕	定义良好(模 2^n)	相同,但 debug 下同样 panic
显式回绕	无标准写法	wrapping_add/sub/mul/...
饱和运算	手动 if 判断	saturating_add/sub/mul/...
溢出检查	靠 if (x > INT_MAX - y) 手写	checked_add 等返回 Option

2.2.2 除法与取模:与 C 相同的截断语义

整数除法向零截断、取模结果与被除数同号——**Rust 与 C99 完全一致** (Python 是向下取整,注意别混用)。另外两个 C 程序员注意不到的细节:**除零**在 Rust 中是 panic(debug 与 release 都 panic),不是 C 那种"得到 NaN 或崩溃或 UB";**取模除数也必须非零**。浮点除零得到无穷(与 C 相同,符合 IEEE 754):

```
fn main() {
    // 除法向零截断:与 C 相同
    println!("{}", 7 / 2); // 3
    println!("{}", -7 / 2); // -3(向零,不是向下:-3.5 的向零是 -3)
    // 取模:符号跟随被除数,与 C 相同、与 Python 不同
    println!("{}", 7 % 2); // 1
    println!("{}", -7 % 2); // -1(Python 会给出 1,注意!)
    println!("{}", 7 % -2); // 1

    // 浮点除零:IEEE 754 的无穷/NaN(与 C 相同)
    println!("{}", 1.0 / 0.0); // inf
    println!("{}", -1.0 / 0.0); // -inf
    println!("{}", 0.0 / 0.0); // NaN

    // 整数除零:panic(先注释掉再试)
    // let z = 10 / 0;
    // 生产代码用 checked_div 防炸
    let safe = 10i32.checked_div(0); // None
    println!("{safe:?}");
}
```

⚠ 易错点 1:把"溢出会悄悄回绕"的 C 习惯带进 Rust

在 C 里, `int a = INT_MAX; a++;` 静默变成 `INT_MIN`,你的程序继续跑,然后在某个诡异的地方爆炸。Rust 默认在 debug 模式下让这种溢出当场 panic,把错误暴露在它发生的位置——这是特性不是缺陷。如果线上需要回绕语义(哈希、计数器),请显式写 `wrapping_add`,并加注释说明为什么允许回绕;审计代码时一眼就能看清你的意图。

§2.3 位运算

2.3.1 全套位运算符

`& | ^ ! << >>` 与 C 语义一致。差异有二:① 位运算符只能用于整数类型 (C 里可以对指针/浮点做位操作,危险但合法;Rust 直接编译错误);② 右移区分类型:无符号数逻辑右移(高位补 0),有符号数算术右移(高位补符号位)——与 C 的"实现定义"不同,Rust 把它固定下来:

```
fn main() {
    let a: u8 = 0b1100_0011;
    let b: u8 = 0b1010_1010;
    println!("{:08b}", a & b);    // 10000010 按位与
    println!("{:08b}", a | b);    // 11101011 按位或
    println!("{:08b}", a ^ b);    // 01101001 按位异或
    println!("{:08b}", !a);       // 00111100 按位取反
    println!("{:08b}", a << 2);    // 00001100 左移:低位补 0,高位丢弃
    println!("{:08b}", a >> 2);    // 00110000 逻辑右移:高位补 0

    // 有符号右移是算术右移:负数补 1
    let n: i8 = -8;                // 二进制:11111000
    println!("{}", n >> 1);        // -4(补符号位)
    let u: u8 = 0b1000_0000;
    println!("{}", u >> 1);        // 64(补 0)

    // 注意:! 是一元位取反;逻辑非用 !b 吗?不,逻辑非是 ! 作用于 bool
    let flag = true;
    println!("{}", !flag);         // false(! 在 bool 上就是逻辑非)
}
```

2.3.2 生产场景:标志位与位掩码

```
// 权限标志:C 里典型写法是 #define READ (1<<0)
const READ: u32 = 1 << 0;    // 0b001
const WRITE: u32 = 1 << 1;   // 0b010
const EXEC: u32 = 1 << 2;    // 0b100

fn main() {
    let mut perms: u32 = 0;
    perms |= READ | WRITE;     // 置位:加上读和写
    perms &= !EXEC;            // 清位:去掉执行
    perms ^= WRITE;            // 翻转:写权限取反
    println!("0b{:03b}", perms); // 0b001:只剩 READ

    if perms & READ != 0 {      // 判断某一位
        println!("有读权限");
    }
    if perms & WRITE != 0 {
        println!("有写权限");    // 不会打印:WRITE 被翻转掉了
    }
}
```

工程建议:位运算的四个生产原则

- 标志位用常量命名,禁止裸数字: `READ = 1 << 0` 而非 `1`;C 的 `#define` 在 Rust 里对应 `const`
- 移位量必须是"无符号且小于位宽"的常量或已知值; `1u32 << 32` 在 Rust 中直接 panic,不会像 C 那样静默出错
- 位运算与比较混用一律加括号,哪怕你知道优先级
- 频繁用到位操作的专业领域(协议解析、编解码),用位字段结构体替代裸掩码,可读性翻倍(第14章 newtype 模式)

§2.4 比较与逻辑

2.4.1 == 的值语义与浮点比较

Rust 的 `==` 只做值比较:整数比数值、字符串比内容(`String` / `&str` 直接可比,不需要 C 的 `strcmp`)、结构体默认不可比——需要 `#[derive(PartialEq)]` (第6章)。C 的 "`==` 比较指针地址"与"比较字符串要 `strcmp`"的双轨制在 Rust 中不存在。浮点的坑与 C 相同:NaN 不等于任何数、`0.1 + 0.2 != 0.3`:

```
fn main() {
    // 字符串直接 ==:内容比较(不是指针比较)
    let a = String::from("hello");
    let b = String::from("hello");
    println!("{}", a == b);           // true

    // NaN 的自我否定:与 C 相同,但 Rust 可以显式判断
    let nan = f64::NAN;
    println!("{}", nan == nan);       // false
    println!("{}", nan.is_nan());     // true:官方推荐写法

    // 二进制浮点误差:0.1 + 0.2 是 0.30000000000000004
    let x = 0.1 + 0.2;
    println!("{}", x == 0.3);         // false

    // 正确姿势:比较差值的绝对值是否小于阈值
    let eps = 1e-9;
    println!("{}", (x - 0.3).abs() < eps); // true
}
```

2.4.2 短路与无隐式真值

`&&` 与 `||` 的短路求值语义与 C 完全一致(左边就能决定结果时,右边不求值)。但 **条件必须是 bool**:C 的 `if (ptr)`、`if (n)`、`while (c)` 在 Rust 全部编译错误,必须显式写出判断。另外 `&&` / `||` 返回 `bool`,绝不返回操作数(与 Python/JS 不同):

```
fn main() {
    let a = false;
    if a && side_effect() {           // side_effect 不会被调用
        println!("不会执行");
    }
    let b = true;
    if b || side_effect() {           // 同上:短路
        println!("会执行");
    }

    // C 的 if (ptr) / if (fd) 必须显式化:
    let fd: Option<i32> = Some(3);    // 第1章的 Option:要么有值要么没有
    if fd.is_some() {
        println!("文件描述符存在");
    }

    // 逻辑运算结果永远是 bool,不能当数用
    let r = 1 < 2 && 3 < 4;
    println!("{}", r);               // true
}

fn side_effect() -> bool {
    println!("side_effect 被调用了!");
    true
}
```

⚠ 易错点 2: 短路没生效的错觉与链式比较

两件事容易搞混: ① `&&` 的右操作数必须是表达式而不是语句, `a && b = 1` 这种 C 写法在 Rust 是编译错误; ② C 的 `a < b < c` (先算 `a < b` 得 0/1, 再与 `c` 比) 在 Rust 中同样编译错误——必须写成 `a < b && b < c`。这两条都是“编译器强制你写清楚”, 不是陷阱而是保护。

§2.5 赋值: 复合赋值、解构绑定、没有 ++

2.5.1 赋值是语句, 不是表达式

C 的经典惯用法 `while ((c = getchar()) != EOF)` 依赖“赋值是表达式”。Rust 中**赋值不产生值**, 它是语句——上面这行直接编译错误。连锁反应: 连锁赋值 `a = b = c`、条件里赋值、`if (x = f())` 全部消失。好处是杜绝了“想写 `==` 却写成 `=`”的经典事故——编译器会拒绝这种代码。

2.5.2 复合赋值: 与解构绑定

复合赋值 `+= -= *= /= %= &= |= ^= <<= >>=` 与 C 语义相同, 且**左侧的位置表达式只求值一次** (C 里 `a[i++] += 1` 是 UB, 因为它两次计算下标)。Rust 还允许在 `let` 中解构绑定:

```
fn main() {
    let mut x = 10;
    x += 5;           // 15
    x *= 2;           // 30
    x >>= 1;          // 15(算术右移)
    x ^= 0xFF;        // 240
    println!("{}", x);

    // 解构绑定: 一次把元组拆开
    let pair = (3, 7);
    let (a, b) = pair; // a = 3, b = 7
    println!("{a} {b}");

    // 解构数组与带 _ 的占位
    let [first, second, _] = [1, 2, 3];
    println!("{first} {second}");
}
```

2.5.3 没有 ++ 与 --: 为什么? 替代是什么?

Rust 刻意不提供 `++ / --`。理由有二: ① 前置/后置的返回值差异 (`x++` 返回旧值) 是 C 里 `i = i++ + i++` 一类未定义行为与可读性灾难的源头; ② Rust 推崇“表达式只做一件事”, `x += 1` 意图比 `x++` 更直白。替代写法: 普通自增 `x += 1`; for 循环用范围 `for i in 0..n` (第3章); 先取旧值再自增这种需求, 用显式两行:

```
fn main() {
    let mut count = 0;
    count += 1;           // 替代 count++
    count -= 1;           // 替代 count--

    // 需要"旧值"的场景(C 的 y = x++):
    let mut x = 5;
    let y = x;             // 先取旧值
    x += 1;                // 再自增
    println!("{y} {x}");   // 5 6

    // C 的 for (i = 0; i < n; i++):
    for i in 0..5 {        // 第3章详解,这里先感受
        print!("{i} ");
    }
    println!();
}
```

⚠ 易错点 3:把 C 的"语句即表达式"习惯搬过来

三类代码在 Rust 中是编译错误,而且都曾经是 C 的"日常":① `while ((c = getchar()) != EOF)` ——赋值不是表达式;② `x = y = z` 连锁赋值;③ `return (x = 5)` 这种"赋值兼返回"。改写思路都一样:把赋值拆成语句,把值存进变量再用。难受两天,换来的是"条件里绝不混赋值"的终身安全。

§2.6 表达式与语句:if/loop/match 都是表达式

2.6.1 C 的三目?: 在 Rust 中是 if 表达式

Rust 没有 `?:`,但 `if` 本身就是表达式:每个分支是一个代码块,块的最后一个表达式(不带分号)就是分支的值。要求:所有分支类型必须一致(否则编译错误)。注意 `else` 不可省略——if 表达式必须给出所有情况的值:

```
fn main() {
    let score = 75;
    // C: int grade = score >= 60 ? 1 : 0;
    let grade = if score >= 60 { "及格" } else { "不及格" };
    println!("{grade}");

    // if-else 链:C 的 else if 链同样成立
    let level = if score >= 90 {
        "A"
    } else if score >= 80 {
        "B"
    } else if score >= 70 {
        "C"
    } else {
        "D"
    };
    println!("{level}");

    // 只用 if 做"副作用"时,分支返回 () 即可(后面不带值)
    let mut v = Vec::new();
    if score > 90 { v.push("优秀"); }
    println!("{:?}", v);
}
```

2.6.2 代码块也是表达式:{} 有值

```
fn main() {
    // 代码块的值 = 最后一个表达式(不带分号)
    let x = {
        let y = 3;
        let z = 4;
        y * z           // 没有分号:这就是代码块的值
    };
    println!("{x}");    // 12

    // 带分号则是 (),常见于"初始化一段临时逻辑"
    let _nothing = { let t = 1; t + 1; }; // 值为 ()
    println!("hello");
}
```

★ 重点:分号决定"是表达式还是语句"

这是全书写 Rust 最常见的"卡亮点":块内最后一个表达式带分号 → 块的值是 ();不带分号 → 块的值是表达式的值。编译错误 "expected `i32`, found `()`" 十有八九就是这个:你在 `if` 分支末尾多写了分号。遇到立刻检查:最后一个表达式有没有多余的分号。

§2.7 特色运算符:?. 范围、求值顺序

2.7.1 ? 运算符:错误传播

? 是 Rust 的标志性运算符:当表达式返回 Result 时,成功则取出值继续,失败则立即把错误"向上返回"。它消灭的正是 C 代码里铺天盖地的 `if (err) return err;` 样板(第9章详解,这里先看形态):

```
use std::num::ParseIntError;

// ? 要求所在函数返回 Result(或 Option)
fn parse_and_double(text: &str) -> Result<i32, ParseIntError> {
    let n: i32 = text.trim().parse()?; // 失败:立刻 return Err
    Ok(n * 2)                          // 成功:拿到 n 继续算
}

fn main() {
    // 调用方可以继续用 ? 向上传,也可以 match 处理
    match parse_and_double(" 21 ") {
        Ok(v) => println!("结果:{v}"), // 42
        Err(e) => println!("错误:{e}"),
    }
    match parse_and_double("abc") {
        Ok(v) => println!("结果:{v}"),
        Err(_) => println!("解析失败"),
    }
}
```

2.7.2 范围运算符 .. 与 ..=

..`表示半开区间(含头不含尾), ..= 表示闭区间。它是循环与切片(第3章)的基石,C 的 for (i = 0; i < n; i++) 在 Rust 中写作 for i in 0..n:`

```
fn main() {
    for i in 1..5 { print!("{i} "); } // 1 2 3 4
    println!();
    for i in 1..=5 { print!("{i} "); } // 1 2 3 4 5
    println!();

    // 与数组切片结合(第1章见过)
    let arr = [10, 20, 30, 40, 50];
    println!("{:?}", &arr[1..3]); // [20, 30]
    println!("{:?}", &arr[..2]); // [10, 20]
    println!("{:?}", &arr[2..]); // [30, 40, 50]
}
```

2.7.3 求值顺序:C 的"未指定"变成"从左到右"

C 里 `f() + g()` 先调谁、`a[i] = i++` 先算谁,都是未指定行为;Rust 明确规定:二元运算符的操作数、函数实参都按从左到右求值;赋值先求值右侧,再求值左侧。副作用顺序可预测,这是编写"表达式内带副作用"代码(虽然不建议)时安心的前提:

```
fn main() {
    // 从左到右:先打印 "左" 再打印 "右"
    let r = left() + right();
    println!("结果:{r}");
}

fn left() -> i32 {
    println!("左操作数先求值");
    1
}

fn right() -> i32 {
    println!("右操作数后求值");
    2
}
```

🚀 工程建议:表达式的生产准则

- 一行只干一件事:避免"表达式里带副作用"(如 print 里调用函数、下标里 i++)——Rust 虽规定顺序,但可读性仍差
- 优先用 let 中间变量拆解长表达式,比注释更有效
- 算术溢出有明确语义后,审计代码时重点找 as 截断与 unwrap(第9章),而不是猜测溢出行为
- 用 clippy(cargo clippy)做静态检查:它会对 `x & 1 == 0` 这类写法给出更可读的替代建议

本章速查表

主题	结论 / 写法
算术	+ - * / % 与 C 相同;除法向零截断;-7 % 2 = -1(与 Python 不同)
溢出	debug 模式 panic;release 回绕;wrapping/saturating/checked 三种显式选择
除零	整数除零、取模除零都是 panic;checked_div 返回 Option;浮点除零得 inf/NaN
自增自减	无 ++/--;x += 1;需要旧值就"先取后加"两行
位运算	& ^ ! << >>;只用于整数;无符号逻辑右移、有符号算术右移
位标志	const 命名 + 置位 / &= ! 清位 / ^ 翻转 / & != 0 判断
比较	== 是值比较:String 直接 ==;NaN != NaN;浮点比较用 (a-b).abs() < eps
逻辑	&& 短路且返回 bool;条件必须是 bool,无隐式真值
赋值	赋值是语句不是表达式;复合赋值左侧只求值一次
解构	let (a, b) = pair;let [a, b, _] = arr;
三目	无 ?::用 if 表达式:let m = if a { x } else { y };
表达式	if/match/loop/{ } 都是表达式;最后无分号者即值;分支类型必须一致
特色	? 错误传播(函数须返回 Result/Option);.. 半开区间、..= 闭区间
优先级	& 高于 == (C 相反);位运算与比较混用一律加括号
求值顺序	从左到右;赋值先右后左;无 C 的"未指定"

最小必会清单

- 能说出 Rust 从 C 删掉的四个运算符/语法(++ -- ?: 逗号、赋值当表达式)与新增的两个(.. 与 ?)
- 能写出 debug/release 两种模式下的溢出行为,以及 wrapping/saturating/checked 三种方法名
- 能口算 $-7 / 2$ 与 $-7 \% 2$,并解释与 Python 的差异
- 能把 C 的 `grade = score >= 60 ? 'A' : 'F'` 改写为 Rust if 表达式
- 能解释 `x & 1 == 0` 在 C 与 Rust 中的解析差异
- 能写出浮点比较的正确姿势并解释为什么
- 能解释为什么 `while ((c = getchar()) != EOF)` 在 Rust 中无法编译
- 能给出一段用位运算做权限标志的完整代码
- 能说出 if 表达式所有分支的类型要求,以及"分号决定表达式还是语句"
- 能说出 Rust 规定的求值顺序,以及它与 C 的差异

自测题

Q 1. `let x = i32::MAX; let y = x + 1;` 在 debug 模式与 release 模式下分别发生什么?C 里 `int y = INT_MAX + 1;` 呢?

✓ 参考答案

Rust debug:panic(attempt to add with overflow),进程终止;release:回绕为 `i32::MIN`。C 是未定义行为:结果不可预测,取决于编译器与优化。Rust 用"确定性"取代"UB":要么在出错现场终止,要么按补码回绕,且可用 `wrapping_add` 显式声明意图。

Q 2. 计算 $-7 / 2$ 与 $-7 \% 2$,并说明与 Python 的差异。

✓ 参考答案

$-7/2 = -3$ (向零截断), $-7\%2 = -1$ (符号跟随被除数),与 C99 相同。Python 里 $-7//2 = -4$ (向下取整), $-7\%2 = 1$ 。差异根源:Python 要求" $a = (a//b)*b + a\%b$ "且余数非负;Rust/C 允许余数为负。写跨语言移植代码时务必显式测试。

Q 3. 用 Rust 表达 C 的 `int m = a > b ? a : b;`,并要求 if 表达式的两个分支类型一致。若写成 `let m = if a > b { a } else { "大" };` 会怎样?

✓ 参考答案

`let m = if a > b { a } else { b };`。分支类型不一致(`i32` vs `&str`)会编译错误,报 "if and else have incompatible types"。这正是 Rust 强制所有分支必须给出同类型值的体现。

Q 4. `x & 1 == 0` 在 C 与 Rust 中的解析分别是什么?为什么会这样?

✓ 参考答案

C 中 `==` 优先级高于 `&`, 解析为 `x & (1 == 0)`, 即 `x & 0`, 恒为 0——经典 bug 来源, 必须写 `(x & 1) == 0`。Rust 中 `&` 优先级高于 `==`, 解析为 `(x & 1) == 0`, 语义正确。工程上仍建议加括号。

Q 5. 为什么 Rust 没有 ++ 与 --? C 的 `y = x++` 如何改写?

✓ 参考答案

前置/后置返回值差异带来大量可读性灾难与未定义行为(如 `i = i++ + i++`)。Rust 用 `x += 1` 表达自增; 需要旧值时显式两行: `let y = x; x += 1;`。牺牲一点简洁, 换取“表达式不再隐藏副作用”。

Q 6. 下面代码输出什么? `let mut x = 4; x >>= 1; x |= 1; println!("{}", x);` 有符号与无符号右移有什么区别?

✓ 参考答案

`x = 2` (4 逻辑右移 1 位), 再 `2 | 1 = 3`, 输出 3。有符号右移是算术右移(负数补 1), 无符号右移是逻辑右移(补 0): `-8i8 >> 1 = -4`, 而 `0b1000_0000u8 >> 1 = 64`。C 中这是实现定义行为, Rust 固定了规则。

章末实验题: 表达式求值计算器

实验 2: 表达式求值计算器

实验目标: 实现一个命令行表达式计算器, 输入字符串如 `"2 + 3 * 4"`、`"1 << 8"`、`"-5 % 3"`, 输出计算结果; 正确处理优先级、括号、错误报告与溢出。

实验要求:

- 任务 1: 分词器 `tokenize`: 把输入拆成数字与运算符 Token (覆盖: 字符遍历、整数字面量解析、Option/Result 雏形)
- 任务 2: 递归下降解析器, 按优先级分层 (覆盖: 优先级表 §2.1、一元负号、括号)
- 任务 3: 算术运算用 `checked_add/checked_sub/checked_mul` 防溢出, 除零与取模除零报错 (覆盖: 溢出三兄弟 §2.2)
- 任务 4: 位运算符 `& | ^ << >>` 接入解析器 (覆盖: 位运算 §2.3、移位量校验)
- 任务 5: `main` 中对输入做短路校验并逐条执行测试用例 (覆盖: 短路 `&& ||` §2.4、复合赋值 `+=` 等、`match` 处理 Result)
- 任务 6: 用 `if` 表达式与 `==` 对比两个表达式的计算结果并输出 (覆盖: `if` 表达式与比较 §2.5–2.6)

实验环境: `cargo new calc_rs`, 代码写入 `src/main.rs`, `cargo run` 运行; 自备 10 条测试表达式 (含故意写错的)。

第一步 一步: 先写 Token 枚举与 `tokenize`; 注意 `<<` 是两个字符, 读到单个 `<` 要报错

第二步 二步: Parser 持有一个 Token 数组与下标; 每个 `parse_xxx` 函数返回 `Result<i64, String>`, 内部用 ? 传播下层错误

第三步 三步: 按 `|` 最低, 然后 `^`、`&`、移位、加减、乘除、一元负号最高"的顺序嵌套实现

第四步 四步: `main` 里逐个跑测试用例, 用 `match` 打印 "表达式 = 结果" 或错误信息

第五步 五步: 追加任务 6: 解析两个表达式并用 `if` 表达式输出比较结果

✓ 参考答案要点

```
// ===== 实验 2:表达式求值计算器 =====
// 覆盖知识点:优先级与结合性($2.1)、溢出处理($2.2)、除法取模($2.2)、
//          位运算($2.3)、短路与复合赋值($2.4/2.5)、
//          if 表达式与 == 比较($2.6)、? 错误传播与范围($2.7)

#[derive(Debug, Clone, PartialEq)]
enum Token {
    Num(i64),
    Plus, Minus, Star, Slash, Percent,
    Amp, Pipe, Caret, Shl, Shr,
    LParen, RParen,
}

/// 分词:把源字符串切成 Token 序列
fn tokenize(src: &str) -> Result<Vec<Token>, String> {
    let chars: Vec<char> = src.chars().collect();
    let mut tokens = Vec::new();
    let mut i = 0;
    while i < chars.len() {
        let c = chars[i];
        match c {
            ' ' | '\t' => {} // 空白跳过
            '0'..'9' => {
                let mut n: i64 = 0;
                while i < chars.len() && chars[i].is_ascii_digit() {
                    n = n.checked_mul(10).ok_or("数字字面量溢出")?
                        .checked_add(chars[i] as i64 - '0' as i64)
                        .ok_or("数字字面量溢出")?;
                    i += 1;
                }
                tokens.push(Token::Num(n));
                continue; // 已前进,跳过底部 i += 1
            }
            '+' => tokens.push(Token::Plus),
            '-' => tokens.push(Token::Minus),
            '*' => tokens.push(Token::Star),
            '/' => tokens.push(Token::Slash),
            '%' => tokens.push(Token::Percent),
            '&' => tokens.push(Token::Amp),
            '|' => tokens.push(Token::Pipe),
            '^' => tokens.push(Token::Caret),
            '<' => {
                if chars.get(i + 1) == Some(&'<') { tokens.push(Token::Shl); i += 1; }
                else { return Err(format!("位置 {i}:单独的 < 不是合法运算符")); }
            }
            '>' => {
                if chars.get(i + 1) == Some(&'>') { tokens.push(Token::Shr); i += 1; }
                else { return Err(format!("位置 {i}:单独的 > 不是合法运算符")); }
            }
            '(' => tokens.push(Token::LParen),
            ')' => tokens.push(Token::RParen),
            _ => return Err(format!("不支持的字符 '{c}'")),
        }
        i += 1;
    }
}
```

```

    Ok(tokens)
}

/// 递归下降解析器:优先级从低到高 | → ^ → & → 移位 → 加减 → 乘除 → 一元负号
struct Parser {
    tokens: Vec<Token>,
    pos: usize,
}

impl Parser {
    fn new(tokens: Vec<Token>) -> Self {
        Parser { tokens, pos: 0 }
    }

    fn peek(&self) -> Option<&Token> { self.tokens.get(self.pos) }

    fn next(&mut self) -> Option<Token> {
        let t = self.tokens.get(self.pos).cloned();
        self.pos += 1;
        t
    }

    fn parse_or(&mut self) -> Result<i64, String> {
        let mut left = self.parse_xor()?;
        while self.peek() == Some(&Token::Pipe) {
            self.next();
            let right = self.parse_xor()?;
            left |= right;           // 复合赋值
        }
        Ok(left)
    }

    fn parse_xor(&mut self) -> Result<i64, String> {
        let mut left = self.parse_and()?;
        while self.peek() == Some(&Token::Caret) {
            self.next();
            let right = self.parse_and()?;
            left ^= right;
        }
        Ok(left)
    }

    fn parse_and(&mut self) -> Result<i64, String> {
        let mut left = self.parse_shift()?;
        while self.peek() == Some(&Token::Amp) {
            self.next();
            let right = self.parse_shift()?;
            left &= right;
        }
        Ok(left)
    }

    fn parse_shift(&mut self) -> Result<i64, String> {
        let mut left = self.parse_add()?;
        loop {

```

```

        match self.peek() {
            Some(Token::Shl) => {
                self.next();
                let r = self.parse_add()?;
                let r = u32::try_from(r).map_err(|_| "移位置必须非负")?;
                left = left.checked_shl(r).ok_or("左移溢出")?;
            }
            Some(Token::Shr) => {
                self.next();
                let r = self.parse_add()?;
                let r = u32::try_from(r).map_err(|_| "移位置必须非负")?;
                left = left.checked_shr(r).ok_or("移位置过大")?;
            }
            _ => break,
        }
    }
    Ok(left)
}

fn parse_add(&mut self) -> Result<i64, String> {
    let mut left = self.parse_mul()?;
    loop {
        match self.peek() {
            Some(Token::Plus) => {
                self.next();
                let r = self.parse_mul()?;
                left = left.checked_add(r).ok_or("加法溢出")?;
            }
            Some(Token::Minus) => {
                self.next();
                let r = self.parse_mul()?;
                left = left.checked_sub(r).ok_or("减法溢出")?;
            }
            _ => break,
        }
    }
    Ok(left)
}

fn parse_mul(&mut self) -> Result<i64, String> {
    let mut left = self.parse_unary()?;
    loop {
        match self.peek() {
            Some(Token::Star) => {
                self.next();
                let r = self.parse_unary()?;
                left = left.checked_mul(r).ok_or("乘法溢出")?;
            }
            Some(Token::Slash) => {
                self.next();
                let r = self.parse_unary()?;
                if r == 0 { return Err("除数为零".into()); }
                left /= r; // 截断除法,与 C 相同
            }
            Some(Token::Percent) => {

```

```

        self.next();
        let r = self.parse_unary()?;
        if r == 0 { return Err("取模除数为零".into()); }
        left %= r;
    }
    _ => break,
}
}
Ok(left)
}

fn parse_unary(&mut self) -> Result<i64, String> {
    if self.peek() == Some(&Token::Minus) {
        self.next();
        let v = self.parse_unary()?;
        return v.checked_neg().ok_or("取负溢出");
    }
    match self.next() {
        Some(Token::Num(n)) => Ok(n),
        Some(Token::LParen) => {
            let v = self.parse_or()?;           // 括号内重新按优先级解析
            match self.next() {
                Some(Token::RParen) => Ok(v),
                _ => Err("缺少右括号".into()),
            }
        }
        _ => Err("语法错误:缺少操作数".into()),
    }
}

}

/// 顶层入口:解析整条表达式,并拒绝"多余内容"
fn eval(src: &str) -> Result<i64, String> {
    let tokens = tokenize(src)?;                // ? 错误传播
    let mut p = Parser::new(tokens);
    let value = p.parse_or()?;
    if p.peek().is_some() {
        return Err("语法错误:表达式后有多余内容".into());
    }
    Ok(value)
}

fn main() {
    // 短路校验:空输入或超长输入直接跳过(覆盖 && 短路)
    let cases = [
        "2 + 3 * 4",
        "(2 + 3) * 4",
        "10 / 3",
        "-7 / 2",
        "-7 % 2",
        "255 & 240",
        "1 | 2 | 4",
        "1 << 8",
        "8 >> 1",
        "-5 + 3",
    ];
}

```



```

        "100 / 0",
        "10 % 0",
        "2 +",
        "1 + 2 +",
        "",
    ];
    for case in cases {
        let t = case.trim();
        if t.is_empty() || t.chars().count() > 64 { // 短路:左边为真就不再算右边
            println!("跳过空/超长输入");
            continue;
        }
        match eval(t) {
            Ok(v) => println!("{case:<10} = {v}"),
            Err(e) => println!("{case:<10} 错误: {e}"),
        }
    }

    // 任务 6:比较两个表达式的值,用 if 表达式输出结论(覆盖 == 与 if 表达式)
    let a = eval("1 + 2 * 3").unwrap_or(0);
    let b = eval("(1 + 2) * 3").unwrap_or(0);
    let conclusion = if a == b { "两个表达式结果相同" } else { "结果不同" };
    println!("{a} 与 {b}:{conclusion}");
}

```

设计说明:整个解析器刻意**不含任何 if/else 嵌套超过一层**——优先级靠函数嵌套表达,错误靠 ? 向上传播,这是 Rust 表达式文化的浓缩:每个 parse_xxx 返回 Result,内部零样板错误处理。运行后把 "255 & 240" 改成 "255 & 0xFF" 试试?你的 tokenize 目前不支持十六进制——这就是任务清单里"自备测试表达式"的意义:让程序自己暴露薄弱点,再回头修。

下一章预告:第3章 流程控制——match 的穷尽性检查、if let/while let 糖、带标签的 break、for 与迭代器。C 的 switch 只能等值比较整型,Rust 的 match 能把任意类型拆开看——那才是真正的"模式匹配"。

第3章 流程控制

建议学时:3-4 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Rust 的全部流程控制原语:if/else、match、if let、while let、loop、while、for——与 C 的 if/switch/for/while/do-while/goto 逐一对照
- 理解 match 相对 C 的 switch 的三个飞跃:穷尽性检查、模式绑定、守卫条件;C 的 fallthrough 与"只能整型"两大致命缺陷在 Rust 中如何被消灭
- 掌握 if let/while let 语法糖,以及 let-else 现代写法
- 掌握三种循环的定位:loop(无限)、while(条件)、for(遍历);会用 loop+if break 模拟 C 的 do-while
- 掌握 break 带值、continue、带标签 break 'outer——Rust 对 C 的 goto 的优雅替代
- 掌握 for 与迭代器:0..n 范围遍历、iter()/iter_mut()/into_iter() 三形态、循环变量每轮新绑定(没有 Go 1.22 前的经典坑)
- 理解"遍历中修改容器"的迭代器失效问题在 Rust 中如何被借用检查器在编译期拦截

💡 从 C 到 Rust:本章主题如何迁移

C 的 switch 有两个与生俱来的缺陷:一是 fallthrough——忘记写 break 就掉进下一个分支,这是 C 生产代码里最经典的 bug 之一;二是它只能对整数做等值比较。Rust 的 match 把这两个缺陷一起消灭:分支之间自动隔离(没有 fallthrough),而且能对任意类型做"模式匹配"——解构结构体、匹配区间、绑定变量、加守卫条件。你在 C 里用一长串 else-if 和 switch 拼出来的逻辑,在 Rust 里是一段整洁的 match。

Rust 对"非穷尽"采取零容忍:match 必须覆盖所有可能,否则编译失败。这看似苛刻,实则是最有价值的安全网——C 里"忘了写 default"只是默默漏掉一种情况;Rust 里编译器会指着那一种情况问你:它该怎么办?你被迫把每种输入都安排明白,这正是生产代码需要的严谨。

C 的 goto 被视为禁忌:滥用会写出意大利面条式代码,但某些场景(深层嵌套中跳出)确实需要它。Rust 的态度是"用更好的原语替代":带标签的 break 'outer 让"跳出两层循环"有了安全优雅的写法;提前返回 让错误处理不再需要跳转。goto 的能力被保留,混乱被移除。

最后是循环哲学:C 的 for 是"下标驱动的循环"(i 从 0 到 n),Rust 的 for 是"迭代器驱动的循环"(依次取每个元素)。前者把注意力放在"怎么数",后者把注意力放在"要什么"——配合 0..n 范围与迭代器方法链,大多数循环在 Rust 里更短、更不易错,而且编译器保证你不会写错边界。

📦 前置知识自查

- C 的 if/else、switch/case/break/default、for/while/do-while 的完整语义
- C 的 fallthrough 陷阱、悬挂 else、for(;;) 无限循环
- C 的 goto 与"提前返回"两种跳出风格
- C 的数组遍历 for (i = 0; i < n; i++) 与指针遍历
- 第1章内容:Option、元组、Vec、枚举;第2章内容:if 是表达式、.. 范围

🚀 本章学习路线

1. **第一阶段(约1小时):**§3.2–§3.3,重点体验 match 的穷尽性与模式绑定,把 C 的 switch 习惯改过来
2. **第二阶段(约1小时):**§3.4–§3.5,练习三种循环与带标签 break、break 带值
3. **第三阶段(约1.5小时):**§3.6 迭代器与 for 的三种形态,再完成章末实验
4. **验收标准:**能用 match 重写任意 if-else 链;能写出"跳出两层循环"与"至少执行一次"的代码;对每个 C 流程控制语句能说出 Rust 对应物

本章知识导图

- §3.1 总览:流程控制差异清单
- §3.2 if 与 if 表达式:条件必须 bool、无悬挂 else
- §3.3 match:C 的 switch 进化版(穷尽/绑定/守卫/区间/多模式)
- §3.4 if let 与 while let:Option 的语法糖
- §3.5 三种循环:loop / while / for、do-while 模拟、break 带值、标签
- §3.6 for 与迭代器:0..n、iter/iter_mut/into_iter、循环变量新绑定
- §3.7 goto 的替代:提前返回与标签 break;迭代器失效问题
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§3.1 总览:与 C 的差异清单

C 的写法	Rust 的写法	差异要点
if (cond) {...} else {...}	if cond {...} else {...}	无括号;条件必须是 bool
switch (n) { case 1: ... }	match n { 1 => ..., _ => ... }	无 fallthrough;必须穷尽;可匹配任意类型
for (i = 0; i < n; i++)	for i in 0..n	范围迭代,编译器保证边界
while (cond)	while cond	相同
do {...} while (cond)	loop {...; if !cond { break } }	Rust 无 do-while,用 loop+break 模拟
for(;;) / while(1)	loop	无限循环专用关键字
break / continue	break / continue	相同;另支持带标签与 break 带值
goto label	'label: loop {...} + break 'label	仅用于跳出嵌套循环,比 goto 安全

§3.2 if 与 if 表达式

3.2.1 条件必须 bool;没有悬挂 else

第2章已强调: `if 1`、`if ptr` 都是编译错误。这里补一个 C 的经典坑的消失:C 的悬挂 else(else 就近配对被误读),在 Rust 里因为大括号必须成对出现而**根本不可能发生**——每个 if 的块边界清清楚楚:

```
fn main() {
    let hour = 9;
    // C 里缩进暗示、编译器就近配对的经典悲剧,在 Rust 中无法构造
    if hour < 12 {
        println!("上午");
    } else if hour < 18 {
        println!("下午");
    } else {
        println!("晚上");
    }
}
```

3.2.2 if 表达式的完整形态

第2章已见 if 表达式的"取两个值"形态;这里补两个生产细节:① if-else 链求值时,从上到下第一个条件成立的分支生效;② 只做副作用时,分支返回 ()——Rust 允许省略 else:

```
fn main() {
    let price = 120;
    // if 表达式:所有分支类型一致(这里是 &str)
    let label = if price >= 100 {
        "贵"
    } else {
        "便宜"
    };
    println!("{label}");

    // 只做副作用:允许没有 else(此时类型是 ())
    let mut cart = Vec::new();
    if price <= 200 {
        cart.push("奶茶");
    }
    println!("{:?}", cart);

    // 经典误用:C 的三目被嵌套使用—Rust 建议拆开
    let grade = if price > 80 { "高" } else { "低" };
    println!("{grade}");
}
```

⚠ 易错点 1:if 表达式分支类型不一致,或忘了"无分号才是值"

编译错误 "if and else have incompatible types" 与第2章讲过的 "expected i32, found ()" 是本章最高频的两个报错。检查顺序:① 每个分支的最后一个表达式有没有多写分号;② 分支是否真的返回同类型;③ 是不是忘了 else(缺 else 时类型是 (),而外面又把它当值用。三分钟内定位,永不慌张。

§3.3 match:C 的 switch 进化版

3.3.1 穷尽性:编译器逼你覆盖所有情况

match 的每个分支无需 break(自动隔离),但整个 match 必须**穷尽**所有可能——漏掉任何情况都是编译错误。对整数,要么列出所有可能性,要么用通配 `_` 收尾。与 C 对照:C 忘写 default 是"悄悄漏掉",Rust 忘写 `_` 是"编译失败,必须处

理":

```
fn main() {
    let n = 3;
    match n {
        0 => println!("零"),
        1 => println!("一"),
        // 注释掉下面一行试试:编译器报"非穷尽"
        _ => println!("其他数"),
    }

    // match 是表达式:整个 match 的值赋给变量
    let word = match n {
        0 => "zero",
        1 => "one",
        _ => "many",
    };
    println!("{word}");

    // 多模式组合:一个分支匹配多个值(C 里要连写几个 case)
    let c = 'x';
    match c {
        'a' | 'e' | 'i' | 'o' | 'u' => println!("元音"),
        'a'..'z' => println!("小写字母"), // 区间模式
        _ => println!("其他字符"),
    }
}
```

3.3.2 模式绑定与守卫

这是 match 相对 switch 的质变:模式可以"拆开"被匹配的值,并把其中一部分绑定为变量——C 的 switch 永远做不到,你只能 case 值然后手动取字段。守卫 `if` 给模式加上附加条件:

```
// 带数据的枚举(第1章见过):模式绑定在此发力
enum Shape {
    Circle(f64),
    Rect { w: f64, h: f64 },
}

fn area(shape: &Shape) -> f64 {
    match shape {
        // 绑定:把圆括号里的 f64 绑给 r
        Shape::Circle(r) => 3.14159 * r * r,
        // 结构体模式:字段名直接绑定
        Shape::Rect { w, h } => w * h,
    }
}

fn main() {
    let shapes = vec![Shape::Circle(2.0), Shape::Rect { w: 3.0, h: 4.0 }];
    for s in &shapes {
        println!("面积:{}", area(s));
    }

    // 守卫:模式之外再加条件
    let score = 85;
    match score {
        s if s >= 90 => println!("优秀"),
        s if s >= 80 => println!("良好"),
        s if s >= 60 => println!("及格"),
        _ => println!("不及格"),
    }
}
```

3.3.3 与 C 的 switch 对照表

对比项	C 的 switch	Rust 的 match
fallthrough	不写 break 就穿透(经典 bug 源)	分支自动隔离,无穿透
可匹配类型	只能整型(及 char 提升)	任意类型:枚举/结构体/元组/区间/引用
穷尽性	无检查,漏 case 或 default 静默	必须穷尽,否则编译错误
绑定变量	case 只做等值比较	模式绑定 + 守卫 if + 多模式
表达式性	语句	表达式,可直接产生值
default	default 分支	_ 通配(可绑定:_x 表示"要值但忽略")

⚠ 易错点 2:把 C 的 fallthrough 习惯带进 match,或误用 _ 遮蔽

① 以为"执行完这个分支继续下一个",不需要——Rust 分支之间没有穿透,写 `=> println!("a"); println!("b")` 是同一个分支的两条语句。② `_` 是"忽略所有未列出的值",`_x` 是"绑定但故意不用";在模式里把 `_` 当变量用会编译错误(`_` 不能读)。③ 忘了模式绑定是"借用还是移动"——match 一个 `&Shape` 时绑定的是引用,`r * r` 直接可用;match 拥有所有权时,`String` 等字段会被 move 走(第5章),之后就不能再用原值了。

§3.4 if let 与 while let

当 match 只有一个分支有意义时(最常见:检查 Option 有没有值),完整 match 显得冗长: `match opt { Some(v) => ..., None => () }`。语法糖 `if let` 专治这种"只想处理一种情况"的场景;匹配失败时可选 `else` 分支:

```
fn main() {
    let opt: Option<i32> = Some(7);
    // 完整 match
    match opt {
        Some(v) => println!("完整写法:{v}"),
        None => (),
    }
    // if let:同一逻辑的精简写法
    if let Some(v) = opt {
        println!("糖写法:{v}");
    }
    // 带 else:处理失败情况
    let none: Option<i32> = None;
    if let Some(v) = none {
        println!("{v}");
    } else {
        println!("没有值");
    }
}
```

while let:循环直到失败

```
fn main() {
    // 经典场景:从容器里不断取元素,直到取空
    let mut stack = vec![10, 20, 30];
    // C 的等价物:while (top != -1) { x = stack[top--]; }
    while let Some(v) = stack.pop() {
        println!("弹出:{v}");
    }
    println!("栈空了:{v}", stack.is_empty()); // true

    // 另一个场景:迭代器逐个取(第4章会细讲闭包与迭代器)
    let mut iter = [1, 2, 3].into_iter();
    while let Some(x) = iter.next() {
        print!("{x} ");
    }
    println!();
}
```

let-else:失败就提前退出(1.65+ 的现代写法)

```
fn parse_port(text: &str) -> u16 {
    // let-else:解构失败时执行 else 块并退出函数
    let Some(port) = text.trim().parse::<u16>().ok() else {
        println!("端口号无效,使用默认 8080");
        return 8080;
    };
    // 到这里 port 一定可用
    port
}

fn main() {
    println!("{}", parse_port("9999")); // 9999
    println!("{}", parse_port("abc"));  // 使用默认值 8080
}
```

工程建议:if let / while let / let-else 的选择

- 只有一种情况要处理:if let;两种都要处理:match(可读性高于 if-let-else)
- 解构失败意味着"函数无法继续":let-else,一目了然
- 循环消费一个 Option 序列:while let
- 生产代码里 if let Some(x) = map.get(key) 是最高频的写法之一,务必形成肌肉记忆

\$3.5 三种循环:loop / while / for

3.5.1 loop:无限循环专用

Rust 为"无限循环"专门造了 `loop` 关键字(C 的 `for(;;)` / `while(1)`),语义清晰;更重要的是 `break` 可以带值:loop 整体是一个表达式,退出时把值"算"出来——这是 C 无法表达的:

```
fn main() {
    // C 的 for(;;) / while(1) 在 Rust 里:
    loop {
        println!("无限循环(用 break 退出)");
        break;
    }

    // break 带值:loop 求出一个结果(C 需要临时变量 + 多次判断)
    let mut n = 1;
    let answer = loop {
        n += 1;
        if n * n > 1000 {
            break n; // 把 n 作为 loop 的值带出去
        }
    };
    println!("第一个平方大于 1000 的数是 {answer}({answer}^2 = {})", answer * answer);
}
```


3.5.2 while 与 do-while 模拟

```
fn main() {
    // while:与 C 完全相同,条件为假可能一次都不执行
    let mut i = 0;
    while i < 3 {
        print!("{i} ");
        i += 1;
    }
    println();

    // C 的 do-while 在 Rust 里没有,用 loop + if break 模拟:
    // 先执行循环体,再检查条件
    let mut tries = 0;
    loop {
        tries += 1;
        println!("第 {tries} 次尝试");
        if tries >= 3 {
            break;           // 等价于 C 的 while(条件) 判断失败退出
        }
    }
}
```

场景	C	Rust
无限循环	while (1) / for (;;)	loop
条件循环	while (cond)	while cond
至少执行一次	do { } while (cond)	loop { ...; if !cond { break; } }
固定次数	for (i = 0; i < n; i++)	for i in 0..n
遍历容器	for (p = head; p; p = p->next)	for x in iterable
跳出多层	goto 或标志位	'outer: loop + break 'outer

3.5.3 break/continue 与标签

```
fn main() {
    // continue:跳过本轮剩余部分,与 C 相同
    for i in 0..6 {
        if i % 2 == 0 {
            continue;           // 偶数跳过
        }
        print!("{i} ");         // 只打印奇数
    }
    println!();

    // 带标签 break:跳出外层循环(goto 的安全替代)
    let mut count = 0;
    'outer: for i in 1..=5 {
        for j in 1..=5 {
            count += 1;
            if i * j > 10 {
                println!("在 i={i}, j={j} 处跳出,共执行 {count} 次");
                break 'outer;    // 直接跳出外层 for
            }
        }
    }

    // 带标签 continue:跳到外层循环的下一迭代
    'filter: for i in 0..5 {
        for j in 0..5 {
            if i + j >= 4 {
                continue 'filter; // 跳过 i 的剩余 j,进入下一个 i
            }
            print!("{i},{j} ");
        }
    }
    println!();
}
```

⚠ 易错点 3:标签语法与 break 的误用

三个高频错误:① 标签写法是 `'outer:` (单引号在冒号左边),写在循环语句之前;写反或漏引号都编译错误。② `break` 不带标签时只能退出最内层循环——想退出两层,必须给外层加标签。③ `loop` 中 `break` 带值后,`loop` 的值类型必须一致; `let x = loop { break 1; break 2; }` 没问题,但 `break;` (无值)与 `break 1;` 混用会类型错误。

§3.6 for 与迭代器

3.6.1 范围遍历:C 的 for(i) 的现代替换

```
fn main() {
    // C: for (int i = 0; i < n; i++)
    // Rust:0..n 半开区间,编译器保证边界正确
    for i in 0..5 {
        print!("{i} ");
    }
    println();

    // C: for (int i = 0; i <= n; i++)
    for i in 0..=5 {
        print!("{i} ");
    }
    println();

    // 倒序:C 的 for (i = n-1; i >= 0; i--)——注意这里 C 的经典下溢坑
    for i in (0..5).rev() {
        print!("{i} ");
    }
    println();
}
```

C 的 `for (i = n-1; i >= 0; i--)` 有一个著名的坑:`i` 是 `int` 时条件永远成立(下溢),必须改用 `size_t` 再小心处理;
`(0..5).rev()` 一行搞定且绝无此问题。

3.6.2 迭代器的三种形态:iter / iter_mut / into_iter

对容器 `v` 做 `for` 遍历有三种意图:只读(`&v / v.iter()`)、改写(`&mut v / v.iter_mut()`)、搬走(`v.into_iter()`)——对应第5章的借用三态,这里先看用法:

```
fn main() {
    let mut v = vec![1, 2, 3];

    // 形态一:只读遍历(推荐:for x in &v)
    for x in &v {
        print!("{x} ");
    }
    println();

    // 形态二:改写每个元素
    for x in &mut v {
        *x *= 10;           // 注意:此处 x 是 &mut i32,要解引用
    }
    println!("{v:?}");      // [10, 20, 30]

    // 形态三:把元素全部搬走(循环后 v 不可再用,第5章解释)
    let total: i32 = v.into_iter().sum();
    println!("总和:{total}");
    // println!("{v:?}");    // 编译错误:v 已被搬空
}
```

3.6.3 循环变量每轮都是新绑定

Go 1.22 之前的经典 bug:循环变量是同一个变量,闭包捕获的是它的地址,循环结束后全部指向最后一个值。Rust 的 `for i in 0..n` 中 **每一轮迭代 `i` 都是独立的新绑定**,捕获安全,此坑不存在:

```
fn main() {
    // 每轮 i 都是新绑定:收集到闭包里也各自独立(第4章讲闭包)
    let mut captured = Vec::new();
    for i in 0..5 {
        captured.push(move || i * i);    // 每个闭包拥有自己的 i
    }
    for f in &captured {
        print!("{}", f());
    }
    println();                          // 0 1 4 9 16
}
```

§3.7 goto 的替代与迭代器失效

3.7.1 Rust 没有 goto:替代方案清单

- **跳出嵌套循环**:带标签 `break/continue('outer: loop + break 'outer)`,覆盖 `goto` 90% 的合法用途
- **提前退出函数**:`return`——Rust 里"守卫 + 提前返回"是主流风格,见下方 `is_prime` 示例
- **错误处理跳转**:`?` 运算符(第2章)与 `match`(本章),替代 C 的 `goto err_cleanup` 模式
- **状态机逻辑**:`loop + match`(状态转移表),替代 `goto` 拼状态跳转

```
// 提前返回风格:C 的 goto 清理模式被"每层提前 return"替代
fn classify(n: i32) -> &'static str {
    if n < 0 {
        return "负数";          // 提前返回,无需 goto 跳转
    }
    if n == 0 {
        return "零";
    }
    if n % 2 == 0 {
        return "正偶数";
    }
    "正奇数"
}

fn main() {
    for n in [-3, 0, 4, 7] {
        println!("{n}: {}", classify(n));
    }
}
```

3.7.2 迭代器失效问题:C++ 的经典噩梦在 Rust 中不存在

C++ 里"遍历 vector 的同时 push/erase"会让迭代器失效,造成悬垂访问;C 里则是在遍历链表时删除节点,轻则跳过元素重则崩溃。Rust 的借用规则(第5章)在 **编译期** 直接禁止"一边借用一边修改":

```
fn main() {
    let mut v = vec![1, 2, 3];

    // 下面的代码编译不过:for x in &v 是共享借用,
    // v.push 需要可变借用,两者同时存在即冲突
    // for x in &v {
    //     v.push(*x);          // 编译错误:cannot borrow `v` as mutable
    // }

    // 正确做法:先算好增量,再一次性修改
    let doubled: Vec<i32> = v.iter().map(|x| x * 2).collect();
    v.extend(doubled);
    println!("{v:?}");        // [1, 2, 3, 2, 4, 6]
}
```

★ 重点:流程控制的 Rust 心法

- 能用表达式就不用语句;if/match/loop 优先"算出值",而不是"改变量"
- match 优先于 if-else 链:分支多、有数据要拆开、需要穷尽检查时,一律 match
- 循环优先 for + 迭代器;需要条件退出用 while;无限循环用 loop;do-while 用 loop+break
- 编译器是你最好的伙伴:穷尽性、借用冲突、边界错误——全都编译期发现

本章速查表

主题	结论 / 写法
if	条件必须 bool;无括号;if-else 是表达式,分支类型必须一致
match 分支	<code>n => expr</code> ;多模式 <code>a b</code> ;区间 <code>1..=9</code> ;守卫 <code>s if cond</code> ;通配 <code>_</code>
match 穷尽	必须覆盖所有可能,否则编译错误; <code>_</code> 处理剩余
模式绑定	<code>Some(v)</code> 、 <code>Circle(r)</code> 、 <code>Rect { w, h }</code> 把数据拆出来绑给变量
if let	<code>if let Some(v) = opt { ... } else { ... }</code> ;只关心一种情况的糖
while let	<code>while let Some(v) = stack.pop()</code> ;循环取到失败为止
let-else	<code>let Some(v) = f() else { return 默认值; }</code> ;失败提前退出
loop	无限循环;break 可带值,loop 整体是表达式
while	条件循环,可能一次都不执行
do-while	Rust 没有; <code>loop { 体; if !cond { break; } }</code> 模拟
for	<code>for i in 0..n</code> ; <code>for x in &v</code> (只读)/ <code>&mut v</code> (改写)/ <code>v</code> (搬走)
break/continue	break 退出当前循环;continue 跳下一轮;支持 'label: 标签
break 带值	<code>let ans = loop { ...; break 42; }</code>
goto	无;替代:标签 break、提前返回、? 运算符
迭代器失效	借用检查器编译期禁止"遍历中修改容器"
循环变量	每轮新绑定,闭包捕获无 Go 1.22 前的经典坑

最小必会清单

- 能把任意 if-else 链改写成 match,并说出 match 的三个相对 switch 的优势
- 能解释"match 必须穷尽"在生产中的价值,并写出 `_` 通配
- 能写出区间模式、多模式 `|`、带守卫 if 的 match 各一个
- 能区分 if let 与 match 的适用场景,并写 while let 消费一个 Option 序列
- 能写出三种循环各自的最佳场景,以及 do-while 的模拟写法
- 能写 break 带值取出 loop 的结果
- 能给两层循环加标签,并写出 `break 'outer` 与 `continue 'filter`
- 能写出 for 的三种形态(`iter`/`iter_mut`/`into_iter`)并说明区别
- 能解释为什么"遍历中修改容器"在 Rust 中编译不过
- 能说出 Rust 用哪些机制替代 C 的 goto,并各给一个例子

自测题

Q 1. 下面的 match 为什么编译错误? `let n = 5; match n { 1 => "一", 2 => "二" }` 如何修复?

✅ 参考答案

非穷尽: `n` 是 `i32`, 可以取无数个数, 只列出 1 和 2 无法覆盖, 编译器报 "non-exhaustive patterns"。修复: 加通配分支 `_ => "其他"` (或 `_x` 如果要用到它)。这正是 "编译器逼你处理所有情况" 的体现——C 里忘了 `default` 只是静默。

Q 2. C 的 `switch` 的 `fallthrough` 与 Rust 的 `match` 有什么关系? 在 `match` 里如何实现 "多个值共用一段代码"?

✅ 参考答案

`match` 分支之间自动隔离, 不存在 `fallthrough`——C 忘写 `break` 的 bug 在 Rust 不可能发生。多个值共用代码用多模式: `1 | 2 | 3 => ...`, 或区间 `1..=3 => ...`。注意: 分支内不能 "往下掉", 但模式本身可以罗列多个情况。

Q 3. 写一个 `loop` 表达式: 从 10 开始递减, 找到第一个能被 7 整除的数, 并把它作为 `loop` 的值。

✅ 参考答案

`let mut n = 10; let ans = loop { if n % 7 == 0 { break n; } n -= 1; };` 结果 7。要点: `break` 带值; `n` 是 `mut`; 若 `n` 为负会一直减——生产里应加边界条件 (这也说明 `break` 带值让 "循环求值" 成为一等公民)。

Q 4. C 的 `do-while` 循环如何用 Rust 改写? 给出 "至少执行一次" 的完整示例。

✅ 参考答案

`loop { 循环体; if !条件 { break; } }`。例如: `let mut t = 0; loop { t += 1; if t >= 3 { break; } }`。与 `while` 的区别: `while` 可能一次都不执行, 此写法保证至少执行一次循环体。

Q 5. 为什么 Rust 的 `for` 循环不存在 Go 1.22 之前的 "循环变量捕获" bug?

✅ 参考答案

Go 1.22 前, `for` 循环变量是循环外声明的一个变量, 每轮复用, 闭包捕获的是同一个地址, 循环后全部指向末值。Rust 的 `for i in 0..n` 每轮迭代创建全新的绑定, 闭包各自捕获独立的 `i`。语言设计层面消灭, 而非靠程序员小心。

Q 6. 如何在 Rust 里 "遍历数组的同时修改它"? 直接 `for x in &v` 里 `push` 会怎样?

✅ 参考答案

直接 `push` 编译错误: `for x in &v` 是共享借用, `push` 需要可变借用, 借用冲突 (第 5 章细讲)。正确做法: 先计算增量 (`let extra: Vec<_> = v.iter().map(...).collect();`) 再 `v.extend(extra)`。C++ 的迭代器失效在 Rust 中由编译器拦截。

章末实验题:文本菜单 + 素数筛选 + 数组统计

实验 3:菜单驱动的控制台工具集

实验目标:实现一个菜单驱动的控制台程序,包含素数筛选、数组统计、二维查找三个工具,强制使用 match 菜单、三种循环、break/continue 与带标签跳转。

实验要求:

- 任务 1:主循环用 loop + match 实现菜单分发,无效输入用 continue 回到菜单,退出用 break(覆盖:loop 无限循环、match 穷尽与 _ 通配、continue/break)
- 任务 2:is_prime 用 while 循环 + 提前返回实现(覆盖:while、goto 替代——提前返回)
- 任务 3:sieve 用 for i in 2..=n 范围遍历收集素数(覆盖:for + 范围、Vec 积累)
- 任务 4:数组统计用 for 遍历 + if 分支,并用 if 表达式生成结论字符串(覆盖:for 迭代器遍历、if 表达式)
- 任务 5:用 while let 消费一个栈直到空,并统计弹出值的和(覆盖:while let 语法糖)
- 任务 6:二维查找用嵌套 for + 带标签 break 'outer(覆盖:标签 break 跳出多层)
- 任务 7:输入解析用 if let/let-else 风格处理失败(覆盖:if let、let-else)

实验环境:cargo new menu_tool,代码写入 src/main.rs;用 cargo run 交互测试,再准备一批非数字输入验证健壮性。

第一步 **一步:**写 read_u32 辅助函数(读一行、trim、parse),失败返回 Option;主菜单里用 let-else 风格处理

第二步 **二步:**先实现纯函数 is_prime 与 sieve,不碰任何 I/O——纯函数最好测,也最容易验证正确性

第三步 **三步:**实现任务 4 的统计,输出时用 {:>4} 对齐素数表格

第四步 **四步:**任务 6 的关键:给外层 for 加 'outer: 标签,内层 break 'outer 才生效

第五步 **五步:**把每个工具放在 match 的独立分支里,分支过长时提取成函数(生产习惯)

参考答案要点


```

// ===== 实验 3:菜单驱动的控制台工具集 =====
// 覆盖知识点:loop 无限循环与菜单(§3.5)、match 穷尽与 _ 通配(§3.3)、
//          if 表达式(§3.2)、if let / while let / let-else(§3.4)、
//          while 与提前返回(§3.7)、for + 范围遍历(§3.6)、
//          break 带值 / continue / 带标签 break 'outer'(§3.5)

use std::io::{self, Write};

/// 素数判断:while + 提前返回(C 的 goto 检查模式的现代替代)
fn is_prime(n: u32) -> bool {
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;           // 偶数直接排除,提前返回
    }
    let mut i = 3;
    while i * i <= n {
        if n % i == 0 {
            return false;       // 提前返回:不需要跳转清理
        }
        i += 2;
    }
    true
}

/// 收集 [2, limit] 内所有素数:for + 范围遍历
fn sieve(limit: u32) -> Vec<u32> {
    let mut primes = Vec::new();
    for n in 2..=limit {
        if is_prime(n) {
            primes.push(n);
        }
    }
    primes
}

/// 从 stdin 读一行并解析为 u32,失败返回 None
fn read_u32() -> Option<u32> {
    let mut line = String::new();
    io::stdin().read_line(&mut line).ok()?; // ? 在 Option 上同样可用
    line.trim().parse().ok()
}

fn main() {
    // 主菜单:loop + match 驱动(无限循环的标准写法)
    loop {
        println!();
        println!("===== 工具菜单 =====");
        println!("(1) 素数筛选(2..=N)");
        println!("(2) 数组统计");
        println!("(3) 二维查找(标签 break)");
    }
}

```

```

println!("4) 退出");
print!("请选择:");
let _ = io::stdout().flush();

// let-else:输入无效则回菜单(覆盖:let-else 提前退出)
let Some(choice) = read_u32() else {
    println!("输入无效,请重新输入");
    continue;           // 回到菜单
};

match choice {
    1 => {
        println!("请输入上限 N:");
        let _ = io::stdout().flush();
        if let Some(n) = read_u32() { // if let:只有成功才继续
            let primes = sieve(n);
            println!("2 到 {n} 之间有 {} 个素数:", primes.len());
            for (idx, p) in primes.iter().enumerate() {
                print!("{p:>4}");
                if (idx + 1) % 10 == 0 {
                    println();           // 每行 10 个
                }
            }
            println();
        } else {
            println!("请输入整数");
        }
    }
    2 => {
        let data = [12, 5, 8, 99, 3, 42, 7];
        let mut max = data[0];
        let mut sum: i32 = 0;
        let mut evens = 0;
        for &x in &data { // 只读迭代器遍历
            if x > max { max = x; } // if 语句形式
            sum += x;
            if x % 2 == 0 { evens += 1; }
        }
        // while let:弹出直到栈空
        let mut stack = vec![1, 2, 3, 4];
        let mut popped_sum = 0;
        while let Some(v) = stack.pop() {
            popped_sum += v;
        }
        // if 表达式:生成结论字符串
        let verdict = if max > 50 { "最大值超过 50" } else { "最大值未超过 50" };
        println!("最大值 {max}, 总和 {sum}, 偶数个数 {evens}, 弹出和 {popped_sum}");
        println!("{verdict}");
    }
    3 => {
        // 带标签 break:一次性跳出两层循环(goto 的安全替代)
        let target = 7;
        let mut found: Option<(u32, u32)> = None;
        'outer: for row in 0..4 {
            for col in 0..4 {

```

```

        let v = row * 4 + col;
        if v == target {
            found = Some((row, col));
            break 'outer;      // 跳出外层 for
        }
    }
}

match found {
    Some((r, c)) => println!("找到 {target}:第 {r} 行第 {c} 列"),
    None => println!("未找到 {target}"),
}

}

4 => {
    println!("再见!");
    break;                      // 退出主菜单循环
}

_ => {
    println!("无效选项,请重试");
    continue;                  // 明确回到菜单
}

}

}

// break 带值演示:loop 直接算出一个结果
let mut n = 1;
let first = loop {
    n += 1;
    if n * n > 1000 {
        break n;
    }
};
println!("第一个平方大于 1000 的数是 {first}");
}

```

设计说明:菜单循环刻意把每种流程控制都放进真实场景——match 管分发(穷尽性保证任何选项都有去处)、if let 管输入解析(失败自然回退)、while let 管栈消费、'outer' 管多层跳出、break 带值管"循环求值"。把任务 1 的分支改成一个没有 _ 的 match 试试:编译器立刻报非穷尽——这就是本章学的核心价值。

下一章预告:第4章 函数与方法——C 的函数指针时代如何被 Rust 的函数、方法、闭包全面取代;参数传递的"所有权移动"与"借用"二分法;Fn/FnMut/FnOnce 三态闭包;以及 Result 错误返回约定。准备好了吗?第5章的钥匙就藏在参数传递里。

第4章 函数与方法

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握函数定义语法:fn 关键字、参数、返回类型、表达式与语句的区分
- 深入理解 参数传递的所有权模型:传值拷贝 vs 所有权移动 vs 借用——C 的"传值 + 指针模拟引用"如何被 Rust 的显式语义取代
- 掌握多返回值(元组)与解构接收;了解返回引用的限制(第5章展开生命周期)
- 掌握方法:impl 块、self/&self/&mut self 三态、关联函数(构造函数惯例)
- 掌握闭包语法与 捕获三态 Fn / FnMut / FnOnce,理解 move 关键字,体会闭包如何超越 C 的函数指针
- 掌握高阶函数:map/filter/fold、函数作为参数与返回值
- 理解 Rust 没有重载与默认参数,掌握生产替代方案(泛型、配置结构体、不同函数名)
- 了解递归与栈溢出、main 的形态(env::args 替代 argc/argv)、Result 错误返回约定

💡 从 C 到 Rust:本章主题如何迁移

C 的函数只有一种"传参哲学":值传递。想修改外部,就传指针;想传大结构体,也传指针。于是每个函数签名都要靠程序员心算"这个指针会不会被改、生命周期到哪"——信息全在注释和约定里,编译器一概不管。Rust 把参数传递分成三种显式语义:按值移动(move)、不可变借用(&T)、可变借用(&mut T),写进签名,由编译器强制执行。C 程序员最需要改的习惯是:"传指针"在 Rust 里要明确回答——你要借来看,还是要借去改,还是要整个拿走?

C 的函数指针是二等公民:能存、能传,但捕获不了上下文(回调需要额外的 void* 参数手工传递状态)。Rust 的闭包把"函数 + 捕获的环境"打包成一个整体,而且捕获方式受控(三态 Fn/FnMut/FnOnce)。你在 C 里见过的 `qsort(data, n, sizeof(int), cmp)` 式回调,在 Rust 里变成 `data.sort_by(|a, b| a.cmp(b))` ——更短,且不依赖任何全局状态。

Rust 没有重载、没有默认参数、没有可变参数(printf 式)。初看像倒退,实际是把"同名不同义"的暧昧从语言里赶走:重载造成的隐式转换与歧义(C++ 里 `f(1)` 该调哪个 `f`?)在 Rust 中不存在,替代品是泛型(第10章)、配置结构体和 Builder。函数签名于是成为精确契约:看到 `fn f(x: &str) -> Result<i32, Err>`,你就知道它借用一个字符串、可能失败——这比 C 里任何注释都可靠。

最后是"入口与出口":Rust 的 main 没有 argc/argv(用 `env::args()` 获取),函数没有 `errno`(错误通过返回值 `Result` 显式传播,第9章详解)。整章的主线可以概括为一句话:函数的一切约定,都从"注释里的口头协议"变成"签名里的类型契约"。

📦 前置知识自查

- C 的函数定义、值传递、指针参数、返回语句
- C 的函数指针:typedef、回调(如 `qsort` 的 `cmp`)
- C 的 `argc/argv`、`errno` 错误返回约定
- C 的 `struct` 与"传结构体指针"惯用法
- 第1-3章:所有权伏笔、Option、match、迭代器基本遍历

本章学习路线

1. **第一阶段(约1小时):**§4.1–§4.3,吃透参数传递三态——本章最核心的概念
2. **第二阶段(约1.5小时):**§4.4–§4.6,方法与闭包;闭包三态多写几遍直到形成直觉
3. **第三阶段(约1.5小时):**§4.7–§4.8 与章末实验,把高阶函数用进真实管线
4. **验收标准:**能说出"传 &T、传 &mut T、传 T"各适用什么场景;能写出 Fn/FnMut/FnOnce 三个闭包示例

本章知识导图

- §4.1 函数定义与调用:fn、参数、返回、表达式风格
- §4.2 参数传递:所有权移动 vs 借用(全对照表)
- §4.3 返回值:多返回值、引用返回与生命周期预告
- §4.4 方法:impl、self 三态、关联函数
- §4.5 闭包:语法、捕获三态 Fn/FnMut/FnOnce、move
- §4.6 高阶函数:函数指针、map/filter/fold、回调
- §4.7 无重载无默认参数:替代方案;递归与栈溢出
- §4.8 入口与错误返回:main、env::args、Result 约定
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§4.1 函数定义与调用

4.1.1 基本形态

函数以 `fn` 开头,参数必须注解类型,返回类型用 `->` 声明(没有返回值时省略)。函数体是表达式风格:最后一个不带分号的表达式就是返回值,等价于 C 的 `return`:

```
// C: int add(int a, int b) { return a + b; }
fn add(a: i32, b: i32) -> i32 {
    a + b                // 无分号:这是函数的返回值
}

// 无返回值:C 的 void;实际返回单元类型 ()
fn greet(name: &str) {
    println!("你好,{name}");
    // 没有 return 语句
}

// 显式 return 也合法:提前返回时用
fn abs(n: i32) -> i32 {
    if n < 0 {
        return -n;        // 提前返回
    }
    n
}

fn main() {
    println!("{}", add(3, 4));    // 7
    greet("张三");
    println!("{}", abs(-5));      // 5
}
```

4.1.2 语句与表达式:函数体是"表达式串"

每个函数体可以看作一列"表达式 + 分号":带分号的是语句(执行副作用,值被丢弃),最后一个不带分号的表达式是函数的值。C 程序员最常见的编译错误 "expected `)` , found `i32`" 或相反,根源都是"最后一行该不该加分号"——检查顺序:函数声明了 `-> i32`,最后一行就必须是 `i32` 表达式,不能加分号。

示例 4-1 三种返回值写法对照

```
fn f1() -> i32 {
    return 42;                // C 风格:显式 return
}

fn f2() -> i32 {
    42                        // Rust 风格:尾表达式
}

fn f3() -> i32 {
    let x = 40;
    x + 2                    // 尾表达式可以是任意表达式
}

fn main() {
    println!("{}", f1(), f2(), f3());
}
```

§4.2 参数传递:移动与借用(本章核心)

4.2.1 C 的"值 + 指针"二分 vs Rust 的"移动 + 借用"三分

C 的规则很简单:一切按值拷贝;想影响外部就传指针。但"传指针"到底是只读还是可写,靠的是注释和程序员自觉。Rust 把参数传递显式分成三种,写进类型签名:

C 的写法	Rust 的写法	语义
void f(int x)	fn f(x: i32)	传值:内部修改不影响外部(C 同理)
void f(const int *p)	fn f(x: &i32)	不可变借用:只看不改(C 靠 const 约定)
void f(int *p)	fn f(x: &mut i32)	可变借用:可修改外部(编译器强制独占)
void f(Item *p)(大结构体)	fn f(x: &Item)	只读借用,不拷贝(替代 C 的传指针省拷贝)
(无对应)	fn f(x: Item)	所有权移动:数据被搬进函数,调用后原变量失效

4.2.2 移动语义:值被"搬走"而不是"拷贝"

第5章会系统讲所有权,这里先看行为:对 String、Vec 等"拥有堆数据"的类型,按值传参意味着所有权从调用方转移到函数内部;调用后原变量不再可用(编译器报 moved 错误)。对 i32 等 Copy 类型,传参就是拷贝,原变量照常可用:

```
fn take(s: String) {
    println!("收到:{s}");
} // 离开作用域,String 被自动释放(第5章的 drop)

fn main() {
    let name = String::from("张三");
    take(name);                // 所有权被搬进 take
    // println!("{name}");     // 编译错误:value borrowed after move
    let n = 5;
    add_and_print(n);          // i32 是 Copy:只是拷贝
    println!("{n}");           // 5,照常可用
}

fn add_and_print(x: i32) {
    println!("{}", x + 1);
}
```

4.2.3 借用参数:&T 与 &mut T

当函数只需要"看"数据时,传 &T (不可变借用)——不拷贝大对象,也不搬走所有权;需要"改"外部数据时,传 &mut T。C 里 f(&x) 之后 x 的命运全靠约定,Rust 里签名直接写死:

```
// 只读借用:可以传数组、Vec、String 的任何"视图"
fn total(v: &[i32]) -> i32 {
    let mut s = 0;
    for &x in v {
        s += x;
    }
    s
}

// 可变借用:直接修改外部数据,不需要 C 的解引用链
fn bump(x: &mut i32, by: i32) {
    *x += by;           // *x 解引用,与 C 的 *p 相同
}

fn main() {
    let arr = [1, 2, 3, 4];
    println!("{}", total(&arr));           // 10:传 &arr 借出

    let mut score = 80;
    bump(&mut score, 5);                   // 修改外部变量
    println!("{}", score);                 // 85
}
```

⚠ 易错点 1:传参时漏写 &,或把 &mut 当 & 用

三种报错天天见:① `fn f(x: &i32)` 却写 `f(x)` ——编译器提示 "expected &i32, found i32",补上 `&x`;② 函数签名是 `&mut i32`,调用处必须 `&mut x`,且调用前 `x` 必须声明为 `mut`;③ 把可变借用的值再传给只读函数没问题(可以同时借用),但反过来——函数内先拿了 `&mut` 又想读——要按借用规则排队(第5章详讲)。先记住一句话:函数要"看"就传 `&`,要"改"就传 `&mut`,要"拿走"就传值。

§4.3 返回值

4.3.1 多返回值:C 的"输出参数"被元组取代

C 里函数要返回多个结果,要么用输出参数 `int div(int a, int b, int *q, int *r)`,要么定义一个临时结构体。Rust 用元组直接返回,调用处解构:

```
// 返回 (商, 余数):元组类型写进签名
fn div_mod(a: i32, b: i32) -> (i32, i32) {
    (a / b, a % b)
}

fn main() {
    let (q, r) = div_mod(17, 5);           // 解构接收:q=3, r=2
    println!("商 {q},余数 {r}");

    // 也可以整体存下再取下标
    let result = div_mod(20, 3);
    println!("{}", ... {}, result.0, result.1);
}
```


4.3.2 返回引用:生命周期问题初现

函数可以返回借用,但编译器要能证明"返回的引用不会悬垂"。这个证明就是生命周期标注(第5章详讲),这里先看两个形态:返回参数里的借用(合法),返回局部变量的借用(编译错误):

```
// 合法:返回的引用来自参数(编译器能证明它活得比函数长)
fn first_word(text: &str) -> &str {
    match text.find(' ') {
        Some(pos) => &text[..pos],
        None => text,
    }
}

fn main() {
    let s = String::from("hello world");
    let w = first_word(&s);
    println!("第一个单词:{w}");

    // 编译错误:返回局部变量的引用(悬垂引用!)
    // fn bad() -> &String {
    //     let s = String::from("x");
    //     &s    // s 在函数结束时被释放,引用悬空
    // }
}
```

§4.4 方法:impl 块

4.4.1 方法与 self 三态

C 把"对结构体的操作"写成 `rect_area(&r)` 之类的自由函数;Rust 允许把函数挂在类型上成为**方法**,第一个参数是 `self`。self 有三种形态: `self`(拿走所有权)、`&self`(只读)、`&mut self`(可变) ——与参数传递三态完全一致:

```

struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
    // &self:只读方法(C 的 const 指针版本)
    fn area(&self) -> f64 {
        self.width * self.height
    }

    // &mut self:修改自身
    fn scale(&mut self, factor: f64) {
        self.width *= factor;
        self.height *= factor;
    }

    // self:消费自身(拿走所有权)
    fn into_square(self) -> Rectangle {
        Rectangle { width: self.width, height: self.width }
    }
}

fn main() {
    let mut r = Rectangle { width: 3.0, height: 4.0 };
    println!("面积:{}", r.area());           // 12(调用后 r 仍可用)
    r.scale(2.0);                             // 修改 r
    println!("放大后:{}", r.area());         // 48
    let sq = r.into_square();                  // r 被消费
    // println!("{}", r.area());              // 编译错误:r 已 move
    println!("正方形:{}", sq.area());        // 64
}

```

4.4.2 关联函数:没有 self 的"静态方法"

不接收 self 的函数挂在 impl 块里,称为**关联函数**,用 `::` 调用;惯例上以 `new` 命名的关联函数充当构造函数——这替代了 C 里"手写 init 函数 + 调用者负责初始化"的脆弱模式:

```

struct Point {
    x: f64,
    y: f64,
}

impl Point {
    // 关联函数(构造函数惯例):Point::new(...)
    fn new(x: f64, y: f64) -> Point {
        Point { x, y }
    }

    fn origin() -> Point {
        Point { x: 0.0, y: 0.0 }
    }

    fn dist(&self, other: &Point) -> f64 {
        let dx = self.x - other.x;
        let dy = self.y - other.y;
        (dx * dx + dy * dy).sqrt()
    }
}

fn main() {
    let a = Point::new(1.0, 1.0);           // 关联函数:类型名::函数名
    let b = Point::origin();
    println!("距离:{}", a.dist(&b));
}

```

工程建议:方法 vs 自由函数的选择

- 操作"明显属于"这个类型(读字段、改字段):方法。C 的 `strlen(s)` 在 Rust 里是 `s.len()`
- 两个类型协同的操作(如 `point.dist(&other)`):方法更自然
- 与类型关系不强的纯计算:自由函数,避免"万物皆方法"的 Java 式强迫症
- 构造函数一律叫 `new`,是社区惯例;需要"解析/尝试构造"时用 `from_str` / `try_from` (第1章见过)

§4.5 闭包:捕获环境的函数

4.5.1 语法:参数与体

```

fn main() {
    // C 函数指针:int (*f)(int);Rust 闭包:
    let double = |x| x * 2;           // 参数类型可推导(常用)
    let greet = |name: &str| { println!("你好,{name}") }; // 多语句用 {} 包裹
    println!("{}", double(4));        // 8
    greet("李四");
}

```

闭包的类型无法显式书写(编译器自动推断),这反而是优点:你只需要写参数与体。闭包与 C 函数指针的本质区别:闭包能捕获定义处的变量(上下文),函数指针不能。C 要模拟捕获,只能靠额外的 `void*` 参数。

4.5.2 捕获三态:Fn / FnMut / FnOnce(必会)

闭包按"如何使用捕获的变量"分成三类,对应三个 trait(第6章详解 trait;这里记住行为和名字):

trait	捕获方式	能调用几次	例子
Fn	只读借用(&T)	任意多次	x x + base(只读 base)
FnMut	可变借用(&mut T)	任意多次	x { total += x; }(改 total)
FnOnce	拿走所有权(move)	一次	move drop(owner)

```
fn main() {
    // Fn:只"看"不"动"
    let base = 10;
    let add = |x| x + base;           // 捕获 &base
    println!("{}", add(1), add(2)); // 可反复调用

    // FnMut:闭包内部修改捕获的变量(计数器模式)
    let mut total = 0;
    let mut accumulate = |x: i32| {
        total += x;                 // 捕获 &mut total
    };
    accumulate(5);
    accumulate(7);
    println!("累计:{total}");       // 12(借用已结束)

    // FnOnce:把值搬进闭包,之后外部不可再用
    let owner = String::from("机密数据");
    let consume = move || println!("{}", owner); // move:彻底搬走
    consume();
    // consume();           // 编译错误:FnOnce 只能调用一次
    // println!("{}", owner); // 编译错误:已被搬走

    // 判断口诀:用到值就得 move(FnOnce);改值就是 FnMut;只看就是 Fn
}
```

4.5.3 move 关键字:主动搬入

闭包默认"按需借用"捕获;用 `move` 强制把所有捕获值搬进闭包。最典型的场景是 把数据交给新线程(第9章)——线程可能活得比函数长,借用必然悬垂,必须 `move`:

```
fn main() {
    let data = vec![1, 2, 3];
    // 不 move:闭包借用了 data,离开作用域后 data 被释放,闭包悬垂
    // let bad = || println!("{}", data);

    // move:data 的所有权进入闭包,闭包生命周期自持
    let owned = move || println!("{}", data);
    owned();           // [1, 2, 3]
    // println!("{}", data); // 编译错误:已 move 进闭包
}
```

⚠ 易错点 2: 闭包调用次数与捕获方式的错配

经典报错:① "cannot move out of `owner`, a captured variable in an `Fn` closure"——闭包没写 move 却要消费值,给闭包加 move 即可;② "cannot borrow `total` as mutable, as it is not declared as mutable"——闭包要改捕获变量,闭包本身也要声明 mut;③ 把一个 FnOnce 闭包存进 Vec 想循环调用——编译器告诉你只能调一次,要么换 FnMut,要么重新设计。三态的区分不是考试题,是编译器每天都在执行的检查:它知道你的闭包会怎么用捕获的变量。

§4.6 高阶函数:函数作为参数与返回值

4.6.1 函数指针类型 fn 与回调

函数名本身就是值,类型写作 `fn(i32) -> i32` (与 C 的函数指针——对应,但写法不带星号)。接受函数参数的函数叫高阶函数:

```
// 接受"处理函数"作参数:C 的 void apply(int *a, int n, int (*f)(int))
fn apply_each(data: &[i32], f: fn(i32) -> i32) -> Vec<i32> {
    let mut out = Vec::new();
    for &x in data {
        out.push(f(x));
    }
    out
}

fn triple(x: i32) -> i32 {
    x * 3
}

fn main() {
    let nums = [1, 2, 3, 4];
    let tripled = apply_each(&nums, triple); // 传函数名
    println!("{tripled:?}");                // [3, 6, 9, 12]

    // 函数指针也是值:可以存、可以传
    let f: fn(i32) -> i32 = triple;
    println!("{}", f(5));                    // 15
}
```

4.6.2 迭代器方法链:map / filter / fold

标准库把"遍历 + 变换 + 归约"做成了迭代器方法,配合闭包使用——这是 C 里必须手写循环三件套(map 之于 for 循环,filter 之于 if 判断,fold 之于累加):

```
fn main() {
    let data = [3, 5, 8, 12, 15, 20];

    // map:每个元素变换(等价 C 的"循环里 push 新值")
    let doubled: Vec<i32> = data.iter().map(|x| x * 2).collect();
    println!("{doubled:?}");

    // filter:按条件筛选(等价 C 的"循环里 if 判断再 push")
    let evens: Vec<i32> = data.iter().filter(|&&x| x % 2 == 0).collect();
    println!("{evens:?}");

    // fold:归约成单个值(等价 C 的累加器)
    let sum: i32 = data.iter().fold(0, |acc, &x| acc + x);
    println!("{sum}");

    // 链式组合:筛选 → 变换 → 求和,一次遍历
    let result: i32 = data.iter()
        .filter(|&&x| x > 5)
        .map(|x| x * x)
        .fold(0, |acc, x| acc + x);
    println!("{result}");    // 8^2+12^2+15^2+20^2 = 713
}
```

注意 `filter` 的闭包参数是 `&&i32` (迭代器产生引用,filter 又传引用),写法 `|&&x|` 直接解构——这是 Rust 迭代器中最常见的一个"仪式感",见多了自然记住。

§4.7 没有重载与默认参数:替代方案

4.7.1 重载的替代

C 没有重载(靠不同函数名),C++ 有静态重载。Rust 没有重载,但有更系统的替代:① 泛型(第10章)——`fn max<T: PartialOrd>(a: T, b: T) -> T` 一个函数服务所有类型;② 枚举参数——把"同操作不同细节"收进一个枚举;③ 不同函数名——`from_bytes`、`from_str` 明确区分。结论:重载带来的"同名歧义"被消灭,代价是函数名略长。

4.7.2 默认参数的替代:配置结构体

```
// C 的典型场景:draw(color, width, style) 想省略参数—用宏或重载;
// Rust 的做法:把可选项收进配置结构体,配合 Default 与更新语法
struct DrawOpts {
    color: u32,
    width: u32,
    style: &'static str,
}

impl DrawOpts {
    fn default() -> Self {
        DrawOpts { color: 0x000000, width: 1, style: "solid" }
    }
}

fn draw(opts: &DrawOpts) {
    println!("画 {},颜色 0x{:06X},{}px", opts.style, opts.color, opts.width);
}

fn main() {
    let defaults = DrawOpts::default();          // 全默认
    draw(&defaults);

    // 只改一个参数:..defaults 更新语法(第14章详解)
    let red = DrawOpts { color: 0xFF0000, ..defaults };
    draw(&red);
}
```

4.7.3 递归与栈溢出

```
fn factorial(n: u64) -> u64 {
    if n <= 1 {
        return 1;
    }
    n * factorial(n - 1)
}

fn main() {
    println!("{}", factorial(5));    // 120

    // 深递归会栈溢出:Rust 与 C 一样使用系统栈(默认 8MB),
    // 且不保证尾递归优化(TCO)—深递归请改写为循环
    let mut result: u64 = 1;
    let mut i = 20;
    while i > 1 {
        result *= i;
        i -= 1;
    }
    println!("{}", result);
}
```

§4.8 入口与错误返回约定

4.8.1 main 的形态与命令行参数

```
use std::env;

// C 的 int main(int argc, char *argv[]) → Rust 的 fn main()
fn main() {
    // argc/argv 由 env::args() 提供:返回迭代器
    let args: Vec<String> = env::args().collect();
    println!("程序名:{}", args[0]);
    if args.len() > 1 {
        for (i, a) in args.iter().enumerate().skip(1) {
            println!("参数 {i}:{a}");
        }
    } else {
        println!("没有参数;用法:程序名 <参数1> ...");
    }
}
```

4.8.2 错误返回约定:Result 取代 errno

C 的错误返回约定靠 `errno` + 返回码,调用链上每层都要"检查、传播、清理",漏一层就静默失败。Rust 的约定是 **函数签名里直接声明可能失败的返回值类型 `Result<T, E>`** (第9章系统讲):

```
// C: int divide(int a, int b, int *out); 返回 errno,成功与否查全局
// Rust:失败与否写在类型里,调用方必须处理
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        return Err("除数为零".to_string());
    }
    Ok(a / b)
}

fn main() {
    match divide(10.0, 2.0) {
        Ok(v) => println!("结果:{v}"),
        Err(e) => println!("错误:{e}"),
    }
    match divide(1.0, 0.0) {
        Ok(v) => println!("结果:{v}"),
        Err(e) => println!("错误:{e}"),
    }
}
```

★ 重点:本章贯穿线索——签名即契约

回顾全章:参数是 `&T` 还是 `&mut T` 还是 `T`,方法取 `&self` 还是 `self`,闭包是 `Fn` 还是 `FnOnce`,返回值是 `i32` 还是 `Result<i32, E>` —— 每个决定都写在签名里,编译器逐条执行。C 程序里那些"看注释才知道的约定",在 Rust 里都是类型。写函数时先想清楚签名,再写体;这是 Rust 生产开发的头号习惯。

⚠ 易错点 3:需要"函数做参数"时,优先闭包而不是 fn 指针

fn 指针类型 `fn(i32) -> i32` 不能捕获环境(它只是裸函数地址);闭包能捕获。生产里"回调参数"的签名优先写成泛型 `F: Fn(...)` 或 `&impl Fn(...)`,除非明确不需要捕获。对照 C:函数指针是唯一选项,所以 C 的回调都要靠"上下文指针"参数传状态;Rust 的闭包把"上下文"打包进闭包自身,少一个容易接错的 `void*`。

本章速查表

主题	结论 / 写法
函数定义	<code>fn name(a: i32, b: i32) -> i32 { ... }</code> ;无返回值省略 <code>-></code>
返回值	最后一个不带分号的表达式; <code>return</code> 用于提前返回
传值	<code>fn f(x: i32):Copy</code> 类型拷贝; <code>String/Vec</code> 等是所有权移动
不可变借用	<code>fn f(x: &T)</code> :只看不改;不拷贝大对象
可变借用	<code>fn f(x: &mut T)</code> :可修改外部;调用处写 <code>&mut x</code>
多返回值	<code>fn f() -> (i32, i32)</code> ;调用处 <code>let (a, b) = f();</code>
返回引用	只能返回参数/外部借来的引用;返回局部引用是编译错误
方法	<code>impl T { fn m(&self) {} }</code> ;三态 <code>self / &self / &mut self</code>
关联函数	<code>impl T { fn new() -> T {...} }</code> ;调用 <code>T::new()</code>
闭包	<code> x x + 1</code> ;可捕获上下文;参数类型多可推导
Fn	只读捕获,可多次调用
FnMut	可变捕获(闭包要声明 <code>mut</code>),可多次调用
FnOnce	<code>move</code> 捕获,只能调用一次
高阶函数	<code>fn</code> 类型可作参数; <code>map/filter/fold</code> 方法链
重载/默认参数	无!替代:泛型、配置结构体 + <code>Default</code> 、不同函数名
递归	支持,但无 <code>TCO</code> 保证;深递归用循环改写
main / 参数	<code>fn main();env::args()</code> 替代 <code>argc/argv</code>
错误返回	<code>Result<T, E></code> 写进签名; <code>match</code> 或 <code>?</code> 处理(第9章)

最小必会清单

- 能写出三种参数形态(传值/&/&mut)并说出各自语义与适用场景
- 能解释"String 传参后原变量失效"并说出原因(所有权移动)
- 能用元组返回多个值并在调用处解构
- 能写一个 `struct + impl`,包含 `&self` 方法、`&mut self` 方法、关联函数 `new`
- 能写出 `Fn/FnMut/FnOnce` 各一个闭包示例,并解释捕获方式差异
- 能说出 `move` 关键字的作用与"交给线程"场景
- 能用 `map/filter/fold` 组合完成一次数据管线处理

- 能说出 Rust 无重载/无默认参数时三种替代方案
- 能用 `env::args()` 读取命令行参数
- 能写出返回 `Result` 的函数并用 `match` 处理

自测题

Q 1. `fn f(s: String) { ... }` 与 `fn g(s: &String) { ... }` 调用后,原变量还能用吗?为什么?

✓ 参考答案

`f` 按值传参:`String` 的所有权被搬进函数,调用后原变量失效(编译错误 `moved`)。 `g` 只借用,`&String` 不转移所有权,原变量照常使用。选择依据:函数是否需要拿走数据;只需查看时永远优先 `&`。

Q 2. C 的 `void swap(int *a, int *b)` 用 Rust 怎么写?与 C 相比少做了什么?

✓ 参考答案

`fn swap(a: &mut i32, b: &mut i32) { std::mem::swap(a, b); }`。与 C 相比:不需要手动解引用赋值(用标准库 `swap`),调用处必须写 `&mut x`——编译器保证两个可变借用不冲突,杜绝 C 里"同一个变量传两个指针"的别名问题。

Q 3. 下面闭包属于 `Fn`、`FnMut` 还是 `FnOnce`? `let c1 = |x| x * 2;`、`let mut n = 0; let c2 = |x| { n += x; };`、`let s = String::from("a"); let c3 = move || drop(s);`

✓ 参考答案

`c1` 不捕获任何变量(或只读),是 `Fn`; `c2` 修改捕获的 `n`,是 `FnMut`; `c3` 用 `move` 搬走 `s` 并消费,是 `FnOnce`。判断顺序:会不会消费值(会→`FnOnce`)、会不会修改(会→`FnMut`)、否则 `Fn`。

Q 4. 用 `map/filter/fold` 把 `[1, 2, 3, 4, 5, 6]` 中偶数的平方求和(期望 `4+16+36=56`)。

✓ 参考答案

`let sum: i32 = [1,2,3,4,5,6].iter().filter(|&&x| x % 2 == 0).map(|x| x * x).fold(0, |acc, x| acc + x);` 结果 56。注意 `filter` 的闭包形参是 `&&i32`。

Q 5. Rust 没有默认参数,`draw(color, width, style)` 想省略参数时该怎么办?

✓ 参考答案

把可选项收进配置结构体 `DrawOpts`,实现 `Default`,调用处 `DrawOpts { color: 0xFF0000, ..defaults }` 只覆盖需要的字段。相比 C++ 默认参数,类型契约更清晰:配置本身是类型,IDE 与编译器都能参与检查。

Q 6. 为什么 Rust 的闭包能超越 C 的函数指针?C 里如何模拟"捕获"?

✓ 参考答案

函数指针只是地址,不能携带上下文;C 用"回调 + `void*` 参数"模拟,类型安全全靠自觉。Rust 闭包把"代码 + 捕获的环境(按 `Fn/FnMut/FnOnce` 受控)"打包,类型系统管理捕获方式,编译器检查借用与所有权。生产中最直接的好

处:sort_by、线程 spawn、迭代器方法链都无需手工传递上下文。

章末实验题:可配置的数组统计管线

实验 4:可配置的数组统计管线

实验目标:实现一个"配置驱动"的数据统计管线:用配置结构体控制统计规则,用函数 + 闭包 + 高阶函数完成多路统计,覆盖全章函数知识点。

实验要求:

- 任务 1:定义 StatsConfig 配置结构体与 Stats 结果元组(覆盖:无默认参数的替代——配置结构体 §4.7)
- 任务 2:analyze 函数借用输入与配置,一次遍历完成多项统计并返回元组(覆盖:借用参数 &[i32] §4.2、多返回值 §4.3)
- 任务 3:实现带 &self/&mut self 方法的 Counter 计数器类型(覆盖:方法三态 §4.4)
- 任务 4:用闭包实现"偶数计数"(FnMut 捕获可变变量)与"加偏移"(move 捕获)(覆盖:闭包三态 §4.5)
- 任务 5:用 map/filter/fold 组合完成过滤与求和(覆盖:高阶函数 §4.6)
- 任务 6:main 入口读取命令行参数作为数据来源之一,失败给出提示(覆盖:main 与 env::args §4.8、Result 匹配)

实验环境:cargo new stats_pipeline;数据先用固定数组,任务 6 再用 env::args 读取。

第一步 **一步:**先写纯函数 analyze:签名 `fn analyze(data: &[i32], cfg: &StatsConfig) -> Stats`,内部 for + if 过滤

第二步 **二步:**实现 Counter 与它的三个方法;在 main 里用可变方法累加

第三步 **三步:**写 FnMut 闭包(计数)与 move 闭包(偏移),调用并打印

第四步 **四步:**任务 5 用方法链一行完成;与任务 2 的手写循环对比可读性

第五步 **五步:**最后接 env::args:把第 2 个参数解析为 i32 加入数据,失败打印提示

参考答案要点

```
// ===== 实验 4:可配置的数组统计管线 =====
// 覆盖知识点:函数定义与表达式风格($4.1)、借用参数与移动($4.2)、
//           多返回值元组($4.3)、方法三态($4.4)、
//           闭包三态与 move($4.5)、高阶函数 map/filter/fold($4.6)、
//           配置结构体替代默认参数($4.7)、main 与 env::args、Result($4.8)

use std::env;

/// 统计配置:把"默认参数"做成结构体($4.7 的替代方案)
struct StatsConfig {
    title: String,
    min: i32,      // 低于此值忽略
    step: i32,     // 只统计 step 的倍数
}

/// 统计结果:元组即"多返回值"
type Stats = (usize, i32, f64, i32); // (数量, 总和, 平均, 最大)

/// 一次遍历完成所有统计:借用输入与配置,不搬走任何所有权
fn analyze(data: &[i32], cfg: &StatsConfig) -> Stats {
    let mut count: usize = 0;
    let mut sum: i32 = 0;
    let mut max: i32 = i32::MIN;
    for &x in data {
        if x < cfg.min {
            continue;
        }
        if x % cfg.step != 0 {
            continue;
        }
        count += 1;
        sum += x;
        if x > max {
            max = x;
        }
    }
    let avg = if count > 0 { sum as f64 / count as f64 } else { 0.0 };
    (count, sum, avg, max)
}

/// 计数器:方法三态演示(&self 只读、&mut self 修改)
struct Counter {
    total: i32,
    calls: i32,
}

impl Counter {
    /// 关联函数:构造函数惯例
    fn new() -> Self {
        Counter { total: 0, calls: 0 }
    }

    fn add(&mut self, x: i32) {
        self.total += x;
        self.calls += 1;
    }
}
```

```

    }

    fn snapshot(&self) -> (i32, i32) {
        (self.total, self.calls)
    }
}

fn main() {
    // 任务 6 前置:读取命令行参数作为额外数据
    let args: Vec<String> = env::args().collect();
    let extra: Option<i32> = args.get(1).and_then(|s| s.parse().ok());

    let mut data = vec![3, 5, 8, 12, 15, 20, 23, 30, 31, 40, 42];
    match extra {
        Some(v) => data.push(v),
        None => println!("提示:可传入一个整数作为额外数据"),
    }

    // 任务 2:借用配置,返回多值,解构接收
    let cfg = StatsConfig {
        title: String::from("不小于 10 的偶数"),
        min: 10,
        step: 2,
    };
    let (count, sum, avg, max) = analyze(&data, &cfg);
    println!("{count}: 数量 {count}, 总和 {sum}, 平均 {avg:.1}, 最大 {max}", cfg.title);

    // 任务 3:Counter 方法使用
    let mut counter = Counter::new();
    for &x in &data {
        counter.add(x);
    }
    let (total, calls) = counter.snapshot();
    println!("计数器:累计 {total},被调 {calls} 次");

    // 任务 4a:FnMut 闭包—可变捕获计数
    let mut even_count = 0i32;
    let mut tally = |x: i32| {
        if x % 2 == 0 {
            even_count += 1;
        }
    };
    for &x in &data {
        tally(x);
    }
    println!("偶数个数(闭包计数):{even_count}");

    // 任务 4b:move 闭包—把 base 搬进闭包
    let base = 1000;
    let offset = move |x: i32| x + base;
    let shifted: Vec<i32> = data.iter().map(|&x| offset(x)).collect();
    println!("加 {base} 偏移后的前 3 个:{:?}", &shifted[..3]);

    // 任务 5:高阶函数方法链—筛选、变换、归约
    let sum_of_squares: i32 = data.iter()

```

```

        .filter(|&&x| x % 3 == 0)          // 只留 3 的倍数
        .map(|&x| x * x)                  // 平方
        .fold(0, |acc, x| acc + x);      // 求和
println!("3 的倍数的平方和:{sum_of_squares}");

// 函数指针作参数:与 C 的 qsort 回调对照
let tripled: Vec<i32> = data.iter().map(|&x| triple(x)).collect();
println!("翻三倍的前 3 个:{:?}", &tripled[..3]);
}

/// 普通函数:fn 类型,可传给高阶函数
fn triple(x: i32) -> i32 {
    x * 3
}

```

设计说明:实验把"配置化"贯穿始终——统计规则(阈值、步长)是配置而不是硬编码,这正对应生产里"策略可配置"的需求;闭包与高阶函数负责数据流变换,方法与函数各司其职。验证方式:改变 `cfg.min/step` 观察统计变化;不带参数运行一次,再带参数运行一次,确认 `env::args` 路径生效。

下一章预告:第5章 内存管理——全册的灵魂章节。C 的 `malloc/free` 时代有三大痛点:泄漏、悬垂指针、重复释放;Rust 用"所有权三法则 + 借用 + 生命周期"把这三类事故全部变成编译错误。`Box/Rc/Arc/RefCell` 各司其职,`unsafe` 划定安全边界。这一章读透了,后面的每一章都会轻松很多。

第5章 内存管理

建议学时:5-8 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言(本章是全册灵魂)

🎯 本章学习目标

- 回顾 C 的内存模型(栈/堆/静态区)与手动管理的三大痛点:泄漏、悬垂指针、重复释放
- 掌握 **所有权三法则**:每值一所有者、move 转移、作用域结束 drop——并能用"所有者"视角分析任何一段代码
- 分清 move 与 Copy/Clone: `i32` 为什么可以随便拷,String 为什么不行;与 C 的 `memcpy` 对照
- 掌握 **借用规则**:`&T` 可多个、`&mut T` 必须独占;理解编译器如何让 `use-after-free` 从运行期事故变成编译期错误
- 理解生命周期标注: `'a` 是编译期检查,不改变运行时行为;会读会写函数签名里的生命周期
- 掌握智能指针家族: `Box` / `Rc` / `Arc` / `Weak` 各司其职,与 C 的 `malloc/free`、C++ 的 `unique/shared/weak_ptr` 对照
- 掌握内部可变性: `Cell` 与 `RefCell`,理解"共享 + 可变"的合法组合 `Rc<RefCell<T>>`
- 理解 `Drop`(RAII 的 Rust 形态)与 `unsafe` 的边界:什么时候用、安全抽象原则

💡 从 C 到 Rust:本章主题如何迁移

C 程序员对内存管理有最深刻的体会:`malloc` 之后必须记得 `free`,但"记得"靠的是纪律。`free(p)` 之后又用 `p`?悬垂指针,静默出错。`malloc` 忘 `free`?泄漏,内存缓慢耗尽。同一个指针 `free` 两次?double free,崩溃或安全漏洞。这三种事故占 C 内存类 bug 的绝大多数,而编译器只会袖手旁观。Rust 的核心承诺就一句话:让编译器代替你的纪律——把"释放时机"从程序员的记忆里搬进类型系统,从源头消灭这三类事故。

机制上,Rust 没有垃圾回收(GC),它走的路线是"**所有权(ownership)**":每个值有且只有一个所有者,所有者离开作用域,值就被自动释放。听起来简单,但它的力量在于"编译器强制"四个字——你想违反?编译错误。于是释放时机的正确性不再是"运行久了才发现的泄漏",而是"编译期就定案的契约"。这与 C++ 的 RAII 精神一致,但 C++ 还允许你手动 `new/delete` 绕开纪律;Rust 没有"绕开"的通道,除非你主动走进 `unsafe`。

接下来是借用与生命周期。C 的指针只回答一个问题:"地址在哪?"Rust 的引用多回答两个:"能不能改?"与"活多久?"`&T` 表示只读共享,`&mut T` 表示独占可写;`'a` 表示引用的存活范围。于是"free 之后还用"变成"引用活得比数据长"的编译错误,"两个线程同时写"变成"可变借用冲突"的编译错误。第9章你会看到,并发安全几乎免费地继承自这套规则。

最后是工具与边界。堆分配交给 `Box`,共享所有权交给 `Rc/Arc`,可变共享交给 `RefCell`,析构逻辑交给 `Drop`——每个需求一个明确工具,不再有"自己造轮子"的空间。而 `unsafe` 是唯一的后门:FFI、性能临界、底层库用它,但纪律是"**unsafe 必须封装在安全接口里,并写清安全前提**"。本章读完,你将获得一个 C 程序员最珍贵的转变:从"我小心地管内存"到"编译器替我管,我专注业务"。

前置知识自查

- C 的 malloc/free/calloc/realloc 的完整语义;谁负责释放、何时释放
- C 的悬垂指针、use-after-free、double free、内存泄漏的产生机制
- C 的栈上变量生命周期(函数返回即失效)与静态区(程序期存活)
- C 的结构体赋值(位拷贝)与 memcpy
- 第1章的类型与 String/Vec 的所有权伏笔;第4章的参数传递三态

本章学习路线

1. **第一阶段(约2小时,建立心智模型):**\$5.1–\$5.3,把"所有者"视角刻进脑子——本章后面全靠它
2. **第二阶段(约2小时,借用与生命周期):**\$5.4–\$5.5,这是最容易反复编译失败的部分,多写多报错多读报错
3. **第三阶段(约2小时,工具与实战):**\$5.6–\$5.8,重点搞清楚 Rc 与 Arc、RefCell 与 &mut 的边界
4. **验收标准:**给任意一段含指针/引用的代码,能用"所有者 + 借用 + 生命周期"三句话讲清它的内存行为

本章知识导图

- §5.1 C 的内存模型与三大痛点(malloc/free 的代价)
- §5.2 所有权三法则:每值一所有者、move、drop
- §5.3 move 与 Copy/Clone:哪些类型可以"随便拷"
- §5.4 借用规则:&T 共享、&mut T 独占、NLL
- §5.5 生命周期标注:'a、省略规则、'static
- §5.6 智能指针:Box / Rc / Arc / Weak(对照 C++ 三件套)
- §5.7 内部可变性:Cell / RefCell / Rc<RefCell<T>>
- §5.8 Drop 与 unsafe 的边界
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§5.1 C 的内存模型与三大痛点

5.1.1 内存三区回顾

C 程序的内存分三块:**栈**(函数调用帧,自动分配自动回收)、**堆**(malloc/free 手动管理)、**静态区**(全局变量与字符串字面量,程序期存活)。Rust 的内存模型完全相同,没有 GC,没有额外运行时——区别只在于**堆内存的释放时机由谁决定**。对照如下:

C 的做法	Rust 的做法
malloc(size) 返回 void*	Box::new(v)、Vec::new() 等:类型安全,无 void*
free(p) 手动释放	所有者离开作用域,自动 drop
忘记 free → 泄漏	编译器保证每个值必被释放,无泄漏通道
free 后再用 → 悬垂指针(UB)	借用检查:引用活得比数据长 = 编译错误
free 两次 → double free(崩溃)	一个值只有一个所有者,释放且仅释放一次
手动 realloc 搬数据	Vec 按需增长,自动重新分配并搬移

⚠ 易错点 1:以为 Rust 是"带 GC 的语言"或"不用管内存"

Rust 没有 GC:栈上值随作用域弹出,堆上值由所有者决定生死,一切在编译期安排妥当——但"安排"是编译器的职责,不是自动运行的回收线程。写 Rust 仍然要思考生命周期,只是思考的结果以类型的形式固化下来、由编译器验证。正因为如此,Rust 才有接近 C 的性能与可预测的释放时机,这是它敢自称"系统级语言"的底气。

§5.2 所有权三法则

5.2.1 法则一:每个值有且只有一个所有者

所有者(owner)是"持有这个值"的变量:局部变量拥有它绑定的值,结构体拥有它的字段,Vec 拥有它缓冲区里的元素。**任何时候,一个值只有一个所有者**——没有 C 里"两个指针指向同一块内存"的别名共享。这条法则直接消灭了 double free:释放权只属于唯一的所有者,释放且仅释放一次。

5.2.2 法则二:所有者离开作用域,值被自动释放(drop)

```
fn main() {
    {
        let s = String::from("临时数据");
        println!("使用中:{s}");
    } // 作用域结束:s 的所有者消失,堆内存被自动释放
    // println!("{s}"); // 编译错误:value borrowed after move / 超出作用域

    let v = vec![1, 2, 3]; // v 的所有者是 main 的函数体作用域
    println!("{v:?}");
} // 函数结束,v 连同堆缓冲区一起被释放
```

对照 C: `char *s = malloc(...)` 之后,释放指令 `free(s)` 必须由你写在某个地方;Rust 把"释放"绑定在作用域结束时自动发生——你根本写不出"忘了 free"的代码,因为 free 不是可选项,是作用域的固定收尾动作。

5.2.3 法则三:所有权可以转移(move)

把值赋给另一个变量、或作为参数传进函数,所有权就发生转移——原变量失效。这就是第4章"传 String 后原变量不可用"的原因:

```
fn take_ownership(s: String) {
    println!("现在由我持有:{s}");
} // s 在这里被释放

fn main() {
    let a = String::from("数据甲");
    let b = a;                // move:a 的所有权转移到 b
    // println!("{a}");        // 编译错误:a 已失效(use of moved value)
    println!("{b}");

    let c = String::from("数据乙");
    take_ownership(c);        // move:所有权进入函数
    // println!("{c}");        // 编译错误:c 已失效
}
```

★ 重点:三法则的"内存故事"

把三法则连起来读一遍: 一个值, 一个主人; 主人走, 值就亡; 想易主, 必须搬。这套规则下, 编译器可以静态地算出每一块内存的确切生死时刻: 不会泄漏(每个值都有人释放)、不会重复释放(释放者唯一)、不会悬垂(引用规则见 §5.4)。对比 C 的三个事故, 全部在编译期被排除——这就是"安全"的运作方式。

§5.3 move 与 Copy / Clone

5.3.1 Copy: 位拷贝后, 原值仍可用

为什么 `let b = a;` 对 `i32` 不报错、对 `String` 报错? 因为 `i32` 实现了 **Copy** 标记: 赋值时做一次位拷贝, 原值仍然有效。这对应 C 的结构体赋值/按值传参——但 C 里所有类型都是"位拷贝"的, 而 Rust 只有显式声明 `Copy` 的类型才这样。内置 `Copy` 类型:

Copy 类型(赋值=拷贝)	非 Copy 类型(赋值=move)
所有整数: <code>i8-i128</code> 、 <code>u8-u128</code> 、 <code>isize/usize</code>	<code>String</code> (拥有堆缓冲区)
浮点 <code>f32/f64</code> 、 <code>bool</code> 、 <code>char</code>	<code>Vec<T></code> (拥有堆缓冲区)
定长数组 <code>[i32; 5]</code> 、元组(元素全 <code>Copy</code>)	<code>Box<T></code> (拥有堆上对象)
<code>&T</code> 引用(复制的是地址)	<code>&mut T</code> (不可 <code>Copy</code> , 独占)

```
#[derive(Clone, Copy)]    // 自定义类型声明 Copy(derive 第6章细讲)
struct Point2 {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point2 { x: 1, y: 2 };
    let q = p;              // Copy:浅拷贝,p 仍可用
    println!("{}", p.x, q.y);

    let s1 = String::from("hello");
    let s2 = s1.clone();    // Clone:显式深拷贝(内容复制,所有权各自持有)
    println!("{s1} {s2}");  // 两个都可用——但这是"深拷贝",有开销
}
```

5.3.2 与 C 的 memcpy 对照

Copy 本质上就是一次 memcpy,但比 C 多做两件事:① 只有"复制后仍安全"的类型(无堆指针、无析构义务)才允许 Copy——String 复制指针而不复制缓冲区会 double free,所以被禁止;② 原值是否仍可用由编译器追踪,而不是靠记忆。C 里 `struct A b = a;` 之后两个结构体都能用、但各自析构语义全凭自觉;Rust 把"能拷贝的"与"只能搬移的"分得清清楚楚。

⚠ 易错点 2:给"带堆数据"的类型随手写 `#[derive(Copy)]`

编译错误会当场教育你:"the trait `Copy` may not be implemented for this type; field `data` does not implement `Copy`"——String、Vec 字段天然禁止 Copy。这不是限制,是保护:Copy 承诺"复制后双方独立",而共享同一个堆缓冲区违背这个承诺。需要"复制内容"用 `clone()`;需要"共享所有权"用 `Rc`(§5.6)——按需选择,不要硬造 Copy。

§5.4 借用规则

5.4.1 借用:只借不拿

数据被 move 走后就消失了,但函数常常只需要"看一眼"。Rust 允许通过引用临时借用数据,借用人不是所有者,不承担释放义务——对应的 C 概念是"受控的指针":

```
fn main() {
    let v = vec![1, 2, 3];
    let r1 = &v;          // 不可变借用:共享
    let r2 = &v;          // 第二个不可变借用:允许
    println!("{r1:?} {r2:?}"); // 多个读者共存

    let mut w = vec![4, 5, 6];
    let m = &mut w;       // 可变借用:独占
    m.push(7);
    println!("{m:?}");
}
```

5.4.2 两条铁律

★ 重点:借用规则(整章的核心)

- 规则一:任意时刻,要么有任意多个不可变借用(&T),要么有一个可变借用(&mut T),不能同时存在
- 规则二:借用必须活得比数据短(数据被释放后,借用必须已消失)

规则一消灭"数据竞争"(多个写者);规则二消灭"悬垂引用"(use-after-free)。两条规则都由编译器在编译期检查——C 里这两类 bug 靠人盯,盯不住。

```
fn main() {
    let mut v = vec![1, 2, 3];

    let r1 = &v;
    let r2 = &v;
    println!("{r1:?} {r2:?}");          // 最后一次使用 r1、r2

    // 借用已结束(NLL:非词法作用域),现在可以可变借用了
    let r3 = &mut v;
    r3.push(4);

    // 下面的写法编译错误:不可变借用 r4 还活着时想可变借用
    // let r4 = &v;
    // let r5 = &mut v;          // error[E0502]: cannot borrow as mutable
}
```

NLL(Non-Lexical Lifetimes,非词法生命周期)是 2018 年引入的改进:借用结束于"最后一次使用",而不是整个作用域。这让"先借后改"的合法代码不再被误伤——上面 r1/r2 用完后立刻可以 &mut。

5.4.3 与 C 的指针对照

C 的写法	Rust 的写法	编译器做什么
void f(const T *p)	fn f(p: &T)	只读借用;多个可共存
void f(T *p)	fn f(p: &mut T)	独占;同一时刻最多一个
void *p = malloc(...); free(p); 再用 p	借用必须短于数据生命	悬垂引用 = 编译错误
两个指针指向同一内存,一个写一个读	读写借用互斥	数据竞争 = 编译错误

⚠ 易错点 3:被 E0502 搞烦了,想用"拷贝"绕过借用

借用冲突的典型解法不是 clone,而是 **缩小借用范围**:把只读用的部分先算完(如上例的 println),或把读取逻辑提取成小函数,或改用迭代器闭包让借用局部化。clone 只在"数据小、拷贝便宜"时作为最后手段。见到 E0502/E0503/E0597,深呼吸,先问自己:哪个借用还活着?让它先走。

§5.5 生命周期标注

5.5.1 为什么需要 'a

看这个函数: `fn longest(x: &str, y: &str) -> &str`——返回的引用到底借自 `x` 还是 `y`? 编译器无法自己猜(两个都可能),于是需要你标注:返回值的存活期与两个参数中最短的那个绑定。语法是在引用类型上写 `'a`:

```
// 'a 是"生命周期参数":声明"x、y 与返回值活得一样久"
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("短");
    let s2 = String::from("更长的字符串");
    let r = longest(&s1, &s2); // 返回的引用在 s1、s2 内都有效
    println!("{}", r);
}
```

5.5.2 关键认知:生命周期是编译期检查,不改变运行时

★ 重点:生命周期不产生任何运行时代码

`'a` 是纯粹的编译期概念:它不改变数据的布局、不产生额外的运行时开销、没有"生命周期对象"存在于内存中。它只是给编译器的一份证明书,声明引用之间的存活关系。你写的 `'a` 会被编译成零条指令。运行时只有一件事是真的:数据在所有者离开作用域时被释放——而这与你标注的 `'a` 无关。

正因为检查发生在编译期,违反规则就是编译错误,而且错误信息会指明是哪一次借用越界。看一个"悬垂引用"的活例子——C 里这是静默 UB,Rust 里是明确的编译错误:

```
fn main() {
    let r;
    {
        let s = String::from("短暂的数据");
        r = longest(&s, &s); // 借用 s,但 s 马上要被释放
    }                       // s 在这里释放
    // println!("{}", r);   // error[E0597]: `s` does not live long enough
}
```

5.5.3 省略规则与 'static

实际代码里 `'a` 并不常见,因为编译器有**生命周期省略(lifetime elision)**规则:单入参函数(如 `fn first_word(s: &str) -> &str`)、方法(`&self` 借用)都自动补全,不用写。只有"多引用入参 + 返回引用"这种组合才必须显式标注。`'static` 是最长的生命周期:程序全程存活——字符串字面量的默认类型就是 `&'static str`:

```
fn main() {
    // 字符串字面量活在静态区,类型是 &'static str
    let greeting: &'static str = "你好,Rust";
    println!("{}",greeting);

    // 普通 String 的引用不是 'static:
    let s = String::from("运行时数据");
    // let p: &'static str = &s;    // 编译错误:无法保证存活到程序结束

    // 'static 的真实场景:配置常量、错误消息、static 全局(第1章)
    static APP_TAG: &'static str = "v1.0";
    println!("{}",APP_TAG);
}
```

工程建议:生命周期标注的三个生产场景

- **函数返回借用时:**写签名前先想"返回的引用借自哪个参数",需要标注就标注;省略规则能覆盖就别写
- **结构体持有引用时:**struct 字段是 &T 必须写生命周期(如 `struct View<'a> { data: &'a [u8] }`);能用 owned 类型(String/Vec)就尽量不用引用——少一类编译烦恼
- **多写多错多修:**生命周期报错是"证明书不全",补标注或调整结构,不是修改逻辑——先确认数据谁活着,再改代码

§5.6 智能指针:Box / Rc / Arc / Weak

5.6.1 Box:把值放到堆上

Box 是"堆上单个值"的所有者,与 C 的 `malloc` 一一对应,但无需 `free`——离开作用域自动释放:

```
fn main() {
    // C: struct Node *n = malloc(sizeof(struct Node));
    let n = Box::new(42);
    println!("{}", *n);           // 解引用,与 C 的 *p 相同

    // Box 常用来打破递归类型(递归大小无法确定,必须装箱):
    // enum List { Nil, Cons(i32, Box<List>) } // 见下方示例
    println!("指针大小:{{}} 字节", std::mem::size_of:::<&i32>());
}

// 递归类型的标准写法:Box 让 List 大小确定(指针 + 数据)
enum List {
    Nil,
    Cons(i32, Box<List>),
}

fn main2() {
    let list = List::Cons(1, Box::new(List::Cons(2, Box::new(List::Nil))));
    let _ = list;
    println!("链表构建完成");
}
```

注意上面有两个 main——教学展示请二选一;实际工程一个 crate 只能有一个 main。这正是"宁简勿错"的演示:递归类型(树、链表)是 Box 的头号生产场景,因为无限递归大小在编译期无法确定,必须装箱成指针。

5.6.2 Rc:单线程引用计数共享

当多个地方需要"共同持有"同一份数据时(图结构、配置共享),Box 不够用——它只能有一个所有者。Rc 提供引用计数:每 clone 一次计数 +1,计数归零才释放。对应 C++ 的 `std::shared_ptr`,但 Rc 是单线程的(线程安全版本是 Arc):

```
use std::rc::Rc;

fn main() {
    let a = Rc::new(String::from("共享配置"));
    let b = Rc::clone(&a);          // 计数 2;注意是 Rc::clone,不是深拷贝
    let c = Rc::clone(&a);          // 计数 3
    println!("强计数:{}", Rc::strong_count(&a));

    drop(b);                        // 计数 2
    println!("drop 后计数:{}", Rc::strong_count(&a));
    println!("{}", c);              // 数据仍有效
    drop(c);
    drop(a);                        // 计数归零:堆上 String 被释放
}
```

5.6.3 Arc:多线程版本的 Rc

Arc = Atomic Rc,计数操作是原子的,可以跨线程共享(第9章与 Mutex 组合使用)。用法与 Rc 相同,只是 `use std::sync::Arc`。对应 C++ 的 `shared_ptr` (C++ 的 `shared_ptr` 天生线程安全计数)。

5.6.4 Weak:打破循环引用的钥匙

引用计数的致命弱点:如果 A 持有 B、B 又持有 A,两者的计数永远不小于 1,内存永远不被释放——这就是引用计数的"泄漏"。C++ 的 `shared_ptr` 同样有这个问题。解法是让环中的一方持有弱引用 **Weak**:Weak 不增加强计数,访问时先 `upgrade()` 试试对方还活着没有:

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,    // 父节点用弱引用:不成环
    children: RefCell<Vec<Rc<Node>>>, // 子节点用强引用
}

impl Node {
    fn new(name: &str) -> Rc<Node> {
        Rc::new(Node {
            name: name.to_string(),
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(Vec::new()),
        })
    }

    fn add_child(parent: &Rc<Node>, child: &Rc<Node>) {
        *child.parent.borrow_mut() = Rc::downgrade(parent); // 父指针:弱
        parent.children.borrow_mut().push(Rc::clone(child)); // 子指针:强
    }
}

fn main() {
    let root = Node::new("根");
    let leaf = Node::new("叶");
    Node::add_child(&root, &leaf);
    println!("root 强计数:{}", Rc::strong_count(&root));
    println!("leaf 强计数:{}", Rc::strong_count(&leaf));

    // 通过弱引用访问父节点:先 upgrade 确认存活
    let parent = leaf.parent.borrow();
    if let Some(p) = parent.upgrade() {
        println!("leaf 的父节点:{}", p.name);
    }
    drop(root); // root 释放后
    // parent.upgrade() 将返回 None,不会产生悬垂访问
    println!("root 已释放,升级结果:{}", parent.upgrade().is_some());
}

```

C	C++	Rust
malloc/free	new/delete、RAII	Box::new + 作用域自动 drop
(手写计数结构)	std::shared_ptr	Rc(单线程)/ Arc(多线程)
(手写弱引用)	std::weak_ptr	Weak(配合 upgrade 检查)
void* + 手动类型转换	unique_ptr<T>	Box<T>:类型安全,无 void*

§5.7 内部可变性:Cell 与 RefCell

5.7.1 冲突的两种解法:换借用方式,或换容器

借用规则说"共享 &T 不可写"。但生产里有合法需求:多个持有者共享一份数据,其中一方要改(统计计数、缓存)。Rust 提供**内部可变性(interior mutability)**容器,把"修改"从编译期检查挪到运行期检查:

容器	适用场景	检查时机
Cell<T>	T 是 Copy 类型;按值读写 get/set	编译期(无借用概念,按值取放)
RefCell<T>	T 任意类型;borrow()/borrow_mut() 取引用	运行期(冲突时 panic)
&mut T 直接借用	普通可变借用	编译期

```
use std::cell::{Cell, RefCell};

fn main() {
    // Cell:按值操作,适合计数器
    let count = Cell::new(0);
    count.set(count.get() + 1);
    println!("计数:{}", count.get());           // 1

    // RefCell:借用规则推迟到运行期
    let data = RefCell::new(String::from("hello"));
    data.borrow_mut().push_str(", world");
    println!("{}", data.borrow());               // hello, world

    // 运行期检查:borrow 未结束时 borrow_mut 会 panic(不是编译错误!)
    let r = data.borrow();
    // data.borrow_mut(); // 运行期 panic:already borrowed
    drop(r);                                     // 先归还借用
    data.borrow_mut().push('!');
    println!("{}", data.borrow());
}
```

5.7.2 组合拳:Rc<RefCell<T>> = 共享 + 可变

单独 Rc 不能改(共享借用只读),单独 RefCell 不能共享(一个所有者)。两者组合,就是 Rust 的"共享可变状态"标准答案——对应 C 里"多个指针共享 + 手动加锁/约定"的场景(单线程版):

```

use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    // 多个持有者 + 内部可变:图、缓存、UI 状态的标准组合
    let shared = Rc::new(RefCell::new(vec![1, 2, 3]));
    let a = Rc::clone(&shared);
    let b = Rc::clone(&shared);

    a.borrow_mut().push(4);      // 通过 a 修改
    b.borrow_mut().push(5);      // 通过 b 修改
    println!("{:?}", shared.borrow()); // [1, 2, 3, 4, 5]
}

```

⚠ 易错点 4:以为 RefCell 让"借用检查失效",随便写

RefCell 把检查从编译期挪到运行期,检查仍然存在,只是时机变了:borrow 与 borrow_mut 同时活跃就 panic,而且 panic 是崩溃不是错误处理。生产教训:① 尽量保持 borrow 的生命周期短(取用即 drop);② 不要在持有 borrow 的引用时调用可能再 borrow 的函数;③ 多线程场景请用 Mutex 而不是 RefCell(第9章)。RefCell 不是后门,是"把违反规则的后果从编译错误换成运行期崩溃"的慎重让步。

§5.8 Drop 与 unsafe 的边界

5.8.1 Drop:释放时的"最后动作"

Drop trait 定义"值被释放时运行的代码",是 Rust 的 RAII 形态:资源(文件、锁、连接)的获取与释放绑定在对象生命周期上。对应 C++ 的析构函数;C 里对应的则是"记得在每条 return 路径上 close 文件"的痛苦:

```

struct FileHandle {
    path: String,
}

impl Drop for FileHandle {
    fn drop(&mut self) {
        // C 里这里必须手动 close(fd),且每条出错路径都要记得
        println!("关闭文件:{}", self.path);
    }
}

fn main() {
    {
        let f = FileHandle { path: String::from("config.toml") };
        println!("使用中:{}", f.path);
    } // 作用域结束:自动调用 drop
    println!("文件已自动关闭,无需手动清理");
}

```

生产中的 Drop 场景:文件句柄、数据库连接、锁的释放、性能计时器。第9章的 Mutex 锁也靠 Drop 自动释放——你不需要写 unlock,作用域结束就是 unlock。

5.8.2 unsafe:唯一的后门,以及它的边界

Rust 的安全保证在 **unsafe** 块/函数中被暂停,允许你做四件事:解引用裸指针、调用 unsafe 函数(FFI)、访问可变静态变量、实现 unsafe trait。什么时候用:

场景	是否建议 unsafe	理由
调用 C 库(FFI)	必须	C 接口本身就是裸指针;安全封装后对外
极致性能临界(如哈希表热点)	极少	先 profile,再决定;绝大多数场景用不到
绕过借用规则"图方便"	禁止	违背安全承诺,且大概率是设计问题
业务代码	禁止	业务里出现 unsafe 是明显的坏味道

```
fn main() {
    let mut x = 42;
    let raw: *mut i32 = &mut x;      // 裸指针:与 C 的 int* 一致
    unsafe {
        *raw += 1;                    // 解引用裸指针必须在 unsafe 里
    }
    println!("{x}");                  // 43

    // 安全抽象原则:unsafe 只藏在函数内部,对外接口是安全的
    let v = read_or_zero(raw);
    println!("读到的值:{v}");
}

/// 安全外壳:内部用 unsafe,外部调用者无需感知
fn read_or_zero(p: *const i32) -> i32 {
    if p.is_null() {
        return 0;                    // 先检查空指针,再进 unsafe
    }
    unsafe { *p }
}
```

★ 重点:unsafe 的四条纪律

1. 最小化:unsafe 块越小越好,一行一行地圈定

2. 封装:unsafe 必须藏在安全函数内部,对外只暴露安全接口

3. 注释:每个 unsafe 块写明"为什么这里是安全的"(安全前提)

4. 隔离:用独立的模块收纳 unsafe 代码,便于审计(第13章 mod 组织)

记住: unsafe 不是"关掉安全检查",而是"你向编译器承诺:这块我自己保证正确"——承诺要兑现,否则就是给自己埋雷。

本章速查表

主题	结论 / 写法
内存三区	栈(自动)/堆(所有者管理)/静态区(程序期);无 GC
所有权法则1	每个值有且只有一个所有者
所有权法则2	所有者离开作用域,值自动 drop(无泄漏通道)
所有权法则3	赋值/传参转移所有权(move);原变量失效
Copy	整数/浮点/bool/char/定长数组/&T:赋值=位拷贝;String/Vec/Box 不是
Clone	s.clone() 显式深拷贝,双方各自持有
借用规则1	多个 &T 或一个 &mut T,不可同时存在
借用规则2	借用必须短于数据生命(悬垂 = 编译错误 E0597)
NLL	借用结束于最后一次使用,而非作用域结束
生命周期	'a 是编译期证明,零运行时开销;多引用返回需标注;省略规则覆盖常见场景
'static	程序全程存活;字符串字面量的默认生命周期
Box	堆上单值;递归类型(链表/树)必用
Rc / Arc	引用计数共享(单/多线程);Rc::clone 只增计数不深拷贝
Weak	弱引用防循环;upgrade() 检查存活;对应 C++ weak_ptr
Cell / RefCell	内部可变性;RefCell 运行期检查,冲突即 panic
Rc<RefCell<T>>	共享 + 可变的单线程标准组合
Drop	作用域结束自动调用;RAII:资源绑定生命周期
unsafe	FFI/裸指针/性能临界;最小化 + 封装 + 注释 + 隔离

最小必会清单

- 能背出所有权三法则,并用"所有者"视角解释任意一段代码的释放时机
- 能说出 5 种 Copy 类型与 3 种非 Copy 类型,并解释 String 为什么不能 Copy
- 能解释 move 与 Clone 的区别,以及各自的开销
- 能背出借用两条铁律,并解释它们如何消灭数据竞争与悬垂引用
- 能说出 NLL 的含义,并写出"先借后改"的合法代码
- 能写出带 'a 的函数签名,并说出生命周期在运行时的开销(0)
- 能说明 Rc 与 Arc 的区别、Weak 解决什么问题
- 能写出 Rc<RefCell<T>> 的共享可变状态代码
- 能说出 RefCell 与 &mut 的检查时机差异(编译期 vs 运行期)
- 能说出 unsafe 的四个使用场景与四条纪律

自测题

Q 1. 以下代码打印什么?为什么没有泄漏? `fn main() { let a = String::from("x"); { let b = a; } println!("{}", b); }`

✓ 参考答案

这个程序编译失败:a 的所有权在 `let b = a` 时 `move` 给 b,b 在内部块结束即释放;随后 `println!("{}", b)` 虽然不引用 a,但 a 已被 `move`——不,`println!("{}", b)` 不引用 a 时其实可以编译!(NLL 后 `move` 检查也按使用点算)。严谨答案:若 `println` 引用 a 则编译错误;若不引用,编译通过,b 的 `String` 在块结束时被释放,无泄漏。核心:释放自动发生,不需要也写不出 `free`。

Q 2. 为什么 `let y = x;` 对 `i32` 合法、对 `String` 需要 `clone` 或 `move`?这与 C 的结构体赋值有何不同?

✓ 参考答案

`i32` 实现了 `Copy`:赋值是位拷贝,双方独立。`String` 拥有堆缓冲区,位拷贝会让两个 `String` 指向同一块缓冲,释放时 `double free`——所以赋值=`move`(转移所有权)。C 里所有类型赋值都是位拷贝,struct 含指针时双份指针指向同一堆内存,谁释放、何时释放全凭约定;Rust 用 `Copy` 标记把"能安全拷贝的"与"只能搬移的"分开,由编译器强制。

Q 3. 下面的代码错在哪? `let mut v = vec![1,2,3]; let r = &v; v.push(4); println!("{}", r);`

✓ 参考答案

不可变借用 `r` 还活着时调用 `v.push` 需要可变借用,违反铁律一:E0502。修复:让 `r` 先"退休"——把 `println` 提到 `push` 之前,或不用 `r`。这正是借用冲突的标准解法:缩小借用范围,而不是 `clone`。

Q 4. 生命周期 'a 在运行时存在吗?它影响性能吗?

✓ 参考答案

不存在、不影响。'a 是纯编译期概念:给编译器证明引用之间的存活关系,零运行时开销、零内存布局影响。数据真正释放的时机只有一条:所有者离开作用域。写 'a 不是写运行时逻辑,是写证明书。

Q 5. Rc 与 Arc 的区别?循环引用为什么泄漏?如何破?

✓ 参考答案

`Arc` 的计数操作原子化,可跨线程;`Rc` 只能单线程。循环引用:两对象互相强引用,各自计数 ≥ 1 ,永远不为 0,内存不释放(引用计数的固有缺陷,C++ `shared_ptr` 同病)。破法:让环上的一方持 `Weak`——`Weak` 不增加强计数,且升级前先检查存活。

Q 6. RefCell 的借用检查在什么时机?违反规则的结果与 `&mut` 有何不同?

✓ 参考答案

`RefCell` 在运行期检查:`borrow_mut` 时若已有活跃 `borrow`(或其逆),直接 `panic`;编译期检查只验证"borrow 的引用不逃逸"。而 `&mut` 在编译期检查,违反即编译错误。所以 `RefCell` 是把"违反规则的代价"从编译错误换成运行

期崩溃——能用 `&mut` 就用 `&mut`, `RefCell` 只用于"多个所有者共享可变状态"的合法需求。

章末实验题:所有权与引用计数观察实验

实验 5:内存行为观察——所有权、引用计数与内部可变性

实验目标:写一个"内存行为示波器":用 `Drop` 打印观察释放时机,用 `Rc/Weak` 观察引用计数变化,用 `RefCell` 体验内部可变性,并做一组所有权重构练习。

实验要求:

- 任务 1:定义带 `Drop` 打印的 `Tracker` 结构体,创建栈上对象与 `Box` 堆上对象,观察 `drop` 顺序(覆盖:所有权三法则 §5.2、`Drop` §5.8、`Box` §5.6)
- 任务 2:做 `move` 实验:`i32` 赋值后双方可用,`String` 赋值后原变量失效(注释掉错误行观察报错)(覆盖:`move` 与 `Copy` §5.3)
- 任务 3:写 `swap` 函数(`&mut` 参数)并调用;再尝试"两个不可变借用 + 可变借用"的冲突代码,观察 `E0502`(覆盖:借用规则 §5.4)
- 任务 4:写带生命周期标注的 `longer` 函数,并构造"借用超过数据生命"的失败示例(覆盖:生命周期 §5.5)
- 任务 5:用 `Rc` + `Weak` 构造父子节点,打印强计数/弱计数,`drop` 父节点后验证 `weak.upgrade()` 返回 `None`(覆盖:`Rc/Arc/Weak` §5.6)
- 任务 6:用 `RefCell` 实现访问计数,验证共享引用下也能修改(覆盖:内部可变性 §5.7)

实验环境:`cargo new mem_observe`;每完成一个任务运行一次 `cargo run`,观察打印顺序与报错信息。

第一步 一步:先写 `Tracker` 与 `Drop impl`,验证"作用域结束自动释放"——这是所有观察的地基

第二步 二步:把失败示例写成注释(如被 `move` 的变量、`borrow` 冲突),运行确认编译错误,再解开修复——"故意编译失败"是本章最有效的学习法

第三步 三步:`Rc` 实验重点看 `strong_count` 的变化与 `drop` 的时机

第四步 四步:`Weak` 实验先构造"父子互持"版本,观察泄漏,再改成弱引用版本对比

第五步 五步:`RefCell` 实验故意触发一次运行期 `panic`,观察它与编译期错误的不同之处

参考答案要点

```

// ===== 实验 5:内存行为观察 =====
// 覆盖知识点:所有权三法则($5.2)、move 与 Copy($5.3)、借用规则($5.4)、
//             生命周期标注($5.5)、Box/Rc/Weak($5.6)、
//             内部可变性 RefCell($5.7)、Drop 析构与释放观察($5.8)

use std::cell::RefCell;
use std::rc::{Rc, Weak};

/// 观察 Drop:每次释放打印一行($5.8)
struct Tracker {
    name: String,
}

impl Drop for Tracker {
    fn drop(&mut self) {
        println!(">>> [drop] {} 被释放", self.name);
    }
}

/// 带访问计数的共享节点($5.6 + $5.7)
struct Node {
    name: String,
    visits: RefCell<u32>,          // 内部可变:共享引用下也能改
}

impl Node {
    fn new(name: &str) -> Rc<Node> {
        Rc::new(Node {
            name: name.to_string(),
            visits: RefCell::new(0),
        })
    }

    fn visit(&self) {
        *self.visits.borrow_mut() += 1; // 运行期借用检查
    }

    fn count(&self) -> u32 {
        *self.visits.borrow()
    }
}

/// 生命周期标注:返回两个引用中较长者($5.5)
fn longer<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

/// 借用演示:交换两个值($5.4)
fn swap(a: &mut i32, b: &mut i32) {
    let t = *a;
    *a = *b;
    *b = t;
}

fn main() {

```

```

// ---- 任务 1:所有权与 Drop 时机 ----
{
    let t = Tracker { name: String::from("栈上对象") };
    println!("使用栈上对象:{}", t.name);
} // 此处应打印 drop

let heap = Box::new(Tracker { name: String::from("堆上对象") });
println!("Box 内对象:{},指针大小 {} 字节",
    heap.name, std::mem::size_of_val(&heap));
drop(heap); // 显式释放:应打印 drop

// ---- 任务 2:move 与 Copy ----
let a = 5;
let b = a; // Copy:双方可用
println!("Copy 后:{a} {b}");
let s1 = String::from("数据");
let s2 = s1; // move:s1 失效
println!("move 后:{s2}");
// println!("{s1}"); // 编译错误:use of moved value

// ---- 任务 3:借用规则 ----
let mut x = 10;
let mut y = 20;
swap(&mut x, &mut y);
println!("swap 后:x={x} y={y}");

let v = vec![1, 2, 3];
let r1 = &v;
let r2 = &v; // 多个只读借用:允许
println!("共享借用:{r1:?} {r2:?}");
// let m = &mut v; // 编译错误:仍有活跃只读借用

// ---- 任务 4:生命周期 ----
let s = String::from("hello world");
let short = "hi";
println!("较长者:{}", longer(&s, short));

// ---- 任务 5:Rc / Weak 计数观察 ----
let root = Node::new("根节点");
let leaf = Node::new("叶节点");
let weak_leaf: Weak<Node> = Rc::downgrade(&leaf);
println!("叶节点:强计数 {},弱计数 {}",
    Rc::strong_count(&leaf), Rc::weak_count(&leaf));

root.visit();
root.visit();
leaf.visit();
println!("根访问 {} 次,叶访问 {} 次(RefCell 生效)",
    root.count(), leaf.count());

if let Some(l) = weak_leaf.upgrade() {
    println!("弱引用升级成功,访问次数:{}", l.count());
}
drop(leaf); // 强计数归零,节点释放
println!("叶节点释放后,弱引用升级是否成功:{}",

```



```

        weak_leaf.upgrade().is_some()); // false:不会悬垂

// ---- 任务 6:RefCell 与借用冲突的两种后果 ----
let cell = RefCell::new(10);
{
    let b = cell.borrow();           // 只读借用存活中
    println!("borrow 中:{b}");
    // cell.borrow_mut();           // 运行期 panic,而非编译错误!
}                                   // b 在此归还
*cell.borrow_mut() += 5;
println!("修改后:{}", cell.borrow());
}

```

设计说明:实验的节奏是"先看对的,再撞错的"——任务 2/3/6 各留了一个注释掉的失败示例,建议逐个解开运行,亲眼确认三类错误(use of moved value、E0502 借用冲突、RefCell 运行期 panic)在编译期/运行期的不同表现。全部跑通后,把 root/leaf 的强引用关系改成"父子互持"再看计数,就能亲手复现循环引用泄漏。

下一章预告:第6章 面向对象——Rust 没有继承!C 用 struct+函数指针模拟多态,Rust 用 trait 定义共享行为、dyn Trait 做动态分派、impl Trait 做静态分派,组合优于继承。运算符重载、访问控制、设计原则,一章讲透。

第6章 面向对象与 trait

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:熟悉 C 的结构体与函数指针,了解"多态"的概念(不要求写过 OOP)

🎯 本章学习目标

- 回顾 C 用"结构体 + 函数指针表"模拟多态的两种做法,理解其样板代码与类型安全之痛
- 掌握 **trait** 的本质:一组行为契约——定义、为类型实现、默认方法、调用方式
- 理解"无继承"的设计:数据与行为分离,接口通过 trait 表达,复用通过组合实现
- 掌握静态分派:泛型 + trait bound、impl Trait 参数与返回值(对应 C++ 模板,零虚表开销)
- 掌握动态分派: **dyn Trait** 与 **vtable** 的内存布局,与 C 函数指针表、C++ 虚函数对照
- 掌握组合优于继承:has-a 结构体嵌套,替代 C++/Java 的 is-a 继承链
- 会用 `std::ops` 实现运算符重载,理解 `Display/Debug/PartialEq` 等标准 trait 的用法
- 了解 trait 继承与默认方法:接口层面的"继承",用扩展与组合而不是类继承

💡 从 C 到 Rust:本章主题如何迁移

C 程序员对"多态"并不陌生——只是名字不叫多态。常见做法有两种:**tag 分发**(struct 里放一个 kind 字段,switch 分发)和**函数指针表**(struct 里放一个函数指针数组,把方法挂上去)。前者每次加新类型要改 switch,后者每次加新方法要改所有表。两个方案都靠约定维护:编译器不知道"面积函数"该有几个参数、哪个字段配哪个函数。C 的 `void*` 更是把类型检查整个扔掉了。

Rust 的答案叫 **trait**:先声明"会算什么(方法签名)",再让具体类型声明"我实现了它"。编译器全程检查:调用方拿到的是实现了某 trait 的类型,字段与方法——对应,`void*` 消失。这解决了 C 样板代码与类型安全的问题,却保留了两样 C 程序员珍惜的东西:**零开销与确定性**。没有 GC、没有反射、没有运行时类型信息包袱——trait 要么在编译期展开(静态分派,像 C++ 模板),要么放进一张显式的函数指针表(动态分派,像 C 的 `vtbl`,只是由编译器生成)。

另一个关键决裂是"无继承"。C 本身没有继承,所以这对你不是损失;但如果你写过一点 C++/Java,请把"is-a"思维换成"has-a":`Car` 里放一个 `Engine` 字段,而不是 `Engine` 的子类。行为共享用 trait(接口),数据复用用组合(字段),两头都不靠"继承基类"这个又耦合又脆弱的机制。本章的路线:先看懂 C 的痛点(\$6.1),再学 trait(\$6.2-\$6.4 由浅入深),然后学组合思维(\$6.5),最后是运算符重载与 trait 家族(\$6.6-\$6.7)。

📦 前置知识自查

- C 的结构体、函数指针、函数指针数组;typedef 函数指针类型
- C 的 `void*` 与类型强转的后果(类型安全缺失)
- 第4章的 `impl` 方法、`self` 三态(`&self` / `&mut self` / `self`)、关联函数
- 第5章的 `Box` 与所有权——本章的 `Box<dyn Trait>` 依赖它们
- (可选)C++ 的 `virtual` 与虚表概念、Java 的 `interface`,用于对照

本章学习路线

1. **第一阶段(约1.5小时):**§6.1–§6.2, trait 的定义、实现与默认方法——先会"声明契约"
2. **第二阶段(约2小时):**§6.3–§6.4, 静态分派与动态分派——理解"编译期展开"与"运行期查表"的取舍, 这是生产选择的灵魂
3. **第三阶段(约1.5小时):**§6.5–§6.7, 组合思维、运算符重载、标准 trait——把 trait 用进日常代码
4. **验收标准:**给你一个"形状"需求, 能徒手写出 trait + 两种分派 + 组合式扩展, 并解释各自开销

本章知识导图

- §6.1 C 的多态困境: tag 分发与函数指针表
- §6.2 trait: 行为契约的定义、实现与默认方法
- §6.3 静态分派: 泛型 + trait bound、impl Trait
- §6.4 动态分派: dyn Trait、vtable 与对象安全
- §6.5 组合优于继承: has-a 思维替代 is-a
- §6.6 运算符重载与标准 trait: Add / Display / PartialEq
- §6.7 trait 继承与默认方法: 接口层面的扩展
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§6.1 C 的多态困境: tag 分发与函数指针表

6.1.1 方案一: tag 分发

struct 里放一个类型标签, 所有函数开头 switch 一下:

```
// C: tag 分发——"多态"的朴素实现
typedef enum { SHAPE_CIRCLE, SHAPE_RECT } ShapeKind;

typedef struct {
    ShapeKind kind;
    double a;      // 圆: 半径; 矩形: 宽
    double b;      // 矩形: 高(圆时未使用)
} Shape;

double shape_area(const Shape *s) {
    switch (s->kind) {
        case SHAPE_CIRCLE: return 3.14159 * s->a * s->a;
        case SHAPE_RECT:   return s->a * s->b;
    }
    return 0;           // 新类型忘了加分支? 静默返回 0
}
```

痛点一望即知: 加一个新类型(三角形)要改 switch 的每一个地方(面积、周长、打印.....), 漏改一处就静默出错; 字段 a/b 的含义随 kind 变化, 全靠注释约定, 编译器无从检查。

6.1.2 方案二:函数指针表(vtbl)

```
// C:函数指针表—把"方法"挂到结构体上
typedef struct Shape Shape;
typedef struct {
    double (*area)(const Shape *self);    // 方法一
    void (*print)(const Shape *self);    // 方法二
} ShapeVtbl;

struct Shape {
    const void *data;                    // 指向具体对象(void*:类型安全已丢失)
    const ShapeVtbl *Vtbl;              // 虚函数表指针
};

// 调用:c->Vtbl->area(c) — 每张表都要手工维护,每个函数都要传 self
```

痛点:每加一个方法,所有类型的表都要改;void* 让编译器再也帮不上忙;self 参数靠人肉传递,传错对象编译通过、运行崩溃。C++ 用 virtual 把这张表藏进编译器,但代价是继承体系本身的复杂性。**Rust 的做法:声明契约(trait)由编译器生成表,类型检查全程在线**——这是本章的支点。

C 的多态手段	痛点	Rust 的对应
switch + kind 字段	加类型要改所有分支,漏改静默错	match 穷尽性检查(第3章)让漏改变编译错误
手写函数指针表	表与类型不绑定,void* 丢类型	trait 由编译器生成 vtable,类型全程保留
函数指针参数	签名不携带"来自哪个对象"	fn 指针、闭包(第4章)或 impl Trait

§6.2 trait:行为契约的定义与实现

6.2.1 声明契约

trait 只声明"有哪些行为"(方法签名),不包含数据。数据永远在 struct/enum 里;trait 只描述"实现者保证能做到的事":

```
// 契约:任何实现了 Shape 的类型,必须提供面积与名称
trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> &'static str;
}

// 数据在具体类型里,不在 trait 里
struct Circle { r: f64 }
struct Rect { w: f64, h: f64 }

// 为 Circle 兑现契约
impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.r * self.r
    }
    fn name(&self) -> &'static str {
        "圆形"
    }
}

impl Shape for Rect {
    fn area(&self) -> f64 { self.w * self.h }
    fn name(&self) -> &'static str { "矩形" }
}

fn main() {
    let c = Circle { r: 2.0 };
    println!("{}", c.name(), c.area());
}
```

6.2.2 默认方法:契约带"免费午餐"

trait 里的方法可以带默认实现,实现者只需覆盖需要定制的部分——这是"接口默认方法"(Java 8 同款)的来源,也是后面 trait 继承的原料:

```
trait Greeter {
    fn name(&self) -> String;           // 必须实现

    fn greet(&self) -> String {         // 默认实现:免费赠送
        format!("你好,{}", self.name())
    }
}

struct 小明;
impl Greeter for 小明 {
    fn name(&self) -> String { String::from("小明") }
}

fn main() {
    let x = 小明;
    println!("{}", x.greet());         // 你好,小明!
}
```

⚠ 易错点 1:想把"字段"写进 trait

trait 里只能有方法,不能有字段。常见冲动: `trait Shape { fn area(&self) -> f64; let r: f64; }` ——不行。正确分工: 数据进 struct,行为进 trait;需要"取半径"就声明一个 `fn radius(&self) -> f64` 方法,由各实现者返回自己的字段。这正是 C 的教训:状态与行为分离,表里不放数据。

§6.3 静态分派:泛型 + trait bound

6.3.1 泛型函数:编译期展开,零开销

要求"任意实现了 Shape 的类型"参与计算,用泛型 + trait bound。编译器为每种具体类型生成一份专门的代码(单态化 **monomorphization**,与 C++ 模板一致):

```
// T 被约束:必须实现 Shape。面积之和无需知道具体类型
fn total_area<T: Shape>(shapes: &[T]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

// 等价写法:impl Trait 参数(语法糖,意义相同)
fn total_area2(shapes: &[impl Shape]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

// impl Trait 返回值:隐藏具体类型,只暴露"实现了 Shape"
fn make_circle() -> impl Shape {
    Circle { r: 1.0 }
}

fn main() {
    let shapes = vec![Circle { r: 1.0 }, Circle { r: 2.0 }];
    println!("总面积:{}", total_area(&shapes));

    let c = make_circle();
    println!("返回的隐藏类型:{}", c.name());
}
```

静态分派的特征:类型在编译期确定,没有查表、没有间接跳转,直接内联调用。与 C 的差别:C 里同一份代码操作不同结构体要么靠宏,要么靠 `void*` 丢类型;Rust 里类型信息全程保留,且生成的是"按类型特化"的代码,性能与手写每种类型的专用函数相同。

§6.4 动态分派:dyn Trait 与 vtable

6.4.1 什么时候必须动态分派

集合里放"不同类型的对象"时,泛型做不到——`Vec<Circle>` 只能装圆形。Rust 用 **trait 对象**: `Box<dyn Shape>` (堆上)或 `&dyn Shape` (借用)。运行期通过 vtable 查找方法,与 C 的函数指针表、C++ 的虚函数是同一个机制,只是表由编译器生成、类型安全:

```
fn main() {
    // 异构集合:圆形、矩形装进同一个 Vec
    let shapes: Vec<Box<dyn Shape>> = vec![
        Box::new(Circle { r: 1.0 }),
        Box::new(Rect { w: 2.0, h: 3.0 }),
    ];

    for s in &shapes {
        println!("{}", s.name(), s.area());
    }

    // 借用形式的 trait 对象(不拥有)
    let c = Circle { r: 5.0 };
    let obj: &dyn Shape = &c;
    println!("通过 &dyn 调用:{}", obj.name());
}
```

6.4.2 vtable 的内存真相

★ 重点:trait 对象的内存布局(与 C 的 vtbl 完全同构)

`Box<dyn Shape>` 在内存里是 两个指针(胖指针):① 指向堆上数据的指针;② 指向 vtable 的指针。vtable 是一张表,装着该类型实现的每个方法地址(以及 drop、size 等信息)。调用 `s.area()` 时:先按表地址取 area 的函数指针,再跳转——一次间接调用,与 C 的手写表、C++ 的 virtual 调用开销完全相同。对照:

C 手写 vtbl	C++ virtual	Rust dyn Trait
手工维护表 + void*	编译器生成虚表	编译器生成 vtable
类型安全丢失(void*)	类型安全(静态)	类型安全(静态)
self 参数手动传	this 隐式传	&self 显式传(与普通方法一致)
每类型一份表	每类型一份虚表	每类型一份 vtable
手工增删方法	加方法要改基类	trait 增方法,实现者统一适配
表 + 数据分开存	对象内嵌虚表指针	胖指针:数据指针 + 表指针

6.4.3 对象安全:不是所有 trait 都能 dyn

能放进 dyn 的 trait 必须满足**对象安全**:所有方法都不能有泛型参数(泛型方法无法放进固定大小的表)、不能返回 Self(返回类型因实现而异)、方法不能要求 Sized。违反的典型:

```
// 反例:泛型方法—编译器无法为 dyn 生成 vtable
trait Bad {
    fn compare<T>(&self, other: T) -> bool;    // 泛型方法
}
// let x: Box<dyn Bad>;    // 编译错误:E0038 the trait cannot be made into an object

// 正例:所有方法的具体类型都在签名里固定
trait Good {
    fn compare(&self, other: &dyn Good) -> bool;    // 可以
}

fn main() {
    println!("对象安全:泛型方法无法进 vtable,所以被编译器拒绝");
}
```

⚠ 易错点 2:处处用 dyn,性能与设计双双受损

能静态就不要动态: 泛型/impl Trait 是默认选择,dyn 只在"异构集合""插件化接口""运行期决定类型"时使用。dyn 有两点代价:① 每次调用一次间接跳转(微乎其微,但热路径可感知);② 最关键——失去内联与特化,编译器无法按具体类型优化。生产经验:接口面优先写成泛型;只有当"类型集合在运行期才闭合"(插件、导出表)时才上 dyn。C 程序员换一种说法就懂:能写死类型就别走函数指针。

§6.5 组合优于继承:has-a 思维替代 is-a

6.5.1 继承的问题

C++/Java 的继承给"复用"开了捷径,也埋了雷:基类一旦改变,所有子类的行为跟着变(**脆弱的基类问题**);菱形继承带来重复与歧义;深层继承链让"这个对象到底做什么"越来越难回答。C 没有继承,反而绕开了这些——Rust 延续这个务实传统:数据复用用"包含"(字段),行为复用用"trait",不提供类继承。

6.5.2 组合的标准写法

```
struct Engine {
    power: u32,           // 单位:马力
}

impl Engine {
    fn new(power: u32) -> Engine { Engine { power } }
    fn start(&self) -> String {
        format!("引擎启动,功率 {} 马力", self.power)
    }
}

// 汽车"拥有"引擎:has-a 组合,而不是"是"引擎的子类
struct Car {
    brand: String,
    engine: Engine,        // 关键:嵌套字段
}

impl Car {
    fn new(brand: &str, power: u32) -> Car {
        Car {
            brand: String::from(brand),
            engine: Engine::new(power),
        }
    }

    fn drive(&self) -> String {
        format!("{}", self.brand, self.engine.start())
    }
}

fn main() {
    let car = Car::new("东风", 300);
    println!("{}", car.drive());
}
```

6.5.3 行为复用走 trait,数据复用走组合

需要"Car 也能被驾驶"时,别建"交通工具"基类——声明 trait 让 Car 实现:

```

trait Drivable {
    fn drive(&self) -> String;
}

// Car 实现"可驾驶"契约,同时保留自己的 Engine 字段
impl Drivable for Car {
    fn drive(&self) -> String {
        format!("{}", self.brand, self.engine.start())
    }
}

// 另一个类型同样实现,互不打扰—这是继承做不到的解耦
struct 自行车;
impl Drivable for 自行车 {
    fn drive(&self) -> String { String::from("脚踏驱动,零排放") }
}

fn go_for_a_ride(vehicle: &impl Drivable) {
    println!("出发:{}", vehicle.drive());
}

fn main() {
    go_for_a_ride(&Car::new("东风", 300));
    go_for_a_ride(&自行车);
}

```

C++/Java 继承	问题	Rust 组合方案
class Car : public Vehicle	基类改动波及全部子类	struct Car { engine: Engine } 字段包含
纯虚接口	接口与实现绑定在类层级里	trait 独立于类型声明,可按需实现
菱形继承	重复与歧义	不存在(无继承,无菱形)
向上转型传递依赖	隐式耦合	泛型 + trait bound:显式契约

§6.6 运算符重载与标准 trait 家族

6.6.1 运算符重载:经 std::ops

Rust 的运算符(+, -, ==, 索引等)都是 trait:实现了 `std::ops::Add` 就能用 +。与 C++ 的 operator 重载对应,但每个运算符绑定固定的 trait 与语义,不能发明新符号:

```

use std::ops::Add;

#[derive(Debug, Clone, Copy)]
struct Point {
    x: f64,
    y: f64,
}

// 实现 + :self 是"被加数"(Add 消费两个值)
impl Add for Point {
    type Output = Point;

    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}

fn main() {
    let a = Point { x: 1.0, y: 2.0 };
    let b = Point { x: 3.0, y: 4.0 };
    println!("{:?}", a + b);           // Point { x: 4.0, y: 6.0 }
}

```

6.6.2 显示与比较: Display / Debug / PartialEq

```

use std::fmt;

// Display:{ } 格式,面向用户
impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

// PartialEq== 与 !=,可自定义比较规则(这里用容差)
impl PartialEq for Point {
    fn eq(&self, other: &Self) -> bool {
        (self.x - other.x).abs() < 1e-9
        && (self.y - other.y).abs() < 1e-9
    }
}

fn main() {
    let a = Point { x: 1.0, y: 2.0 };
    let b = Point { x: 1.0, y: 2.0 };
    println!("{a}");           // (1, 2):走 Display
    println!("{a} == {b} : {}", a == b); // true:走 PartialEq

    // Debug({:?}) 常用 derive 自动生成,手写用于定制
    println!("{:?}", a);       // 需要 #[derive(Debug)]
}

```

工程建议:derive 优先,手写兜底

Debug/Clone/Copy/PartialEq 等"机械性"trait 用 `#[derive(...)]` 自动生成(第10、12章详解);需要定制语义(容差比较、脱敏打印)才手写 `impl`。规则:先 `derive`,不够再手写,永远不要手写机械实现。

易错点 3:运算符重载里误用"消费"还是"借用"

`impl Add for Point` 用 `fn add(self, ...)` 消费两个值。若你的类型很贵且语义是"计算不搬走",应实现 `impl Add for &Point` (对引用加法),或两者都实现。最常见的坑:实现了 `Add for Point` 后, `a + b` 会把 `a` 和 `b` 都 `move` 掉,后续再用 `a` 就编译错误。对策:类型是 `Copy`(如上例 `derive(Clone, Copy)`)就无所谓;非 `Copy` 的昂贵类型,优先实现引用版本,或提供方法而非运算符。

§6.7 trait 继承与默认方法:接口层面的扩展

6.7.1 trait 可以"继承"trait

trait 之间可以声明超集关系:实现子 trait 必须先实现父 trait。这近似 Java 的接口继承,但不带来数据耦合——仍是纯行为契约:

```

trait Shape {
    fn area(&self) -> f64;
}

// Drawable 是 Shape 的超集:实现者必须先实现 Shape
trait Drawable: Shape {
    fn draw(&self) -> String;
}

struct Circle { r: f64 }

impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.r * self.r
    }
}

impl Drawable for Circle {
    fn draw(&self) -> String {
        format!("画圆,半径 {}", self.r)
    }
}

// 泛型约束写法:要求"既是 Shape 又是 Drawable"
fn draw_all<T: Shape + Drawable>(items: &[T]) {
    for it in items {
        println!("{}", it.draw(), it.area());
    }
}

fn main() {
    draw_all(&[Circle { r: 2.0 }]);
}

```

6.7.2 扩展 trait:给别人的类型加方法

trait 最有生产价值的特性: **可以为任何类型实现任何 trait** (孤儿规则:二者至少一个在本 crate)。这让你不修改第三方代码,就能给它的类型挂上自己的行为——C 里想给别人的 struct 加"方法",只能包一层或改源码:

```
// 假设 Point 来自第三方库,我们无权改它

// 自己声明的扩展 trait
trait Distance {
    fn distance_to(&self, other: &Point) -> f64;
}

impl Distance for Point {          // 类型是别人的,trait 是自己的:合法
    fn distance_to(&self, other: &Point) -> f64 {
        let dx = self.x - other.x;
        let dy = self.y - other.y;
        (dx * dx + dy * dy).sqrt()
    }
}

fn main() {
    let p = Point { x: 0.0, y: 0.0 };
    let q = Point { x: 3.0, y: 4.0 };
    println!("距离:{}", p.distance_to(&q));    // 5
}
```

C++/Java	Rust
抽象基类 / 接口	trait(纯行为契约,无数据)
接口默认方法(Java 8+)	trait 默认方法
接口继承	trait 继承(父 trait 约束)
为既有类加接口	扩展 trait(孤儿规则保护全局一致性)
虚函数调用	dyn Trait 的 vtable 查找
模板特化	泛型单态化
装饰器/适配器	组合:包一层 struct 再实现 trait

本章速查表

主题	结论 / 写法
trait 是什么	一组方法签名的契约;不含数据;实现者声明"我能做到"
定义/实现	<code>trait Name { fn m(&self); } / impl Name for Type { ... }</code>
默认方法	trait 内带实现体;实现者可覆盖,不覆盖则用默认
静态分派	泛型 + trait bound: <code>fn f<T: Trait>(x: &T);</code> 编译期单态化,零虚表
impl Trait	参数/返回位置的语法糖:隐藏具体类型,保留 trait 契约
动态分派	<code>Box<dyn Trait> / &dyn Trait;</code> 运行期查 vtable,一次间接调用
胖指针布局	数据指针 + vtable 指针;与 C 手写 vtbl、C++ 虚函数同构
对象安全	方法无泛型参数、不返回 Self;违反则不能 dyn(错误 E0038)
继承	无类继承;行为复用走 trait,数据复用走字段组合
组合	<code>struct Car { engine: Engine } —— has-a 替代 is-a</code>
运算符重载	<code>std::ops::Add</code> 等;注意 self 消费与 <code>&self</code> 两种形态
Display / Debug	<code>{}</code> 与 <code>{:?}</code> ;Debug 常 derive,Display 常手写(定制)
PartialEq	实现 <code>==</code> ;定制规则(容差)手写,机械比较用 derive
trait 继承	<code>trait Sub: Super { ... };</code> 要求先实现父 trait
扩展 trait	类型与他 trait 至少一方是本 crate(孤儿规则);第三方类型可加方法

最小必会清单

- 能说清 trait 与 struct 的分工:数据进 struct,行为进 trait
- 能徒手写出 trait 定义、默认方法、impl 实现与调用
- 能解释静态分派与动态分派的机制差异(展开 vs 查表)与开销差异
- 能在"异构集合"场景正确使用 `Box<dyn Trait>`,并能解释胖指针布局
- 能说出对象安全的两条主要规则,并解释为什么泛型方法不能进 vtable
- 能说出"无继承"的设计:组合(字段)+ trait(契约)分别替代什么
- 能实现 `std::ops::Add` 并为自定义类型定制 Display
- 知道 derive 优先原则:Debug/Clone/Copy/PartialEq 先 derive 再手写
- 能写出 trait 继承(父 trait 约束)与扩展 trait(为第三方类型加方法)

自测题

Q 1. trait 与 C 的函数指针表有什么本质区别?

✓ 参考答案

本质区别是"契约由谁维护":C 的表是程序员手写的,与类型无绑定,void* 丢掉类型信息,字段/方法对应关系靠约定;trait 是编译器参与的类型契约——实现时逐方法检查签名,调用时类型安全,表(如需要 vtable)由编译器生成。同一份代码里,C 靠"记得",Rust 靠"类型保证"。

Q 2. 静态分派与动态分派各有什么开销与限制?生产上如何选择?

✓ 参考答案

静态(泛型/impl Trait):编译期单态化,可内联、可特化,零运行时开销;代价是"编译出一份份专用代码"(代码膨胀,一般可控)、调用方必须知道具体类型。动态(dyn):运行期查 vtable,一次间接跳转,无法内联;换来异构集合与运行期才确定的类型。选择:默认静态;只有异构集合、插件化、运行期决定类型时才用 dyn。

Q 3. 为什么 Vec<dyn Shape> 编译不过,而 Vec<Box<dyn Shape>> 可以?

✓ 参考答案

dyn Shape 是"胖指针"(数据指针 + vtable 指针),大小不固定、不可直接拷贝,不能作为 Vec 的元素按值存储;Box<dyn Shape> 把胖指针装箱到堆上,Box 本身是定长指针,可以放入 Vec。同理,Vec<&dyn Shape> 也可行(借用形式的胖指针)。

Q 4. 一个方法带泛型参数,为什么 trait 就不能做成 dyn?

✓ 参考答案

vtable 是固定大小的函数指针数组,每行存一个确定签名的方法。泛型方法实际是"方法族"(每个具体 T 一份),无法填进固定槽位;而且调用时需要知道 T 才能选实现。所以对象安全要求方法无泛型参数。需要"多态 + 泛型行为"时,改用泛型约束(静态)而不是 dyn(动态)。

Q 5. "组合优于继承"在 Rust 里具体怎么落地?

✓ 参考答案

两层落地:① 数据复用:struct Car { engine: Engine }——把要复用的状态作为字段包含,而不是继承基类;② 行为复用:trait 声明契约,各类型独立实现,相互解耦;需要"既是 A 又是 B"时用 trait bound(Shape + Drawable)或组合。没有 is-a 层级,只有 has-a 结构与行为契约。

Q 6. 运算符重载时,"消费 self"与"借用 &self"两种实现有什么后果?

✓ 参考答案

impl Add for Point(消费)调用 a + b 后 a、b 被 move,无法再用;若 Point 实现 Copy 则无感。impl Add for &Point(借用)调用 &a + &b,原值保留,适合昂贵非 Copy 类型。选择依据:值语义小类型(坐标、颜色)常两者都实现;引用语义类型只实现借用版,或用方法替代运算符。

章末实验题:几何图形系统

实验 6:几何图形系统——trait 全流程实战

实验目标:实现一个"几何图形系统",把本章所有知识点串起来:trait 契约、默认方法、静态分派、动态分派、运算符重载、组合、trait 继承。

实验要求:

- 任务 1:定义 Shape trait(area/name 方法)与 Drawable: Shape(增加 draw 方法)(覆盖:trait 定义 §6.2、trait 继承 §6.7)
- 任务 2:实现 Circle、Rect、Triangle 三个图形(覆盖:impl 契约 §6.2)
- 任务 3:写泛型 total_area 与 impl Trait 参数函数各一个,验证静态分派(覆盖:静态分派 §6.3)
- 任务 4:用 Vec<Box<dyn Drawable>> 建异构场景,统一绘制与计算(覆盖:动态分派 §6.4)
- 任务 5:为 Point 实现 Add(向量加)与 Display(覆盖:运算符重载 §6.6)
- 任务 6:Scene 结构体组合 shapes 与 label 字段,实现 Show 行为(覆盖:组合 §6.5)

实验环境:cargo new geometry;完成后 cargo run,输出应包括:各图形面积、异构列表总面积、每个图形的绘制语句、两个 Point 相加结果。

第一步 **一步:**先写 Point 与其 Add/Display,建立"值类型"地基

第二步 **二步:**写 Shape 与三个图形类型,跑通"各算各的面积"

第三步 **三步:**加 Drawable(继承 Shape),给每个图形补 draw

第四步 **四步:**先写泛型函数(静态),再改造成 dyn 版本,对比两者的使用方式差异

第五步 **五步:**组合 Scene 收尾——体会"数据嵌套 + 行为契约"替代继承的完整形态

参考答案要点

```
// ===== 实验 6:几何图形系统 =====
// 覆盖知识点:trait 契约与默认方法($6.2)、trait 继承($6.7)、
//          静态分派与 impl Trait($6.3)、动态分派 dyn($6.4)、
//          组合优于继承($6.5)、运算符重载与 Display($6.6)

use std::fmt;
use std::ops::Add;

// ---- 任务 1:trait 契约与 trait 继承 ----
trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> &'static str;
}

trait Drawable: Shape {                // 父 trait:必须先实现 Shape
    fn draw(&self) -> String;
}

// ---- 任务 2:三个图形实现契约 ----
struct Circle { r: f64 }
struct Rect { w: f64, h: f64 }
struct Triangle { base: f64, height: f64 }

impl Shape for Circle {
    fn area(&self) -> f64 { std::f64::consts::PI * self.r * self.r }
    fn name(&self) -> &'static str { "圆形" }
}

impl Shape for Rect {
    fn area(&self) -> f64 { self.w * self.h }
    fn name(&self) -> &'static str { "矩形" }
}

impl Shape for Triangle {
    fn area(&self) -> f64 { self.base * self.height / 2.0 }
    fn name(&self) -> &'static str { "三角形" }
}

impl Drawable for Circle {
    fn draw(&self) -> String { format!("画圆:半径 {:.1}", self.r) }
}

impl Drawable for Rect {
    fn draw(&self) -> String {
        format!("画矩形:{:.1} x {:.1}", self.w, self.h)
    }
}

impl Drawable for Triangle {
    fn draw(&self) -> String {
        format!("画三角形:底 {:.1} 高 {:.1}", self.base, self.height)
    }
}

// ---- 任务 3:静态分派(两种写法) ----
```

```

fn total_area<T: Shape>(shapes: &[T]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

fn describe(shape: &impl Drawable) -> String {
    format!("{}", shape.name(), shape.draw())
}

// ---- 任务 5:运算符重载与 Display ----
#[derive(Clone, Copy)]
struct Point { x: f64, y: f64 }

impl Add for Point {
    type Output = Point;
    fn add(self, other: Point) -> Point {
        Point { x: self.x + other.x, y: self.y + other.y }
    }
}

impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({:.1}, {:.1})", self.x, self.y)
    }
}

// ---- 任务 6:组合:场景拥有图形列表(has-a) ----
struct Scene {
    label: String,
    shapes: Vec<Box<dyn Drawable>>, // 异构:动态分派
}

impl Scene {
    fn new(label: &str) -> Scene {
        Scene { label: String::from(label), shapes: Vec::new() }
    }

    fn add(&mut self, s: Box<dyn Drawable>) {
        self.shapes.push(s);
    }

    fn render(&self) {
        println!("=== 场景:{} ===", self.label);
        let mut total = 0.0;
        for s in &self.shapes {
            println!("{}", s.draw());
            total += s.area(); // dyn 上的方法调用
        }
        println!("场景总面积:{:.2}", total);
    }
}

fn main() {
    // 静态分派:同构集合
    let circles = [Circle { r: 1.0 }, Circle { r: 2.0 }];
    println!("两个圆面积和:{}", total_area(&circles));
}

```

```
// 动态分派:异构集合
let mut scene = Scene::new("小树林");
scene.add(Box::new(Circle { r: 2.0 }));
scene.add(Box::new(Rect { w: 3.0, h: 4.0 }));
scene.add(Box::new(Triangle { base: 2.0, height: 3.0 }));
scene.render();

// 运算符重载与 Display
let p1 = Point { x: 1.0, y: 2.0 };
let p2 = Point { x: 3.0, y: 4.0 };
println!("向量和:{}", p1 + p2);
}
```

设计说明:任务 3 与任务 4 是本章的对照核心——`total_area(&[Circle; 2])` 在编译期展开为圆专用代码,而 `Scene` 里的 `draw/area` 走 `vtable` 查找;两者同一套 `trait`,互不干扰。试着把 `fn add(&mut self, s: Box<dyn Drawable>)` 改成 `s: T` (泛型)就会遇到"异构容器装不下"的限制,这一步亲手撞过,就真正理解了两种分派的边界。

下一章预告:第7章 文件与 IO——C 用 `fopen/fread/fwrite` 与 `errno`,Rust 用 `std::fs` 与 `Result`。路径、读写、目录遍历、序列化与标准流,一次性讲清。

第7章 文件与 IO

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:熟悉 C 的 fopen/fread/fwrite/fclose、FILE*、errno 与标准流 stdin/stdout

🎯 本章学习目标

- 理解 C 的 FILE* 世界与 Rust 的 **Result + RAII** 世界的对应关系:打开失败不再返回 NULL,而是 Result
- 掌握路径类型 Path/PathBuf:为什么不能用 String 拼接路径,join 与 display 的正确用法
- 掌握文件读写的三种姿势:一次性读入、流式按行读、二进制读写,以及各自的适用场景
- 掌握 BufReader/BufWriter 缓冲语义,与 C 的 setvbuf/fread 缓冲机制对照
- 掌握 OpenOptions 的精细控制:创建/追加/截断,对应 C 的 "w"/"a"/"a+" 模式
- 掌握目录遍历 read_dir 与递归删除/创建,对照 C 的 opendir/readdir 与平台 API
- 会用 serde_json 做结构化的序列化与反序列化,告别手写字符串解析
- 掌握标准流 stdin/stdout/stderr 与命令行参数,main 返回 Result 的错误传播约定

💡 从 C 到 Rust:本章主题如何迁移

C 的 IO 函数是"全局状态 + 错误码"的样板:fopen 失败返回 NULL(还要 errno 分辨原因),fread 返回"读取的字节数"(要自查是否等于请求数),fclose 失败还可能静默丢数据,忘了 fclose 就是泄漏。每个 FILE* 是"不透明指针",打开模式靠字符串 "rb"/"a+" 约定。这套 API 你用了十年,深知其中暗坑:缓冲区没刷、错误码没查、路径太长、跨平台换行.....

Rust 把同一件事重新组织成三个支柱:**第一,错误成为类型**——每个 IO 调用返回 Result,失败信息(路径、原因)随值携带,不再靠 errno 猜;? 运算符把"失败就提前返回"写成一行(第9章详述,本章先用)。**第二,资源由类型管理**——File 离开作用域自动关闭(RAII,第5章已铺垫),没有 fclose,没有泄漏,没有"双关"。**第三,路径是类型,不是字符串**——Path 区分于 String,把"拼接分隔符""检查存在""平台差异"收进类型系统。

对照着学最省力:本章每一节都从"C 是怎么写的"出发,给出 Rust 对应写法与差异表。你先看到 fopen 与 File::open 的对应,再看到 BufReader 与 setvbuf 的对应,然后看到 serde_json 如何终结手写解析——这就是本章的迁移路线。

📦 前置知识自查

- C 的文件 API:fopen 模式字符串、fread/fwrite 返回值、fseek/feof、errno 与 perror
- C 的标准流 stdin/stdout/stderr、printf/fprintf/sprintf 家族
- 第4章的 main 函数与 Result 返回约定、第5章的 RAII/Drop 概念
- 第2章的运算符与表达式(本章大量使用 ? 与解构)
- (可选)JSON 的基本结构:对象、数组、字符串、数字

本章学习路线

1. **第一阶段(约1小时):**§7.1–§7.2,文件打开与路径类型——先会用 `File::open/fs::write` 读写文件
2. **第二阶段(约1.5小时):**§7.3–§7.4,缓冲读写、`OpenOptions` 与目录操作——生产 IO 的主力
3. **第三阶段(约1.5小时):**§7.5–§7.7,`serde_json`、标准流与错误传播——把 IO 与数据类型打通
4. **验收标准:**不查文档能写一个"按行读日志、统计、写汇总"的工具,且每个 IO 调用都带 `?` 或显式处理

本章知识导图

- §7.1 `FILE*` 世界 vs `Result` 世界:打开与关闭
- §7.2 路径类型:`Path` 与 `PathBuf`
- §7.3 读写的三种姿势:一次性、流式、二进制
- §7.4 精细控制:`OpenOptions` 与缓冲语义
- §7.5 目录操作:遍历、创建、删除
- §7.6 结构化数据:`serde_json` 序列化
- §7.7 标准流、命令行参数与错误传播
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§7.1 FILE* 世界 vs Result 世界:打开与关闭

7.1.1 C 的样板代码

```
// C:每个文件操作都要查错误码,忘了就静默出错
#include <stdio.h>

int copy_file(const char *src, const char *dst) {
    FILE *in = fopen(src, "rb");
    if (in == NULL) {                // 打开失败:返回 NULL,errno 告诉原因
        perror("open src");
        return -1;
    }
    FILE *out = fopen(dst, "wb");
    if (out == NULL) {
        perror("open dst");
        fclose(in);                // 别忘了关掉第一个!
        return -1;
    }

    char buf[4096];
    size_t n;
    while ((n = fread(buf, 1, sizeof buf, in)) > 0) {
        if (fwrite(buf, 1, n, out) != n) {
            perror("write");
            fclose(in); fclose(out);
            return -1;
        }
    }
    fclose(in); fclose(out);        // 每一处提前返回都要记得关
    return 0;
}
```

这段代码的每一行都在跟"错误"搏斗:NULL 检查、errno、fclose 的成对出现。少一个 fclose 是泄漏,少一个检查是静默错误。

7.1.2 Rust 的对应:Result 携带错误,Drop 自动关闭

```
// Rust:同样的文件复制,错误是类型,资源是 RAII
use std::fs;
use std::io;

fn copy_file(src: &str, dst: &str) -> io::Result<()> {
    let data = fs::read(src)?;          // 失败返回 Err,包含路径与原因
    fs::write(dst, data)?;              // 失败同样提前返回
    Ok(())
}

fn main() {
    match copy_file("a.txt", "b.txt") {
        Ok(()) => println!("复制成功"),
        Err(e) => eprintln!("复制失败:{e}"),    // e 里带着"哪个文件、什么错误"
    }
}
```

对照一目了然:

C	Rust	差异要点
fopen 返回 FILE* 或 NULL	File::open 返回 Result<File>	NULL 意味着"错误与类型混在一起";Result 让错误成为类型的一部分
errno + perror	io::Error(含路径、错误码、系统消息)	错误随值传递,不会因线程/调用被覆盖
fclose 手动调用	Drop 自动关闭	没有泄漏,没有 double-close
模式字符串 "rb"/"a+"	OpenOptions 方法链	模式编译期检查,拼错字符串?C 会静默失败,Rust 报编译错误
fread 返回值要自查	read_to_end/read_to_string 一次到位	读完没读全是内部逻辑,不留给调用方
feof/ferror 判断结束	read 返回 Ok(0) 或 Err	EOF 与错误用类型区分,不是状态位

⚠ 易错点 1:File::open 只能读,不能写

File::open(path) 等价 C 的 "rb"(只读)。想写文件、想追加、想创建,必须用 File::create(覆盖写,等价 "wb")或 OpenOptions(§7.4)。生产代码最常见的报错就是"用 File::open 打开后调用 write,得到 'cannot borrow as mutable' 或 PermissionDenied"——先想清楚你要哪种模式。

§7.2 路径类型:Path 与 PathBuf

7.2.1 为什么不用 String 表示路径

C 里路径就是 char 数组,拼接靠 sprintf("s%s/config.ini", dir)。Windows 与 Unix 分隔符不同、编码不同,同一段代码换个平台就崩。Rust 用两个类型解决:**Path**(只读借用,对应 &str)与 **PathBuf**(可增长的拥有型,对应 String)。分隔符由 join 自动处理,路径操作(取文件名、取父目录、加后缀)都是类型的方法:


```

use std::path::{Path, PathBuf};

fn main() {
    // C:char dir[256]; sprintf(dir, "%s/logs/app.log", base);
    let base = Path::new("/var/data");
    let log = base.join("logs").join("app.log");    // 自动拼分隔符
    println!("完整路径:{}", log.display());        // /var/data/logs/app.log

    // 取文件名与父目录(对照 basename/dirname)
    println!("文件名:{}", log.file_name().unwrap().to_string_lossy());
    println!("父目录:{}", log.parent().unwrap().display());

    // 换扩展名:生成另一个路径,原路径不动
    let bak = log.with_extension("log.bak");
    println!("备份路径:{}", bak.display());

    // 判断路径属性(对照 stat)
    let p = Path::new("Cargo.toml");
    println!("存在:{} 是文件:{} 是目录:{}",
        p.exists(), p.is_file(), p.is_dir());

    // 转为拥有型 PathBuf(用于长期保存)
    let owned: PathBuf = log.to_path_buf();
    println!("owned:{}", owned.display());
}

```

7.2.2 display 与 to_string_lossy

Path 可能不是合法 UTF-8(Unix 文件名可以是任意字节)。打印用 `display()` (按当前平台编码显示),转字符串用 `to_string_lossy()` (非法字节替换为 U+FFFD)。不要直接用 `to_str()`——它返回 `Option`,遇到非 UTF-8 是 `None`,生产代码里要先处理。

C 的路径操作	Rust 对应
<code>sprintf(dir, "%s/%s", base, name)</code>	<code>Path::join:base.join(name)</code>
<code>basename(3)</code>	<code>path.file_name() -> Option<&OsStr></code>
<code>dirname(3)</code>	<code>path.parent() -> Option<&Path></code>
<code>strchr(name, '.') 改扩展名</code>	<code>path.with_extension("txt")</code>
<code>stat()</code> 判断类型	<code>path.is_file() / is_dir() / exists()</code>
字符串拼出来的分隔符	<code>join</code> 自动按平台处理

§7.3 读写的三种姿势:一次性、流式、二进制

7.3.1 一次性读入:小文件首选

```
use std::fs;

fn main() -> std::io::Result<()> {
    // 整个文件读成字符串(按 UTF-8 解码)
    let text = fs::read_to_string("config.toml"?);
    println!("配置内容:\n{}", text);

    // 整个文件读成字节(二进制或不确定编码)
    let bytes = fs::read("logo.png"?);
    println!("图片大小: {} 字节", bytes.len());

    // 一次写入(覆盖)
    fs::write("out.txt", "第一行\n第二行\n");

    Ok(())
}
```

适用场景:配置文件、日志单条、体积已知的小文件(经验阈值:几十 KB 以内)。大文件一次性读入会吃满内存——那就用流式。

7.3.2 流式按行读:BufReader + lines

```
use std::fs::File;
use std::io::{BufRead, BufReader};

fn main() -> std::io::Result<()> {
    // C 的 getline 循环对应:BufReader 提供缓冲的按行迭代
    let file = File::open("access.log"?);
    let reader = BufReader::new(file);

    let mut line_no = 0u32;
    for line in reader.lines() {           // 逐行,不整文件进内存
        let line = line?;                 // 每一行都可能 IO 失败(解码/读盘)
        line_no += 1;
        if line.contains("ERROR") {
            println!("{line_no}: {line}");
        }
    }
    println!("共扫描 {line_no} 行");
    Ok(())
}
```

7.3.3 二进制流式读:read_exact 与循环

```
use std::fs::File;
use std::io::{Read};

fn main() -> std::io::Result<()> {
    let mut file = File::open("data.bin")?;

    // 固定大小结构:一次读满(对照 fread 后自查返回值)
    let mut header = [0u8; 16];
    file.read_exact(&mut header)?; // 读不够 16 字节直接报 UnexpectedEof
    println!("头 16 字节:{header:02X?}");

    // 长度不确定的数据:循环读,read 返回 0 表示 EOF
    let mut rest = Vec::new();
    file.read_to_end(&mut rest)?;
    println!("剩余 {} 字节", rest.len());
    Ok(())
}
```

⚠ 易错点 2:read 与 read_exact 的区别

`read(&mut buf)` 返回"实际读到的字节数",可能小于 `buf` 长度(非阻塞、缓冲边界、信号中断),必须循环;而 `read_exact(&mut buf)` 保证读满或返回错误——解析固定大小结构(文件头、报头、帧)永远用 `read_exact`,不要自己写循环。这是 C 的 `fread` 返回值检查在 Rust 里的类型化版本。

§7.4 精细控制:OpenOptions 与缓冲语义

7.4.1 OpenOptions:打开模式的类型化版本

```
use std::fs::OpenOptions;

fn main() -> std::io::Result<()> {
    // C: fopen(path, "a") — 追加写
    let mut log = OpenOptions::new()
        .append(true)           // 追加
        .create(true)           // 不存在则创建
        .open("app.log")?;
    use std::io::Write;
    writeln!(log, "[{}] 服务启动", "2026-08-26")?;

    // C: fopen(path, "w") — 截断覆盖
    let mut out = OpenOptions::new()
        .write(true)
        .create(true)
        .truncate(true)
        .open("report.txt")?;
    writeln!(out, "月度报告")?;

    // C: fopen(path, "r+") — 读写,不截断
    let _both = OpenOptions::new()
        .read(true)
        .write(true)
        .open("data.bin")?;

    Ok(())
}
```

7.4.2 BufWriter:攒批写,减少系统调用

```
use std::fs::File;
use std::io::{BufWriter, Write};

fn main() -> std::io::Result<()> {
    // 裸 File::write 每次调用都是一次系统调用;BufWriter 攒满再刷
    let file = File::create("big.txt")?;
    let mut w = BufWriter::new(file);

    for i in 0..100_000 {
        writeln!(w, "第 {i} 行数据")?;    // 先进内存缓冲
    }
    w.flush()?;                            // 最后统一落盘(或 Drop 时自动刷)
    println!("写完");
    Ok(())
}
```

与 C 对照: `setvbuf(stdout, NULL, _IOFBF, 4096)` 之后 `printf` 的行为,就是 `BufWriter` 的默认行为。区别: Rust 的缓冲是 **显式类型**——你拿到 `File` 就是无缓冲直写,想要缓冲就包一层 `BufWriter`,编译器帮你记账,不会出现"忘了 `setvbuf`

导致性能灾难"的隐式状态。

工程建议:日志/上报类写操作,顺序保证

崩溃恢复类场景(交易日志、追加式审计)要"写一条刷一条",不能攒批。做法:不用 `BufWriter` 直接写 `File`,或每行后显式 `flush()`;性能敏感且允许丢尾巴的场景(统计输出)才用 `BufWriter`。C 里同样纠结 `fflush`,但 Rust 把选择放在了明面上。

§7.5 目录操作:遍历、创建、删除

7.5.1 列目录与递归遍历

```
use std::fs;

fn main() -> std::io::Result<()> {
    // 对照 C:opendir/readdir/closedir
    println!("== 当前目录内容 ==");
    for entry in fs::read_dir(".")? {
        let entry = entry?;           // 每个条目本身可能出错
        let name = entry.file_name(); // OsString:不是 UTF-8 也安全
        println!("{}", name.to_string_lossy());
    }

    // 递归统计:目录树中所有 .rs 文件的行数(手写递归,对照 C 的递归遍历)
    fn count_rs_lines(dir: &std::path::Path) -> std::io::Result<(u32, u32)> {
        let mut files = 0u32;
        let mut lines = 0u32;
        for entry in fs::read_dir(dir)? {
            let entry = entry?;
            let path = entry.path();
            if path.is_dir() {
                let (f, l) = count_rs_lines(&path)?; // 递归
                files += f;
                lines += l;
            } else if path.extension().and_then(|e| e.to_str()) == Some("rs") {
                files += 1;
                lines += std::io::BufRead::lines(
                    std::io::BufReader::new(fs::File::open(&path)?)
                ).count() as u32;
            }
        }
        Ok((files, lines))
    }

    let (f, l) = count_rs_lines(std::path::Path::new("src"))?;
    println!("src 下 {f} 个 rs 文件,共 {l} 行");
    Ok(())
}
```

7.5.2 创建与删除

```
use std::fs;

fn main() -> std::io::Result<()> {
    // 递归创建目录(对照 mkdir -p)
    fs::create_dir_all("backup/2026/08"?);

    // 删除文件
    fs::remove_file("tmp/old.tmp"?);

    // 递归删除目录树(危险操作,生产慎用)
    fs::remove_dir_all("tmp"?);

    // 重命名/移动
    fs::rename("a.txt", "archive/a.txt"?);

    println!("目录操作完成");
    Ok(())
}
```

C / POSIX	Rust std::fs
opendir / readdir / closedir	fs::read_dir 迭代器(自动关闭)
mkdir -p(循环建)	fs::create_dir_all
rmdir	fs::remove_dir
unlink	fs::remove_file
rm -rf(手写递归)	fs::remove_dir_all
rename(2)	fs::rename
stat 判断类型	path.is_file() / is_dir() / metadata()

⚠ 易错点 3:is_dir() 与 read_dir 的组合陷阱

`path.is_dir()` 本身做一次系统调用;在遍历中, `entry.path()` 之后再调 `is_dir` 是"先取路径再 stat"。更高效且避免 TOCTOU(时间差攻击)的做法:用 `entry.file_type()` 的 `is_dir()`,它来自 `readdir` 已经拿到的事实。另外注意:文件系统是活的——遍历中途文件被删,`is_dir` 可能返回 `false`,别把它当权威,IO 错误要留好? 出口。

§7.6 结构化数据:serde_json 序列化

7.6.1 C 的世界:手写解析,苦不堪言

C 读 JSON 要么用第三方库,要么手写字符串切割。手写的典型:strstr 找冒号、atoi 转数字、转义字符自己处理——每加一个字段改一处解析。Rust 的标准做法:serde 框架 + serde_json, 结构体上写 `derive`,序列化与反序列化自动生成(`derive` 原理第10/12章详解,这里直接用):

```

use serde::{Deserialize, Serialize};

// 一个 derive,换来 to_json 与 from_json 两套能力
#[derive(Debug, Serialize, Deserialize)]
struct Config {
    host: String,
    port: u16,
    workers: u32,
    tags: Vec<String>,          // 数组字段
    debug: bool,               // 布尔字段
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let cfg = Config {
        host: "0.0.0.0".into(),
        port: 8080,
        workers: 4,
        tags: vec!["api".into(), "prod".into()],
        debug: false,
    };

    // 序列化:结构体 -> JSON 字符串(pretty 便于人类阅读)
    let json = serde_json::to_string_pretty(&cfg)?;
    println!("{}", json);

    // 反序列化:JSON 字符串 -> 结构体;字段缺失/类型不对直接报错
    let text = r#"{"host":"127.0.0.1","port":9090,"workers":2,
        "tags":["dev"],"debug":true}"#;
    let cfg2: Config = serde_json::from_str(text)?;
    println!("解析结果:{cfg2:?}");

    // 宽松场景:先解析成通用值再取字段
    let v: serde_json::Value = serde_json::from_str(text)?;
    println!("host = {}", v["host"]);
    Ok(())
}

```

工程建议:serde 是事实标准,不是"框架绑架"

serde + serde_json 在 crates.io 上是 Rust 生态的基石级依赖(几乎所有网络、配置、数据库库都在用)。Cargo.toml 加两行:serde = { version = "1", features = ["derive"] } 与 serde_json = "1"。字段命名习惯是 snake_case,对接外部 camelCase 的 JSON 时用 #[serde(rename_all = "camelCase")] 或逐字段 #[serde(rename = "...")]。

§7.7 标准流、命令行参数与错误传播

7.7.1 stdin/stdout/stderr 与命令行参数

```
use std::env;
use std::io::{self, BufRead, Write};

fn main() -> io::Result<()> {
    // 命令行参数:对照 C 的 argv[1..];env::args() 第一个是程序名
    let args: Vec<String> = env::args().collect();
    if args.len() < 2 {
        eprintln!("用法:{<关键词>", args[0]); // 标准错误,对照 fprintf(stderr,...)
        std::process::exit(1);                // 对照 exit(1)
    }
    let keyword = &args[1];

    // 按行读 stdin:对照 getline(stdin) 循环
    let stdin = io::stdin();
    for line in stdin.lock().lines() {
        let line = line?;
        if line.contains(keyword) {
            // 显式写 stdout(对照 fputs(stdout)),flush 保证立即输出
            let mut out = io::stdout().lock();
            writeln!(out, "{line}")?;
        }
    }
    Ok(())
}
```

7.7.2 main 返回 Result:错误传播的完整链路

生产约定的主力形态:main 返回 `Result<(), Box<dyn std::error::Error>>`,所有 IO 调用一路 `?` 到底。程序失败时,系统打印错误信息并以非零码退出——这替代了 C 里"每个函数返回 int + 逐层 if 检查 + 顶层 perror"的全套仪式:


```

use std::fs;
use std::io::{self, BufRead, BufReader};

// 读文件并统计非空行数;失败信息随类型一路传播
fn count_nonempty(path: &str) -> io::Result<u32> {
    let reader = BufReader::new(fs::File::open(path)?);
    let mut n = 0;
    for line in reader.lines() {
        let line = line?;
        if !line.trim().is_empty() {
            n += 1;
        }
    }
    Ok(n)
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let path = "notes.txt";
    let n = count_nonempty(path)?;      // 打开失败:整条链快速返回
    println!("{path} 非空行数:{n}");
    Ok(())
}

// 运行:./tool; 若 notes.txt 不存在,输出:
// Error: Os { code: 2, kind: NotFound, message: "系统找不到指定的文件。" }
// 退出码非 0 — 与 C 的 exit(EXIT_FAILURE) 语义一致,但信息完整

```

C	Rust
printf / puts	println!
fprintf(stderr, ...)	eprintln!
fprintf(stdout, ...) + fflush	write!(io::stdout(), ...)? 或 println!
getchar / getline(stdin)	io::stdin().lock().lines() / read_line
argv / argc	env::args()(跳过程序名后即 argv[1..])
exit(1) + perror	std::process::exit(1) 或 main 返回 Err(自动打印)
errno 跨函数传递	? 把 io::Error 一路向上,类型明确

本章速查表

主题	结论 / 写法
打开只读	<code>File::open(path)? -> Result<File></code> (等价 <code>"rb"</code>)
覆盖写	<code>fs::write(path, data)?</code> 或 <code>File::create(path)?</code> (等价 <code>"wb"</code>)
追加写	<code>OpenOptions::new().append(true).create(true).open(path)?</code>
一次性读文本	<code>fs::read_to_string(path)?</code> (UTF-8,小文件)
一次性读字节	<code>fs::read(path)? -> Vec<u8></code>
按行流式读	<code>BufReader::new(File::open(p)?.lines())</code>
二进制定长读	<code>file.read_exact(&mut [u8; N])?</code> (读不满报 <code>UnexpectedEof</code>)
缓冲写	<code>BufWriter::new(File::create(p)?);</code> 写完 <code>flush()</code> 或 <code>Drop</code> 自动刷
路径拼接	<code>Path::new(base).join("a").join("b");</code> 打印用 <code>display()</code>
取文件名/父目录	<code>path.file_name()</code> / <code>path.parent()</code> (返回 <code>Option</code>)
换扩展名	<code>path.with_extension("bak")</code>
列目录	<code>for entry in fs::read_dir(dir)? { let e = entry?; }</code>
递归建目录	<code>fs::create_dir_all("a/b/c")?(<code>mkdir -p</code>)</code>
删除/移动	<code>fs::remove_file</code> / <code>fs::remove_dir_all</code> / <code>fs::rename</code>
JSON 序列化	<code>#[derive(Serialize, Deserialize)] + serde_json::to_string / from_str</code>
命令行参数	<code>env::args().collect::<Vec<_>>()</code> ; <code>args[0]</code> 是程序名
标准错误	<code>eprintln!("{}", ...);</code> 退出码: <code>std::process::exit(1)</code>
main 错误出口	<code>fn main() -> Result<(), Box<dyn std::error::Error>> + 全程 ?</code>

最小必会清单

- 能说出 `File::open` 与 `File::create`、`OpenOptions` 三种打开方式分别对应 C 的哪些模式
- 能写出"读文件 -> 处理 -> 写文件"的完整主链,且每个 IO 调用要么 ? 要么显式 `match`
- 知道小文件用 `read_to_string/read`,大文件用 `BufReader` 流式;能解释内存差异
- 知道 `read_exact` 与 `read` 的区别,固定结构解析必用 `read_exact`
- 能徒手写出 `BufWriter` 的"创建 -> 循环写 -> `flush`"三段式
- 能用 `Path::join` 拼接路径,知道为什么不用字符串拼,知道 `display()` 与 `to_string_lossy`
- 能写 `read_dir` 遍历并递归统计目录树;知道 `file_type()` 比 `is_dir()` 少一次 `stat`
- 能用 `serde derive` 定义配置结构体,`from_str` 与 `to_string_pretty` 双向转换
- 能写出 `main -> Result` 的错误传播主链,与 C 的逐层 `errno` 检查对照
- 能说出 `RAII` 自动关闭如何消灭 `fclose` 遗漏与 `double-close` 两类 bug

自测题

Q 1. `File::open("x.txt")` 失败时,你能拿到什么信息?与 C 的 `fopen` 返回 `NULL + errno` 有什么本质不同?

✓ 参考答案

拿到一个 `io::Error` 值,包含:错误种类(`NotFound/PermissionDenied` 等)、系统错误码、可读消息(带路径)。本质不同:错误是值的一部分,随调用链传递,不会被其他调用覆盖;`NULL` 只是一个哨兵,细节全靠 `errno` 这个全局状态。Rust 的 `Err` 可以一路? 传播,在顶层统一打印,中途不需要逐层检查。

Q 2. 一次性读入与流式按行读,各自适用什么场景?判断依据是什么?

✓ 参考答案

一次性(`read_to_string/read`)适合小文件(经验阈值几十 KB 内)、需要整份内容做处理(解析、替换)的场景,代码最短;流式(`BufReader.lines()`)适合大文件(日志、数据导出),内存占用与文件大小无关,只与单行长度有关。判断依据:文件是否可能超过内存预算、是否需要全部内容同时在场。

Q 3. `read(buf)` 与 `read_exact(buf)` 有什么区别?什么场景必须用 `read_exact`?

✓ 参考答案

`read` 返回实际读到的字节数(可能小于请求数,如缓冲边界、非阻塞),需要循环直至读满;`read_exact` 保证填满缓冲区或返回错误(`UnexpectedEof`)。解析定长结构(文件头、帧头、固定记录)必须用 `read_exact`——C 的“`fread` 后自查返回值”被吸收进了类型保证,杜绝“结构体只读了一半”的静默 bug。

Q 4. 用字符串直接拼路径会出什么问题?`Path::join` 解决了哪些?

✓ 参考答案

问题:分隔符需自己处理(Windows 用 `\` 而 Unix 用 `/`),拼接时容易出“多一个少一个斜杠”,非 UTF-8 文件名无法用 `String` 表示。`join` 解决:按平台自动插入正确分隔符;`Path` 内部可容纳非 UTF-8(`OsStr`);`display()` 提供安全的显示转换。路径从此是类型,不是文本。

Q 5. `serde` 的 `derive` 相比手写字符串解析,在生产上赢了什么?

✓ 参考答案

三点:① 正确性——转义、嵌套、数字边界、类型不匹配全部由库处理并给出错误,手写解析的漏洞(越界、转义漏处理)不复存在;② 维护成本——结构体加字段,序列化自动跟进,手写解析要同步改两处;③ 类型安全——`from_str` 直接产出强类型结构体,后续代码不用再手工转换与检查。

Q 6. 一个程序既要“正常输出到终端”,又要“把日志写进文件”,用 Rust 怎么写?

✓ 参考答案

双流并存:println!/write!(stdout) 走标准输出;日志用 OpenOptions 追加模式打开文件 + BufWriter(需要立即落盘则每次 flush),eprintln! 走标准错误。与 C 的 stdout/stderr 分工一致,只是文件句柄是 Result 而来、作用域结束自动关闭。把"输出目的地"抽成 trait 或闭包,就能实现"同一条消息双写"的典型架构(第6章 trait 的用武之地)。

章末实验题:日志解析统计工具

实验 7:日志解析统计工具

实验目标:写一个命令行工具,读入一个日志文件,统计各级别(INFO/WARN/ERROR)的条数,把 ERROR 详情与统计汇总写入输出文件。

实验要求:

- 任务 1:命令行参数解析——程序名、输入路径、输出路径(覆盖:env::args §7.7)
- 任务 2:输入路径存在性检查,不存在则打印错误并退出非零(覆盖:Path 操作 §7.2)
- 任务 3:流式按行读入,解析"时间戳 级别 消息"格式(覆盖:BufReader §7.3)
- 任务 4:HashMap 统计各级别条数;收集 ERROR 行(覆盖:数据组织)
- 任务 5:汇总与错误详情写入输出文件(覆盖:BufWriter/fs::write §7.3)
- 任务 6:全程错误传播——main 返回 Result,打开失败信息可见(覆盖:错误传播 §7.7)
- 加分:用 serde_json 把统计结果序列化后写入(覆盖:serde_json §7.6)

日志样例行: 2026-08-26 10:00:01 INFO 服务启动、 2026-08-26 10:01:22 ERROR 数据库连接超时

第一步 一步:cargo new logstat;先在 main 里用 env::args 拿到输入输出路径

第二步 二步:写 parse_line 函数,把一行拆成(时间戳, 级别, 消息)

第三步 三步:搭起读入循环,BufReader.lines() 逐行调用 parse_line,先用 println 验证解析

第四步 四步:加 HashMap 统计与 ERROR 收集

第五步 五步:写汇总文件,跑通全流程;再把 parse_line 的返回值改成 Result,处理坏行

参考答案要点

```
// ===== 实验 7:日志解析统计工具 =====
// 覆盖知识点:命令行参数($7.7)、路径检查($7.2)、流式读($7.3)、
//          缓冲写($7.3)、错误传播($7.7)、serde_json 汇总($7.6)

use std::collections::HashMap;
use std::env;
use std::fs;
use std::io::{BufRead, BufReader, BufWriter, Write};
use std::path::Path;

/// 日志行解析结果
#[derive(Debug)]
struct LogLine {
    timestamp: String,
    level: String,
    message: String,
}

/// 解析一行:"2026-08-26 10:00:01 INFO 服务启动"
/// 时间戳含空格,所以从右侧切:最后两个字段是"级别 消息"
fn parse_line(line: &str) -> Option<LogLine> {
    let line = line.trim();
    if line.is_empty() {
        return None;
    }
    // 前 19 个字符是 "YYYY-MM-DD HH:MM:SS"
    if line.len() < 20 {
        return None;
    }
    let timestamp = line[..19].to_string();
    let rest = &line[20..]; // "INFO 服务启动"
    let mut parts = rest.splitn(2, ' '); // 只拆一次:级别 + 整段消息
    let level = parts.next()?.to_string();
    let message = parts.next()?.to_string();
    Some(LogLine { timestamp, level, message })
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // ---- 任务 1:命令行参数 ----
    let args: Vec<String> = env::args().collect();
    if args.len() != 3 {
        eprintln!("用法:{<输入日志> <输出汇总>", args[0]);
        std::process::exit(2);
    }
    let in_path = &args[1];
    let out_path = &args[2];

    // ---- 任务 2:输入存在性检查 ----
    if !Path::new(in_path).is_file() {
        eprintln!("错误:找不到日志文件 {in_path}");
        std::process::exit(1);
    }

    // ---- 任务 3 + 4:流式读入与统计 ----
    let reader = BufReader::new(fs::File::open(in_path)?);
```

```

let mut counts: HashMap<String, u32> = HashMap::new();
let mut errors: Vec<LogLine> = Vec::new();
let mut bad_lines = 0u32;
let mut total = 0u32;

for line in reader.lines() {
    let line = line?; // 读行失败:传播
    match parse_line(&line) {
        Some(log) => {
            total += 1;
            *counts.entry(log.level.clone()).or_insert(0) += 1;
            if log.level == "ERROR" {
                errors.push(log);
            }
        }
        None => bad_lines += 1, // 坏行:计数,不中断
    }
}

// ---- 任务 5:写汇总文件 ----
let file = fs::File::create(out_path)?;
let mut w = BufWriter::new(file);

writeln!(w, "=== 日志统计汇总 ===");
writeln!(w, "输入文件:{}, in_path)?;
writeln!(w, "总行数:{total} 坏行:{bad_lines}");
writeln!(w, "--- 各级别条数 ---");
for (level, n) in &counts {
    writeln!(w, "{level}: {n}");
}
writeln!(w, "--- ERROR 详情(共 {} 条)---", errors.len());
for e in &errors {
    writeln!(w, "[{}] {}", e.timestamp, e.message);
}
w.flush()?; // 显式刷盘

// ---- 加分:JSON 版本汇总(覆盖 serde_json) ----
#[derive(serde::Serialize)]
struct Summary {
    total: u32,
    bad_lines: u32,
    counts: HashMap<String, u32>,
    error_count: usize,
}

let summary = Summary {
    total,
    bad_lines,
    counts,
    error_count: errors.len(),
};

let json_path = format!("{}", out_path);
fs::write(&json_path, serde_json::to_string_pretty(&summary)?);

println!("完成:汇总写入 {out_path},JSON 写入 {json_path}");

```

```
    Ok(())  
}
```

设计说明: `parse_line` 用 `splitn(2, ' ')` 只拆一次, 保证消息里的空格不被切断——这是日志解析的生产常识; 坏行不 `panic` 而是计数, 保证工具在脏数据面前不崩溃; 所有 IO 通过 `?` 传播, `main` 返回 `Result`, 失败时系统打印完整错误并退出非零。试着删掉 `is_file` 检查, 对比“打开失败”的错误信息形态, 理解两种错误处理风格的差异。

下一章预告: 第8章 网络通信——C 的 `socket` 三件套(`socket/bind/connect`)与 Rust 的 `std::net`; TCP/UDP、HTTP 客户端、粘包拆包与最小服务端, 一次讲清。

第8章 网络通信

建议学时:4–6 小时 | **难度:**★★★★☆ | **读者背景:**熟悉 C 的 socket 编程(socket/bind/listen/accept/connect/send/recv)与基本的 TCP/UDP 概念

本章学习目标

- 掌握 std::net 的 TcpListener/TcpStream 与 C 的 socket 三件套的完整对应关系
- 掌握客户端 connect 与服务端 bind/listen/accept 循环的 Rust 写法,错误全程 Result
- 理解 TCP 是字节流而非消息流: 粘包/拆包的本质 与行协议、长度前缀两种解法
- 掌握读写超时与半关闭(shutdown),对照 C 的 SO_RCVTIMEO 与 shutdown(2)
- 掌握 UDP 的 send_to/recv_from,对照 sendto/recvfrom 与无连接语义
- 会用 request 写 HTTP 客户端,理解阻塞客户端与异步客户端的选择
- 能手写一个 ≤40 行的最小 HTTP 服务端,读懂请求行与响应格式
- 理解 TIME_WAIT 与端口复用在生产中的意义,以及 Rust 标准库的对应立场

从 C 到 Rust:本章主题如何迁移

C 的网络编程是"初始化仪式 + 全局错误"的双重折磨:WSAStartup(Windows)/socket() 后还要 bind、listen、accept 一步步来,每步都检查负数返回值;错误码藏在 errno/WSAGetLastError 里;还得自己管理 fd 生命周期,close 漏了就 fd 泄漏。Rust 的 std::net 把这些压扁成三个类型:TcpListener(监听)、TcpStream(连接)、UdpSocket(数据报)。bind 与 listen 合为一个构造器,TcpListener::bind 返回 Result;accept 返回 (TcpStream, SocketAddr)。套接字是 RAII:离开作用域自动关闭,不存在 fd 泄漏。

但网络编程的真正难点与语言无关,反而是字节流语义:TCP 没有消息边界,你在 C 里遇到的"一次 recv 只收到半条消息""两条消息粘在一起",在 Rust 里一模一样——因为这是 TCP 的物理事实,不是 API 缺陷。Rust 能帮你的,是提供 BufReader/BufWriter 与类型化的流读写,把"按行读网络数据"写成与读文件相同的样子(第7章的知识直接复用)。本章的策略:先建立 API 对照(\$8.1–\$8.3),再攻克字节流语义(\$8.5),然后 UDP 与 HTTP(\$8.6–\$8.7)——实验做成回声服务器,亲手验证粘包与拆包。

前置知识自查

- C 的 socket 编程:socket/bind/listen/accept/connect/send/recv/close,阻塞 IO 语义
- TCP 与 UDP 的区别:连接 vs 无连接、字节流 vs 数据报、可靠性
- 第5章的线程与所有权(本章每连接一线程要用 move 闭包)
- 第7章的 BufReader/Write 与 ? 错误传播(网络读写与文件读写同一套接口)
- HTTP 基本结构:请求行/请求头/空行/正文;状态码 200/404

本章学习路线

1. **第一阶段(约1.5小时):**§8.1–§8.3,客户端与服务端的骨架——先跑通"连上、发一句、收一句"
2. **第二阶段(约1.5小时):**§8.4–§8.5,超时与字节流边界——网络编程真正的生产门槛,粘包拆包
3. **第三阶段(约1.5小时):**§8.6–§8.7,UDP 与 HTTP——覆盖生产网络编程的两个高频场景
4. **验收标准:**能徒手写多客户端回声服务器 + 行协议客户端,并解释为什么不会粘包

本章知识导图

- §8.1 socket 三件套 vs std::net 三类型
- §8.2 客户端:connect 与读写
- §8.3 服务端:bind/accept 循环与每连接一线程
- §8.4 超时与半关闭:别让连接"永久卡死"
- §8.5 粘包与拆包:TCP 字节流的正确食用方式
- §8.6 UDP:send_to 与 recv_from
- §8.7 HTTP:request 客户端与最小服务端
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§8.1 socket 三件套 vs std::net 三类型

8.1.1 C 的初始化仪式

```
// C:一个监听套接字的完整仪式
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int fd = socket(AF_INET, SOCK_STREAM, 0);          // 1. 建套接字
if (fd < 0) { perror("socket"); return -1; }

int opt = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof opt); // 2. 端口复用

struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);                      // 3. 网络字节序(大端)
addr.sin_addr.s_addr = INADDR_ANY;

if (bind(fd, (struct sockaddr *)&addr, sizeof addr) < 0) { perror("bind"); return -1; }
if (listen(fd, 128) < 0) { perror("listen"); return -1; } // 4. 开始监听

// 之后才能 accept(...) — 每一步都要查错,端口字节序要人肉 htons
```

8.1.2 Rust 的压缩形态

```
// Rust:bind + listen 一步到位,端口/地址自动处理字节序
use std::net::TcpListener;

fn main() -> std::io::Result<()> {
    // 绑定 0.0.0.0:8080(INADDR_ANY),失败即返回错误
    let listener = TcpListener::bind("0.0.0.0:8080")?;
    println!("监听中:{}", listener.local_addr()?);
    // 套接字随 listener 离开作用域自动关闭(RAII)
    Ok(())
}
```

对照:地址字符串(含端口)交给 ToSocketAddrs 解析,字节序由类型处理,没有 htons/ntohs、没有 SO_REUSEADDR 样板——标准库把"常见正确做法"做进了构造器。C 的 socket/fd 生命周期靠人记得 close,Rust 的 TcpListener/TcpStream 是 RAII 值,作用域结束即关闭。

C / POSIX	Rust std::net
socket() + setsockopt + bind + listen	TcpListener::bind("addr:port")? 一步完成
htons / ntohs 字节序转换	类型自动处理(ToSocketAddrs 解析)
accept() 返回 fd	listener.accept()? -> (TcpStream, SocketAddr)
connect()	TcpStream::connect("addr:port")?
send / recv / read / write	stream.read / write / read_exact / write_all
close(fd)	Drop 自动关闭(RAII)
errno / WSAGetLastError	io::Result 携带完整错误
fd 泄漏 / double-close	类型系统排除

§8.2 客户端:connect 与读写

```
use std::io::{Read, Write};
use std::net::TcpStream;

fn main() -> std::io::Result<()> {
    // 连接(对照 connect(2));失败信息含原因:拒绝、超时、无路由...
    let mut stream = TcpStream::connect("127.0.0.1:8080")?;
    println!("已连接:{}", stream.peer_addr()?);

    // 发送:write_all 保证整段发出(对照 send 返回值的循环检查)
    stream.write_all(b"GET / HTTP/1.1\r\nHost: 127.0.0.1\r\n\r\n")?;

    // 接收:循环读到 EOF(服务端关闭写方向后 read 返回 Ok(0))
    let mut buf = [0u8; 1024];
    let mut total = 0usize;
    loop {
        let n = stream.read(&mut buf)?;
        if n == 0 {
            break; // EOF:对端关闭
        }
        total += n;
        if let Ok(text) = std::str::from_utf8(&buf[..n]) {
            print!("{text}");
        }
    }
    println!("\n共接收 {total} 字节");
    Ok(())
}
```

对照要点:C 的 send/recv 返回值语义直接迁移

read 返回 0 = 对端关闭(EOF),负数 = 错误;write_all 封装了"部分写入要循环"的麻烦。C 程序员最常犯的"recv 只收了一半就当完整消息",在 Rust 里依然要自己处理——这正是 §8.5 的主题。唯一改变的是:错误不再是负数和 errno,而是 Result。

§8.3 服务端:bind/accept 循环与每连接一线程

```
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

/// 处理单个客户端连接:逐行读,原样写回(行协议回声)
fn handle_client(mut stream: TcpStream) -> std::io::Result<()> {
    // 读需要借用 stream,写需要可变借用:先克隆一份
    let mut reader = BufReader::new(stream.try_clone()?);

    for line in reader.lines() {
        let line = line?;
        if line == "quit" {
            break;
        }
        // 写回"回声:xxx";换行补上,保持行协议
        writeln!(stream, "回声:{line}")?;
    }
    println!("连接关闭:{}", stream.peer_addr()?);
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080")?;
    println!("服务端就绪,等待连接...");

    // 对照 C 的 while(1) { accept(); 处理; }:Rust 用迭代器 + 线程
    for stream in listener.incoming() {
        let stream = stream?;           // accept 失败:打印后继续等
        println!("新连接:{}", stream.peer_addr()?);

        // 每连接一个线程;stream 被 move 进闭包(所有权转移)
        thread::spawn(move || {
            if let Err(e) = handle_client(stream) {
                eprintln!("处理连接出错:{e}");
            }
        });
    }
    Ok(())
}
```

8.3.1 错误处理的对照

C 的 `accept` 失败(`errno=EMFILE` 等)要么终止要么靠全局 `errno` 判断;Rust 里 `stream = stream?` 一旦失败整个服务退出——生产上通常要"失败继续等"。上述代码中 `stream?` 直接传播,体现的是"教学清晰";生产模式是把 `accept` 的错误也打印后 `continue`(如下):

```
fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080");
    for stream in listener.incoming() {
        match stream {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(e) = handle_client(stream) {
                        eprintln!("处理连接出错:{e}");
                    }
                });
            }
            Err(e) => eprintln!("accept 失败(继续监听):{e}"),
        }
    }
    Ok(())
}
```

⚠ 易错点 1:同一 TcpStream 上"边读边写"要 try_clone

BufReader::new(stream) 会把 stream 的读借用拿走;再想 writeln!(stream) 就"双重借用"编译失败。标准做法:stream.try_clone() 复制一份句柄,读用克隆、写用原柄。注意 try_clone 是操作系统层面的 dup——两个句柄共享同一个连接,不违反任何规则。C 里同样可以 dup(fd),只是没人告诉你这么用。

§8.4 超时与半关闭:别让连接永久卡死

```
use std::io::{Read, Write};
use std::net::TcpStream;
use std::time::Duration;

fn main() -> std::io::Result<()> {
    let mut stream = TcpStream::connect("10.0.0.5:22")?;

    // 读超时:5 秒内无数据到达则 Err(TimedOut)(对照 SO_RCVTIMEO)
    stream.set_read_timeout(Some(Duration::from_secs(5)))?;
    // 写超时:防止对端不读导致发送缓冲区塞满卡死(对照 SO_SNDTIMEO)
    stream.set_write_timeout(Some(Duration::from_secs(5)))?;

    stream.write_all(b"hello")?;

    // 半关闭:只关写方向,还能继续读(对照 shutdown(fd, SHUT_WR))
    // 意义:通知对端"我说完了",对端读到 EOF 后可以收尾
    stream.shutdown(std::net::Shutdown::Write)?;

    let mut buf = [0u8; 256];
    loop {
        match stream.read(&mut buf) {
            Ok(0) => break, // 对端也关闭了
            Ok(n) => println!("收到:{}", String::from_utf8_lossy(&buf[..n])),
            Err(e) if e.kind() == std::io::ErrorKind::TimedOut => {
                eprintln!("读超时,放弃");
                break;
            }
        }
        Err(e) => return Err(e),
    }
}
Ok(())
}
```

超时是生产网络的生存技能:无超时的阻塞读,对端崩溃后连接会挂到系统超时(几分钟),日志里全是"僵尸连接"。任何面向不可信对端的 read,都应该有超时。C 的 SO_RCVTIMEO 语义一致,但 Rust 把超时变成了每个流的显式设置,并区分错误种类(TimedOut 可以单独捕获)。

§8.5 粘包与拆包:TCP 字节流的正确食用方式

8.5.1 问题的本质

TCP 是字节流,不是消息流。发送方 write("你好") 两次,接收方可能一次 read 就拿到 "你好你好"(粘包),也可能两次 read 各拿一半(拆包)。这是内核缓冲与分段的物理事实,任何语言都一样。C 的解决方案是"应用层协议":要么每条消息以换行结束(行协议),要么头部带长度(长度前缀)。Rust 不改变这个事实,只把工具做得更顺手。

8.5.2 方案一:行协议(文本消息)

```
// 服务端:按行读,天然拆包;写回时补 \n,天然定界
use std::io::{BufRead, BufReader, Write};
use std::net::TcpListener;

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:9000")?;
    for stream in listener.incoming() {
        let mut stream = stream?;
        let mut reader = BufReader::new(stream.try_clone()?);

        // lines() 内部处理"半行/多行"的切分—粘包拆包的第一道防线
        for line in reader.lines() {
            let line = line?;
            println!("收到消息:{}", line.trim_end());
            writeln!(stream, "已收到:{line}")?;    // 响应同样以 \n 结尾
        }
    }
    Ok(())
}
```

8.5.3 方案二:长度前缀(二进制消息)

```
use std::io::{Read, Write};
use std::net::TcpStream;

// 协议:4 字节大端长度 + 正文
fn send_frame(stream: &mut TcpStream, payload: &[u8]) -> std::io::Result<()> {
    let len = (payload.len() as u32).to_be_bytes(); // 对照 htonl
    stream.write_all(&len)?; // 长度在前
    stream.write_all(payload)?; // 正文在后
    Ok(())
}

fn recv_frame(stream: &mut TcpStream) -> std::io::Result<Vec<u8>> {
    // 读 4 字节头:read_exact 保证整帧头(拆包时自动等齐)
    let mut head = [0u8; 4];
    stream.read_exact(&mut head)?;
    let n = u32::from_be_bytes(head) as usize; // 对照 ntohl
    if n > 1024 * 1024 { // 防炸弹:超限直接拒绝
        return Err(std::io::Error::new(
            std::io::ErrorKind::InvalidData, "帧过长"));
    }
    let mut body = vec![0u8; n];
    stream.read_exact(&mut body)?; // 读齐 n 字节
    Ok(body)
}

fn main() -> std::io::Result<()> {
    let mut stream = TcpStream::connect("127.0.0.1:9000")?;
    send_frame(&mut stream, b"第一条");
    send_frame(&mut stream, b"第二条"); // 连续两条也不会粘
    let reply = recv_frame(&mut stream)?;
    println!("服务端回复:{}", String::from_utf8_lossy(&reply));
    Ok(())
}
```

⚠ 易错点 2:长度前缀不校验,是生产事故之源

帧头给出 4GB 长度,你 allocate 4GB 内存然后 read——这就是"内存炸弹"。读长度后必须立即与上限比较(如 1MB),超限直接报错或断开。这与 C 里检查 ntohl 后的长度同理,只是 C 程序员通常要吃过一次苦头才记得。

★ 重点:粘包拆包在 Rust 里的解法不变,但工具变了

拆包(半条消息)由 read_exact 与 BufReader 消化:read_exact 等齐为止,中间由 lines() 自动攒行。粘包由协议定界解决:行协议看 \n,长度前缀看头部。C 需要手写"攒缓冲 + 找边界"的循环;Rust 的 BufRead trait(第7章已见)把这个循环封装成标准接口,网络与文件共用一套——这正是"一个 io 模型打通文件与网络"的体现。

§8.6 UDP:send_to 与 recv_from

```
use std::net::UdpSocket;

fn main() -> std::io::Result<()> {
    // UDP:无连接。bind 后直接收发(对照 socket + bind + sendto/recvfrom)
    let socket = UdpSocket::bind("127.0.0.1:0"?); // 端口 0:系统分配
    socket.connect("127.0.0.1:9000"?);           // 可选:固定对端(简化 send)

    // 发送(对照 sendto):数据报原子性由系统保证,一次一报
    socket.send(b"UDP 测试包"?);

    // 接收(对照 recvfrom):recv_from 返回 (数据, 来源地址)
    let mut buf = [0u8; 2048];
    let (n, src) = socket.recv_from(&mut buf)?;
    println!("来自 {src}:{n}", String::from_utf8_lossy(&buf[..n]));
    Ok(())
}
```

```
// UDP 服务端:无 accept,循环 recv_from
use std::net::UdpSocket;

fn main() -> std::io::Result<()> {
    let socket = UdpSocket::bind("127.0.0.1:9000"?);
    let mut buf = [0u8; 2048];
    loop {
        let (n, src) = socket.recv_from(&mut buf)?;
        // 每个包独立处理:无连接、无会话(对照 DNS、NTP 等协议模型)
        println!("收到 {src} 的 {n} 字节");
        let reply = format!("回:{n}\n", String::from_utf8_lossy(&buf[..n]));
        socket.send_to(reply.as_bytes(), src)?; // 回复要带来源地址
    }
}
```

场景	TCP	UDP
连接状态	面向连接,三次握手	无连接,直接发报
边界	字节流(要协议定界)	数据报(系统保边界)
可靠性	重传/排序/流控	可能丢、可能乱序
Rust 类型	TcpStream / TcpListener	UdpSocket(收发一体)
典型用途	HTTP、数据库、长连接	DNS、NTP、实时音视频、探活

§8.7 HTTP:request 客户端与最小服务端

8.7.1 HTTP 客户端:request(阻塞版)

```
// Cargo.toml 依赖:request = { version = "0.11", features = ["blocking"] }
// 用 request 的 blocking 客户端:同步 API,与 C 的 libcurl 用法同构
use std::time::Duration;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // GET 请求(对照 curl 的最简用法)
    let resp = request::blocking::get("https://example.com/");
    println!("状态码:{}", resp.status());
    println!("响应头:content-type = {}", resp.headers()["content-type"]
        .to_str()?);

    let body = resp.text()?; // 正文作为 UTF-8 文本
    println!("正文长度:{} 字符", body.len());

    // POST + JSON(对照 libcurl 的 POST 配置)
    let client = request::blocking::Client::builder()
        .timeout(Duration::from_secs(10)) // 全局超时
        .build()?;
    let resp = client
        .post("https://httpbin.org/post")
        .json(&serde_json::json!({"name": "rust", "n": 42}))
        .send()?;
    println!("POST 状态码:{}", resp.status());
    Ok(())
}
```

8.7.2 最小 HTTP 服务端(≤40 行,零依赖)

```
// 零依赖最小 HTTP 服务端:只处理 GET,返回固定页面
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};

fn handle(mut stream: TcpStream) -> std::io::Result<()> {
    let mut reader = BufReader::new(stream.try_clone()?);
    // 请求行:GET /path HTTP/1.1(读一行即可演示)
    let mut request_line = String::new();
    reader.read_line(&mut request_line)?;
    println!("请求:{request_line.trim()}");

    // 构造响应:状态行 + 头 + 空行 + 正文
    let body = "<h1>你好,Rust 服务端!</h1>\n";
    let response = format!(
        "HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=utf-8\r\nContent-Length: {}\r\nConnection: close\r\n\r\n{}",
        body.len(), body
    );
    stream.write_all(response.as_bytes())?;
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080")?;
    println!("HTTP 服务在 http://127.0.0.1:8080/ 运行");
    for stream in listener.incoming() {
        let stream = stream?;
        std::thread::spawn(move || {
            let _ = handle(stream); // 单请求即关:Connection: close
        });
    }
    Ok(())
}
// 浏览器或 curl http://127.0.0.1:8080/ 即可看到页面
```

工程建议:生产 HTTP 服务用框架,别裸写

上面的 40 行服务端是为了看懂"HTTP 就是带格式的 TCP 字节流"。生产环境请直接上生态框架(axum、actix-web 或 tide),它们处理路由、并发、中间件、请求体解析。但理解本节仍不可替代:遇到"响应格式不对"Content-Length 不一致"的疑难杂症,能回到字节流层面推理,才是网络工程师的本事。

8.7.3 TIME_WAIT 与端口复用

C 服务端在 bind 前要 setsockopt(SO_REUSEADDR),否则服务重启时 TIME_WAIT 状态(主动关闭方在 2×MSL 内保留端口)会让 bind 失败。Rust 标准库 TcpListener::bind 不暴露这个选项——在 Unix 上它默认不设置 SO_REUSEADDR,重启服务可能碰到 AddrInUse。需要该行为时,用 socket2 库显式设置;或者把"主动关闭"留给对端(服务端先关连接时自己是主动方)。生产部署中,优雅重启(先关监听、等存量连接结束)是比 SO_REUSEADDR 更彻底的方案。

⚠ 易错点 3:对端崩溃后,read 不会立即返回

TCP 连接上对端进程崩溃,内核可能几十分钟后才发 RST;这期间的 read 会一直阻塞。生产客户端必须:① 设读写超时(\$8.4);② 需要"对端还活着吗"时用应用层心跳。C 的教训(SO_KEEPALIVE、select 超时)原样适用——区别是 Rust 把超时做成每个流的显式配置,忘了设就等着连接挂死,而错误种类(TimedOut)可以直接捕获分流。

本章速查表

主题	结论 / 写法
监听套接字	TcpListener::bind("0.0.0.0:8080")?(bind+listen 一步)
接受连接	listener.accept()? -> (TcpStream, SocketAddr);或 for s in listener.incoming()
客户端连接	TcpStream::connect("127.0.0.1:8080")?
发送	stream.write_all(bytes)?(保证全量;对照 send 循环)
接收	read 返回 Ok(0) = EOF;定长读用 read_exact
边读边写	stream.try_clone()? 后,BufReader 用克隆,写用原柄
读/写超时	set_read_timeout / set_write_timeout(Some(Duration));TimedOut 单捕
半关闭	stream.shutdown(Shutdown::Write)?(对照 shutdown(2))
粘包拆包	TCP 是字节流;行协议用 lines(),二进制用"长度前缀 + read_exact"
长度上限	读长度后立即校验上限,防内存炸弹
UDP 收发	UdpSocket::bind;send_to / recv_from -> (n, src)
HTTP 客户端	reqwest(blocking 特性):Client::builder().timeout().build()
HTTP 服务端	手写 ≤40 行(教学)或 axum/actix-web(生产)
套接字关闭	RAII:作用域结束自动关闭;无 fd 泄漏

最小必会清单

- 能写出 TcpListener::bind + incoming 循环的服务端骨架,错误全程 Result
- 能写出 TcpStream::connect + write_all + read 循环的客户端骨架,识别 EOF
- 知道边读边写要 try_clone,并能解释它为什么是 dup 而非复制数据
- 能说出 TCP 无消息边界,粘包拆包的本质;能用行协议或长度前缀解决
- 会写 read_exact 读定长帧头 + 长度校验防炸弹
- 知道读写超时的设置方法与 TimedOut 的捕获方式
- 能用 UdpSocket 收发,并说出与 TCP 在边界、可靠性上的差异
- 会用 reqwest blocking 客户端发 GET/POST,处理状态码与正文
- 能逐字段解释 HTTP 响应的"状态行 + 头 + 空行 + 正文"
- 知道 TIME_WAIT 与 AddrInUse 的关系,以及生产上的优雅重启思路

自测题

Q 1. TcpListener::bind("127.0.0.1:8080") 一步完成了 C 的哪几步?省略了什么?

✓ 参考答案

完成:socket() + bind() + listen()(以及地址解析与字节序处理)。未完成:SO_REUSEADDR 等 setsockopt 选项(标准库不暴露,需要时用 socket2);accept 循环与每连接处理(那是业务代码)。省略掉的是"必须正确的仪式",留下的是"业务可选的调优"。

Q 2. read 返回 Ok(0) 意味着什么?在 C 里对应什么?哪些场景会导致它迟迟不来?

✓ 参考答案

Ok(0) = EOF:对端已关闭连接(或 shutdown 了写方向),不会再发数据。对应 C 的 recv 返回 0。迟迟不来的场景:对端进程崩溃但连接未感知(内核还没发 RST)、对端只是不发送数据(长连接挂起)、网络故障。所以生产读循环要么有超时,要么有应用层心跳。

Q 3. 为什么 TCP 会粘包,而 UDP 不会?粘包能"关掉"吗?

✓ 参考答案

TCP 是面向字节流的:内核把多次 write 的数据按需合并/切分放进报文,接收方看到的只是字节序列,没有"消息"概念,所以多段数据可能粘在一次 read 里(粘包)或一段数据分多次 read(拆包)。UDP 以数据报为单位收发,系统保证一次 send 对应一次 recv 的边界。粘包是 TCP 的本质特性,无法关闭,只能由应用层协议定界(行协议/长度前缀)解决。

Q 4. 帧协议里,为什么读长度之后必须校验上限?

✓ 参考答案

长度字段来自网络,由对端控制。若不校验,恶意或损坏的帧头给出 4GB,程序会申请 4GB 内存并阻塞等待数据——内存炸弹与资源耗尽攻击。对策:与协议上限比较,超限立即报错/断开。这与 C 的教训一致,但 read_exact + 显式上限让防线写起来只有三行。

Q 5. 每连接一线程的服务端,线程里为什么用 move 闭包?如果不用会怎样?

✓ 参考答案

stream 在 main 的循环里创建,线程闭包要长期持有它。move 把 stream 的所有权转移进新线程——这是 Rust 保证"多线程不同时访问同一所有权"的机制(第9章详述)。不用 move,闭包只能借用循环变量,而循环会覆盖它,编译错误(borrowed value does not live long enough)——编译器强迫你做出正确选择。

Q 6. 服务端重启时报 Address already in use,原因是什么?Rust 怎么处理?

✓ 参考答案

旧进程主动关闭连接时进入 TIME_WAIT 状态(持续约 $2 \times \text{MSL}$),端口未完全释放;而 Rust 标准库的 bind 默认不设置 SO_REUSEADDR,于是新进程 bind 失败。对策:① 用 socket2 设置 SO_REUSEADDR(与 C 相同);② 优雅重启:

先关闭监听、等待存量连接自然结束再退出(生产首选);③ 改端口。

章末实验题:回声服务器与行协议客户端

实验 8:回声服务器与行协议客户端

实验目标:实现一个行协议回声系统:服务端接受多客户端,逐行读入并原样回复;客户端连接后连续发送若干行消息,校验回复。

实验要求:

- 任务 1:服务端 bind + incoming 循环,每连接一线程(覆盖:服务端骨架 §8.3)
- 任务 2:用 BufReader + lines() 读行,回复加前缀并补换行(覆盖:行协议 §8.5)
- 任务 3:客户端连接、循环发送、逐行读回并校验(覆盖:客户端 §8.2)
- 任务 4:设置读超时,测试"服务端不回复"时的超时路径(覆盖:超时 §8.4)
- 任务 5:客户端支持命令行指定"消息条数",批量发送模拟粘包压力(覆盖:粘包理解 §8.5)
- 加分:服务端统计每个连接的消息数,连接结束时打印(覆盖:每连接状态)

运行方式:两个终端:cargo run --bin server(或先启动服务端),再 cargo run --bin client -- 5(发送 5 条)。

第一步 **一步:**cargo new echo 后,建 src/bin/server.rs 与 src/bin/client.rs 两个入口

第二步 **二步:**先写服务端:bind、打印地址、incoming 循环 + 线程 + 行协议回声

第三步 **三步:**写客户端:connect、循环发送 "消息 N"、逐行读回并断言前缀

第四步 **四步:**加超时设置,故意停掉服务端,观察客户端超时退出而不是永久挂起

第五步 **五步:**把客户端改成"一次写出全部消息"(write_all 一段含多行的数据),观察服务端能否逐行还原——验证 BufReader 拆包

参考答案要点

```
// ===== 实验 8:回声服务器与行协议客户端 =====
// 覆盖知识点:服务端骨架与每连接线程(§8.3)、行协议拆包(§8.5)、
// 客户端读写(§8.2)、超时(§8.4)、粘包验证(§8.5)

// ---- src/bin/server.rs ----
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

fn handle_client(mut stream: TcpStream) -> std::io::Result<()> {
    let mut reader = BufReader::new(stream.try_clone()?);
    let mut count = 0u32;
    for line in reader.lines() {
        let line = line?;
        count += 1;
        if line == "quit" {
            break;
        }
        // 回声:原样 + 前缀;补 \n 维持行协议
        writeln!(stream, "echo:{line}")?;
    }
    // 加分:连接结束时统计
    println!("连接 {} 关闭,共处理 {} 条消息",
        stream.peer_addr()?);
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080");
    println!("回声服务端监听 {}", listener.local_addr()?);
    for stream in listener.incoming() {
        match stream {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(e) = handle_client(stream) {
                        eprintln!("处理连接出错:{e}");
                    }
                });
            }
            Err(e) => eprintln!("accept 失败(继续监听):{e}"),
        }
    }
    Ok(())
}

// ---- src/bin/client.rs ----
use std::io::{BufRead, BufReader, Write};
use std::net::TcpStream;
use std::time::Duration;

fn main() -> std::io::Result<()> {
    // 消息条数来自命令行
    let n: u32 = std::env::args()
        .nth(1).unwrap_or_else(|| "3".into())
        .parse().expect("条数必须是数字");

```

```

let mut stream = TcpStream::connect("127.0.0.1:8080");
// 任务 4:读超时 5 秒
stream.set_read_timeout(Some(Duration::from_secs(5)));
let mut reader = BufReader::new(stream.try_clone()?);

// 任务 5:批量发送——一次 write_all 含多行,验证服务端逐行还原
let mut batch = String::new();
for i in 1..=n {
    batch.push_str(&format!("消息 {i}\n"));
}
stream.write_all(batch.as_bytes())?;

// 逐行读回校验
for i in 1..=n {
    let mut line = String::new();
    match reader.read_line(&mut line) {
        Ok(0) => {
            eprintln!("服务端提前关闭(第 {i} 条)");
            std::process::exit(1);
        }
        Ok(_) => {
            let expect = format!("echo:消息 {i}");
            assert!(line.trim() == expect,
                "第 {i} 条回复不匹配:got '{}' want '{expect}'",
                line.trim());
            println!("✓ {line}", );
        }
        Err(e) if e.kind() == std::io::ErrorKind::TimedOut => {
            eprintln!("读超时:服务端无响应(第 {i} 条)");
            std::process::exit(2);
        }
        Err(e) => return Err(e),
    }
}
// 收尾:发 quit 让服务端优雅关闭该连接
stream.write_all(b"quit\n")?;
println!("全部 {n} 条校验通过");
Ok(())
}

```

设计说明:客户端一次写出整批消息,是验证"拆包"的标准手法——TCP 可能把这批数据切成任意大小的段,服务端的 `lines()` 必须逐行还原,任何一行错位都会让断言失败。把断言换成 `println` 再跑一遍,你会看到"粘包"其实从没被消除,只是被协议层正确地重建了。超时分支用 `ErrorKind::TimedOut` 单独捕获——这正是"错误是类型"在生产中的价值:C 里同一套 `errno` 要区分五六种含义,Rust 直接按 `kind` 分流。

下一章预告:第9章 并发与异常处理——C 的 `errno` 与 `pthread` 对应 Rust 的 `Result`? 与 `std::thread`; `Send`/`Sync`、`Mutex`/原子、通道与 `tokio` 异步入门。

第9章 并发与异常处理

建议学时:5-7 小时 | 难度:★★★★★ | 读者背景:熟悉 C 的 `errno` 错误处理与 `pthread` 线程(或至少了解线程概念),听说过数据竞争

本章学习目标

- 掌握错误处理三件套的分工:可恢复错误用 `Result`,不可恢复用 `panic!`,“快速传播”用 `?`(对照 `errno` 与 `abort`)
- 掌握 `std::thread` 的创建、`join` 与 `move` 闭包,对照 `pthread_create/pthread_join`
- 理解 `Send/Sync` 两个标记 `trait`:数据竞争从“运行期灾难”变成“编译期拒绝”
- 掌握 `Arc<Mutex<T>>` 组合:共享可变状态的唯一正道;理解锁中毒
- 掌握 `RwLock` 与原子类型,对照 C11 的 `atomic` 与读写锁
- 掌握 `mpsc` 通道:多生产者单消费者模型,对照 `pipe` 与消息队列
- 理解死锁的成因与预防:锁顺序、持锁作用域、无锁优先
- 掌握 `tokio` 异步入门:`async/await` 的直觉、与线程的取舍、何时用

从 C 到 Rust:本章主题如何迁移

这一章是两个世界的正面碰撞。错误处理:C 用 `errno`(全局变量,可被任意函数覆盖)、返回值(`-1 + errno`)、`abort()`。Rust 用三种显式形态:`Result`(可恢复,值是数据)、`panic!`(不可恢复,栈展开或 `abort`)、`?`(传播,一行代替三行样板)。关键不是“哪种更好”,而是“类型系统把错误分成了几类,每类各有出口”——C 里所有错误都是“负返回值 + `errno`”,于是经常忘了查。

并发:C 的 `pthread` 给你一把刀,也给你全部伤害自己的能力:共享变量不加锁是数据竞争(未定义行为)、加错锁是死锁、忘记 `join` 是悬垂。Rust 的态度是“能静态保证的,绝不留到运行期”:所有权(第5章)保证了每个值只有一个所有者,于是——数据要么单线程独享(`move` 进线程),要么多线程共享(必须 `Send/Sync` 且经互斥/原子访问)。`Send/Sync` 是两个自动推导的标记 `trait`:你的类型天然声明“我能/不能安全地跨线程传递”。写错并发代码不再是“偶尔崩溃的玄学”,而是“编译器拒绝的必然”。本章从头到尾,你要带着一个对照表看:C 的每一次并发冒险,在 Rust 里对应哪个类型、哪条规则。

前置知识自查

- C 的 `errno`、`perror`、`strerror`;函数返回 `-1` 表示失败的惯例;`abort()` 与 `exit()`
- `pthread` 的基本使用:`pthread_create/join/mutex_lock/unlock`(至少见过)
- 第5章的所有权与 `move` 语义——本章线程的 `move` 闭包直接依赖它
- 第8章的每连接一线程模式(已用过 `thread::spawn`)
- (可选)C11 的 `atomic` 头文件、生产者-消费者模型的直觉

本章学习路线

1. **第一阶段(约1.5小时):**§9.1–§9.2,错误处理三件套与线程——先跑通"多线程 + Result"
2. **第二阶段(约2小时):**§9.3–§9.4,Send/Sync、Mutex 与原子——本章最难也最值钱的部分,反复对照 C 的数据竞争
3. **第三阶段(约1.5小时):**§9.5–§9.6,通道与死锁——生产者-消费者模型落地
4. **第四阶段(约1小时):**§9.7,tokio 异步入门——知道"什么时候该换武器"
5. **验收标准:**能徒手写一个"多生产者多消费者 + 共享计数"的程序,并说出每个 Arc/Mutex/通道选择的原因

本章知识导图

- §9.1 错误处理三件套:Result / ? / panic!(vs errno)
- §9.2 线程:std::thread 与 move 闭包(vs pthread)
- §9.3 Send 与 Sync:数据竞争的编译期防线
- §9.4 共享可变状态:Arc<Mutex<T>>、RwLock 与原子
- §9.5 通道:mpsc 生产者-消费者
- §9.6 死锁:成因、识别与预防
- §9.7 tokio 异步入门:async/await
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§9.1 错误处理三件套:Result / ? / panic!(vs errno)

9.1.1 C 的 errno 之痛

```
// C:错误与返回值混在一起,errno 是全局状态
#include <errno.h>
#include <string.h>

int read_config(const char *path, char *buf, size_t n) {
    FILE *f = fopen(path, "r");
    if (f == NULL) return -1;           // 原因在 errno 里
    if (fread(buf, 1, n, f) != n) {    // 读不完整
        fclose(f);
        return -1;                     // errno 可能已被内部调用覆盖!
    }
    fclose(f);
    return 0;
}

// 调用方:if (read_config(...) == -1) { perror("config"); }
// 问题:多线程下 errno 是线程局部的还好,但"谁覆盖了 errno"永远说不清
```

errno 的三宗罪:错误与返回值分离(要查两处)、全局状态(线程内可被任何库函数改写)、错误码没有携带上下文(不知道是哪个文件、哪个操作)。

9.1.2 Rust 的三件套

```
use std::fs;

/// 可恢复错误:返回 Result。错误携带上下文(路径 + 原因),随值传递
fn read_config(path: &str) -> std::io::Result<String> {
    let text = fs::read_to_string(path)?;    // ? :失败就带着错误提前返回
    if text.trim().is_empty() {
        // 业务规则错误:也是可恢复的,用自定义错误表达
        return Err(std::io::Error::new(
            std::io::ErrorKind::InvalidData,
            format!("配置文件 {path} 为空")));
    }
    Ok(text)
}

/// 不可恢复错误:panic!。用于"前置条件被破坏,程序无法继续"
fn main() {
    let cfg = match read_config("app.toml") {
        Ok(text) => text,
        Err(e) => {
            // 顶层兜底:打印完整错误并优雅退出(对照 perror + exit(1))
            eprintln!("启动失败:{e}");
            std::process::exit(1);
        }
    };

    // 断言式 panic:不可达条件。release 构建仍检查(对照 assert(3))
    assert!(cfg.contains("server"), "配置文件缺少 server 段");
    println!("配置加载成功");
}
```

C	Rust	语义
返回 -1 + errno	Result<T, E>:Ok(T) / Err(E)	错误是值,带上下文,随调用链传递
perror / strerror	错误 Display 打印(含路径/原因)	信息完整,不猜
逐层 if (ret < 0) return -1	?	一行完成"失败即返回"
abort()	panic!(默认栈展开,可配 abort)	不可恢复;但 panic 前可运行 Drop 清理
assert(cond) 宏	assert!(cond) / assert_eq!	不变式检查,失败即 panic
全局错误码(errno)	错误随值传递	无全局状态,线程/库调用互不污染

⚠ 易错点 1:unwrap/expect 是"开发期 OK,生产期雷"

unwrap() 在 Err 时直接 panic。开发与原型随便用;生产代码在"错误可预期"的路径上(文件缺失、网络失败、解析失败)一律用 ? 或 match, unwrap 只留给"逻辑上不可能失败"的调用(如写死的字面量解析)。经验法则:release 构建里每个 unwrap 都是一个潜在的 panic 点,代码审查时逐一点名。

§9.2 线程:std::thread 与 move 闭包

```
use std::thread;
use std::time::Duration;

fn main() {
    // 对照 pthread_create:spawn 接收闭包(线程体),返回句柄
    let handle = thread::spawn(|| {
        for i in 1..=3 {
            println!("工作线程:{i}");
            thread::sleep(Duration::from_millis(200));
        }
    });

    // 主线程继续干活
    for i in 1..=3 {
        println!("主线程:{i}");
        thread::sleep(Duration::from_millis(150));
    }

    // 对照 pthread_join:等待工作线程结束;返回值是 Result
    handle.join().expect("工作线程 panic 了");

    // 带返回值的线程:闭包的最后一个表达式就是线程的"退出码"
    let calc = thread::spawn(|| 6 * 7);
    let answer = calc.join().expect("计算线程崩溃");
    println!("计算结果:{answer}");
}
```

9.2.1 move:把数据送进线程

```
use std::thread;

fn main() {
    let data = String::from("这是要给线程的数据");

    // 不 move:闭包借用 data,而 data 属于主线程—生命周期冲突,编译错误
    // let t = thread::spawn(|| println!("{data}")); // ERROR: may outlive...

    // move:把 data 的所有权转移进新线程(第5章的转移语义)
    let t = thread::spawn(move || {
        println!("线程收到:{data}"); // data 归线程所有,主线程不能再碰
    });
    // println!("{data}"); // ERROR: value borrowed after move

    t.join().unwrap();
}
```

C 的"传参给线程"靠 malloc + 指针 + 约定"谁 free";Rust 的 move 闭包把所有权转移显式化,编译器保证:要么数据在线程里,要么在创建者手里,不会两边同时悬垂。这就是"数据竞争预防"的第一道闸门。

§9.3 Send 与 Sync:数据竞争的编译期防线

9.3.1 两个标记 trait 的定义

Send:类型可以安全地转移所有权到另一个线程(TcpStream、String、Box 都是 Send)。**Sync**:类型可以安全地被多个线程同时借用(&T 可以跨线程,即 T: Sync)。普通类型自动满足二者;例外:Rc(引用计数非原子)、裸指针、RefCell 等内部可变类型是 !Send!/Sync 的典型。你几乎不用写"impl Send",编译器自动推导,但报错时,错误信息就是在告诉你"哪个类型为什么不安全"。

```
use std::thread;

fn main() {
    // Rc 不是 Send:引用计数不是原子的,跨线程转移会造成计数错乱
    let rc = std::rc::Rc::new(42);
    // thread::spawn(move || println!("{}", rc)); // ERROR: Rc<i32> cannot be sent
    // 因为 Rc 的 clone 不保证原子性,另一个线程 drop 它时计数可能已错乱

    // 换成 Arc(原子引用计数)就合法—见 §9.4
    let arc = std::sync::Arc::new(42);
    thread::spawn(move || println!("Arc 可以跨线程:{}", arc)).join().unwrap();

    println!("Send/Sync 编译期防线演示完毕");
}
```

★ 重点:C 的数据竞争 vs Rust 的数据竞争

C/C++ 里两个线程无锁读写同一变量是未定义行为——可能今天正常、明天崩溃、后天在客户现场炸。Rust 里同样的意图是编译错误:

- 共享可变数据:必须放进 Arc<Mutex<T>>(或原子),编译器强制你加锁;
- 只读共享:&T 本身就允许跨线程(Sync),因为不可变借用没有竞争;
- 独占转移:move 进线程,所有权保证互斥。

结果是:数据竞争类 bug 在 Rust 程序里出现的概率从"随机的"变成"编译期不可能的"——除非你用 unsafe(第5章已述,unsafe 是唯一后门)。

§9.4 共享可变状态:Arc<Mutex<T>>、RwLock 与原子

9.4.1 Arc + Mutex:标准组合

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // Arc:原子引用计数,多线程共享所有权(对照 shared_ptr)
    // Mutex:互斥锁,提供可变访问(对照 pthread_mutex_t)
    let counter = Arc::new(Mutex::new(0u64));
    let mut handles = vec![];

    for _ in 0..8 {
        // 每线程克隆一个 Arc 句柄(计数 +1),锁本体只有一个
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            for _ in 0..1000 {
                // lock() 返回锁保护的可变引用;作用域结束自动解锁
                let mut n = c.lock().unwrap();
                *n += 1;
            }
        })));
    }

    for h in handles {
        h.join().unwrap();
    }
    // 8 线程 × 1000 次,结果必须是 8000—无竞争
    println!("最终计数:{}", *counter.lock().unwrap());
}
```

对照 C:pthread_mutex_lock/unlock 的成对出现是"忘了 unlock 就死锁"的温床;Rust 的锁是 RAII——lock() 返回的 MutexGuard 离开作用域自动解锁,编译器保证解锁发生在所有引用之后。死锁依旧可能(§9.6),但"忘了解锁"这类 bug 被类型消灭了。

9.4.2 锁中毒:panic 留下的烂摊子

```
// 若持锁线程 panic, 锁进入"中毒"状态: 后续 lock() 返回 Err
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let m = Arc::new(Mutex::new(vec![1, 2, 3]));

    let m2 = Arc::clone(&m);
    let t = thread::spawn(move || {
        let mut v = m2.lock().unwrap();
        v.push(4);
        panic!("持锁期间崩溃");           // 解锁前 panic → 锁中毒
    });
    let _ = t.join();

    // 中毒后:unwrap 会 panic; 生产做法是接管中毒状态继续用
    let v = m.lock().unwrap_or_else(|e| e.into_inner());
    println!("数据仍在:{v:?}");           // 数据完好, 只是锁标记了中毒
}
```

9.4.3 RwLock 与原子类型

```
use std::sync::{Arc, RwLock};
use std::sync::atomic::{AtomicUsize, Ordering};

fn main() {
    // 读写锁:读多写少的配置/缓存场景(对照 pthread_rwlock)
    let config = Arc::new(RwLock::new(String::from("v1")));

    // 读:可多个线程同时 hold 读锁
    let r1 = config.read().unwrap();
    println!("读锁读取:{}", *r1);
    drop(r1); // 显式释放,再拿写锁

    // 写:独占(与 Mutex 同强度,但读路径并发)
    {
        let mut w = config.write().unwrap();
        w.push_str("-hotfix");
    }

    // 原子计数:无锁,单指令完成(对照 C11 atomic_fetch_add)
    let hits = AtomicUsize::new(0);
    let mut handles = vec![];
    for _ in 0..4 {
        let h = &hits;
        handles.push(std::thread::spawn(move || {
            for _ in 0..500 {
                h.fetch_add(1, Ordering::Relaxed); // 原子自增
            }
        }));
    }
    for h in handles { h.join().unwrap(); }
    println!("原子计数:{}", hits.load(Ordering::Relaxed)); // 2000
}
```

同步原语	C 对应	Rust 类型
互斥锁	pthread_mutex_t	std::sync::Mutex(RAI1 守卫)
读写锁	pthread_rwlock_t	std::sync::RwLock
共享所有权	shared_ptr(引用计数)	Arc(原子计数)/ Rc(单线程)
原子整数	C11 atomic_int / fetch_add	AtomicUsize / AtomicI64 等
内存序	memory_order_*	Ordering::Relaxed/AcqRel/SeqCst
条件变量	pthread_cond_t	std::sync::Condvar(少用,通道常更简单)

⚠ 易错点 2:持锁期间做 IO/长时间计算

MutexGuard 的作用域要 **尽量短**:拿到值拷贝出来就 drop 锁,再做耗时工作。持锁 sleep/网络 IO/大循环 = 所有等待线程排队 = 吞吐归零,还容易诱发死锁(\$9.6)。C 程序员常犯"锁包住整个函数",Rust 的显式作用域(花括号)把"锁的存活范围"写进代码,审查时一目了然。

§9.5 通道:mpsc 生产者-消费者

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    // mpsc:multi-producer single-consumer 多生产者单消费者
    let (tx, rx) = mpsc::channel();

    // 生产者 1
    let tx1 = tx.clone(); // Sender 可以克隆(多生产者)
    thread::spawn(move || {
        for i in 1..=3 {
            tx1.send(format!("生产者1-第{i}条")).unwrap();
            thread::sleep(Duration::from_millis(50));
        }
    });

    // 生产者 2(原 tx 移入)
    thread::spawn(move || {
        for i in 1..=2 {
            tx.send(format!("生产者2-第{i}条")).unwrap();
        }
    });

    // 注意:两个 sender 都 drop 后,消费者 recv 返回 Err(通道关闭)

    // 消费者:主线程收,阻塞直到有消息或通道关闭
    while let Ok(msg) = rx.recv() {
        println!("消费:{msg}");
    }
    println!("所有生产者已关闭通道");
}
```

对照 C:这等价"线程安全队列 + 条件变量"的手写组合;mpsc 把生产者侧做成可克隆的 Sender,消费者侧 recv 阻塞、try_recv 非阻塞、全部 Sender 关闭后 recv 返回 Err——这个"优雅关闭"信号在 C 里要自己实现。send 返回 Result:接收端已关闭时 send 失败(Err),这正是"通道对方死了,别再写"的类型化表达。

§9.6 死锁:成因、识别与预防

```
// 经典死锁:两个线程互等对方持有的锁
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let a = Arc::new(Mutex::new(0u32));
    let b = Arc::new(Mutex::new(0u32));

    let a1 = Arc::clone(&a); let b1 = Arc::clone(&b);
    let t1 = thread::spawn(move || {
        let _ga = a1.lock().unwrap(); // 先拿 a
        thread::sleep(std::time::Duration::from_millis(10));
        let _gb = b1.lock().unwrap(); // 再要 b → 可能等死
    });

    let a2 = Arc::clone(&a); let b2 = Arc::clone(&b);
    let t2 = thread::spawn(move || {
        let _gb = b2.lock().unwrap(); // 先拿 b
        thread::sleep(std::time::Duration::from_millis(10));
        let _ga = a2.lock().unwrap(); // 再要 a → 与 t1 互相等待
    });

    // 上面两个线程大概率互卡(锁顺序不一致)
    t1.join().unwrap();
    t2.join().unwrap();
    println!("死锁演示(若程序挂住,说明互等发生)");
}
```

★ 重点:死锁预防三原则

- **锁顺序一致**:多把锁时,所有线程按同一全局顺序加锁(如按地址排序)——本示例的修复就是 t1/t2 都先拿 a 再拿 b;
- **持锁范围最小化**:拿锁、拷贝数据、放锁,再处理业务(§9.4 易错点 2);
- **避免持锁等待通道**:recv 阻塞在持锁状态下 = 互相等待的死锁配方;需要同步时,考虑"锁外收发,锁内只改状态"。

工具层面:lock() 返回的守卫无法跨作用域逃逸(类型系统),所以"忘了解锁"被消灭;剩余死锁主要来自"多锁顺序"与"持锁阻塞",这两类要靠设计与审查——Rust 不保证消灭,但把范围缩小到可推理的境地。

§9.7 tokio 异步入门:async/await

9.7.1 为什么需要异步

每连接一线程的模型,连接数上万时线程开销(每线程约 1MB 栈 + 调度)成为瓶颈。异步编程:单个线程上跑大量"可挂起任务",IO 等待时让出线程给别的任务。C 的 epoll 事件循环 + 状态机是同一思路,但状态机要手写;Rust 的 **async/await** 让编译器生成状态机。tokio 是事实标准运行时:

```
// Cargo.toml:tokio = { version = "1", features = ["full"] }
// 异步函数:执行到 .await 时若未就绪,让出线程,稍后继续
async fn fetch_page(url: &str) -> Result<String, reqwest::Error> {
    let body = reqwest::get(url).await?.text().await?;
    Ok(body)
}

#[tokio::main]                                // 运行时入口宏
async fn main() {
    // 同时发起多个任务,共享少量线程(对照 epoll + 事件驱动)
    let urls = ["https://example.com", "https://example.org"];
    let mut handles = vec![];
    for u in urls {
        // tokio::spawn:并发执行异步任务
        handles.push(tokio::spawn(async move {
            match fetch_page(u).await {
                Ok(body) => println!("{}", u, body.len()),
                Err(e) => eprintln!("{}", e, u),
            }
        }));
    }
    for h in handles {
        let _ = h.await;                // 等待任务完成
    }
    println!("全部完成");
}
```

9.7.2 线程 vs 异步:选型准则

维度	std::thread	tokio async
并发规模	数百至数千(每线程 1MB 栈)	数万至数十万(任务为堆对象)
代码形态	同步调用,自然阅读	async/await,传染性(调用链全异步)
CPU 密集任务	天然并行(多核多线程)	需 spawn_blocking 送线程池
IO 密集任务	线程等待时占用栈与调度	等待时零占用,吞吐高
适用场景	脚本、小规模、CPU 计算	高并发服务端、网关、代理

入门建议:先线程,后异步

本章的线程知识是异步的地基:async 任务同样遵循所有权、Send/Sync(await 的任务必须是 Send 才能跨线程调度)、共享状态同样用 Arc<Mutex>。生产高并发服务直接上 tokio 生态(axum 等);脚本与小工具用 std::thread 更简单。不要“为了异步而异步”——单机小工具用线程,是工程判断力。

易错点 3:Mutex 保护"数据",不是"代码段"

持锁时间越长,竞争越大,死锁风险越高(\$9.6)。正确姿势:锁内只做“取出/放入数据”的最小操作,计算与 IO 全在锁外。对照 C 的“一把大锁包住整个函数”的坏习惯——Rust 的守卫作用域是显式花括号,把“锁的存活范围”变成可见的代码结构,审查时一眼能看出锁内有没有多余操作。

本章速查表

主题	结论 / 写法
可恢复错误	<code>fn f() -> Result<T, E>; Ok(value) / Err(e)</code>
快速传播	表达式后加 <code>? :Err</code> 则提前返回(需要返回类型是 <code>Result</code>)
不可恢复	<code>panic!("消息") / assert!(cond) / unreachable!()</code>
顶层兜底	<code>main</code> 返回 <code>Result</code> 或 <code>match</code> 后 <code>eprintln + process::exit</code>
创建线程	<code>thread::spawn(move {...}) -> JoinHandle</code>
等待线程	<code>handle.join() -> Result</code> (线程 <code>panic</code> 时为 <code>Err</code>)
Send / Sync	自动推导; <code>Rc</code> /裸指针/ <code>RefCell</code> 不满足; <code>Arc</code> / <code>String</code> / <code>&T</code> 满足
共享可变	<code>Arc<Mutex<T>>::clone</code> 句柄, <code>lock()</code> 得守卫,作用域结束自动解锁
锁中毒	持锁 <code>panic</code> 后 <code>lock()</code> 返回 <code>Err</code> ; <code>unwrap_or_else(e e.into_inner())</code> 接管
读写锁	<code>RwLock::read()</code> 并发读, <code>write()</code> 独占写(读多写少)
原子	<code>AtomicUsize::new / fetch_add / load(Ordering)</code>
通道	<code>mpsc::channel() -> (Sender, Receiver)</code> ; <code>Sender</code> 可 <code>clone</code> ; <code>recv</code> 阻塞
通道关闭	全部 <code>Sender drop</code> 后 <code>recv</code> 返回 <code>Err</code> ; <code>send</code> 到已关闭通道也 <code>Err</code>
死锁预防	锁顺序一致;持锁作用域最小;避免持锁 <code>recv</code> /长 <code>IO</code>
异步	<code>#[tokio::main] + async fn + .await</code> ; <code>tokio::spawn(async move {})</code>

最小必会清单

- 能说出 `Result` / `?` / `panic!` 的分工,并解释为什么它们取代了 `errno + abort`
- 知道生产代码 `unwrap` 的边界:只留给"逻辑上不可能失败"的调用
- 能写 `thread::spawn(move || ...) + join`,并解释 `move` 的必要性
- 能解释 `Send` 与 `Sync` 的差别(转移所有权 vs 跨线程借用),各举一个反例类型
- 能徒手写 `Arc<Mutex<T>>` 的计数器模式,并说出 `RAII` 自动解锁的意义
- 知道锁中毒是什么、怎么接管;知道 `RwLock` 与 `Mutex` 的适用场景
- 能用 `AtomicUsize` 写无锁计数,知道 `Ordering` 至少认识 `Relaxed/SeqCst`
- 能用 `mpsc` 写生产者-消费者,理解通道关闭信号(`recv Err`)
- 能说出死锁三大成因(锁顺序/持锁阻塞/互等)与预防手段
- 能说出线程与 `tokio` 的选型准则,并能写一个最小 `async main`

自测题

Q 1. 为什么 `errno` 不适合生产错误处理? `Rust` 的 `Result` 是怎么解决这些问题的?

 参考答案

`errno` 是全局状态:任何库函数都可能改写它,错误与返回值分离导致"查漏",多线程虽线程局部但无法携带上下文(哪个文件、哪个操作)。Result 把错误作为值随调用链显式传递:不会静默覆盖;错误携带路径/原因;? 让传播写成一行的同时保留完整信息;类型系统强制调用方处理(不处理就编译警告)。

Q 2. panic! 与 abort() 有什么本质区别?什么场景宁可 panic?

✓ 参考答案

`abort` 立即终止,不执行任何清理;`panic` 默认"栈展开",沿途执行 `Drop`(文件句柄、锁守卫、缓冲区都会被释放),可配置为 `abort(panic = "abort"` 剖面)。宁可 `panic` 的场景:前置条件被破坏且无法恢复(配置非法、状态机进入不可能分支、断言失败)——此时快速失败优于带着坏状态继续跑。生产上"外部输入错误"应走 `Result`, "内部逻辑错误"走 `panic`。

Q 3. 为什么 Rc 不能跨线程,而 Arc 可以?编译错误提示了什么?

✓ 参考答案

`Rc` 的引用计数增减不是原子操作:两个线程同时 `clone/drop` 会错乱计数,造成双重释放或提前释放(未定义行为)。`Arc` 的计数用原子指令,保证跨线程安全。编译错误 "`Rc<i32> cannot be sent between threads safely`" 直接点明类型与原因——这正是 `Send/Sync` 自动推导的价值:`C` 里同样的错误要等运行期崩溃,而且崩溃点与根源相距甚远。

Q 4. Arc<Mutex<T>> 与"全局 static mut + pthread_mutex"相比,结构上强在哪?

✓ 参考答案

三点:① 生命周期:锁与数据绑定在一个类型里,访问必须经过 `lock()`,编译器防止"绕过锁直接读写数据"(`C` 里 `static mut` 可以被直接访问,锁形同虚设);② 解锁保证:`MutexGuard RAII`,离开作用域自动解锁,不存在忘解锁的死锁;③ 所有权:`Arc` 句柄的 `clone/转移` 受所有权规则管理,不会出现悬垂指针指向已释放的锁。

Q 5. mpsc 的 recv 返回 Err 意味着什么?这个信号在 C 的手写队列里怎么表达?

✓ 参考答案

意味着所有 `Sender` 已被 `drop`(或发送失败被忽略后),不会再有任何消息到达——通道"优雅关闭"。`C` 的手写队列需要自己实现"关闭标志 + 条件变量广播",消费者醒来后还要检查标志并区分"队列空"与"队列关闭"两种状态;`mpsc` 用 `Err` 一个类型把这两种情况区分开,生产者侧 `send` 的 `Err` 则表达"接收端已消失,别再写"。

Q 6. 一个高并发 HTTP 服务,什么时候该用 tokio,什么时候用 std::thread 就够了?

✓ 参考答案

取决于并发规模与任务类型:数千并发连接、大量 `IO` 等待(网络/磁盘/下游调用)的场景,`tokio` 的"任务让出线程"带来数量级优势;`CPU` 密集或数百并发以内的服务,`std::thread` 每连接一线程简单直接。混合形态:`tokio` 服务里 `CPU` 密集段用 `spawn_blocking` 送回线程池。判断口诀:`IO` 密集 + 大规模并发 = 异步;`CPU` 密集 + 小规模 = 线程。

章末实验题:生产者-消费者任务队列

实验 9:生产者-消费者任务队列

实验目标:实现一个任务队列系统:多个生产者向通道投放任务,固定数量的 worker 线程消费并处理,共享计数器统计结果,主线程汇总。

实验要求:

- 任务 1:定义 Task 类型(编号 + 载荷),生产者线程循环投放(覆盖:线程 §9.2、通道 §9.5)
- 任务 2:固定 4 个 worker 线程,从接收端 recv 取任务并"处理"(sleep 模拟耗时)(覆盖:消费者模式 §9.5)
- 任务 3:Arc<AtomicUsize> 共享"已完成数"与"错误数"计数(覆盖:原子 §9.4)
- 任务 4:所有生产结束后,主线程 drop 自己的 Sender 使通道关闭,worker 感知 Err 退出(覆盖:通道关闭 §9.5)
- 任务 5:主线程 join 全部 worker,输出汇总(覆盖:join §9.2)
- 加分:任务内容为表达式字符串时,解析失败的任务计入错误数而不是崩溃(覆盖:Result 错误处理 §9.1)

运行方式:cargo run;预期输出 4 个 worker 各自处理的条数与总数、错误数。

第一步 **一步:**设计 Task 与结果类型,确定生产者数量、任务总数、worker 数量为常量

第二步 **二步:**建通道,生产者线程 move 各自的 Sender 克隆并投放

第三步 **三步:**写 worker:loop 中 recv,Err 时 break(通道关闭)

第四步 **四步:**加 Arc<AtomicUsize> 计数,处理每条任务后 fetch_add

第五步 **五步:**主线程 drop tx 后 join 全部 worker,打印汇总,验证多次运行计数一致

参考答案要点

```

// ===== 实验 9:生产者-消费者任务队列 =====
// 覆盖知识点:线程与 move($9.2)、mpsc 通道与优雅关闭($9.5)、
//          原子计数($9.4)、join 汇总($9.2)、Result 错误处理($9.1)

use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::mpsc::{self, Receiver, Sender};
use std::sync::Arc;
use std::thread;
use std::time::Duration;

const PRODUCERS: usize = 3;      // 生产者数量
const TASKS_PER_PRODUCER: u32 = 5; // 每生产者任务数
const WORKERS: usize = 4;        // 消费者(worker)数量

/// 任务:编号 + 载荷表达式
struct Task {
    id: u32,
    expr: String,
}

/// 处理一个任务:解析"a+b"并返回结果(失败计入错误,不崩溃)
fn process(task: &Task) -> Result<i32, String> {
    // 简单格式:"数字+数字"
    let parts: Vec<&str> = task.expr.split('+').collect();
    if parts.len() != 2 {
        return Err(format!("任务 {} 格式错误:{}", task.id, task.expr));
    }
    let a: i32 = parts[0].trim().parse()
        .map_err(|_| format!("任务 {} 左操作数非法", task.id))?;
    let b: i32 = parts[1].trim().parse()
        .map_err(|_| format!("任务 {} 右操作数非法", task.id))?;
    Ok(a + b)
}

/// 生产者:连续投放任务,随后 drop 自己的 Sender
fn producer(tx: Sender<Task>, id: usize) {
    for i in 0..TASKS_PER_PRODUCER {
        let task = Task {
            id: id as u32 * 100 + i,
            expr: format!("{}", id * 7 + i as usize, i + 1),
        };
        // send 失败 = 接收端已关闭:直接返回(优雅终止)
        if tx.send(task).is_err() {
            eprintln!("生产者 {}:通道已关闭,停止", id);
            return;
        }
    }
}

/// 消费者:循环取任务,Err 表示通道关闭,统计后退出
fn worker(rx: Receiver<Task>, done: &AtomicUsize, failed: &AtomicUsize) {
    loop {
        match rx.recv() {
            Ok(task) => {
                // 模拟处理耗时(对照真实 IO/计算)

```

```

        thread::sleep(Duration::from_millis(10));
        match process(&task) {
            Ok(result) => {
                done.fetch_add(1, Ordering::Relaxed);
                println!("任务 {}:{} = {}", task.id, task.expr, result);
            }
            Err(msg) => {
                failed.fetch_add(1, Ordering::Relaxed);
                eprintln!("{msg}");
            }
        }
    }
    Err(_) => break,    // 所有生产者已关闭:通道结束
}
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let (tx, rx) = mpsc::channel::<Task>();
    let done = Arc::new(AtomicUsize::new(0));
    let failed = Arc::new(AtomicUsize::new(0));

    // 启动 workers:每个 worker 一个接收端副本(Receiver 不可克隆,
    // 所以用 Arc<Receiver> 共享 — 生产上更常用多通道分发)
    let rx = Arc::new(rx);
    let mut worker_handles = vec![];
    for _ in 0..WORKERS {
        let rx = Arc::clone(&rx);
        let done = Arc::clone(&done);
        let failed = Arc::clone(&failed);
        worker_handles.push(thread::spawn(move || {
            worker((*rx).clone(), &done, &failed);
        }));
    }

    // 启动生产者
    let mut prod_handles = vec![];
    for id in 0..PRODUCERS {
        let tx = tx.clone();
        prod_handles.push(thread::spawn(move || producer(tx, id)));
    }
    // 主线程持有最后的 tx:立即 drop,使全部 sender 关闭
    drop(tx);
    for h in prod_handles {
        h.join().expect("生产者崩溃");
    }
    // 生产者全部退出后,所有 Sender 已 drop → worker 的 recv 返回 Err
    for h in worker_handles {
        h.join().expect("worker 崩溃");
    }

    let total = done.load(Ordering::Relaxed);
    let bad = failed.load(Ordering::Relaxed);
    println!("\n=== 汇总 ===");
    println!("成功:{total} 失败:{bad} 总任务:{}",

```



```
        PRODUCERS * TASKS_PER_PRODUCER as usize);
    if total + bad == PRODUCERS * TASKS_PER_PRODUCER as usize {
        println!("✓ 任务全部处理完毕,计数一致");
    } else {
        println!("X 计数不一致!丢失任务");
    }
    Ok(())
}
```

设计说明:三个关键点值得反复咀嚼——① `drop(tx)` 是"优雅关闭"的开关:主线程作为最后的 `Sender` 持有者主动放弃,workers 集体收到 `Err` 退出,没有任何标志位和广播;② `process` 返回 `Result`,坏任务计入 `failed` 而不是让 worker 崩溃——这是 §9.1 "可恢复错误走 `Result`"的生产形态;③ 计数用原子而非 `Mutex`,因为"自增"是单条指令——选型时先想"这个操作需要多步吗",单步就用原子。把 `done` 的类型换成 `Mutex` 再跑一遍,感受两种写法的差异,你就完成了本章的最后一课。

下一章预告:第10章 泛型与元编程——C 的宏与 `void*` 对应 Rust 的泛型系统;trait bound、关联类型、const 泛型、单态化,以及 `macro_rules!` 与过程宏的元编程世界。

第10章 泛型与元编程

建议学时:5-7 小时 | **难度:**★★★★★ | **读者背景:**熟悉 C 的 #define 宏、void* 与函数指针,了解 C++ 模板或 Java 泛型(任一即可,不了解也不影响)

🎯 本章学习目标

- 回顾 C 的"泛型"两大困境:预处理宏的文本替换之坑与 void* 的类型丢失
- 掌握泛型函数与泛型结构体,理解 **单态化**:与 C++ 模板相同的"按类型展开"机制,零运行时开销
- 掌握 trait bound 与 where 子句:把"任意类型"变成"满足契约的类型"
- 掌握关联类型(iterator 的 Item)与泛型参数的选型直觉
- 掌握 const 泛型:把"数组长度"变成类型的一部分,对照 C 的数组衰减
- 掌握 macro_rules! 声明式宏:模式匹配 + 代码生成,理解 vec! 等宏的原理
- 了解过程宏三兄弟(派生/属性/函数宏)与 #[derive] 的编译期原理
- 能在"泛型"与"宏"之间正确选型:类型级复用走泛型,语法级生成走宏

💡 从 C 到 Rust:本章主题如何迁移

C 程序员对"代码复用"有两件武器,都有代价。**宏(#define)**:文本替换,不检查类型,宏参数里出现表达式就踩"双重求值""优先级翻转"的坑,调试时看到的是展开后的天书。**void* + 函数指针**:把类型擦掉,换来通用,代价是丢类型安全——栈可以装任意东西,但取出时要人肉转回正确类型,转错就是 UB。C++ 用模板解决了类型问题,代价是复杂的模板语法与编译错误的天书。

Rust 的泛型把 C++ 模板的"零开销 + 编译期展开"保留下来,把语法与错误信息做成现代形态: `fn f<T: Trait>(x: T)` 读起来像函数,约束写在类型参数上,编译器错误指向具体位置。而宏依然存在(`macro_rules!` 与过程宏),但定位完全不同:宏不再做"假装泛型"的事(那是泛型的活),而是做 **语法层面的代码生成**——`println!` 的格式化、`vec!` 的初始化、`#[derive(Debug)]` 的样板生成。一句话记住本章的分工:**泛型处理"类型参数化",宏处理"语法生成"**。

📦 前置知识自查

- C 的 #define 宏(带参宏)、void*、函数指针、数组参数衰减(数组退化为指针)
- 第6章的 trait 与 trait bound(本章的 <T: Trait> 直接复用)
- 第7章的 serde derive 用法(已接触"宏生成的 impl")
- 第9章的 Result(实验题需要)
- (可选)C++ 模板的"编译期展开"概念,用于对照单态化

🚀 本章学习路线

1. **第一阶段(约1.5小时)**:§10.1–§10.3,从 C 的困境到泛型函数/结构体/trait bound——先会"写约束"
2. **第二阶段(约1.5小时)**:§10.4–§10.5,关联类型与 const 泛型——两个容易混淆的进阶特性
3. **第三阶段(约2小时)**:§10.6–§10.7,声明式宏与过程宏——元编程的世界,理解 derive 原理
4. **验收标准**:能徒手写泛型 Stack + 一个自用的 macro_rules!,并能解释"为什么这里用泛型、那里用宏"

本章知识导图

- §10.1 C 的泛型困境:宏与 void* 的代价
- §10.2 泛型函数与泛型结构体:单态化
- §10.3 trait bound 与 where:约束的力量
- §10.4 关联类型:Iterator 的 Item
- §10.5 const 泛型:长度成为类型
- §10.6 macro_rules! 声明式宏
- §10.7 过程宏与 #[derive] 的原理
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§10.1 C 的泛型困境:宏与 void* 的代价

10.1.1 宏:文本替换的灾难

```
// C:用宏"假装泛型"
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int x = MAX(3, 4);           // 看起来正常
int y = MAX(++i, j);         // 爆炸:i 被求值两次!
double z = MAX(1.5, 2);      // 类型不匹配:静默转换

// 更糟:宏没有类型,定义 MAX 时不检查 a/b 是否为数值
#define SWAP(a, b) do { typeof(a) t = (a); (a) = (b); (b) = t; } while (0)
// typeof 是 GNU 扩展,标准 C 根本写不了通用 SWAP
```

10.1.2 void*:擦掉类型的代价

```
// C:void* 栈—装得下一切,也认不出一切
struct Stack { void **data; int top; int cap; };

void stack_push(struct Stack *s, void *v) { /* ... */ }
void *stack_pop(struct Stack *s) { /* ... */ }

// 使用方必须记住"我装的是什么类型",转错就是 UB
struct Point *p = stack_pop(s);           // 若实际装的是 int*—灾难
// 取出时的类型转换没有任何检查,全靠约定
```

两个方案的共同问题: **编译器无法帮你检查"类型契约"**。宏在编译前替换,void* 在编译后消失。Rust 的泛型在"编译期、类型检查之后"展开——既保留类型信息,又生成专用代码。

C 方案	问题	Rust 对应
#define 宏模拟泛型	文本替换、双重求值、无类型检查	泛型 + 单态化(编译期展开,类型全程在场)
void* 通用容器	取出时类型丢失,转错 UB	Vec<T> / 泛型结构体(类型在签名里)
函数指针 + 回调	签名不携带类型约束	trait bound + 闭包(第4/6章)
宏生成重复代码	无法调试、无作用域	macro_rules! / 过程宏(语法级生成,可见可调)

§10.2 泛型函数与泛型结构体:单态化

10.2.1 泛型函数

```
// 一个函数,适用于"任何类型 T"(没有约束时 T 可以做任何类型)
fn first<T>(list: &[amp;T]) -> &T {
    &list[0]
}

fn main() {
    let nums = vec![10, 20, 30];
    let words = vec!["a", "b"];
    println!("{}", first(&nums));    // T = i32
    println!("{}", first(&words));  // T = &str — 同一函数两种化身
}
```

10.2.2 泛型结构体:类型安全的通用容器

```
// 泛型栈:内部是 Vec,对外是栈语义(对照 C 的 void* 栈)
struct Stack<T> {
    items: Vec<T>,
}

impl<T> Stack<T> {
    fn new() -> Stack<T> {
        Stack { items: Vec::new() }
    }

    fn push(&mut self, v: T) {
        self.items.push(v);
    }

    fn pop(&mut self) -> Option<T> {
        self.items.pop()           // 空栈返回 None,不用哨兵
    }

    fn is_empty(&self) -> bool {
        self.items.is_empty()
    }
}

fn main() {
    let mut ints: Stack<i32> = Stack::new();
    ints.push(1);
    ints.push(2);
    println!("{}", ints.pop().unwrap());

    let mut names: Stack<String> = Stack::new();
    names.push(String::from("张三"));
    // names.push(42);    // ERROR:i32 塞不进 Stack<String> — 类型安全!
}
```

10.2.3 单态化:编译期展开,零开销

单态化(monomorphization):编译器为每个实际用到的 T 生成一份专用代码。`Stack<i32>` 与 `Stack<String>` 编译成两份完全独立的实现,调用处直接内联跳转——与手写两份专用结构体性能完全相同,零虚表、零间接调用。这正是 C++ 模板的机制;Rust 的改进是约束清晰、错误可读。代价:代码体积随"类型 × 函数"组合增长(与 C++ 相同),极端场景用 `&dyn Trait`(第6章)换取单份代码。

§10.3 trait bound 与 where:约束的力量

10.3.1 无约束泛型的边界

```
// 错误:不是所有 T 都能比较(第2章:无隐式强转、无默认比较)
// fn largest<T>(list: &[T]) -> T { let mut m = list[0]; ... m > other ... }
// ERROR: binary operation `>` cannot be applied to type `T`

// 正确:要求 T 可比较(PartialOrd)且可复制(Copy,避免移动语义纠缠)
fn largest<T: PartialOrd + Copy>(list: &[T]) -> T {
    let mut max = list[0];
    for &item in list.iter() {
        if item > max {
            max = item;
        }
    }
    max
}

fn main() {
    let nums = vec![3, 9, 5, 7];
    println!("最大:{}", largest(&nums));           // 9
    let chars = vec!['a', 'z', 'k'];
    println!("最大:{}", largest(&chars));           // z
    // largest(&vec![true, false]);                 // bool 没实现 PartialOrd? 有,但顺序出乎意料——约束已经把"能做
    什么"说清楚了
}
```

10.3.2 where 子句:复杂约束的可读写法

```
// 多个约束挤在尖括号里难读;生产惯例:签名简洁,约束挪到 where
fn summarize<T, U>(a: &T, b: &U) -> String
where
    T: std::fmt::Display,
    U: std::fmt::Display + std::fmt::Debug,
{
    format!("{}", vs {:?})", a, b)
}

fn main() {
    println!("{}", summarize(&42, &"x"));           // 42 vs "x"
}
```

⚠ 易错点 1:约束过多,泛型退化成"脆弱的接口"

约束是契约,也是负担:每加一个 bound,调用方能传的类型就少一类。生产经验:先写最少约束,报错时再加——编译器会告诉你"T 需要实现哪个 trait",照着补即可。反之,为了一处小需求给整个函数挂 5 个 bound,会让函数失去复用价值。

§10.4 关联类型:Iterator 的 Item

10.4.1 为什么 Iterator 不用泛型参数

```
// 标准库 Iterator 的简化定义:Item 是"关联类型"
pub trait Iterator {
    type Item;                                // 关联类型:一个 trait 一个"产出类型"
    fn next(&mut self) -> Option<Self::Item>;
}

// 实现者指定 Item;调用方直接使用
struct Counter { n: u32 }
impl Iterator for Counter {
    type Item = u32;                          // 承诺:产出 u32
    fn next(&mut self) -> Option<Self::Item> {
        self.n += 1;
        if self.n < 5 { Some(self.n) } else { None }
    }
}

fn main() {
    let mut c = Counter { n: 0 };
    for x in c.by_ref().take(3) {             // by_ref:避免消耗计数器
        println!("{}", x);
    }
    // 所有迭代器适配器(sum/filter/map)无需重复写明 Item
    let total: u32 = Counter { n: 0 }.sum();
    println!("总和:{total}");                // 1+2+3+4 = 10
}
```

★ 重点:泛型参数 vs 关联类型,怎么选

同一个 trait 的多个实现,"产出类型"应该用关联类型(Iterator 的 Item);"输入类型"用泛型参数(如第6章的 `Add<Rhs>`——加法可以对接不同类型的右操作数)。直觉:一种类型只产一种 Item,但可以加多种类型的数。一个实现对应"唯一产出"→关联类型;一个实现对应"多组输入组合"→泛型参数。

§10.5 const 泛型:长度成为类型

10.5.1 C 的数组衰减

```
// C:数组当参数就退化成指针,长度信息丢失
int sum_arr(int arr[], int n);               // n 必须靠人传,传错没人管
// int sum_arr(int arr[10]);                 // 写 10 也没用:编译器忽略!
// 想要"长度也类型安全"?C 没有答案(除非宏或模板)
```

10.5.2 const 泛型:长度是类型的一部分

```
// N 是"值参数",在类型层面约束:[u32; N] 长度与 N 绑定
fn sum_arr<const N: usize>(arr: [u32; N]) -> u32 {
    let mut total = 0;
    for &x in arr.iter() {
        total += x;
    }
    total
}

fn main() {
    let a = [1, 2, 3];
    let b = [4, 5, 6, 7, 8];
    println!("{}", sum_arr(a), sum_arr(b));    // N 自动推导 3 和 5

    // 长度错配是编译错误,不是运行期约定
    // let c: [u32; 4] = [1, 2, 3];    // ERROR:长度不符
}
```

对照:泛型参数是"类型",const 泛型是"值(且必须是编译期常量)". 典型用途:固定大小缓冲区、硬件寄存器数组、编译期查表——C 里"宏定义长度"的不变量,在 Rust 里成为类型系统的一部分,长度错配直接编译失败。

§10.6 macro_rules! 声明式宏

10.6.1 模式匹配驱动的代码生成

```
// vec![1, 2, 3] 的真身:标准库宏。
// 声明式宏 = 模式匹配 + 替换,工作方式类似"按语法树匹配的模板"

// 自己实现一个 mini_vec!:匹配"零个或多个表达式,逗号分隔"
macro_rules! mini_vec {
    // 模式:$( $x:expr ),* — 任意多个 expr,用逗号分隔
    ($( $x:expr ),*) => {{
        let mut v = Vec::new();
        $( v.push($x); )*           // 对每个匹配项生成一条 push
        v
    }};
    // 第二个分支:尾部逗号的情况 [1, 2,]
    ($( $x:expr ),* ,) => {
        mini_vec! [ $( $x ),* ]    // 宏内可递归调用
    };
}

fn main() {
    let v = mini_vec![10, 20, 30];    // 展开成 push 三条
    println!("{:?}", v);
    let w = mini_vec![1, 2,];        // 尾逗号也接受
    println!("{:?}", w);
}
```


10.6.2 用宏消灭重复样板

```
// 生产场景:为多个错误变体生成一致的 Display 实现
macro_rules! impl_error {
    ($name:ident, $msg:literal) => {
        impl std::fmt::Display for $name {
            fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
                write!(f, $msg)
            }
        }
        impl std::error::Error for $name {}
    };
}

#[derive(Debug)]
struct TimeoutError;
#[derive(Debug)]
struct BadInputError;

impl_error!(TimeoutError, "操作超时");
impl_error!(BadInputError, "输入非法");

fn main() {
    println!("{}", TimeoutError);    // 操作超时
    println!("{}", BadInputError);  // 输入非法
}
```

⚠ 易错点 2:宏不是函数——优先级、卫生性与调试

宏的展开是"语法树替换"而非"函数调用":参数求值时机由你的模式决定(避免 C 宏的双重求值:每个 `$x` 只展开一次,天然卫生);宏引入的标识符在展开处可见(卫生性限制:局部变量用 `$$` 或小心命名)。调试用 `cargo expand` 看展开结果。规则:能用函数/泛型解决的,别用宏;宏只用于"语法必须变"的场合。

§10.7 过程宏与 `#[derive]` 的原理

10.7.1 过程宏三兄弟

种类	形态	例子
派生宏(derive)	<code>#[derive(Trait)]</code> 生成 impl	<code>#[derive(Debug, Clone, PartialEq)]</code>
属性宏	<code>#[attr]</code> 改写所在条目	<code>#[test]</code> 生成测试入口、 <code>#[tokio::main]</code>
函数宏	名字像函数,生成代码	<code>println!</code> 、 <code>serde_json::json!</code>

10.7.2 #[derive(Debug, Clone)] 到底做了什么

```
// 你写的:
#[derive(Debug, Clone, Copy)]
struct Point { x: f64, y: f64 }

// 编译器看到的(示意):derive 宏读结构体定义,生成样板 impl
// impl std::fmt::Debug for Point {
//     fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
//         f.debug_struct("Point")
//             .field("x", &self.x)
//             .field("y", &self.y)
//             .finish()
//     }
// }
// impl Clone for Point { fn clone(&self) -> Self { *self } }
// impl Copy for Point {}

fn main() {
    let p = Point { x: 1.0, y: 2.0 };
    println!("{:?}", p);           // Point { x: 1.0, y: 2.0 }
    let q = p;                     // Copy:拷贝而非移动
    println!("{:?}", q);
}
```

derive 宏在编译期把结构体的字段列表喂给宏代码,宏输出对应的 impl——字段变更时,生成代码自动跟随,不存在"手写 impl 忘了加字段"的漂移。serde 的 #[derive(Serialize, Deserialize)](第7章)就是同一机制:宏读取字段与属性(rename_all 等),生成完整的序列化代码。自己写过程宏需要独立的 proc-macro crate,入门后用 serde/thiserror 这类成熟宏即可——但理解"宏在编译期读你的代码、生成你的代码",是读懂它们文档的前提。

本章总纲:泛型 vs 宏

泛型:参数是"类型",产出"类型的组合",检查在类型层面。宏:参数是"代码(token)",产出"代码",检查在语法层面。判据:你的抽象能不能用"一个类型参数"表达?能,就泛型;需要"语法变形"(生成 impl、改写函数、编译期 DSL),就宏。两者都做"编译期的事"——这是与 C 的运行时黑客最根本的分野。

易错点 3:泛型与 dyn 混用的"静态/动态"错觉

fn f<T: Trait>(x: T) 与 fn g(x: Box<dyn Trait>) 都能接"实现了 Trait 的类型",但前者是单态化(每个 T 一份代码,可内联),后者是 vtable 查找(一份代码,不可内联)。同一个接口里来回切换两者,是设计混乱的开始:先问"调用点的具体类型是否编译期确定",确定走泛型,不确定走 dyn——选定后保持全链路一致。

本章速查表

主题	结论 / 写法
泛型函数	<code>fn first<T>(list: &[T]) -> &T</code> —— 调用时推断 T
泛型结构体	<code>struct Stack<T> { items: Vec<T> };impl<T> Stack<T> { ... }</code>
单态化	编译期为每个具体类型生成专用代码,零运行时开销(同 C++ 模板)
trait bound	<code>fn f<T: PartialOrd + Copy>(…)</code> —— 多约束用 +
where 子句	约束挪到函数签名下方,复杂约束可读性优先
关联类型	<code>trait Iterator { type Item; ... }</code> —— "产出类型"用关联类型
泛型参数 vs 关联	输入类型用泛型参数(<code>Add<Rhs></code>);唯一产出用关联类型(<code>Item</code>)
const 泛型	<code>fn f<const N: usize>(a: [u32; N])</code> —— 长度成为类型的一部分
声明式宏	<code>macro_rules! name { (\$(\$x:expr),*) => {...} }</code> 模式匹配 + 生成
宏重复	<code>\$(code)*</code> 对匹配的每一项生成一段代码
过程宏	<code>derive</code> (生成 <code>impl</code>)/ 属性(改写条目)/ 函数(生成代码)三类
derive 原理	编译期读结构体字段,生成 <code>impl</code> ;字段变更自动跟随
选型	类型参数化→泛型;语法生成→宏;能函数就别宏

最小必会清单

- 能解释单态化的机制与开销,并说出与 C++ 模板的一致性
- 能徒手写泛型结构体 `Stack<T>` 及 `impl` 块
- 能写 `trait bound`(含多约束)与 `where` 子句,并解释各自适用场景
- 能说出关联类型与泛型参数的选型原则(产出 vs 输入)
- 能写出 `const` 泛型函数并解释它如何消灭"数组长度靠约定"
- 能解释宏与函数/泛型的区别(语法树替换 vs 调用/类型展开)
- 能读懂与编写简单的 `macro_rules!`(至少 `mini_vec` 级别)
- 能说出过程宏三兄弟与 `#[derive]` 的编译期原理
- 知道调试宏展开用 `cargo expand`
- 能在"泛型 vs 宏"之间做出正确选型

自测题

Q 1. 单态化与虚函数调用(第6章 `dyn`)的开销差异是什么?分别适合什么场景?

✅ 参考答案

单态化:编译期生成专用代码,调用直接内联,零间接跳转;代价是代码体积随"类型×函数"组合增长。dyn:运行期查 vtable,一次间接调用,代码单份;代价是无法内联与特化。选型:类型集合编译期已知、性能敏感(数值库、容器)→泛型;类型集合运行期才闭合(插件、异构集合)→dyn。

Q 2. C 的 #define MAX(a,b) 与 Rust 的泛型函数,本质区别在哪?

✓ 参考答案

宏是预处理期的文本替换:不参与类型检查,参数被多次求值(MAX(++i,j) 的经典爆炸),错误在展开后爆发且难定位。泛型是编译期的类型级抽象:参数有类型,约束由编译器检查,每个展开点都经过完整类型检查,错误指向源代码位置。宏"假装泛型"的时代结束了——泛型负责类型,宏退守语法。

Q 3. trait Iterator 为什么用关联类型 Item 而不是泛型参数?

✓ 参考答案

因为"产出类型"对每个实现是唯一的:Counter 只产出 u32,不可能产出 u32 又产出 String。用关联类型,一个 impl 块声明一次 Item,所有方法(next 等)自动共享,调用方无需重复标注。若用泛型参数,一个类型要产出多类元素才能表达,语义错位且所有适配器都要多写一个参数。

Q 4. 数组 [u32; N] 与 C 的数组参数有什么本质差异?const 泛型解决了什么?

✓ 参考答案

C 的数组参数退化为指针,长度信息丢失,只能靠额外的 n 参数靠约定传递。Rust 的 [u32; N] 中长度是类型的一部分:sum_arr<const N: usize>(arr: [u32; N]) 让 N 成为编译期参数,长度错配直接编译错误——"这个数组一定是 4 个元素"从注释变成类型事实,编译器替你维护不变量。

Q 5. macro_rules! 与函数,什么时候必须用宏?

✓ 参考答案

函数在"值层面"复用:输入是值、输出是值。当复用点是"语法形态"时必须用宏:① 生成 impl/derive 样板;② 接受"不限数量、不限类型的参数"(println! 的格式化串);③ 需要在调用点展开上下文代码(第3章的标签、泛型数组字面量)。判断口诀:数据进函数,代码进宏。

Q 6. #[derive(Debug)] 之后给结构体加了一个字段,会发生什么?手写 impl 呢?

✓ 参考答案

derive 生成的 impl 在编译期按当前字段列表重建:加字段后 Debug 输出自动包含新字段,零手工维护。手写 impl Debug 则必须自己加字段的格式化代码,漏加就是"打印与真实结构不一致"的隐蔽漂移。这就是"生成代码跟随类型定义"的价值——也是 serde/thiserror 等宏生态在生产中无可替代的原因。

章末实验题:泛型栈与自用宏

实验 10:泛型栈与自用宏

实验目标:实现一个泛型 Stack(含 trait bound 方法),再用 macro_rules! 写一个"简化初始化"宏,并加一个 const 泛型函数,把本章全部特性串起来。

实验要求:

- 任务 1:泛型 Stack<T>,含 new/push/pop/peek/is_empty(覆盖:泛型结构体 §10.2)
- 任务 2:为 Stack<T> 实现 Display——要求 T: Display,输出栈内元素(覆盖:trait bound §10.3)
- 任务 3:实现 PartialEq——要求 T: PartialEq,两个栈逐元素比较(覆盖:多约束/where §10.3)
- 任务 4:写 stack! 宏:stack![1, 2, 3] 生成带元素的栈(覆盖:macro_rules! §10.6)
- 任务 5:const 泛型函数 fixed_sum<const N: usize> 求 [i32; N] 之和(覆盖:const 泛型 §10.5)
- 加分:用 #[derive(Debug, Clone, PartialEq)] 定义测试用结构体,验证 derive 生成(覆盖:derive 原理 §10.7)

第一步 一步:写 Stack<T> 的骨架与四个基础方法,先跑通 push/pop

第二步 二步:实现 Display:先加 T: Display 约束编译,再把约束换成 where 形式感受差异

第三步 三步:实现 PartialEq(需要 T: PartialEq),并体会"实现 std trait 就是运算符可用"

第四步 四步:写 stack! 宏(注意:生成的是带初始值的 Stack,需要先有 new/push)

第五步 五步:写 const 泛型函数,并尝试用不同长度数组调用验证类型绑定

参考答案要点

```

// ===== 实验 10:泛型栈与自用宏 =====
// 覆盖知识点:泛型结构体与 impl($10.2)、trait bound 与 where($10.3)、
//          const 泛型($10.5)、macro_rules!($10.6)、derive($10.7)

use std::fmt;

// ---- 任务 1:泛型栈 ----
#[derive(Debug, Clone)]
struct Stack<T> {
    items: Vec<T>,
}

impl<T> Stack<T> {
    fn new() -> Stack<T> {
        Stack { items: Vec::new() }
    }

    fn push(&mut self, v: T) {
        self.items.push(v);
    }

    fn pop(&mut self) -> Option<T> {
        self.items.pop()
    }

    fn peek(&self) -> Option<&T> {
        self.items.last()
    }

    fn is_empty(&self) -> bool {
        self.items.is_empty()
    }
}

// ---- 任务 2:Display,需要 T: Display ----
impl<T: fmt::Display> fmt::Display for Stack<T> {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "[");
        for (i, item) in self.items.iter().enumerate() {
            if i > 0 {
                write!(f, ", ");
            }
            write!(f, "{item}")?;
        }
        write!(f, "]")
    }
}

// ---- 任务 3:PartialEq,多约束用 where 写清楚 ----
impl<T> PartialEq for Stack<T>
where
    T: PartialEq,
{
    fn eq(&self, other: &Self) -> bool {
        self.items == other.items
    }
}

```

```

    }
}

// ---- 任务 4:stack! 宏:自动调用 new/push ----
macro_rules! stack {
    ($( $x:expr ),* $(,)? ) => {{
        let mut s = Stack::new();
        $( s.push($x); )*
        s
    }};
}

// ---- 任务 5:const 泛型求和 ----
fn fixed_sum<const N: usize>(arr: [i32; N]) -> i32 {
    let mut total = 0;
    for &x in arr.iter() {
        total += x;
    }
    total
}

// ---- 加分:derive 生成的类型,直接可用于断言 ----
#[derive(Debug, Clone, PartialEq)]
struct Point { x: i32, y: i32 }

fn main() {
    // 泛型栈两种类型互不干扰
    let mut s1 = stack![1, 2, 3];
    println!("s1 = {s1}"); // [1, 2, 3]
    assert_eq!(s1.pop(), Some(3));

    let mut s2 = Stack::<String>::new();
    s2.push(String::from("rust"));
    println!("s2 = {s2}"); // [rust]

    // 栈相等比较:需要 T: PartialEq(编译期保证)
    let a = stack![1, 2];
    let b = stack![1, 2];
    assert!(a == b, "同元素栈应相等");

    // const 泛型:长度进入类型
    println!("sum([1,2,3]) = {}", fixed_sum([1, 2, 3]));
    println!("sum([4,5,6,7]) = {}", fixed_sum([4, 5, 6, 7]));

    // derive 产物:Debug/Clone/PartialEq 全部可用
    let p = Point { x: 1, y: 2 };
    let q = p.clone();
    assert_eq!(p, q);
    println!("Point: {p:?}");
    println!("全部断言通过");
}

```

设计说明:任务 2 与任务 3 是"约束生长"的现场——先写 impl 不加约束,编译器报 "T cannot be formatted/compared",补上 bound 才通过。这种"编译器告诉你缺什么"的节奏,正是生产里写泛型的常态。

`stack!` 宏的 `$(,)?` 分支吸收尾逗号,是与标准库 `vec!` 对齐的细节;注意宏内 `Stack::new()` 的路径是调用处可见的——宏卫生性允许它引用模块路径,但要求调用方能看到该类型,这也是宏与函数的一个微妙差异。

下一章预告:第11章 反射与动态特性——Rust 如实说"没有反射";Any 与 `downcast`、`serde` 的编译期序列化、宏生成与 `trait` 动态分派,如何替代运行期自省。

第11章 反射与动态特性

建议学时:3-5 小时 | **难度:**★★★☆☆ | **读者背景:**熟悉 C 的 sizeof、结构体与类型转换;知道 Java/C#/Python 的反射或自省能力(仅作参考,不强求)

🎯 本章学习目标

- 理解反射的含义(运行期类型自省、按名访问、动态创建),并如实认识: Rust 没有反射
- 理解 Rust 拒绝反射的设计取舍:零开销、静态类型安全、无运行时元数据包袱
- 掌握 `std::any::Any`:运行期类型检查与 `downcast` 的有限能力
- 掌握 `TypeId`:类型身份的比较与使用(对应 C 没有、C++ `typeid` 的有限形态)
- 理解 `dyn Trait` 的 `vtable` 是"契约反射":按 `trait` 调用而非按类型自省
- 理解 编译期"反射":`serde` 的 `derive` 宏在编译期读字段并生成代码,替代运行期自省
- 掌握"反射需求"在 Rust 中的替代方案清单:序列化、打印、插件、配置
- 能在设计时判断:这个"反射需求"是真的需要运行期动态,还是可以用编译期解决

💡 从 C 到 Rust:本章主题如何迁移

先说实话:C 没有反射,所以你带着的经验是"没有也能活"。C 里处理"类型未知的数据"靠三样东西:`sizeof`(编译期大小)、`void*` + 手动转回、以及"数据结构的字段顺序即约定"(结构体本质上是手写的字节布局描述)。Java/Python 的反射则是另一极端:运行期能拿到"这个对象的所有字段名和类型、按名调用方法、动态创建实例"——代价是巨大的运行时元数据与性能开销。

Rust 站在 C 一侧,但把工具升级:类型信息仍然主要在编译期,但你需要"运行期知道类型"的地方,有精确的替代品:`Any` 提供"运行期类型检查 + 安全向下转型";`TypeId` 提供类型身份;`dyn Trait` 提供按契约调用;`serde` 宏提供编译期字段遍历(序列化、打印、比较)。本章的路线是"如实承认没有反射,然后逐项看替代品":§11.1-11.2 认识问题,§11.3-11.4 学 `Any/TypeId/dyn`,§11.5-11.6 学编译期方案,§11.7 给一份"反射需求 × Rust 对策"的决策表。

📦 前置知识自查

- C 的 `sizeof`、结构体布局、`void*` 与类型转换;知道"C 没有运行期类型信息"
- 第6章的 `dyn Trait` 与 `vtable`(本章大量复用)
- 第7章的 `serde derive` 用法、第10章的 `derive` 原理
- 第2章的 `match` 与模式匹配(`downcast` 结果用 `match/if let` 处理)
- (可选)Java 的 `getClass/instanceof`、Python 的 `type()/vars()` 用于对照

🚀 本章学习路线

1. **第一阶段(约1小时):**§11.1-§11.2,反射的图景与 Rust 的取舍——先建立正确预期
2. **第二阶段(约1.5小时):**§11.3-§11.4,`Any/TypeId/dyn Trait`——运行期能做什么、不能做什么
3. **第三阶段(约1.5小时):**§11.5-§11.7,编译期替代方案与决策表——生产里的"伪反射"艺术
4. **验收标准:**拿到一个"想用反射"的需求,能五分钟内给出 Rust 替代方案并说出开销差异

本章知识导图

- §11.1 反射是什么:C 没有,Java 有,Rust 呢?
- §11.2 设计取舍:为什么 Rust 拒绝运行期反射
- §11.3 Any:运行期类型检查与安全转型
- §11.4 Typeld 与 dyn Trait:类型身份与契约调用
- §11.5 编译期"反射":serde 的 derive 机制
- §11.6 打印、比较与调试:Debug 与格式化家族
- §11.7 "反射需求"决策表:生产替代方案清单
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§11.1 反射是什么:C 没有,Java 有,Rust 呢?

11.1.1 反射的三层能力

能力	含义	C	Java/C#	Python	Rust
类型自省	运行期间"这个值是什么类型"	无	getClass()	type()	Any + Typeld(有限)
字段/方法遍历	运行期枚举字段名与类型、按名调用	无	getFields()	vars()/getattr()	无(编译期宏替代)
动态创建	运行期按字符串创建实例	无	newInstance()	globals()[name]	无(工厂函数/宏替代)

结论直白:Rust 在"类型自省"上有精确的最小能力(Any),在"字段遍历与动态创建"上没有运行期能力——这些被移到编译期(宏)。这不是缺失,是取舍:运行期反射的元数据与开销,换不来 Rust 场景里需要的东西。

11.1.2 C 的替代方案:约定与手写

```
// C:没有运行期类型信息,程序员自己造
struct Entry {
    int kind;           // 0=int, 1=double, 2=char*
    union {             // 类型自省的"手工版"
        int i;
        double d;
        char *s;
    } val;
};

// 打印"任意 Entry":switch 是手写反射
void print_entry(const struct Entry *e) {
    switch (e->kind) {
        case 0: printf("%d", e->val.i); break;
        case 1: printf("%f", e->val.d); break;
        case 2: printf("%s", e->val.s); break;
    }
}
```

C 的做法是"标签 + 联合体 + switch",全靠程序员维护:kind 写错、新增类型忘改 switch,都是静默错误。Rust 的 enum(第1/3章)把"kind + union + switch"收进一个类型,编译器保证穷尽性——这是"手写反射"的类型安全版。

§11.2 设计取舍:为什么 Rust 拒绝运行期反射

★ 重点:三个理由,对应三条 C 哲学

- **零开销原则**:Rust 承诺"你不用的功能不付钱"。运行期反射要求每个类型携带元数据(字段名表、类型表),连 int 都要能"自省"——这与 C 的"int 就是 4 字节"哲学背道而驰;
- **静态类型安全**:反射的招牌能力"按字符串访问字段"天然绕过类型检查(Java 的 NoSuchFieldException 是运行期错误)。Rust 的目标是把错误留在编译期;
- **可预测的内存与性能**:没有隐藏的元数据分配、没有运行期类型查找的开销,行为可静态推理——服务端与嵌入式场景的硬要求。

代价是实打实的:序列化、调试打印、通用比较需要显式声明(derive 或手写 impl),"按名字段查配置"要多写代码。本章剩余篇幅就是这些代价的补偿方案。

§11.3 Any:运行期类型检查与安全转型

11.3.1 存异构值,查它是什么

```
use std::any::Any;

fn main() {
    // 任何 'static 类型都能放进 Box<dyn Any>(对照 C 的 void*)
    let mixed: Vec<Box<dyn Any>> = vec![
        Box::new(42i32),
        Box::new(String::from("你好")),
        Box::new(3.14f64),
    ];

    for item in &mixed {
        // downcast_ref:若真是该类型则返回 Some(&T),否则 None
        if let Some(n) = item.downcast_ref::<i32>() {
            println!("i32 值:{n}");
        } else if let Some(s) = item.downcast_ref::<String>() {
            println!("String 值:{s}");
        } else if let Some(d) = item.downcast_ref::<f64>() {
            println!("f64 值:{d}");
        } else {
            println!("未知类型");
        }
    }
}
```

对照 C 的 void*: **downcast** 的"猜类型"是带检查的——猜错返回 None,而不是 UB。C 里 void* 转错类型是未定义行为,运行期可能一切正常也可能炸;Rust 的 downcast 把"猜"变成"可失败的检查",安全地失败。

11.3.2 是/不是 检查与值版 downcast

```
use std::any::Any;

fn type_name_of(val: &dyn Any) -> &'static str {
    if val.is::<i32>() {
        "i32"
    } else if val.is::<String>() {
        "String"
    } else if val.is::<f64>() {
        "f64"
    } else {
        "其他"
    }
}

fn main() {
    let a: Box<dyn Any> = Box::new(100i32);
    println!("类型:{}", type_name_of(&*a));

    // 拥有型 downcast:成功返回 Box<T>,失败返回原 Box
    match a.downcast::<i32>() {
        Ok(n) => println!("取回拥有权:{n}"),
        Err(b) => println!("类型不符,原样退回:{:?}", b.type_id()),
    }
}
```

⚠ 易错点 1:Any 只对 'static 类型生效

`&dyn Any` 的约束是 `T: 'static`:含引用的类型(如 `&str`、带生命周期参数的泛型)不能直接 downcast 到具体借用——因为"它是否还活着"与类型系统纠缠。实践中:异构容器装拥有型值(`Box<dyn Any>`),借用型需求用 `dyn Trait`(§11.4)表达。另外注意 `&str` 本身是 'static 的(字面量),可以 downcast;而 `&String` 的引用不可以。

§11.4 TypeId 与 dyn Trait:类型身份与契约调用

11.4.1 TypeId:类型身份的比较

```
use std::any::{Any, TypeId};

// 类型身份:TypeId 可比较、可哈希(对照 C++ 的 typeid)
fn same_type<T: Any, U: Any>() -> bool {
    TypeId::of::<T>() == TypeId::of::<U>()
}

fn main() {
    println!("i32 与 u32 相同?{}", same_type::<i32, u32>()); // false
    println!("i32 与 i32 相同?{}", same_type::<i32, i32>()); // true

    // 动态侧:拿运行期值的 TypeId 与编译期类型比较
    let val: &dyn Any = &7i32;
    if val.type_id() == TypeId::of::<i32>() {
        println!("运行期确认:是 i32");
    }
}
```

11.4.2 dyn Trait 是"契约反射"

```
// 真正生产级的"异构 + 按能力调用":不是"查类型",而是"按契约调用"
// 需要"打印能力"就声明 Print 契约,实现者各写各的
trait Describable {
    fn describe(&self) -> String;
}

struct Server { name: String }
struct Config { port: u16 }

impl Describable for Server {
    fn describe(&self) -> String {
        format!("服务器:{}", self.name)
    }
}

impl Describable for Config {
    fn describe(&self) -> String {
        format!("配置:端口 {}", self.port)
    }
}

fn main() {
    // vtable 里存的是"契约的方法",不是"类型名"
    let items: Vec<Box<dyn Describable>> = vec![
        Box::new(Server { name: "api-01".into() }),
        Box::new(Config { port: 8080 }),
    ];
    for it in &items {
        println!("{}", it.describe()); // 不需要知道具体类型!
    }
}
```

★ 重点:反射 vs 契约,两种思维的差别

反射问的是"你是谁?"(类型名、字段名),契约问的是"你能干什么?"(trait 方法)。Rust 的哲学是后者更符合生产需求:插件、注册表、消息分发——调用方需要的从来不是类型本身,而是某组行为。dyn Trait 的 vtable(第6章)提供行为分发的全部能力,且无需类型名比较。当"行为"能表达需求时,优先 dyn Trait;只有确实需要"按类型分支处理"时才上 Any。

§11.5 编译期"反射":serde 的 derive 机制

11.5.1 序列化是"反射"的头号需求

```
// Java 反射做序列化:运行期遍历字段名/值
// Rust 做法:编译期由宏读取字段,生成序列化代码
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize)]
struct User {
    id: u32,
    name: String,
    tags: Vec<String>,          // 数组
    active: bool,
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let u = User {
        id: 7,
        name: "李明".into(),
        tags: vec!["rust".into(), "server".into()],
        active: true,
    };

    // 序列化:宏生成的代码遍历字段(编译期完成,运行期零元数据)
    let json = serde_json::to_string_pretty(&u)?;
    println!("{}", json);

    // 反序列化:字段名与结构体对齐,类型不符直接报错
    let back: User = serde_json::from_str(&json)?;
    println!("还原:{:?}", back);
    Ok(())
}
```

11.5.2 宏能做"反射"能做的三件事

反射场景	运行期反射(Java)	编译期宏(Rust)
序列化/反序列化	遍历字段名与值	derive(Serialize, Deserialize) 生成代码
调试打印	toString() 反射拼接	derive(Debug) 生成 fmt
深拷贝/比较	反射遍历字段	derive(Clone, PartialEq) 生成实现
字段改名/别名	@SerializedName 注解 + 反射	#[serde(rename = "...")] 属性宏
动态创建	newInstance(类名)	工厂函数 + match,类型安全

宏的"反射"在编译期完成:字段增减时生成代码自动跟随(第10章的 derive 原理),运行期零元数据、零动态查找——这是 Rust 对"反射需求"的标准回答:能编译期做的,绝不放运行期。

§11.6 打印、比较与调试:Debug 与格式化家族

11.6.1 Debug 是"结构化自省"的公共接口

```
#[derive(Debug)]
struct Point { x: f64, y: f64 }

fn main() {
    let p = Point { x: 1.0, y: 2.0 };
    // {:?} 是结构化输出:字段名与值,调试日志与错误信息的原料
    println!("{:?}", p);
    // {:#?} 美化多行
    println!("{:#?}", p);
}
```

生产意义:给自定义类型实现 Debug(通常 derive),错误信息与日志就自动携带"完整结构化状态"——这是 C 的 printf("%s", strerror) 永远给不了的现场还原能力。

11.6.2 自定义 Debug 与"脱敏"场景

```
use std::fmt;

// 生产常见:Debug 输出脱敏(密码、令牌不打印明文)
struct Credentials { user: String, password: String }

impl fmt::Debug for Credentials {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        f.debug_struct("Credentials")
            .field("user", &self.user)
            .field("password", &"*****")
            .finish()
    }
}

fn main() {
    let c = Credentials {
        user: "admin".into(),
        password: "s3cret".into(),
    };
    println!("{:?}", c);    // Credentials { user: "admin", password: "*****" }
```

工程建议:错误类型实现 Debug + Display + Error

第9章的 `Box<dyn std::error::Error>` 要求错误实现 Debug 与 Display。徒手写三次样板很烦——生产用 `thiserror` 宏:一行 `#[derive(thiserror::Error, Debug)]` 自动生成(这就是"编译期反射"替代手写样板的标准形态)。规则:自定义错误永远走 `thiserror`,别手写。

§11.7 "反射需求"决策表:生产替代方案清单

需求(常见"想用反射"的场景)	Rust 对策	关键点
打印/调试任意对象	<code>#[derive(Debug)] + {?}</code>	字段自动跟随,脱敏可手写 <code>fmt</code>
序列化到 JSON/TOML/二进制	<code>serde derive</code>	编译期生成,字段改名用属性
深拷贝/比较	<code>derive(Clone, PartialEq)</code>	语义特殊时手写 <code>impl</code>
异构集合按类型处理	<code>enum</code> (穷尽)或 <code>Box<dyn Any> + downcast</code>	类型集合闭合用 <code>enum</code> ,开放用 <code>Any</code>
按行为分发(插件/回调)	<code>dyn Trait / trait object</code>	契约优先于类型名
按字符串选类型/函数	<code>match</code> 字符串到工厂函数	编译期穷尽,运行期无反射查找
数据库 ORM	<code>diesel/sqlx</code> 的 <code>derive</code> 宏	查询结果编译期映射结构体
配置驱动代码生成	过程宏读配置生成代码	生成期在编译期,错误也在编译期

⚠ 易错点 2:把"反射"当第一反应

从 Java/Python 转来的程序员,遇到"对象要能描述自己"第一反应是反射。在 Rust 里第一反应应该是:先问"运行时真的需要动态吗?"。类型集合在编译期就确定的(配置文件字段、协议消息、插件清单),一律 `enum + match + derive`——编译器穷尽检查比任何反射都可靠;只有"未知类型来自运行期"(插件 `dll` 加载、用户输入的类型名)才需要 `Any/dyn`,而且量很小。设计时把这句话贴在桌上:反射是运行期的最后手段,不是第一选项。

⚠ 易错点 3:downcast 分支膨胀是设计信号

如果代码里出现"针对 5 种以上类型各写一个 `downcast` 分支",说明你实际需要的是 `enum`(类型闭合时)或 `trait`(行为分发时)——`Any` 是"未知类型集合"的逃生门,不是常规建模工具。分支数量 = 重构信号:3 种以内还能接受,超过就该回到 §14.3 的 `enum` 建模或 §11.4 的 `dyn Trait`。

本章速查表

主题	结论 / 写法
Rust 有没有反射	没有运行期反射;用 Any/TypeId(自省)与宏(字段遍历)替代
存异构值	Vec<Box<dyn Any>>(对照 void* 数组)
运行期查类型	val.is::<T>() -> bool
安全转型(借用)	val.downcast_ref::<T>() -> Option<&T>,猜错返回 None 而非 UB
安全转型(拥有)	boxed.downcast::<T>() -> Result<Box<T>, Box<dyn Any>>
类型身份	TypeId::of::<T>(),可比较;type_id() 拿运行期值身份
Any 的限制	仅 'static 类型;借用型需求用 dyn Trait
行为分发	dyn Trait + vtable:按契约调用,不查类型名
序列化"反射"	derive(Serialize, Deserialize):编译期生成字段代码
调试打印	derive(Debug) + {:?} / {:#?};脱敏手写 fmt
深拷贝/比较	derive(Clone, Copy, PartialEq)
错误类型样板	thiserror 宏生成 Debug + Display + Error
动态创建	工厂函数 + match(编译期穷尽),而非字符串实例化
选型口诀	编译期能做的绝不放运行期;enum + match 优先,Any/dyn 兜底

最小必会清单

- 能准确说出:Rust 没有运行期反射,但有 Any/TypeId 与编译期宏两大替代
- 能解释 Rust 拒绝反射的三个理由(零开销、静态安全、可预测)
- 能用 Box<dyn Any> 存异构值并 downcast 安全取回
- 知道 downcast_ref 与 void* 转型的本质区别(检查 vs UB)
- 能用 TypeId 比较类型身份,知道 type_id() 的用法
- 能区分"按类型处理"(Any)与"按行为处理"(dyn Trait)并正确选型
- 能解释 serde 的 derive 如何替代反射式序列化,及其开销优势
- 能写出脱敏的 Debug 实现
- 知道 thiserror 宏替代手写错误样板
- 拿到"想用反射"的需求,能套决策表给出 Rust 方案

自测题

Q 1. "Rust 没有反射"这句话准确吗?准确的说法是什么?

✅ 参考答案

不准确(或者说只说对一半)。准确说法:Rust 没有"运行期字段遍历与动态创建"式的反射,但有精确的最小替代——Any 提供运行期类型检查与安全转型(自省的一小部分),TypeId 提供类型身份,宏(serde/derive)在编译期完成"字

段遍历"性质的代码生成。所以:没有"运行期反射",有"编译期反射 + 有限运行期自省"。

Q 2. `void*` 与 `Box<dyn Any>` 都"能装任意类型",生产上为什么差距巨大?

✅ 参考答案

`void*` 取回时"转回正确类型"完全靠程序员记忆与约定,转错是未定义行为(可能静默出错);`Box<dyn Any>` 的 `downcast` 是检查过的转型:类型不符返回 `None` 或 `Err`,安全失败。另外 `Any` 带类型身份(可 `is::<T> / type_id`),而 `void*` 连"它是什么"都无法回答。同样的"异构存储",前者是雷区,后者是可推理的类型。

Q 3. 什么场景用 `downcast` 按类型分支,什么场景用 `dyn Trait` 按行为调用?

✅ 参考答案

判据:你拿到值之后要"按类型分别处理"还是"统一调用某组方法"。前者(事件分发、调试器、反序列化分发)用 `Any + downcast`——处理逻辑以类型为分支;后者(插件、回调、图形绘制)用 `dyn Trait`——调用方只依赖契约,不需要知道类型。能用行为表达需求的,永远优先 `dyn Trait`:它不依赖类型名,扩展新类型不用改分发代码。

Q 4. `serde` 的 `derive` 凭什么替代"反射式序列化"?开销与正确性各如何?

✅ 参考答案

`derive` 宏在编译期读取结构体的字段列表,生成逐字段的序列化/反序列化代码。开销:运行期零元数据、零动态查找,直接执行生成的字段代码(与手写序列化同速);正确性:字段增减自动跟随,类型不符编译期报错,而反射式序列化把"字段不存在/类型错"留到运行期抛异常。代价是结构体必须实现 `serde trait`——而这就是 `derive` 一行的事。

Q 5. "按字符串创建对象"在 `Rust` 里如何实现?为什么比反射更安全?

✅ 参考答案

用 `match` 把字符串映射到工厂函数:`match name { "server" => Box::new(Server::new()), "client" => Box::new(Client::new()), _ => return Err(...) }`。安全性:① 映射关系编译期可见,拼写错误在编译期或测试期暴露;② 每个分支返回的类型有签名约束(如 `Box<dyn Plugin>`),错误提前;③ 不存在的名字是显式 `Err`,而反射的 `NoSuchMethod` 是运行期异常。若类型集合由外部插件提供(运行期才闭合),才需要 `dlopen + 符号查找`——那是少量、受控的 `unsafe` 边界。

Q 6. `Debug` 脱敏怎么写?为什么生产日志里必须做?

✅ 参考答案

手写 `impl fmt::Debug`,用 `debug_struct().field()` 逐个输出字段,敏感字段输出掩码(如 `"*****"`),或用 `#[derive(Debug)]` 后对敏感字段标 `#[debug(skip)]`(需要 `debug` 宏辅助库)。必要性:错误与日志会自动打印 `Debug`,一旦 `Credentials` 混入错误链,明文密码就进了日志文件——这是 C 时代 `print` 结构体字段时同样存在的风险,而 `Rust` 的 `Debug` 契约让"脱敏"有了标准实现点。

章末实验题:通用结构体打印机与异构处理

实验 11:通用结构体打印机

实验目标:写一个"通用打印机":接受任意异构值集合,输出每个值的类型信息与内容;再叠加 serde 序列化输出,对比"运行期自省"与"编译期生成"两条路线。

实验要求:

- 任务 1:定义 3-4 个不同结构体(至少一个含嵌套字段)(覆盖:类型设计)
- 任务 2:用 `Box<dyn Any>` 集合存放异构值,遍历输出类型名(自定义映射)(覆盖:Any §11.3)
- 任务 3:`downcast` 到具体类型并打印字段内容,类型不符走兜底(覆盖:`downcast` §11.3)
- 任务 4:用 `TypeId` 精确比较并打印"是否匹配"信息(覆盖:`TypeId` §11.4)
- 任务 5:为结构体 `derive(Serialize)`,把整个集合序列化输出(覆盖:`serde` §11.5)
- 任务 6:写一个 `enum` 版"打印机"对比:类型闭合时 `enum + match` 更优(覆盖:选型 §11.7)

第一步 一步:定义结构体:`Server(name, port)`、`Metric(name, value)`、`Event(id, kind, msg)`

第二步 二步:写 `type_label(&dyn Any)` 函数,先 `is::<T>` 链判断返回标签

第三步 三步:写 `print_any:downcast` 每个类型并打印细节,兜底打印 "未知"

第四步 四步:加入 `TypeId` 分支输出"类型标识一致"验证

第五步 五步:`derive Serialize`,用 `serde_json` 输出整个列表;对比与任务 3 的实现差异

参考答案要点

```
// ===== 实验 11:通用结构体打印机 =====
// 覆盖知识点:Any 与 downcast(§11.3)、TypeId(§11.4)、
//          serde 编译期序列化(§11.5)、enum 选型对照(§11.7)

use std::any::{Any, TypeId};
use serde::Serialize;

// ---- 任务 1:三个结构体 ----
#[derive(Debug, Serialize, Clone)]
struct Server {
    name: String,
    port: u16,
}

#[derive(Debug, Serialize, Clone)]
struct Metric {
    name: String,
    value: f64,
}

#[derive(Debug, Serialize, Clone)]
struct Event {
    id: u32,
    kind: String,
    msg: String,
```

```

}

// ---- 任务 2:类型标签(运行期自省的最小形态) ----
fn type_label(val: &dyn Any) -> &'static str {
    if val.is::<Server>() {
        "Server"
    } else if val.is::<Metric>() {
        "Metric"
    } else if val.is::<Event>() {
        "Event"
    } else {
        "未知"
    }
}

// ---- 任务 3:downcast 打印细节 ----
fn print_any(val: &dyn Any) {
    println!("[{}] ", type_label(val));
    if let Some(s) = val.downcast_ref::<Server>() {
        println!("服务器 {}:{}", s.name, s.port);
    } else if let Some(m) = val.downcast_ref::<Metric>() {
        println!("指标 {} = {:.2}", m.name, m.value);
    } else if let Some(e) = val.downcast_ref::<Event>() {
        println!("事件 #{} {}:{}", e.id, e.kind, e.msg);
    } else {
        println!("(类型未知,无法打印细节)");
    }
}

// ---- 任务 4:TypeId 精确比较 ----
fn is_server(val: &dyn Any) -> bool {
    val.type_id() == TypeId::of::<Server>()
}

// ---- 任务 6:enum 对照版:类型闭合时更优 ----
#[derive(Debug)]
enum Thing {
    S(Server),
    M(Metric),
    E(Event),
}

fn print_thing(t: &Thing) {
    match t {
        Thing::S(s) => println!("enum Server {}:{}", s.name, s.port),
        Thing::M(m) => println!("enum Metric {} = {}", m.name, m.value),
        Thing::E(e) => println!("enum Event #{} {}", e.id, e.msg),
        // 无兜底分支:新增类型时编译器强制你处理 — 反射给不了这个保证
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // 任务 2+3:异构集合 + 运行期自省
    let mixed: Vec<Box<dyn Any>> = vec![
        Box::new(Server { name: "api-01".into(), port: 8080 }),
    ]
}

```

```

Box::new(Metric { name: "cpu", value: 42.5 })),
Box::new(Event { id: 7, kind: "deploy".into(), msg: "发布完成".into() })),
Box::new(42i32),                // 未知类型进列表
];
for item in &mixed {
    print_any(&item);
    if is_server(&item) {
        println!(" (TypeId 确认:是 Server)");
    }
}

// 任务 5:serde 编译期序列化 — 一条路径输出全部,字段自动跟随
let docs: Vec<serde_json::Value> = mixed.iter().map(|item| {
    // 演示:为每个元素生成 "类型 + JSON" 的汇总
    let label = type_label(&item);
    let value = if let Some(s) = item.downcast_ref::<Server>() {
        serde_json::to_value(s).unwrap_or_default()
    } else if let Some(m) = item.downcast_ref::<Metric>() {
        serde_json::to_value(m).unwrap_or_default()
    } else if let Some(e) = item.downcast_ref::<Event>() {
        serde_json::to_value(e).unwrap_or_default()
    } else {
        serde_json::Value::Null
    };
    serde_json::json!({"type": label, "data": value})
}).collect();
println!("JSON 汇总:\n{}", serde_json::to_string_pretty(&docs)?);

// 任务 6:enum 对照 — 类型闭合、编译器穷尽
let things = vec![
    Thing::S(Server { name: "db-01".into(), port: 5432 }),
    Thing::M(Metric { name: "mem", value: 66.6 }),
];
for t in &things {
    print_thing(t);
}
Ok(())
}

```

设计说明:任务 3 与任务 6 的对照是全章的缩影——Any 版的好处是"集合里能混入未知类型",enum 版的好处是"新增类型编译器逼你处理"。生产判断:类型集合由自己掌控且会增长,用 enum(编译期穷尽);集合要容纳"运行期才出现"的类型,用 Any。任务 5 的 serde 路径展示了"编译期反射"的威力:字段改名、嵌套结构变化,序列化自动跟随,而 Any 版的分支代码必须同步修改——这正是 Rust 把"反射"放在编译期的全部理由。

下一章预告:第12章 注解与编译期配置——C 的 #ifdef/_attribute_ 对应 Rust 的属性系统;#[derive]/#[cfg]/#[allow] 与自定义属性宏,编译期开关的正确用法。

第12章 注解与编译期配置

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:熟悉 C 的 `#ifdef/#ifndef`、`__attribute__`、`-Wall/-Werror` 与 `Makefile/CMake` 的条件开关

本章学习目标

- 理解属性(attribute)在 Rust 中的角色: C 的预处理指令 + `__attribute__` + 编译器选项,三者合一
- 掌握属性语法与分类:内置属性按用途分组,能认出与检索
- 掌握 `#[cfg]` 条件编译,与 C 的 `#ifdef/#ifndef` 逐点对照,并掌握 `cfg!()` 运行期判断
- 掌握 `#[derive]` 的常用组合与适用场景(承接第10章的原理)
- 掌握 lint 系统:`#[allow]`/`#[deny]`/`#[warn]` 与 C 的 `-Wall/-Werror` 对照,理解 `#[must_use]`
- 了解自定义属性宏(过程宏 attribute 类):`#[timed]` 式改写函数的原理与写法
- 掌握 Cargo features 特性开关:编译期配置的工程标准(对照 CMake 的选项)
- 能在"运行期配置"与"编译期配置"之间做出正确选择

从 C 到 Rust:本章主题如何迁移

C 的"编译期配置"散落在三个世界:**预处理指令** `#ifdef/#ifndef/#define`(文本级条件)、**编译器属性** `__attribute__((...))`(给编译器的提示,各编译器方言不一)、**编译器选项** `-Wall/-Werror`/宏定义。三者语法不同、作用时机不同、跨平台表现不同:`#ifdef` 是文本替换,能破坏语法(`#define bool int` 的灾难);`__attribute__` 在 GCC 与 MSVC 之间不通用;警告开关换个编译器就失效。

Rust 把这三者统一成一套**属性系统**:以 `#[...]` 形式附着在代码条目上,由编译器在语法解析后统一处理。`#[cfg]` 是类型安全的条件编译(不像 `#ifdef` 那样文本替换);`#[allow]`/`#[deny]` 是每个 crate 可配的 lint 开关;`#[derive]` 是代码生成;自定义属性宏把"编译器扩展"开放给库作者。配套的 Cargo features 提供工程级的"构建选项"——比 CMake 的 `option()` 更结构化(类型化、可组合、可推导)。本章的路线:先看 C 的三个世界如何对应过来(\$12.1-12.2),再逐个掌握 `#[cfg]`(\$12.3)、`#[derive]`(\$12.4)、lint 系统(\$12.5),然后是属性宏(\$12.6)与 features(\$12.7)。

前置知识自查

- C 的 `#ifdef/#ifndef/#else/#endif`、`#define` 宏、`__attribute__((unused/format))`、`-Wall -Werror`
- 第10章的宏与 `derive` 原理(属性宏是过程宏的一种)
- 第11章的 `serde` 属性用法(`#[serde(rename)]`)——你已经用过属性了
- Cargo.toml 的基本结构(第13章会系统讲,本章先用 `features`)
- (可选)Makefile/CMake 的开关变量与条件编译

本章学习路线

1. **第一阶段(约1小时):**§12.1–§12.2,属性系统总览与语法——先建立"三合一"的心智模型
2. **第二阶段(约1.5小时):**§12.3–§12.5,#[cfg]/#[derive]/lint——日常代码里出现频率最高的三个属性族
3. **第三阶段(约1.5小时):**§12.6–§12.7,属性宏与 features——生产级编译期配置的完整形态
4. **验收标准:**能徒手写"平台分支 + 特性开关 + 测试打印开关"三合一的条件编译代码,并说出每个属性在编译期做什么

本章知识导图

- §12.1 C 的三个世界 vs Rust 的属性系统
- §12.2 属性语法:位置、参数与分类
- §12.3 #[cfg] 条件编译与 cfg!()
- §12.4 #[derive] 派生宏的常用组合
- §12.5 lint 系统:#[allow]/#[deny]/#[must_use]
- §12.6 自定义属性宏:#[timed] 的原理
- §12.7 Cargo features:工程级编译开关
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§12.1 C 的三个世界 vs Rust 的属性系统

12.1.1 C 的条件编译:文本级的危险艺术

```
// C:预处理在语法解析之前,条件编译是"文本删减"
#ifdef _WIN32
    #include <windows.h>
    #define PATH_SEP '\\'
#elif defined(__linux__)
    #include <unistd.h>
    #define PATH_SEP '/'
#else
    #error "不支持的平台"
#endif

// 风险:被 #ifdef 删掉的内容不参与语法检查——错误藏在"没编译到的分支"里
#ifdef DEBUG
    printf("debug=%d\n", dbg);    // 拼错变量名?只有开 DEBUG 才炸
#endif
```


12.1.2 Rust:属性由编译器统一处理

```
// Rust:条件编译在语法解析之后—所有分支都先过语法检查,再决定取舍
#[cfg(target_os = "windows")]
fn path_sep() -> char { '\\' }

#[cfg(target_os = "linux")]
fn path_sep() -> char { '/' }

// 关键差异:#ifdef 里写错语法没人管;
// #[cfg] 里两个分支都必须语法正确,只是"类型检查与代码生成"按条件进行
fn main() {
    println!("分隔符:{}", path_sep());
}
```

C 世界	Rust 对应	差异要点
#ifdef / #ifndef / #endif	#[cfg(...)] / #[cfg(not(...))]	Rust 先解析语法再条件取舍;C 先删文本再解析
#define 编译期常量	const / cfg 选项 / features	无文本替换,类型安全
__attribute__((...))	#[...] 属性	统一语法,跨平台一致
-Wall / -Werror / -Wno-xxx	#[warn]/#[deny]/#[allow] Cargo.lints	+ 配置进源码,随 crate 分发
CMake option() 开关	Cargo.toml [features]	类型化、可组合、可被依赖方覆盖
#pragma once / include 守卫	模块系统(crate/mod,第13章)	无头文件,无重复包含问题

§12.2 属性语法:位置、参数与分类

12.2.1 语法形态

```
// 三种写法:
// 1) 无参数:作用于紧随其后的条目
#[derive(Debug)]
struct Point { x: f64, y: f64 }

// 2) 有参数:括号内是参数
#[cfg(target_os = "windows")]
fn win_only() {}

// 3) 多个属性叠放
#[derive(Clone, PartialEq)]
#[allow(dead_code)]
struct Config { port: u16 }

// 4) 作用于整个文件/模块的"内部属性"(!)
#![allow(unused_variables)] // 放在 crate/模块最顶部,作用于整个容器
fn main() {
    let unused = 42; // 不再警告
    println!("ok");
}
```

12.2.2 内置属性的六大用途分组

用途	代表属性	C 对应
条件编译	<code>#[cfg]</code> / <code>#[cfg_attr]</code>	<code>#ifdef</code> / <code>#if</code>
代码生成	<code>#[derive]</code> / <code>#[test]</code>	宏 / 无
lint 控制	<code>#[allow]</code> / <code>#[deny]</code> / <code>#[warn]</code> / <code>#[must_use]</code>	<code>-Wno-xxx</code> / <code>-Werror</code>
入口与外部接口	<code>#[main]</code> / <code>#[no_mangle]</code> / <code>#[repr]</code> / <code>#[link]</code>	<code>extern "C"</code> / 链接选项
布局与 ABI	<code>#[repr(C)]</code> / <code>#[repr(u8)]</code> / <code>#[repr(transparent)]</code>	结构体布局约定
静态保证	<code>#[inline]</code> / <code>#[cold]</code> / <code>#[deprecated]</code>	<code>__attribute__((always_inline))</code>

§12.3 #[cfg] 条件编译与 cfg!()

12.3.1 平台分支的正确姿势

```
// 平台相关代码:cfg 表达式是"真布尔逻辑",支持 and/or/not
#[cfg(all(target_os = "linux", target_arch = "x86_64"))]
fn platform_desc() -> &'static str { "Linux x86_64" }

#[cfg(any(target_os = "windows", target_os = "macos"))]
fn platform_desc() -> &'static str { "Windows 或 macOS" }

#[cfg(not(debug_assertions))]
fn build_profile() -> &'static str { "release" }

#[cfg(debug_assertions)]
fn build_profile() -> &'static str { "debug" }

fn main() {
    println!("{}", platform_desc(), build_profile());
}
```

12.3.2 cfg!():运行期布尔,不是删代码

```
// cfg!() 在编译期求值为 true/false 常量,代码始终存在:
// 适合"分支很小、不值得物理删除"的场景
fn main() {
    if cfg!(debug_assertions) {
        println!("调试构建:启用详细日志");
    } else {
        println!("发布构建:日志降级");
    }

    // 对照 C:#ifdef DEBUG + printf 的成对出现;
    // cfg!() 让"环境判断"变成普通布尔表达式,可在任何表达式位置用
    let threshold: u32 = if cfg!(target_pointer_width = "64") { 1_000_000 } else { 100_000 };
    println!("阈值:{threshold}");
}
```

⚠ 易错点 1:#[cfg] 删条目,cfg!() 保留代码——别混用

#[cfg] 作用于条目时,不匹配的条目 **整个不编译**:其中的类型错误、未定义名称都不会报错(对照 #ifdef 的风险,但语法仍检查);cfg!() 是"编译期常量布尔",代码必须始终合法。判据:整个函数/模块要不要存在→#[cfg];一段逻辑的取值差异→cfg!()。最典型的混用错误:给函数挂 #[cfg(feature = "x")],却在别处无条件调用它——未启用特性时编译错误"function not found",这正是设计意图,也是查错的方向。

12.3.3 cfg_attr:条件化地加属性

```
// #[cfg_attr(条件, 属性)]:条件成立时才附加属性
// 生产场景:release 下为热路径函数附加内联提示
#[cfg_attr(not(debug_assertions), inline)]
fn hot_add(a: u32, b: u32) -> u32 { a + b }

// 或:只有特定平台才派生某 trait
#[cfg_attr(target_os = "windows", derive(Debug))]
struct WinInfo { name: String }

fn main() {
    println!("{}", hot_add(1, 2));
}
```

§12.4 #[derive] 派生宏的常用组合

```
// 生产代码的标准组合(第10章讲原理,这里讲"选型")
#[derive(Debug, Clone, PartialEq)]
struct Task { id: u32, name: String }

#[derive(Debug, Clone, Copy, PartialEq)]
struct Vec2 { x: f64, y: f64 }          // 小值类型:加 Copy

#[derive(Debug, Clone, PartialEq, Eq, Hash)]
struct Key { a: u32, b: u32 }          // 作 HashMap 键:Eq + Hash

fn main() {
    let t1 = Task { id: 1, name: "a".into() };
    let t2 = t1.clone();                // Clone
    assert_eq!(t1, t2);                 // PartialEq
    println!("{:?}", t2);               // Debug

    let v = Vec2 { x: 1.0, y: 2.0 };
    let v2 = v;                         // Copy:隐式拷贝,不用 clone
    println!("{:?} {:?}", v, v2);
}
```

派生 trait	生成能力	适用类型
Debug	{:?} 结构化打印	几乎所有类型(日志/错误链)
Clone	显式深拷贝 .clone()	需要复制的类型
Copy	隐式按位拷贝(值语义)	小值类型:数字、坐标、标志
PartialEq	== / !=	需要比较的类型
Eq + Hash	可作 HashMap/HashSet 键	键类型
PartialOrd / Ord	< > 与排序	需要排序/取极值的类型
Default	default() 零值构造	配置类、可空字段容器

§12.5 lint 系统:#[allow]/#[deny]/#[must_use]

12.5.1 三档级别与 C 的警告开关

```
// 三档:allow(允许)/ warn(警告,默认)/ deny(升级为错误)
#![deny(unsafe_code)]           // 整个 crate 禁止 unsafe(对照 -Werror 的极端版)
#![deny(missing_docs)]          // 公共 API 缺文档即编译失败(生产库的标配)

#[allow(dead_code)]             // 局部豁免:这个结构体暂时没用,别警告
struct Wip { x: u32 }

#[warn(unused_variables)]        // 显式声明某条 lint 的级别(一般用不到,默认已开)

fn main() {
    let tmp = 42;                // 默认 warn:unused_variables
    println!("看,没有警告");
    let _ = tmp;
}
```

12.5.2 #[must_use]:防止"丢了返回值"

```
// C 的经典事故:fclose/strtok/printf 的返回值没人查
// Rust 用 #[must_use] 把"必须使用返回值"写进类型契约

#[must_use = "校验失败会被静默吞掉,务必处理 Result"]
fn validate(input: &str) -> Result<(), String> {
    if input.is_empty() {
        Err("输入为空".into())
    } else {
        Ok(())
    }
}

fn main() {
    // validate("x");           // 警告:unused must_use value
    let _ = validate("x");      // 显式忽略:警告消失(有意为之)

    // 标准库里 #[must_use] 的著名成员:Result、MutexGuard、Iterator
    let v = vec![1, 2, 3];
    let _iter = v.iter().map(|x| x * 2); // 不收集会警告吗?不会——map 是惰性的,
    // 但"算完没保存"的典型错误会警告:std 已把该标的都标了
    println!("{}", validate("x").is_ok());
}
```

工程建议:新项目的 lint 起点

Cargo.toml 里配 [lints] rust = { unsafe_code = "deny", missing_docs = "warn" },把策略随 crate 版本化(第13章 Cargo 全流程会细讲)。从 C 转来的程序员最值得先开的两条:unsafe_code(让 unsafe 边界显式化)与 clippy 全家桶(clippy --fix 自动修)——这相当于把 -Wall -Wextra -Werror 变成默认,并且更智能。

§12.6 自定义属性宏:#[timed] 的原理

12.6.1 需求:给函数自动加计时

```
// 想写这样:任何函数挂上 #[timed],调用时自动打印耗时
// #[timed]
// fn work() { ... }

// 属性宏的职责:读取被标注的函数,在其外层生成"计时包裹代码"。
// 过程宏必须放在独立的 proc-macro crate(设计如此:宏是编译器的插件)
```

12.6.2 完整的最小实现(教学简化版)

```
// ---- timer-macros/src/lib.rs(独立 crate 的完整源码)----
// Cargo.toml 关键段:
// [lib] proc-macro = true
// [dependencies] quote = "1", syn = "2"

use proc_macro::TokenStream;
use quote::quote;

/// #[timed] 属性宏:包裹无参函数,调用时打印耗时
/// 教学简化版:只处理无参数、无返回值的 fn
#[proc_macro_attribute]
pub fn timed(_attr: TokenStream, item: TokenStream) -> TokenStream {
    // 1. 把被标注的条目解析为"函数"AST
    let func = syn::parse_macro_input!(item as syn::ItemFn);
    let name = &func.sig.ident;
    let body = &func.block;

    // 2. 生成新的函数体:原文被包进"计时包裹"
    let output = quote! {
        fn #name() {
            let __start = std::time::Instant::now();
            #body
            println!("[计时] {} 耗时 {:?}",
                    stringify!(&name), __start.elapsed());
        }
    };

    // 3. 输出替换原条目的代码(编译期完成)
    output.into()
}
```

```
// ---- 使用方 main.rs ----
use timer_macros::timed;

#[timed]                                // 宏在编译期改写这个函数
fn work() {
    let mut sum = 0u64;
    for i in 1..=1_000_000 {
        sum += i;
    }
    println!("sum = {sum}");
}

fn main() {
    work();                               // [计时] work 耗时 1.2ms
    work();                               // 每次调用都自动计时
}
```

⚠ 易错点 2:过程宏是"编译器插件",有隔离与版本风险

过程宏 crate 用独立的编译器实例运行,不能与主 crate 共享代码;宏代码里的 panic 会变成编译错误(信息难看)。生产上:优先用成熟宏库(tracing、thiserror、serde),自研属性宏只在你确实需要"改写代码形态"时做——写好文档、做好测试(宏展开测试用 trybuild)。用 cargo expand 预览展开结果,是调试宏的第一工具。

§12.7 Cargo features:工程级编译开关

12.7.1 features 的声明与使用

```
# ---- Cargo.toml(工程配置) ----
# [features] 表:每个特性是一个"开关",可以被默认启用或被依赖方按需打开
#
# [features]
# default = ["std"]           # 默认启用的特性列表
# std = []                    # 空特性:纯粹的标志
# verbose = ["std"]           # 特性可以依赖其他特性
#
# [dependencies]
# serde = { version = "1", optional = true } # 可选依赖,随特性启用
```

```
// ---- src/main.rs(代码侧) ----
// 特性开关在代码里通过 #[cfg] 感知

#[cfg(feature = "std")]
fn io_support() -> &'static str { "标准库可用" }

#[cfg(not(feature = "std"))]
fn io_support() -> &'static str { "no_std 环境" }

// 可选依赖的惯用法:特性名与依赖同名,启用特性才编译依赖
#[cfg(feature = "verbose")]
fn log_detail(msg: &str) {
    println!("[详细] {msg}");
}

#[cfg(not(feature = "verbose"))]
fn log_detail(_msg: &str) {
    // 无操作:特性关闭时调用方代码不变,行为降级
}

fn main() {
    println!("{}", io_support());
    log_detail("处理完成");
}
```

12.7.2 与 CMake 的对照

CMake 概念	Cargo features
option(VERBOSE "... OFF) + if(VERBOSE)	[features] verbose = [] + #[cfg(feature = "verbose")]
add_definitions(-DVERBOSE)	cargo build --features verbose
target_compile_definitions 按目标	#[cfg(target_os = ...)] 平台条件
find_package 可选依赖	[dependencies] xxx = { optional = true }
开关变量作用域(目录级)	特性随 crate 传递:依赖方可覆盖
配置漂移(忘开/错开)	编译期 #[cfg] 校验,未定义特性名编译报错

★ 重点:运行期配置 vs 编译期配置,怎么选

同一份"配置",两条路:运行期配置(env/配置文件,第7章)支持热切换、用户可调;编译期配置(features/#[cfg])减小体积、去掉攻击面、让代码分支成为死代码。判据:值在构建时确定且不需要切换(平台、no_std、嵌入大小)→编译期;运行中可变或用户可调(端口、日志级别、功能开关)→运行期。生产系统常混合:编译期定"支持哪些能力",运行期定"当前启用哪些"。C 程序员要警惕的惯性:#ifdef 开太多会把产品拆成一堆不可测的组合——features 组合爆炸同样存在,用"default 合理 + 文档 + CI 矩阵"控制。

⚠ 易错点 3:feature 名拼写错误是"静默关特性"

Cargo 构建时,未知的 feature 名会报错;但代码里的 `#[cfg(feature = "verbose")]` 与 Cargo.toml 的声明必须严格一致——改名后漏改一处,特性就"静默关闭"(不报错,只是代码不编译进二进制)。对策:CI 里至少跑一次 `--all-features` 构建,把"所有特性全开"的形态纳入验证;这正是 C 的 `#ifdef` 组合爆炸问题在 Rust 里的对应管理手段。

把"运行期配置"与"编译期配置"放进同一套代码,是生产系统的常态:features 决定"这份二进制支持什么"(后端类型、可选依赖),运行期配置决定"这次运行怎么用"(端口、级别、开关)。C 程序员最容易犯的惯性是把一切做成 `#ifdef`——结果产品被拆成几十种互不相同的二进制,测试矩阵爆炸。Rust 的纪律:编译期开关必须数量少、语义正交(default 合理);一旦出现"两个特性必须同时开"的隐式耦合,说明该特性应该合并,或改为运行期配置。特性是"能力",不是"模式"——这个心智模型,是生产级编译期配置的总纲。

本章速查表

主题	结论 / 写法
属性本质	<code>#[...]</code> 附着条目,编译器语法解析后处理;替代 <code>#ifdef + __attribute__ + 编译选项</code>
条件编译	<code>#[cfg(target_os = "windows")]</code> / <code>#[cfg(not(debug_assertions))]</code>
cfg 组合	<code>all(...)</code> / <code>any(...)</code> / <code>not(...)</code> 布尔组合
运行期判断	<code>cfg!(debug_assertions)</code> 编译期常量布尔,代码始终存在
条件属性	<code>#[cfg_attr(cond, attr)]</code> :条件成立才附加属性
代码生成	<code>#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Default)]</code>
lint 三档	<code>#[allow(x)]</code> 放行 / <code>#[warn(x)]</code> 警告 / <code>#[deny(x)]</code> 升级错误
crate 级 lint	<code>#![deny(unsafe_code)]</code> 顶部内部属性,作用于整个 crate
必须使用	<code>#[must_use]</code> 标记类型/函数:忽略返回值即警告
属性宏	<code>#[proc_macro_attribute]</code> 改写被标注条目;独立 proc-macro crate + <code>syn/quote</code>
特性声明	Cargo.toml [features] default = ["std"];verbose = []
特性感知	<code>#[cfg(feature = "verbose")]</code> 与 <code>#[cfg(not(...))]</code> 双分支
可选依赖	[dependencies] serde = { optional = true }
开启方式	<code>cargo build --features verbose</code> ;依赖方用 <code>features = ["verbose"]</code>

最小必会清单

- 能说出 Rust 属性系统统一了 C 的哪三类机制(预处理/ `__attribute__` /编译选项)
- 能写 `#[cfg]` 平台分支与 debug/release 分支,并解释与 `#ifdef` 的语法检查差异
- 能区分 `#[cfg]` 与 `cfg!()`,并说出各自适用场景
- 能列出常用 derive 组合(Debug/Clone/Copy/PartialEq/Eq/Hash/Default)与选型
- 能用 `#[allow]/#[deny]` 控制 lint,说出与 `-Werror` 的关系
- 能解释 `#[must_use]` 防止哪类 C 常见事故
- 能读懂一个简单的 `#[proc_macro_attribute]` 并说出 `syn/quote` 的角色

- 能声明与使用 Cargo features,包括可选依赖
- 能说出"运行期配置 vs 编译期配置"的选型判据
- 知道 cargo expand 与 clippy 这两个配套工具

自测题

Q 1. #[cfg] 与 #ifdef 最本质的差别是什么?对正确性意味着什么?

✓ 参考答案

处理时机:预处理在语法解析前按文本删减,#ifdef 未选中的分支连语法都不检查,错误潜伏在"没编译到的代码"里(开 DEBUG 才爆);#[cfg] 在语法解析后做条件取舍,所有分支都必须先通过语法解析,只有类型检查与代码生成按条件进行。正确性收益:条件分支里的语法错误、括号不匹配等在编译时必然暴露,不存在"暗藏分支"。

Q 2. 什么时候用 #[cfg] 删条目,什么时候用 cfg!() 留代码?

✓ 参考答案

整个条目(函数/模块/impl)在该条件下无意义→#[cfg](删除后连符号都不存在,体积与检查都更省);一段逻辑仅取值不同、代码量小→cfg!()(代码始终合法,表达式位置随意用)。判据之二:cfg!() 里的代码在两种构建中都必须通过类型检查;若分支两边类型差异大或依赖不同依赖,必须用 #[cfg]。

Q 3. #[allow(dead_code)] 与 #![deny(unsafe_code)] 的层级与作用域有什么区别?

✓ 参考答案

前者是"条目属性"(无 !),只作用于紧随其后的那个条目,局部豁免某条 lint;后者是"内部属性"(带 !),放在 crate 或模块顶部,作用于整个容器。工程形态:全局策略用 #![deny/![warn] 写进 crate 根部或 Cargo.lints;局部豁免用条目级 #[allow]——"默认严格、逐点豁免"是 Rust 的 lint 哲学,与 C 的"默认宽松、逐个开告警"相反。

Q 4. #[must_use] 对应的 C 事故是什么?它的检查在哪个阶段?

✓ 参考答案

对应 C 里"返回值被忽略"的经典事故:忘查 fclose/fflush 失败、strtok 返回 NULL 不判、realloc 失败丢原指针。Rust 把"必须使用返回值"写成类型/函数级契约:调用方忽略带 #[must_use] 的返回值,编译器发出警告(unused_must_use),检查发生在编译期。生产上给"返回 Result/安全相关"的函数统一标注,把"忘查"变成编译警告。

Q 5. 过程宏为什么必须放独立 crate?生产上什么情况下才值得自研?

✓ 参考答案

过程宏以"编译器插件"形态运行:它被编译后由编译器加载执行,与主 crate 的依赖图隔离,因此必须独立成 proc-macro crate(这也是 Rust 团队隔离宏崩溃、保持编译期稳定性的设计)。自研的合理场景:你确实需要"读代码、改代码"的元编程(自动注册、按结构体生成状态机、DSL);若只是加样板,先用 macro_rules!(第10章)与成熟库(thiserror/serde)解决。自研时:保持宏行为可预期、提供展开测试。

Q 6. features 与"运行期配置"如何分工?举一个生产实例。

✅ 参考答案

分工:构建期定"能力集合"(编译哪些代码、链接哪些依赖),运行期定"行为取值"。实例:一个服务支持"数据库存储与内存存储"两种后端——用 feature 决定编译哪个后端模块(编译期,体积与攻击面可控),而端口、日志级别、限流阈值用配置文件(运行期,免重编译)。反例警示:把日志级别做成编译期开关,调级要重编译,运维会恨你——级别永远是运行期配置。

章末实验题:计时属性宏与特性开关

📄 实验 12:计时属性宏 + #[cfg] 开关

实验目标:做一个"带计时能力的程序":① 用 #[cfg] 实现 debug/release 两套日志行为;② 用 Cargo feature 控制"详细模式";③ 实现 #[timed] 属性宏(独立 proc-macro crate),给函数自动计时。

实验要求:

- 任务 1:debug/release 双分支的 log 函数(覆盖:#[cfg] §12.3)
- 任务 2:cfg!() 实现阈值选择(覆盖:cfg!() §12.3)
- 任务 3:Cargo.toml 声明 feature "verbose",代码感知开关(覆盖:features §12.7)
- 任务 4:建立 timer-macros 过程宏 crate,实现 #[timed](覆盖:属性宏 §12.6)
- 任务 5:用 #[timed] 标注 2-3 个函数,验证每次调用输出耗时(覆盖:宏使用)
- 加分:在 #[timed] 基础上加 #[cfg_attr] 控制"release 下自动内联"(覆盖:cfg_attr §12.3)

工程结构:workspace 或两个独立 crate:主程序 + timer-macros(依赖路径 "path = "../timer-macros")。

第一步 一步:cargo new timed_demo 与 cargo new timer-macros --lib;后者 Cargo.toml 加 proc-macro = true

第二步 二步:写 timer-macros 的 #[timed](参考 §12.6 源码),先在本 crate 用 trybuild 前手工验证

第三步 三步:主 crate 依赖 timer-macros,写 2-3 个 #[timed] 函数并调用

第四步 四步:写 #[cfg(debug_assertions)] 双分支日志,分别 cargo run 与 cargo run --release 对比输出

第五步 五步:加 [features] verbose = [],双分支 log_detail,用 --features verbose 验证开关

✅ 参考答案要点

```
// ===== 实验 12:计时属性宏与特性开关 =====
// 覆盖知识点:#[cfg] 条件编译(§12.3)、cfg!()(§12.3)、cfg_attr(§12.3)、
//          features 开关(§12.7)、属性宏(§12.6)

// ---- timer-macros/Cargo.toml ----
// [package] name = "timer-macros" version = "0.1.0" edition = "2021"
// [lib] proc-macro = true
// [dependencies] quote = "1" syn = "2"

// ---- timer-macros/src/lib.rs ----
use proc_macro::TokenStream;
use quote::quote;

/// #[timed]:包裹无参函数并打印耗时(教学简化版)
```

```

#[proc_macro_attribute]
pub fn timed(_attr: TokenStream, item: TokenStream) -> TokenStream {
    let func = syn::parse_macro_input!(item as syn::ItemFn);
    let name = &func.sig.ident;
    let body = &func.block;
    quote! {
        fn #name() {
            let __start = std::time::Instant::now();
            #body
            println!("[计时] {} 耗时 {:?}",
                    stringify!(&name), __start.elapsed());
        }
    }
    .into()
}

// ---- timed_demo/Cargo.toml ----
// [package] name = "timed_demo" version = "0.1.0" edition = "2021"
// [features]
// default = []
// verbose = [] # 详细模式开关
// [dependencies]
// timer-macros = { path = "../timer-macros" }

// ---- timed_demo/src/main.rs ----
use timer_macros::timed;

// 任务 1: debug/release 双分支日志
#[cfg(debug_assertions)]
fn log(msg: &str) {
    println!("[DEBUG] {msg}");
}

#[cfg(not(debug_assertions))]
fn log(msg: &str) {
    println!("[INFO] {msg}");
}

// 任务 2: cfg!() 阈值选择(代码始终存在,值编译期定)
fn threshold() -> u32 {
    if cfg!(debug_assertions) { 10_000 } else { 1_000_000 }
}

// 任务 3: feature 感知的详细日志
#[cfg(feature = "verbose")]
fn log_detail(msg: &str) {
    println!("[详细] {msg}");
}

#[cfg(not(feature = "verbose"))]
fn log_detail(_msg: &str) {
    // 关闭时静默:调用方代码无需改动
}

// 任务 5: 计时宏上场

```

```
#[timed]
fn heavy_sum(n: u32) {
    let mut total = 0u64;
    for i in 1..=n {
        total += i as u64;
    }
    println!("sum(1..={n}) = {total}");
}

#[timed]
fn heavy_fib(n: u32) {
    let mut a = 0u64;
    let mut b = 1u64;
    for _ in 0..n {
        let t = a + b;
        a = b;
        b = t;
    }
    println!("fib({n}) = {a}");
}

fn main() {
    log("程序启动");
    log_detail("进入主流程");
    println!("当前阈值:{}", threshold());

    heavy_sum(100_000);
    heavy_fib(40);
    log("程序结束");
}

// 运行:
// cargo run          → [DEBUG]/[INFO] + 两次 [计时] 输出(无详细日志)
// cargo run --release → [INFO] + 阈值变大 + 耗时明显缩短(内联优化)
// cargo run --features verbose → 多出 [详细] 行
```

设计说明:三个开关各演示一种形态——`#[cfg]` 换"函数存在性"(debug/release 各编译一份)、`cfg!()` 换"常量取值"、`feature` 换"能力开关"。注意 `#[timed]` 宏把原函数体嵌进包裹代码:原函数内定义的变量、`println!` 全部照常工作,这正是"属性宏改写代码"与"普通函数封装"的本质区别。扩展练习:把 `#[timed]` 改成支持带参数函数(需要把 `syn::ItemFn` 的 `sig` 原样搬进生成代码),你会更深入理解过程宏的"代码搬运"本质。

下一章预告:第13章 模块与 Cargo 工程——`crate/mod/use/pub` 的组织方式替代头文件体系;Cargo 全流程、`features`、`workspace` 与 `crates.io` 依赖管理。

第13章 模块与 Cargo 工程

建议学时:4–6 小时 | **难度:**★★★☆☆ | **读者背景:**熟悉 C 的头文件体系(#include、extern、static)、Makefile/CMake 构建,写过至少一个多文件 C 工程

🎯 本章学习目标

- 理解 C 头文件体系的痛点(重复包含、顺序依赖、宏污染)与 Rust 模块系统的答案
- 掌握模块三件套: **mod 声明**、**use 导入**、**pub 可见性**,理解"文件即模块"的映射
- 掌握模块树的组织:库 crate 与二进制 crate 的差异、目录模块的两种形态
- 掌握可见性体系:pub / pub(crate) / pub(super),对照 C 的 extern 与 static
- 掌握 use 路径:绝对路径 crate::、相对 super::/self::,pub use 重导出与别名
- 掌握 Cargo 全流程:new/build/run/test/check,依赖管理与语义化版本
- 掌握 workspace 多 crate 工程与 crates.io 的发布/使用流程
- 能独立搭建一个"库 + 二进制 + 第三方依赖"的标准 Rust 工程

💡 从 C 到 Rust:本章主题如何迁移

C 的工程组织是"头文件 + 链接器"的二元体系:每个 .c 文件配一个 .h,#include 把声明文本复制进来;可见性靠 extern(跨文件)与 static(本文件)两个关键字;构建靠 Makefile/CMake 手工描述依赖图。痛处你早已熟悉:头文件重复包含(#pragma once 的补丁)、包含顺序敏感、宏在头文件间互相污染、忘了链接某个 .o 是链接器报"undefined reference"。这些问题的根源是同一个:"编译单元"与"逻辑模块"不是一回事,模块边界靠约定而非语言。

Rust 把工程组织收进语言与工具:模块(mod)是语言的正式公民,文件与模块一一对应,可见性(pub)由编译器检查;Cargo 是"构建系统 + 依赖管理 + 测试 + 文档"一体化的官方工具——你不再写 Makefile,Cargo.toml 声明依赖,锁文件(Cargo.lock)固定版本,一个命令下载构建全部依赖。本章先学模块的语言机制(§13.1–13.5),再学 Cargo 的工程流程(§13.6–13.7),最后实验搭一个"库 + CLI + 第三方依赖"的完整工程——这将是读者进入生产后每天都在用的骨架。

📦 前置知识自查

- C 的多文件工程:#include、头文件守卫、extern、static、链接错误(undefined reference)
- Makefile 或 CMakeLists.txt 的基本结构(至少见过 target/dependency 概念)
- 第10章的 crate/mod 概念引入、第12章的 Cargo.toml 与 features
- 第2章的基本语法(本章的示例代码需要能读懂)
- (可选)Python 的 import/package 或 Java 的 package,用于对照

本章学习路线

1. **第一阶段(约1.5小时):**§13.1–§13.3,模块机制与文件组织——先会"把代码拆进文件"
2. **第二阶段(约1.5小时):**§13.4–§13.5,可见性与 use 路径——模块系统的"接线"功夫
3. **第三阶段(约1.5小时):**§13.6–§13.7,Cargo 全流程与 workspace——工程落地的完整形态
4. **验收标准:**不看教程,能徒手新建 workspace、写库 crate、写 CLI、加依赖、跑测试

本章知识导图

- §13.1 头文件体系 vs 模块系统
- §13.2 模块基础:mod / use / pub
- §13.3 文件组织:文件即模块
- §13.4 可见性:pub 家族与 C 的对照
- §13.5 use 路径与重导出
- §13.6 Cargo 全流程:构建、测试、依赖
- §13.7 workspace 与 crates.io
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§13.1 头文件体系 vs 模块系统

13.1.1 C 的头文件之痛

```
// C:头文件是"文本复制"—每个 .c 编译单元各自复制一份声明
// math.h
#ifndef MATH_H                // 头文件守卫:手写的补丁
#define MATH_H
int add(int a, int b);
int sub(int a, int b);
#endif

// main.c
#include "math.h"              // 声明被复制进来
#include <stdio.h>
int main(void) {
    printf("%d\n", add(1, 2)); // 链接期才知道 add 在 math.c 里
    return 0;
}
// 问题:包含顺序敏感、宏互相污染、链接错误与声明不一致难以追踪
```


13.1.2 Rust 的模块:语言原生的边界

```
// Rust:模块是编译器的正式概念,不是文本复制
// src/main.rs(单文件演示;多文件见 §13.3)
mod math {                               // 声明模块:块内的代码属于 math
    pub fn add(a: i32, b: i32) -> i32 { a + b }
    pub fn sub(a: i32, b: i32) -> i32 { a - b }
}

fn main() {
    println!("{}", math::add(1, 2));    // 通过模块路径访问
}
```

C 的头文件体系	Rust 模块系统
#include 文本复制	mod 声明,模块是语言实体
#ifndef 守卫(手写补丁)	无需守卫:模块只编译一次
包含顺序敏感	声明顺序无关,模块树结构化
宏在头文件间污染	宏只在声明处与 use 处生效
extern:跨文件可见	pub:跨模块可见(显式)
static:本文件私有	默认私有(不写 pub 即私有)
链接期发现缺失符号	编译期发现缺失与未导出

§13.2 模块基础:mod / use / pub

13.2.1 mod 声明与默认私有

```
mod outer {                               // 声明模块 outer
    fn private() {}                       // 不写 pub:仅 outer 内可见
    pub fn public() {}                   // pub:对外可见

    pub mod inner {                       // 模块可以嵌套
        pub fn deep() {}
    }
}

fn main() {
    outer::public();                     // ✓ pub 可见
    // outer::private();                 // ERROR: private function(默认私有)
    outer::inner::deep();                 // ✓ 逐层 pub 才能到达
}
```

Rust 的可见性默认是全私有,需要显式 pub 开放——与 C 的默认全 extern(要 static 才收窄)完全相反。这个“默认私有”直接消灭了一整类“意外暴露内部符号”的问题。

13.2.2 use:把路径"接线"到本地

```
use std::collections::HashMap;           // 把完整路径绑定到短名字

fn main() {
    let mut m = HashMap::new();           // 不用写 std::collections::HashMap
    m.insert(1, "a");
    println!("{:?}", m);

    // 多路径一次性导入
    use std::io::{Read, Write};           // 花括号组
    // use std::fmt::{self, Debug}; // self 表示模块本身

    // 别名:解决命名冲突(对照 C 的 typedef 式缩名)
    use std::collections::HashMap as Map;
    let mut m2: Map<u32, u32> = Map::new();
    m2.insert(1, 2);
    println!("{:?}", m2);
}
```

§13.3 文件组织:文件即模块

13.3.1 单文件模块 → 拆成文件

```
// 规则一:src/main.rs(二进制 crate 根)或 src/lib.rs(库 crate 根)是模块树的根
// 规则二:mod math; 会在同目录找 math.rs 或 math/mod.rs

// ---- src/main.rs ----
mod math;           // 声明"math 模块在 src/math.rs"
mod utils;           // src/utils.rs

fn main() {
    println!("{}", math::add(1, 2));
    println!("{}", utils::greet());
}

// ---- src/math.rs ----
pub fn add(a: i32, b: i32) -> i32 { a + b }

// ---- src/utils.rs ----
pub fn greet() -> &'static str { "你好,模块世界" }
```

13.3.2 目录模块:子模块的两种形态

```
// 形态一:目录 + mod.rs(旧惯例,仍广泛使用)
// src/math/mod.rs 里声明 pub mod complex;
// src/math/complex.rs 是子模块文件
//
// 形态二:目录同名文件 + 子文件(新惯例,2021 edition 支持)
// src/math.rs 里声明 pub mod complex;
// src/math/complex.rs 作为子模块
//
// 两者等价;推荐形态二(与"文件即模块"的直觉一致)

// ---- src/lib.rs(库 crate 根:对外提供 API)----
pub mod math;           // 库的公共 API:math 模块对外可见
pub mod utils;

// ---- src/math.rs ----
pub mod complex;        // 子模块:complex.rs

// ---- src/math/complex.rs ----
pub fn norm(x: f64, y: f64) -> f64 {
    (x * x + y * y).sqrt()
}
```

⚠ 易错点 1:文件里的 pub 与"mod 声明处的 pub"是两回事

文件内函数写 pub 只代表"模块内可见性";模块能否被外部访问,取决于父模块里 mod 声明是否带 pub。即: mod math; 就算 math.rs 里全是 pub fn,外部 crate 也看不到 math——必须 pub mod math;。两层 pub 各管一层,漏掉任何一层都报"module is private"。

§13.4 可见性:pub 家族与 C 的对照

```
// 可见性四级(由宽到窄)
pub fn wide() {}           // 完全公开:本 crate 与所有依赖方可见
pub(crate) fn mid() {}     // 仅本 crate 内可见(二进制/库内部)
pub(super) fn narrow() {}  // 仅父模块可见
fn narrowest() {}         // 默认:仅当前模块及其子模块可见

// 对照 C:
// pub           ≈ 非 static 的全局符号(extern 默认)
// pub(crate)    ≈ 链接器内部符号(不导出到动态符号表)
// 默认私有      ≈ static 修饰的全局/函数

mod api {
    pub fn public_api() {} // 依赖方可调用
    pub(crate) fn internal() {} // 库内部各模块可用,外部不可见
}

fn main() {
    api::public_api();
    api::internal();       // ✓ 同 crate 内可见
}
```

工程建议:pub(crate) 是"库内部共用"的标准答案

C 的"多文件共享但不想对外暴露"靠 static 与符号表技巧;Rust 直接写 `pub(crate) :crate` 内部随意调用,编译成库后外部无法触碰。生产库的 API 设计基线:能 `pub(crate)` 就不 `pub`——`pub` 的每一项都是永久承诺(第13.7节 `semver` 会讲为什么)。

§13.5 use 路径与重导出

13.5.1 路径的三种基准

```
// 绝对路径:从 crate 根开始
use crate::math::complex::norm;

// 相对路径:从当前模块开始
// use self::helper::x;    // self:: 当前模块
// use super::helper::x;   // super:: 父模块

mod helper {
    pub fn x() -> u32 { 1 }
}

mod app {
    // super 引用父模块(crate 根)里的 helper
    use super::helper::x;
    pub fn run() -> u32 { x() }
}

fn main() {
    println!("{}", app::run());
}
```

13.5.2 pub use:重导出,定义"公共门面"

```
// 库作者视角:内部组织 vs 对外门面
// src/lib.rs
mod internal {                                // internal 本身不公开
    pub struct Config { pub port: u16 }
    pub fn load() -> Config { Config { port: 8080 } }
}

// 但对外提供便捷入口:重导出(等价"转发引用",对照头文件的二次声明)
pub use internal::{Config, load};

// 使用方视角:use mylib::{Config, load}; 与库内部结构无关
// 好处:内部重构(改模块名/层级)不影响依赖方——门面稳定,内部自由
```

⚠ 易错点 2:use 与 pub use 的区别

use 只在本模块内绑定名字(私有); pub use 同时把它"加入本模块的公共 API"——依赖方可以 use 你重导出的名字,而无需知道它的真实来源。对照 C:pub use 相当于头文件里 typedef/声明了另一个文件里的符号。库设计惯例:对外只暴露 crate 根的门面(pub use 汇总),内部模块一律 pub(crate)。

§13.6 Cargo 全流程:构建、测试、依赖

13.6.1 常用命令速览

```
# 新建:二进制工程(默认)与库工程
cargo new myapp           # 生成 src/main.rs + Cargo.toml
cargo new --lib mylib      # 生成 src/lib.rs

# 构建与运行
cargo build                # 调试构建 → target/debug/
cargo build --release      # 发布构建 → target/release/
cargo run                  # 构建并运行
cargo check                # 只查类型错误,不生成代码(最快的体检)

# 测试与文档
cargo test                 # 运行全部测试(单元/集成/文档测试)
cargo doc --open           # 生成并打开 API 文档

# 依赖管理
cargo add serde            # 添加依赖(自动写进 Cargo.toml)
cargo update               # 按版本约束更新 Cargo.lock
cargo search json          # 在 crates.io 搜索

# 依赖树与审计
cargo tree                 # 查看依赖树
cargo audit                # 检查已知漏洞(生产 CI 必备)
```

13.6.2 Cargo.toml 核心字段与版本约束

```
# ---- Cargo.toml(注释为教学说明)----
[package]
name = "myapp"
version = "0.1.0"           # 语义化版本:主.次.补丁
edition = "2021"           # 语言版本(第0章提过 2021 edition)
authors = ["you <you@example.com>"]
description = "示例工程"

[dependencies]
serde = { version = "1", features = ["derive"] } # 版本约束 ^1
rand = "0.8"           # ^0.8:允许 0.8.x,不允许 0.9
# 版本语义(semver):^1.2.3 允许 1.2.3 及以上、小于 2.0
# 0.x 系列:^0.8 只允许 0.8.x(0.8.1 是"破坏性"的,语义特殊)

[dev-dependencies]           # 仅测试用的依赖
pretty_assertions = "1"

[profile.release]           # 发布构建优化
opt-level = 3               # 对照 -O3
lto = true                  # 链接时优化
```

13.6.3 测试三形态

```
// 1) 单元测试:与被测代码同文件,#[cfg(test)] 只在测试构建编译
#[cfg(test)]
mod tests {
    use super::*;          // 导入被测模块的私有项(super 指向父模块)

    #[test]
    fn add_works() {
        assert_eq!(add(1, 2), 3);
    }

    #[test]
    fn add_handles_negative() {
        assert!(add(-1, 2) > 0);
    }
}

fn add(a: i32, b: i32) -> i32 { a + b }
```

// 2) 集成测试:tests/ 目录下的文件,以"外部使用方"视角调用库 API

```
// tests/api.rs:
// use mylib::load;
// #[test]
// fn loads_default() { assert_eq!(load().port, 8080); }
```

// 3) 文档测试:代码块里的示例被当作测试运行

```
// /// 计算两数之和
// /// ```
// /// assert_eq!(mylib::add(1, 2), 3);
// /// ```
// pub fn add(a: i32, b: i32) -> i32 { a + b }
```

§13.7 workspace 与 crates.io

13.7.1 workspace:多 crate 的单一工程

```
# ---- 根 Cargo.toml(仅声明成员,不写 [package])----
[workspace]
members = ["math-lib", "math-cli"]
resolver = "2"                # 现代依赖解析

# ---- 目录结构 ----
# myproj/
# └─ Cargo.toml                (workspace 声明)
# └─ math-lib/
#   └─ Cargo.toml
#   └─ src/lib.rs
# └─ math-cli/
#   └─ Cargo.toml              (依赖 math-lib,可用 path 或 workspace = true)
#   └─ src/main.rs

# ---- math-cli/Cargo.toml ----
# [dependencies]
# math-lib = { path = "../math-lib" }  # 路径依赖:本地开发

# 根目录统一操作:
# cargo build                # 构建全部成员
# cargo test                  # 测试全部成员
# cargo run -p math-cli -- 参数  # 运行指定成员

# 多成员共享依赖:根 Cargo.toml 加 [workspace.dependencies]
# rand = "0.8"
# 成员里写:rand = { workspace = true }
```

13.7.2 crates.io:发布与使用

阶段	命令/操作	说明
注册账号	cargo login 令牌	crates.io 网站生成 API 令牌
检查打包	cargo package	本地验证 .crate 内容(会跑 cargo test)
发布	cargo publish	版本号不可重复,一经发布不可覆盖
使用	cargo add 包名	自动选择满足约束的最新版本
锁版本	Cargo.lock	应用工程提交锁文件,固定可复现构建
更新	cargo update	在版本约束内升级
文档	docs.rs 自动构建	发布即文档,与版本绑定

★ 重点:语义化版本与"pub 即承诺"

semver:主版本号变化 = 破坏性变更;次版本 = 新增功能(兼容);补丁 = 修复(兼容)。Rust 生态严格执行:你把 API 标成 pub 并发布 1.0 后,任何破坏性改动都要求升主版本——这就是"pub 是永久承诺"的原因。依赖方读 Cargo.toml 的 ^1.2 就能知道"补丁与次版本更新自动接受,主版本升级需手动"。生产上:锁文件 + CI 固定工具链 + cargo audit,是依赖管理的四件套。

⚠ 易错点 3:同名文件与同名目录的模块冲突

Rust 允许 src/math.rs 与 src/math/ 目录并存:math.rs 里的子模块文件放在 math/ 目录下。若两个位置都想定义"math"本身,会报"file for module found at both"错误。新项目统一采用"math.rs + math/ 子文件"的形态(\$13.3 形态二),别混用 mod.rs 老形态,避免二义;迁移老工程时先删 mod.rs 再搬文件。

依赖管理的完整生产姿势,是"四件套"的组合:① Cargo.lock 提交进版本库——固定每个依赖的确切版本,任何机器、任何时候构建结果一致;② 工具链固定——rust-toolchain.toml 文件钉死 rustc 版本,防止"昨天还编译,今天 rustup update 后挂了";③ cargo audit——在 CI 里跑,扫描依赖树的已知漏洞(CVE),对照 C 世界"依赖靠手工下载、漏洞靠新闻得知"的现状,这是生态级的安全升级;④ 语义化版本纪律——升级依赖前读 changelog,主版本升级(cargo update 不会自动做)要专门安排回归测试。这四个动作加起来,让"依赖"从"不可控的黑盒"变成"可审计、可复现、可升级"的工程资产。C 程序员把 Makefile 里的链接库路径换成 Cargo.toml,只是第一步;真正的迁移是接受"依赖树由工具管理"这一事实,把精力从"怎么链接"挪到"用哪个版本、何时升级"。

最后补一条关于文档的工程常识:docs.rs 为每个发布的 crate 版本自动构建 API 文档, `cargo doc --open` 生成本地文档。Rust 生态的文档是"随代码生成"的——给公共项写 /// 注释,文档、示例测试(doc test)、API 说明三者同步,不存在 C 世界"文档与头文件分家、头文件与实现分家"的三层漂移。进入生产后,阅读依赖代码的方式也因此改变:先读 docs.rs 的 API 页,再点进源码——这是 C 程序员需要主动养成的新习惯。

本章速查表

主题	结论 / 写法
模块声明	<code>mod math;</code> 找 <code>math.rs</code> ; <code>pub mod math;</code> 才对外可见
默认可见性	不写 <code>pub</code> 即私有(本模块及其子模块可见)
可见性四级	<code>pub</code> / <code>pub(crate)</code> / <code>pub(super)</code> / 默认私有
文件即模块	<code>main.rs/lib.rs</code> 是根; <code>mod x</code> 对应 <code>x.rs</code> 或 <code>x/mod.rs</code>
use 导入	<code>use crate::math::add;</code> <code>use super::*;</code> <code>use self::x;</code>
重导出	<code>pub use internal::{A, B};</code> 定义公共门面
别名/组	<code>use std::io::{Read, Write};</code> <code>use x as y;</code>
新建工程	<code>cargo new name</code> (二进制) / <code>cargo new --lib name</code> (库)
日常构建	<code>cargo build</code> / <code>cargo run</code> / <code>cargo check</code> (最快体检)
测试	<code>cargo test</code> ; 单元 <code>#[cfg(test)]</code> + 集成 tests / + 文档示例
加依赖	<code>cargo add serde</code> ; <code>Cargo.lock</code> 锁定可复现版本
版本约束	<code>^1.2</code> = <code>1.2</code> 至 <code><2.0</code> ; <code>0.x</code> 有特殊语义(<code>^0.8</code> = <code>0.8.x</code>)
workspace	<code>[workspace] members = [...]</code> ; 根目录统一 <code>build/test</code>
发布	<code>cargo login</code> → <code>cargo package</code> → <code>cargo publish</code> ; 版本不可覆盖
库 API 纪律	能 <code>pub(crate)</code> 就不 <code>pub</code> ; <code>pub</code> 是 semver 承诺

最小必会清单

- 能解释 Rust 模块系统如何解决头文件的三大痛点(守卫/顺序/污染)
- 能徒手把单文件拆成"根 + 两个模块文件"并正确声明
- 能说出 `pub` 与 `pub(crate)` 的差别,并知道库内部共享用哪个
- 能用 `super::` 与 `crate::` 写出正确的跨模块引用
- 能解释 `use` 与 `pub use` 的区别,并会用重导出搭建公共门面
- 能独立执行 `cargo new` → `add` → `build` → `test` → `run` 完整循环
- 知道单元测试(`#[cfg(test)]`)与集成测试(`tests/`)的分工
- 能读懂 `^1.2` 版本约束并解释 `Cargo.lock` 的作用
- 能搭建双成员 workspace(库 + 二进制)并用 `path` 依赖关联
- 知道 semver 与"pub 即承诺"对库 API 的约束

自测题

Q 1. C 的头文件守卫(`#ifndef`)在 Rust 里对应什么?为什么不需要?

✅ 参考答案

没有对应物。头文件守卫防的是"同一份声明被 `#include` 复制多次";Rust 的模块只编译一次,`mod` 声明是"引用"而非"复制"——同一个模块被多次 `use` 也只是建立引用,不会重复定义。同理,包含顺序问题也不存在:模块树的声明顺序与可见性由编译器统一检查,不依赖文本先后。

Q 2. 为什么 Rust 默认私有,而 C 默认全 `extern`?默认私有带来了什么?

✅ 参考答案

C 的符号默认全局可见,收窄靠 `static`——意味着"忘记收窄"就会污染全局命名空间,链接错误与符号冲突成为常态。Rust 默认私有,开放靠显式 `pub`——编译器强制你为每个对外符号做出决定;带来的直接收益:内部实现细节不可能意外泄漏,公共 API 边界由编译器检查,重构内部结构不影响依赖方(还有 `pub(crate)` 提供"内部共享"的中间档)。

Q 3. 一个库 `crate` 里,内部模块之间共享工具函数,该用什么可见性?

✅ 参考答案

`pub(crate)`:本 `crate` 内任意模块可调用,编译为库后外部不可见。比 `pub` 安全(不给外部承诺)、比默认私有方便(不限制在单模块内)。C 的对应物:链接器内部符号或"声明在内部头文件、不进安装头"——但 C 没有语言级强制,Rust 是编译器保证。

Q 4. `use` 与 `pub use` 对依赖方有什么区别?

✅ 参考答案

`use` 是模块内的私有接线:依赖方看不到、也不该知道。`pub use` 把名字加入模块的公共 API:依赖方可以用你重导出的名字,且无需关心真实来源。库设计意义:内部把实现拆得再碎(模块层级随意调整),只要 `crate` 根的重导出不变,依赖方代码一行不改——`pub use` 是"内部自由重构"与"对外稳定门面"之间的桥梁。

Q 5. `Cargo.lock` 该不该提交进版本库?为什么?

✅ 参考答案

应用工程(二进制)必须提交:锁文件固定每个依赖的确切版本,保证任何机器、任何时候 `cargo build` 得到完全一致的依赖集(可复现构建,对照"供应商 vendoring"的效果)。库工程惯例不提交锁文件(依赖方会用自己的解析);但两者都受益于 CI 里固定工具链版本与 `cargo audit` 检查。C 世界的对照:没有锁文件,构建可复现性完全靠"团队口头约定版本"。

Q 6. 你在维护一个发布到 `crates.io` 的库。想给公共函数换签名,应该怎么做?

✅ 参考答案

按 `semver` 走:破坏性变更(签名、行为、删除 `pub` 项)必须升主版本号(`1.x` $\rightarrow$ `2.0`);若想温和过渡,标准流程是"弃用期":保留旧签名并标 `#[deprecated]`,新增新签名,一个次版本内共存,下个主版本再移除。因为 `pub` 即承诺:依赖方按 `^1.x` 解析,你的 `1.3` 里删掉函数会直接破坏他们的构建——这正是 Rust 生态依赖安全感的来源。

章末实验题:数学库 + CLI + 第三方依赖

实验 13:数学库 + CLI 工具工程

实验目标:搭建一个 workspace 工程:math-lib(库)提供基础数学运算与统计,math-cli(二进制)读取命令行参数调用库,并引入第三方依赖(rand)做随机练习生成。

实验要求:

- 任务 1:workspace 声明两个成员(覆盖:workspace §13.7)
- 任务 2:math-lib 库:模块拆分为 basic(加减乘除)与 stats(均值/方差),全部 pub 导出(覆盖:模块与文件 §13.3、重导出 §13.5)
- 任务 3:库内工具函数用 pub(crate),测试用 #[cfg(test)] 单元测试(覆盖:可见性 §13.4、测试 §13.6)
- 任务 4:math-cli 二进制依赖 math-lib(path 依赖),解析命令行执行"加/均/随机练习"(覆盖:依赖 §13.6)
- 任务 5:cargo add rand,用 rand 生成随机算术练习并自动判分(覆盖:第三方依赖 §13.6)
- 加分:为库写集成测试 tests/api.rs;用 cargo doc --open 生成文档

运行示例: cargo run -p math-cli -- add 3 5 → 8; cargo run -p math-cli -- avg 1 2 3 4 → 2.5; cargo run -p math-cli -- quiz → 随机题目与判分。

第一步 **一步:**建目录 math-ws,写根 Cargo.toml 声明 workspace;cargo new --lib math-lib 与 cargo new math-cli

第二步 **二步:**在 math-lib 建 src/basic.rs 与 src/stats.rs,lib.rs 声明并重导出

第三步 **三步:**写单元测试,cargo test -p math-lib 全绿

第四步 **四步:**math-cli 加 path 依赖,写 main:match 子命令分发

第五步 **五步:**cargo add rand,实现 quiz 子命令;cargo run -p math-cli 验证三种用法

参考答案要点

```
// ===== 实验 13:数学库 + CLI 工程 =====
// 覆盖知识点:workspace(§13.7)、模块与文件(§13.3)、重导出(§13.5)、
// 可见性(§13.4)、测试(§13.6)、依赖管理(§13.6)

// ---- 根 Cargo.toml ----
// [workspace]
// members = ["math-lib", "math-cli"]
// resolver = "2"

// ---- math-lib/Cargo.toml ----
// [package] name = "math-lib" version = "0.1.0" edition = "2021"

// ---- math-lib/src/lib.rs(库根:声明 + 重导出门面)----
pub mod basic;           // 模块文件 src/basic.rs
pub mod stats;           // src/stats.rs
// 门面重导出:依赖方 use math_lib::{add, mean};
pub use basic::{add, div, mul, sub};
pub use stats::{mean, variance};

// ---- math-lib/src/basic.rs ----
/// 加法
```

```

pub fn add(a: i64, b: i64) -> i64 { a + b }
pub fn sub(a: i64, b: i64) -> i64 { a - b }
pub fn mul(a: i64, b: i64) -> i64 { a * b }

/// 除法:除零返回 None(错误进类型,而非崩溃)
pub fn div(a: i64, b: i64) -> Option<i64> {
    if b == 0 { None } else { Some(a / b) }
}

// ---- math-lib/src/stats.rs ----
/// 均值:f64 向量
pub fn mean(xs: &[f64]) -> f64 {
    if xs.is_empty() { 0.0 } else { xs.iter().sum::<f64>() / xs.len() as f64 }
}

/// 方差:与均值的平方偏差的平均
pub fn variance(xs: &[f64]) -> f64 {
    if xs.is_empty() { return 0.0; }
    let m = mean(xs);
    xs.iter().map(|x| (x - m) * (x - m)).sum::<f64>() / xs.len() as f64
}

// ---- math-lib 单元测试:与被测代码同文件 ----
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn add_works() { assert_eq!(add(2, 3), 5); }

    #[test]
    fn div_by_zero_is_none() { assert_eq!(div(1, 0), None); }

    #[test]
    fn mean_and_variance() {
        let xs = [2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0];
        assert!((mean(&xs) - 5.0).abs() < 1e-9);
        assert!((variance(&xs) - 4.0).abs() < 1e-9);
    }
}

// ---- math-lib/tests/api.rs(集成测试:外部视角)----
use math_lib::{add, mean};

#[test]
fn public_api_works() {
    assert_eq!(add(1, 1), 2);
    assert!((mean(&[1.0, 3.0]) - 2.0).abs() < 1e-9);
}

// ---- math-cli/Cargo.toml ----
// [package] name = "math-cli" version = "0.1.0" edition = "2021"
// [dependencies]
// math-lib = { path = "../math-lib" }
// rand = "0.8"

```

```
// ---- math-cli/src/main.rs ----
use rand::Rng;

fn print_usage(prog: &str) {
    eprintln!("用法:");
    eprintln!(" {prog} add <a> <b>      加法");
    eprintln!(" {prog} avg <数...>      均值");
    eprintln!(" {prog} quiz                随机练习");
}

fn main() {
    let args: Vec<String> = std::env::args().collect();
    let prog = args[0].as_str();
    let Some(cmd) = args.get(1).map(String::as_str) else {
        print_usage(prog);
        std::process::exit(2);
    };

    match cmd {
        "add" => {
            let a: i64 = args[2].parse().expect("数字");
            let b: i64 = args[3].parse().expect("数字");
            println!("{}", a + b = {})", a, b, math_lib::add(a, b));
        }
        "avg" => {
            let xs: Vec<f64> = args[2..]
                .iter().map(|s| s.parse().expect("数字")).collect();
            println!("均值:{}", math_lib::mean(&xs));
        }
        "quiz" => {
            let mut rng = rand::thread_rng();
            let mut score = 0u32;
            for _ in 0..5 {
                let a: i32 = rng.gen_range(1..=20);
                let b: i32 = rng.gen_range(1..=20);
                let op = rng.gen_range(0..2); // 0=+, 1=-
                let (question, answer) = if op == 0 {
                    (format!("{}", a + b"), a + b)
                } else {
                    (format!("{}", a - b"), a - b)
                };
                println!("第 {} 题: {question} = ?", score + 1);
                let mut input = String::new();
                std::io::stdin().read_line(&mut input).unwrap();
                if input.trim() == answer.to_string() {
                    score += 1;
                    println!("✓ 正确");
                } else {
                    println!("✗ 答案是 {}", answer);
                }
            }
            println!("得分: {score}/5");
        }
        other => {

```

```
        eprintln!("未知命令:{other}");
        print_usage(prog);
        std::process::exit(2);
    }
}
```

设计说明:本实验的骨架就是生产小工具的完整形态——库与二进制分离(逻辑可测试、可复用)、门面重导出(内部结构自由调整)、pub(crate) 防泄漏、测试三件套、第三方依赖一句话接入。注意两个细节:div 返回 Option 而非崩溃(第9章"错误进类型");let-else(第3章)处理缺失子命令。进阶练习:把 quiz 的输入解析换成 Result(非法输入不 panic)、给 math-lib 加 #[deny(missing_docs)] 并用 cargo doc 验证——那时你就拥有一个"发布级"库的完整姿势了。

下一章预告:第14章 类型系统进阶——Option 空安全、enum 和类型、newtype 模式、类型推导边界与 impl Trait 返回;全书的收官之章。

第14章 类型系统进阶

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已完成前13章;熟悉 C 的类型、NULL、typedef、union 与 enum

🎯 本章学习目标

- 掌握类型推导的边界:什么时候编译器能猜、什么时候必须显式标注
- 掌握 Option 的完整用法,理解 空安全:可选性进入类型系统,消灭 NULL 事故
- 掌握 enum 和类型:用类型表达"选项集合",与 C 的 enum+union+tag 对照
- 掌握 newtype 模式:用类型包装消灭单位混淆与"类型即文档"
- 深化值语义 vs 引用语义:Copy/Clone/借用三态在生产代码里的完整图景
- 理解类型系统与 trait 的交汇:impl Trait 返回、?Sized、类型别名
- 用领域建模案例(订单系统)把全书知识串成完整闭环
- 获得"进入生产后不再有语法困惑"的最后一块拼图

💡 从 C 到 Rust:本章主题如何迁移

全书最后一章,回答一个总问题:C 程序员的类型直觉,在 Rust 里哪里需要升级?答案是三处:**第一,可选性**——C 的 NULL 是"值层面的哨兵",谁都能传、传了才知道错;Rust 的 Option<T> 是"类型层面的事实",函数签名直接声明"这里可能没有值",编译器强迫你处理。**第二,区分度**——C 的 typedef 只是别名,int 换了个名字还是 int,米和秒照样能相加;Rust 的 newtype 让"米"和"秒"成为两个互不相通的类型,单位错误在编译期暴露。**第三,值 vs 引用**——C 只有"值/指针"两态,指针传参的意图(只读/修改/转移)全靠注释;Rust 的值/借用/移动三态(第4章)是类型系统的一部分,函数签名即意图。

这三处升级的共同本质:把"约定"变成"类型"。C 靠注释和纪律维持的约定,在 Rust 里由编译器强制执行。本章前三节讲 Option 与 enum(建模可选与选项),§14.4 讲 newtype(建模区分),§14.5 讲值/引用语义(建模所有权),§14.6 讲与 trait 的交汇,最后 §14.7 用订单领域模型把全书 14 章串成一张生产地图。

📦 前置知识自查

- C 的 NULL、typedef、union、enum、指针传参的三种意图(只读/修改/转移)
- 第1章的 let 与类型标注、第2章的运算符、第3章的 match/if let
- 第4章的参数传递三态、第5章的所有权/借用、第6章的 trait
- 第9章的 Result 与 ?(本章 Option 与 Result 对照)
- 第13章的模块与 crate(本章示例使用库结构)

本章学习路线

1. **第一阶段(约1小时):**§14.1–§14.2,类型推导边界与 Option——把"可能没有"变成类型事实
2. **第二阶段(约1.5小时):**§14.3–§14.4,enum 建模与 newtype——领域建模的两大武器
3. **第三阶段(约1.5小时):**§14.5–§14.6,值/引用语义与 trait 交汇——类型系统的完整图景
4. **第四阶段(约1小时):**§14.7,订单领域模型实验与全书收官——把 14 章串成闭环
5. **验收标准:**给你一个业务需求,能用"enum 建模状态、newtype 建模单位、Option 建模可选、trait 建模行为"给出完整设计

本章知识导图

- §14.1 类型推导:边界与默认规则
- §14.2 Option:空安全 vs C 的 NULL
- §14.3 enum 和类型:状态与选项的建模
- §14.4 newtype 模式:类型即文档
- §14.5 值语义 vs 引用语义:深化
- §14.6 类型系统与 trait 的交汇
- §14.7 收官:订单领域模型与全书地图
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§14.1 类型推导:边界与默认规则

14.1.1 哪里能推导,哪里必须显式

```
fn main() {  
    // 能推导:let 绑定、函数调用结果  
    let x = 42;                // i32(整数默认)  
    let y = 3.14;              // f64(浮点默认)  
    let s = String::from("hi"); // 构造器决定类型  
  
    // 必须显式:函数签名(类型是契约的一部分)  
    // fn f(x) { ... }          // ERROR: 必须写 fn f(x: i32)  
  
    // 必须显式:泛型函数的类型参数(编译器不知道你要哪个)  
    let n: u32 = "42".parse().expect("字面量必然可解析"); // 类型标注决定 parse 的目标  
    // "42".parse();          // ERROR: 无法推断  
  
    // 必须显式:集合元素的类型(空集合无法从内容推断)  
    let empty: Vec<i32> = Vec::new();  
    let m: std::collections::HashMap<u32, String> = Default::default();  
    println!("{n} {empty:?} {:?}", m.len());  
}
```


14.1.2 推导的边界:不跨越函数与闭包体

```
// 推导是"局部"的:一个函数内的类型问题,编译器不会到另一个函数里找答案
fn identity<T>(x: T) -> T { x }

fn main() {
    // 泛型调用需要"推断点":赋值目标或参数类型
    let a: u64 = identity(5);           // T = u64
    let b = identity(5u8);             // 字面量后缀给出 T
    println!("{a} {b}");

    // 闭包参数:能推导时省略标注;需要时显式
    let double = |x| x * 2;            // 从使用处推导
    // let trouble = |x| { 复杂到推断不出 }; // 罕见;显式标注参数即可
    let f = |x: f64| x * 2.0;          // 显式标注参数类型
    println!("{}", double(21), f(1.5));
}
```

⚠ 易错点 1:整数默认 i32,浮点默认 f64——循环与索引的坑

let n = 0; 是 i32;数组长度是 usize。索引/取长度相关的推导冲突(IndexError: expected usize)是 C 转 Rust 高频报错——C 里 int 到处通用,Rust 里 usize 与 i32 不互转(第2章零隐式转换)。对策:循环计数与集合下标统一用 usize;需要数学运算时再显式转换 as i64 等。

§14.2 Option:空安全 vs C 的 NULL

14.2.1 NULL 的灾难与 Option 的答案

```
// C:NULL 是"值层面的哨兵",函数签名看不出会不会返回 NULL
// const char *find_name(int id);    // 可能返回 NULL?看文档才知道
// char *p = find_name(42);
// strcpy(p, "x");                    // p 是 NULL?崩溃(或者更糟:静默)

// Rust:可选性写在签名里,编译器强迫处理
fn find_name(id: u32) -> Option<String> {    // 明确:可能没有
    if id == 42 {
        Some(String::from("小明"))
    } else {
        None                                // 显式表示"没有"
    }
}

fn main() {
    let name = find_name(7);
    // 三种处理方式:
    // 1) match 穷尽(第3章)
    match &name {
        Some(n) => println!("找到:{n}"),
        None => println!("未找到"),
    }
    // 2) 链式处理:map 只对 Some 生效
    let len = name.as_ref().map(|n| n.len()).unwrap_or(0);
    println!("长度:{len}");
    // 3) 不存在的值不可能"被当作存在"使用——类型检查兜底
}
```

14.2.2 Option 的常用组合拳

```
fn main() {
    let a: Option<i32> = Some(10);
    let b: Option<i32> = None;

    // map:Some 则变换,None 保持 None
    println!("{:?}", a.map(|x| x * 2));      // Some(20)
    println!("{:?}", b.map(|x| x * 2));      // None

    // and_then:变换结果也是 Option(展平),对照 C 的"返回值再判空"
    println!("{:?}", a.and_then(|x| if x > 5 { Some(x - 5) } else { None })); // Some(5)

    // unwrap_or / unwrap_or_else:兜底值(延迟计算用闭包)
    println!("{}", b.unwrap_or(0));          // 0
    println!("{}", a.unwrap_or_else(|| 99)); // 10

    // ? 运算符:Option 与 Result 可以互转(第9章 Result)
    // fn f() -> Option<i32> { let x = a?; Some(x + 1) }

    // 判空/取引用
    println!("is_some:{}, is_none:{}", a.is_some(), b.is_none());
    println!("{:?}", a.as_ref());            // Some(&10):借用不移动
}
```

C 的空值处理	Rust Option
NULL 指针 / 哨兵值(-1、EOF)	Option<T>:Some(T) / None,类型级表达
返回值是否可能为空:看文档/注释	签名自带:Option<T> 就是"可能没有"
忘记判空:运行期崩溃或 UB	编译期强迫处理:None 无法当作 T 用
判空后层层 if 缩进	map / and_then / ? 链式,无嵌套
空串/0 作为"伪空"的妥协	None 是唯一空值,语义精确

§14.3 enum 和类型:状态与选项的建模

14.3.1 用类型表达"选项集合"

```
// C 的 enum + union + tag 三件套 → Rust 一个 enum 搞定
// C:
//   enum OrderStatus { PENDING, PAID, SHIPPED, CANCELLED };
//   // 状态值只是数字;switch 忘了处理新状态?编译器不检查

// Rust:enum 是完整的类型,携带数据,穷尽检查
enum OrderStatus {
    Pending,                                // 无数据
    Paid { amount: f64, time: String },    // 带数据的变体(对照 union)
    Shipped(String),                       // 带单个数据:运单号
    Cancelled(String),                     // 原因
}

fn describe(status: &OrderStatus) -> String {
    match status {
        OrderStatus::Pending => "等待支付".to_string(),
        OrderStatus::Paid { amount, .. } => format!("已支付 {amount} 元"),
        OrderStatus::Shipped(no) => format!("已发货,运单 {no}"),
        OrderStatus::Cancelled(reason) => format!("已取消:{reason}"),
        // match 穷尽:以后新增变体,这里编译错误,逼你处理
    }
}

fn main() {
    let s = OrderStatus::Paid { amount: 99.0, time: "10:00".into() };
    println!("{}", describe(&s));
}
```

14.3.2 enum 作为"选项集"的实战形态

```
// 生产高频模式:enum 表示"几种可能的形态",数据随变体走
enum Config {
    Port(u16),                // 单个值
    Addr { host: String, port: u16 }, // 命名结构
}

impl Config {
    fn display(&self) -> String {
        match self {
            Config::Port(p) => format!("只配端口:{p}"),
            Config::Addr { host, port } => format!("{host}:{port}"),
        }
    }
}

fn main() {
    let c1 = Config::Port(8080);
    let c2 = Config::Addr { host: "0.0.0.0".into(), port: 9090 };
    println!("{}", c1.display(), c2.display());
    // C 对照:union + kind 字段 + switch;Rust 把三样收进一个类型,
    // 且"哪个变体有哪些字段"由编译器检查—取错字段直接编译错误
}
```

⚠ 易错点 2:enum 里塞"状态数据" vs 用字段表达

状态机建模的常见分歧:状态相关的数据放 enum 变体里(如上例),与状态无关的数据放 struct 字段里。判据:该数据是否只在某状态下存在?是→放变体;否→放结构体字段。把"所有状态共用的数据"塞进变体,会让 match 每个分支重复处理,还容易漏。生产直觉:先画状态图,再定 enum,最后定 struct 字段。

§14.4 newtype 模式:类型即文档

14.4.1 typedef 的局限与 newtype 的升级

```
// C:typedef 只是别名,米和秒都是 double,照样能相加
// typedef double meters;
// typedef double seconds;
// meters m = 100.0; seconds s = 5.0;
// double v = m / s;    // 合法但..... m/s 是速度,单位混淆编译器不管

// Rust:newtype—包一层,单位成为独立类型
struct Meters(f64);
struct Seconds(f64);

impl Meters {
    fn new(v: f64) -> Meters { Meters(v) }
    fn value(&self) -> f64 { self.0 }
}

fn speed(distance: Meters, time: Seconds) -> f64 {
    distance.value() / time.value()    // 取内部值需显式方法
}

fn main() {
    let d = Meters::new(100.0);
    let t = Seconds::new(5.0);
    println!("速度:{}", speed(d, t));
    // speed(t, d);          // ERROR:类型不匹配—单位错了,编译期报错!
    // let wrong = d + t;    // ERROR:Meters 与 Seconds 不能相加(无 Add impl)
}
```

14.4.2 newtype 的三个生产用途

```
// 用途一:单位/概念区分(上例)
// 用途二:为外部类型附加本 crate 的方法(孤儿规则限制的解药)
use std::collections::HashMap;

struct UserMap(HashMap<u32, String>);    // 包装外部类型

impl UserMap {
    fn new() -> UserMap { UserMap(HashMap::new()) }
    fn insert(&mut self, id: u32, name: &str) {
        self.0.insert(id, name.to_string());
    }
    fn total(&self) -> usize { self.0.len() }
}

// 用途三:给"同一底层类型"的不同角色加保护
struct RawText(String);    // 未净化的文本
struct Sanitized(String);  // 已净化的文本
// 只有 Sanitized 才能进渲染函数 → 数据流的安全阶段在类型里可见

fn main() {
    let mut users = UserMap::new();
    users.insert(1, "张三");
    println!("用户数:{}", users.total());

    let raw = RawText("<script>alert(1)</script>".to_string());
    // render(&raw);    // ERROR:RawText 不是 Sanitized—类型防线
    let _safe = Sanitized(raw.0);    // 显式净化转换
    println!("净化管线类型安全");
}
```

C	Rust
typedef double meters;只是别名	struct Meters(f64);独立类型,互不兼容
单位混淆:编译通过,运行期算错	单位混淆:编译错误
int 的"多个角色"靠命名约定	newtype 让角色成为类型
外部类型不能加方法	newtype 包装后可为包装类型实现 trait
数据流安全靠纪律	RawText/Sanitized 式管道,编译器把关

§14.5 值语义 vs 引用语义:深化

14.5.1 三条语义轴

```
// 轴一:复制(Copy)vs 移动(Move)
// 数字/字符/小值:Copy,赋值即拷贝
let a = 5i32;
let b = a; // 拷贝:b 与 a 都是 5,独立
// 大值/String/Vec:Move,赋值转移所有权
let s = String::from("hi");
let t = s; // 移动:s 失效
// println!("{}", s); // ERROR: use of moved value

// 轴二:共享借用(&T,只读)vs 独占借用(&mut T,可写)
// 规则(第5章):不可变借用可无数;可变借用唯一
// 对应 C:const T* 与 T*—但 Rust 的规则是编译器强制的

// 轴三:所有权(拥有/释放)vs 借用(使用不拥有)
// Box/String/Vec:拥有; &T:借用;生命周期由编译器管

#[derive(Debug, Clone)]
struct Big { data: Vec<u32> } // 大类型:Move + 显式 clone

fn main() {
    let mut big = Big { data: vec![1, 2, 3] };
    let copy_of_big = big.clone(); // 显式深拷贝(对照 memcpy)
    let borrowed = &big; // 共享借用:可同时多个
    println!("{:?} {:?}", borrowed, copy_of_big);

    let exclusive = &mut big; // 独占借用
    exclusive.data.push(4);
    println!("{:?}", big); // 借用结束,回到拥有者
}
```

14.5.2 从 C 的指针三意图到 Rust 的三态

C 的传参意图	C 写法	Rust 三态
只读查看	const T* p	&T(共享借用)
就地修改	T* p	&mut T(独占借用)
转移所有权	T* p + 约定"调用方负责 free"	T(按值传入,所有权转移)
复制一份	memcpy / 手写	T: Clone 的 .clone() / T: Copy

★ 重点:C 的意图靠注释,Rust 的意图是类型

读 C 代码时,你要从 `void f(T* p, const T* q)` 里猜哪个是读、哪个是写、哪个是转移——猜错的代价是悬垂与泄漏。Rust 的函数签名直接回答这三个问题: `f(&T)` 不拥有、`f(&mut T)` 独占改写、`f(T)` 拿走所有权。这三态(第4章)与所有权(第5章)是全书的心跳;本章的深化在于:把三态放进"值语义 vs 引用语义"的完整坐标里——Copy 类型天然值语义,C 的"struct 按值传"在这里得到类型化;引用语义类型(字符串、容器、流)的共享与独占由借用规则管。

§14.6 类型系统与 trait 的交汇

14.6.1 impl Trait 返回:隐藏类型,保留契约

```
// 返回"实现了某 trait 的类型",但不暴露具体类型
fn make_counter() -> impl Iterator<Item = u32> {
    (1..=5) // 具体类型是 RangeInclusive,调用方不需要知道
}

fn main() {
    let total: u32 = make_counter().sum();
    println!("总和:{total}");

    // 与泛型的对照:返回位置 impl Trait 是"匿名具体类型";
    // 调用方只能按 trait 用,不能做 trait 之外的操作——契约即边界
}
```

14.6.2 Sized 与 ?Sized、类型别名

```
// 默认 T: Sized(大小编译期已知);dyn Trait 与 [T] 是不定大小
fn takes_sized<T>(_x: T) {} // T 必须有确定大小(默认)

// ?Sized:允许不定大小(如 &dyn Trait 的借用)
fn accepts_any<T: ?Sized>(_x: &T) {} // &str / &dyn Trait 都行

// 类型别名:与 C typedef 相同(纯别名,不提供 newtype 的安全性)
type Price = f64; // 别名:Price 就是 f64
type Result2<T> = std::result::Result<T, String>; // 简化签名

fn main() {
    let p: Price = 99.9; // 与 f64 完全互通
    println!("{}", p);
    let r: Result2<u32> = Ok(1);
    println!("{}", r);

    // 别名的取舍:C 的 typedef 习惯在 Rust 里的正确位置:
    // 需要"简洁"用 type;需要"区分与安全"用 newtype(§14.4)
}
```

14.6.3 类型系统全景:三股力量

力量	工具	解决的问题
可选性	Option / Result	"可能没有"与"可能失败"进入类型
区分度	newtype / enum / 类型参数	单位、角色、状态不再混淆
行为契约	trait / trait bound / dyn	"能做什么"由类型声明

§14.7 收官:订单领域模型与全书地图

14.7.1 全书 14 章的知识闭环

章节	一句话贡献
1 数据类型与变量	let 绑定、强类型、类型系统带着所有权
2 运算符与表达式	无隐式转换、无 ++、表达式风格
3 流程控制	match 穷尽、if let、loop 带值
4 函数与方法	值/借用/移动三态、闭包三态
5 内存管理	所有权三法则、生命周期、智能指针
6 面向对象与 trait	行为契约、组合优于继承、两种分派
7 文件与 IO	Result 世界、路径类型、serde
8 网络通信	TCP/UDP、字节流定界、HTTP
9 并发与异常	错误三件套、Send/Sync、通道、tokio
10 泛型与元编程	单态化、trait bound、宏
11 反射与动态特性	无反射;Any/宏的编译期替代
12 注解与编译期配置	属性系统、#[cfg]、features
13 模块与 Cargo 工程	模块树、Cargo 全流程、workspace
14 类型系统进阶	Option、enum、newtype、语义三轴(本章)

14.7.2 进入生产后的三条心法

心法一:类型先行

写业务前先定义类型:状态用 enum、可选用 Option、单位用 newtype、行为用 trait——类型定义得越精确,后续 match/分支越少,编译器帮你查的错误越多。C 程序员最划算的习惯升级:写函数前先写签名,签名即文档,签名即测试。

心法二:编译错误是老师的答案

Rust 编译器报错时,报错信息通常指向"缺哪个 trait、哪个生命周期、哪个所有权转移"。生产里 90% 的修复路径:读错误 → 补约束/换借用 → 过。不要与编译器搏斗,顺着它的引导走。

🚀 心法三:unsafe 是边界,不是习惯

FFI、裸指针、极致性能才碰 unsafe,并用 unsafe 块包裹、注释说明不变量(第5章)。Rust 的安全保证是生产级代码的护城河——护城河外的操作,要像 C 一样小心,但范围要缩到最小。

⚠️ 易错点 3:newtype 不是越多越好

newtype 的安全来自"转换要显式",但每个 newtype 都增加一层 .0/方法的样板。判据:两个概念在业务上是否真的"混用即事故"?物理量、货币、ID、已净化文本——是,值得包;同义别名(两个都只是"一段文本")——用 type 别名或直接用 String。过度包装会让代码像裹了多层保鲜膜,可读性被样板反噬。生产经验:先以 type 别名起步,出现"传错角色"的真实事故记录后再升级为 newtype,让类型跟随痛苦而生长。

本章速查表

主题	结论 / 写法
类型推导	let 绑定/调用结果可推导;函数签名、泛型参数、空集合必须显式
整数/浮点默认	i32 / f64;索引与长度用 usize(与 i32 不隐式互转)
Option 形态	Some(T) / None;签名自带"可能没有"
Option 处理	match / if let / map / and_then / unwrap_or / ?
enum 建模	变体可带数据:enum S { A, B {x: f64}, C(String) }
穷尽检查	match 必须覆盖全部变体;新增变体编译报错逼你处理
newtype	struct Meters(f64);互不兼容的类型;单位/角色/外部类型包装
type 别名	type Price = f64;纯别名,无安全性(与 typedef 相同)
语义三轴	Copy/Move;共享借用 &T;独占借用 &mut T;所有权/借用
impl Trait 返回	fn f() -> impl Iterator<Item = u32>;隐藏类型保留契约
?Sized	<T: ?Sized> 允许不定大小(如 &dyn Trait)
类型系统三力	可选性(Option/Result)+ 区分度(newtype/enum)+ 契约(trait)

最小必会清单

- 能说出类型推导的边界(签名/泛型/空集合必须显式)与默认类型规则
- 能用 Option 建模"可能没有",并说出与 C NULL 的本质差别
- 能写出带数据的 enum,并利用 match 穷尽检查获得编译期保障
- 能用 newtype 建模单位/角色,说出与 typedef 的本质差别
- 能说清 Copy/Clone/Move/借用之间的关系与各自场景
- 能从函数签名读出"只读/改写/转移"三态(第4章能力的最终形态)
- 会用 impl Trait 返回隐藏具体类型
- 能区分 type 别名与 newtype 的适用场景
- 能用"类型先行"的方法为业务建模(enum 状态 + newtype 单位 + Option 可选)

- 能画出全书 14 章的知识地图,并说出每章在生产中的角色

自测题

Q 1. 为什么函数参数必须写类型,而 let 绑定可以不写?这对 C 程序员意味着什么?

✓ 参考答案

函数签名是跨调用方的契约:类型是接口的一部分,编译器要据此检查所有调用点;let 绑定是局部事实,可从初始化表达式推导。对 C 程序员:这比 C 更严格也更安全——C 的函数声明(头文件)里类型不齐是常态,靠调用方猜;Rust 的签名即全部契约,猜的部分由编译器完成。

Q 2. Option<T> 与 C 的"返回 NULL / 哨兵值"相比,消除了哪几类 bug?

✓ 参考答案

三类:① 忘记判空——None 无法被当作 T 使用,编译期强制处理;② 语义歧义——C 里 0/-1/空串都当过"空",Option 只有 None 一个空值;③ 误判生命周期——C 的返回指针可能是悬垂(指向已释放栈),Option 的值语义/借用规则在类型层排除。剩下的"选择",用 map/? 链式表达,不产生嵌套缩进。

Q 3. C 的 enum + union + tag 与 Rust 的 enum,编译期保障差在哪里?

✓ 参考答案

C:enum 是数字集合,switch 漏掉新分支不报错(或靠 -Wswitch 部分检查);union 字段与 tag 的对应靠程序员约定,取错字段是 UB。Rust:enum 是代数数据类型,变体与携带数据一一对应,取某变体的数据必须先匹配该变体(编译器检查);match 穷尽性保证新增变体时所有分支都被审计——"状态机漏分支"从运行期灾难变成编译期必报。

Q 4. typedef double meters 与 struct Meters(f64) 在生产上差出什么?

✓ 参考答案

typedef 的 meters 仍是 double:米与秒、价格与数量可以互相赋值、相加,单位错误全部留到运行期(数据算错,最危险的那种 bug)。newtype 的 Meters 是与 f64 互不兼容的独立类型:传错单位编译失败;需要取值必须显式 .0/方法——每个单位转换在代码里可见。生产收益:涉及物理量、货币、ID 的系统,newtype 把一整类事故消灭在编译期。

Q 5. 从函数签名 &T / &mut T / T 分别读出的契约是什么?C 里对应的注释长什么样?

✓ 参考答案

&T:只读共享,调用方保留所有权,可能同时被多处读;&mut T:独占改写,调用期间别无借用;T:所有权转移,调用方不再拥有。C 的对应:const T* / T* / "T* 且约定调用方负责 free"。差别:前两者编译器强制(违反借用规则编译失败),转移在 C 里只是口头约定——所以 C 的悬垂与 double-free 频发,而 Rust 的签名即保证。

Q 6. 一个电商系统里,订单金额、商品 ID、用户 ID 都用 u64 表示,会出什么问题?如何修复?

✅ 参考答案

问题:三者的 u64 互相兼容,传参错位(把商品 ID 当金额、把用户 ID 当商品 ID)编译通过、运行期产生错误业务数据——这是"类型区分度"缺失。修复:newtype 建模:struct Money(u64); struct ProductId(u64); struct UserId(u64);各配实现。从此金额函数只收 Money,ID 函数只收对应 ID,错位在编译期报错。配套:enum 建模订单状态、Option 建模"优惠券可能没有",这就是"类型先行"的完整生产形态。

章末实验题:订单领域模型

📄 实验 14:订单领域模型

实验目标:用本章全部武器建模一个订单系统:newtype 建模金额与 ID、enum 建模订单状态与支付方式、Option 建模可选信息、trait 建模行为;并写一个命令行演示全流程。

实验要求:

- 任务 1:newtype:Money、ProductId、UserId(各自实现 Display)(覆盖:newtype §14.4)
- 任务 2:enum OrderStatus 与 PaymentMethod(带数据变体)(覆盖:enum §14.3)
- 任务 3:struct Order:用户、商品列表、金额、状态、可选优惠券(覆盖:Option §14.2)
- 任务 4:impl 方法:total(汇总金额)、apply_coupon(返回 Result 或 Option)(覆盖:impl 与错误 §9/§14.2)
- 任务 5:trait Billable:Order 与 Invoice 都实现"应付款"行为(覆盖:trait §6)
- 任务 6:main 演示:建订单 → 加优惠券 → 支付 → 状态流转,全程类型安全(覆盖:收官 §14.7)
- 加分:尝试把 Order::total 的金额类型改成 u64 直接相加,观察编译错误(单位防线)

第一步 一步:cargo new order-demo;先写三个 newtype 与 Display

第二步 二步:写两个 enum(状态与支付方式),为状态写描述方法

第三步 三步:写 Order 结构与方法(字段含 Option<Coupon>)

第四步 四步:写 Billable trait,Order 与 Invoice 各自实现

第五步 五步:main 里完整走一遍流程,编译通过后做加分项(去掉 Money 看编译错)

✅ 参考答案要点

```
// ===== 实验 14:订单领域模型 =====
// 覆盖知识点:newtype 建模(§14.4)、enum 状态建模(§14.3)、
//          Option 可选性(§14.2)、trait 行为契约(§6)、
//          impl 方法与错误处理(§9)、收官串联(§14.7)

use std::fmt;

// ---- 任务 1:newtype 三件套:单位与角色独立 ----
struct Money(u64);           // 金额:分
struct ProductId(u64);
struct UserId(u64);

impl fmt::Display for Money {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{:.2} 元", self.0 as f64 / 100.0)
```

```

    }
}

impl fmt::Display for ProductId {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "商品#{}", self.0)
    }
}

impl fmt::Display for UserId {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "用户#{}", self.0)
    }
}

impl Money {
    fn add(&self, other: &Money) -> Money { Money(self.0 + other.0) }
    fn is_zero(&self) -> bool { self.0 == 0 }
}

// ---- 任务 2:enum 状态与支付方式 ----
enum OrderStatus {
    Pending, // 待支付
    Paid { method: PaymentMethod, at: String }, // 已支付(带数据)
    Shipped(String), // 已发货(运单号)
    Cancelled(String), // 已取消(原因)
}

enum PaymentMethod {
    WechatPay,
    Alipay,
    BankCard(String), // 尾号
}

impl OrderStatus {
    fn describe(&self) -> String {
        match self {
            OrderStatus::Pending => "待支付".to_string(),
            OrderStatus::Paid { method, .. } => {
                format!("已支付({})", match method {
                    PaymentMethod::WechatPay => "微信支付",
                    PaymentMethod::Alipay => "支付宝",
                    PaymentMethod::BankCard(last4) => {
                        return format!("已支付(银行卡尾号 {last4})");
                    }
                })
            },
            OrderStatus::Shipped(no) => format!("已发货(运单 {no})"),
            OrderStatus::Cancelled(reason) => format!("已取消:{reason}"),
        }
    }
}

// ---- 任务 2 补充:优惠券(可选信息) ----
struct Coupon {

```

```

    code: String,
    discount: Money,                                // 减免金额
}

// ---- 任务 3:Order 结构 ----
struct Order {
    id: u32,
    user: UserId,
    products: Vec<(ProductId, Money)>,              // 商品与单价
    status: OrderStatus,
    coupon: Option<Coupon>,                          // 可能没有优惠券
}

// ---- 任务 4:订单方法 ----
impl Order {
    fn new(id: u32, user: UserId) -> Order {
        Order {
            id,
            user,
            products: Vec::new(),
            status: OrderStatus::Pending,
            coupon: None,
        }
    }

    fn add_product(&mut self, pid: ProductId, price: Money) {
        self.products.push((pid, price));
    }

    /// 汇总金额:商品总价减去优惠券减免(减不了就报错)
    fn total(&self) -> Result<Money, String> {
        let raw = self.products.iter()
            .fold(Money(0), |acc, (_, p)| acc.add(p));
        match &self.coupon {
            None => Ok(raw),
            Some(c) if c.discount.0 > raw.0 => {
                Err(format!("优惠券 {} 减免超总价", c.code))
            }
            Some(c) => Ok(Money(raw.0 - c.discount.0)),
        }
    }

    /// 支付:仅待支付状态可支付(状态机在类型层把关)
    fn pay(&mut self, method: PaymentMethod) -> Result<(), String> {
        match &self.status {
            OrderStatus::Pending => {
                self.status = OrderStatus::Paid {
                    method,
                    at: "2026-08-26 12:00".to_string(),
                };
                Ok(())
            }
            _ => Err(format!("订单 {} 当前状态不可支付", self.id)),
        }
    }
}

```

```

}

// ---- 任务 5:trait 行为契约:可开票对象 ----
trait Billable {
    fn amount(&self) -> Result<Money, String>;
    fn describe(&self) -> String;
}

impl Billable for Order {
    fn amount(&self) -> Result<Money, String> { self.total() }
    fn describe(&self) -> String {
        format!("订单#{ } { } { }", self.id, self.user, self.status.describe())
    }
}

struct Invoice { no: String, items: Vec<Money> }

impl Billable for Invoice {
    fn amount(&self) -> Result<Money, String> {
        Ok(self.items.iter().fold(Money(0), |a, i| a.add(i)))
    }
    fn describe(&self) -> String {
        format!("发票#{ } 共 { } 项", self.no, self.items.len())
    }
}

fn print_bill(b: &impl Billable) {
    match b.amount() {
        Ok(m) => println!("{ } -> 应付 {m}", b.describe()),
        Err(e) => eprintln!("{ } -> 错误:{e}", b.describe()),
    }
}

// ---- 任务 6:main 全流程演示 ----
fn main() {
    let mut order = Order::new(1, UserId(42));
    order.add_product(ProductId(1001), Money(1999)); // 19.99 元
    order.add_product(ProductId(1002), Money(3000)); // 30 元
    order.coupon = Some(Coupon {
        code: "SAVE5".into(),
        discount: Money(500), // 减 5 元
    });

    print_bill(&order); // 应付 44.99 元
    let _ = order.pay(PaymentMethod::WechatPay);
    print_bill(&order); // 已支付(微信支付)

    // 错误路径:重复支付被拒(状态机)
    match order.pay(PaymentMethod::Alipay) {
        Ok(_) => println!("不该成功"),
        Err(e) => println!("预期错误:{e}"),
    }

    // 发票同样可开票(多态:同一契约,不同实现)
    let invoice = Invoice {

```



```

        no: "INV-001".into(),
        items: vec![Money(1999), Money(500)],
    };
    print_bill(&invoice);

    // 加分演示:直接 u64 相加会被类型拒绝—
    // let bad: u64 = order.total().unwrap().0 + 100; // 需显式 .0,看得见单位
    println!("领域模型演示完毕");
}

```

设计说明:全章的武器在一百行内协同——newtype 保证金额与 ID 不会互换,enum 让订单状态机在类型里显形(match 穷尽保证每个状态都被处理),Option 让"优惠券可能没有"成为签名事实,trait 让订单与发票共用一套"开票"逻辑(第6章组合思维),Result 让"优惠券超抵""状态不可支付"成为显式错误。这就是"类型先行"的完整示范。进阶挑战:给 Money 实现 PartialOrd 与 Add(第6章运算符重载),给 Order 加"只能从未发货状态取消"的状态机守卫——做完这两步,你的 Rust 生产级建模能力就毕业了。

全书完。14 章,从 let 绑定到领域建模,从 malloc/free 到所有权,从 errno 到 Result——愿这本书成为你从 C 到 Rust 的桥,愿你进入生产后不再有语法困惑。